



Guia do usuário da versão 2

AWS Command Line Interface



AWS Command Line Interface: Guia do usuário da versão 2

Copyright © 2024 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

As marcas comerciais e imagens comerciais da Amazon não podem ser usadas no contexto de nenhum produto ou serviço que não seja da Amazon, nem de qualquer maneira que possa gerar confusão entre clientes ou que deprecie ou desprestige a Amazon. Todas as outras marcas comerciais que não pertencem à Amazon pertencem a seus respectivos proprietários, que podem ou não ser afiliados, conectados ou patrocinados pela Amazon.

Table of Contents

.....	ix
Sobre o AWS CLI	1
Sobre a AWS CLI versão 2	1
Manutenção e suporte para as versões principais do SDK	2
Sobre a Amazon Web Services	2
Sobre os exemplos	2
Documentação e recursos adicionais	4
Documentação e recursos da AWS CLI	4
Outros AWS SDKs	4
Conceitos básicos	6
Pré-requisitos	7
Crie uma conta administrativa do IAM ou do IAM Identity Center	7
Próximas etapas	8
Instalar/atualizar	9
Instruções de instalação e atualização da AWS CLI	9
Solução de problemas de erros de instalação e desinstalação da AWS CLI	23
Próximas etapas	23
Versões anteriores	24
Solução de problemas de erros de instalação e desinstalação da AWS CLI	42
Próximas etapas	43
Compilar e instalar usando o código-fonte	43
Por que compilar usando o código-fonte?	43
Etapas rápidas	44
Etapa 1: Configurar todos os requisitos	47
Etapa 2: Configurar a instalação da AWS CLI pelo código-fonte	51
Etapa 3: Compilar a AWS CLI	58
Etapa 4: Instalar a AWS CLI	59
Etapa 5: Verificar a instalação da AWS CLI	60
Exemplos de fluxo de trabalho	61
Solução de problemas de erros de instalação e desinstalação da AWS CLI	64
Próximas etapas	64
Amazon ECR Public/Docker	64
Pré-requisitos	65
Decidir entre o Amazon ECR Public e o Docker Hub	65

Executar as imagens oficiais	65
Observações sobre interfaces e compatibilidade com versões anteriores das imagens oficiais	66
Usar versões e tags específicas	67
Atualizar para a imagem oficial mais recente	68
Compartilhar arquivos de host, credenciais, variáveis de ambiente e configuração	68
Reduzir o comando de execução do Docker	74
Configuração	77
Reunir suas informações de credencial para acesso programático	78
Definir credenciais e configurações novas	79
Usar arquivos de credenciais e configurações existentes	87
Configurar o AWS CLI	88
Precedência de credenciais e configurações	88
Tópicos adicionais nesta seção	89
Configurações do arquivo de configuração e credenciais	90
Formato dos arquivos de credenciais e configuração	90
Onde as definições de configuração ficam armazenadas?	99
Usar perfis nomeados	100
Definir e visualizar as configurações usando comandos	100
Definir novos exemplos de comandos de configuração e credenciais	103
Configurações de arquivo config compatíveis	106
Variáveis de ambiente	125
Como definir variáveis de ambiente	126
Variáveis de ambiente compatíveis da AWS CLI	127
Opções de linha de comando	137
Como usar as opções de linha de comando	138
A AWS CLI comporta opções de linha de comando globais	138
Usos comuns das opções de linha de comando	143
Conclusão de comando	144
Como funciona	144
Configuração do preenchimento automático de comando no Linux ou no macOS	145
Configuração do preenchimento de comandos no Windows	149
Repetições	150
Modos de novas tentativas disponíveis	151
Configuração um modo de nova tentativa	153
Visualização de logs de novas tentativas	155

Usar um proxy HTTP	155
Como usar os exemplos da	156
Autenticar para um proxy	157
Uso de proxy em instâncias do Amazon EC2	158
Solução de problemas	158
Endpoints	158
Definir o endpoint para um único comando	159
Definir um endpoint global para todos os Serviços da AWS	159
Definido para usar endpoints FIPs para todos os Serviços da AWS	161
Configurado para usar os endpoints de pilha dupla para todos os Serviços da AWS	162
Definir endpoints específicos de serviço	163
Precedência de configurações e definições do endpoint	167
Autenticação e credenciais de acesso	168
Precedência de credenciais e configurações	169
Tópicos adicionais nesta seção	170
Autenticação do IAM Identity Center	170
Configurar a atualização automática de tokens	171
Configuração herdada não atualizável	179
Usar um perfil do IAM Identity Center	184
Credenciais de curto prazo	188
Perfis do IAM	189
Pré-requisitos	189
Visão geral do uso de funções do IAM	189
Configurar e usar uma função	191
Usar MFA	193
Funções entre contas e ID externo	195
Especificar um nome de sessão de função para facilitar a auditoria	195
Assumir a função com a identidade da web	196
Limpar as credenciais em cache	197
IAM users	198
Etapa 1: criar o usuário do IAM	198
Etapa 2: obter as chaves de acesso	199
Configurar o AWS CLI	199
Uso de credenciais para metadados de instância do Amazon EC2.	202
Pré-requisitos	202
Configuração de um perfil para metadados do Amazon EC2	202

Credenciais externas	203
Usar a AWS CLI	206
Obter ajuda	206
O comando de ajuda integrado da AWS CLI	207
Guia de referência da AWS CLI	212
Documentação de API	212
Solucionar de problemas de erros	213
Ajuda adicional	213
Estrutura do comando	213
Estrutura do comando	213
Comandos de espera	215
Especificar valores de parâmetro	216
Tipos comuns de parâmetros	217
Aspas com strings	222
Parâmetros de arquivos	227
Gerar um modelo de esqueleto da CLI	230
Sintaxe Simplificada	242
Prompt automático	244
Como funcionam	245
Recursos do prompt automático	245
Modos de prompt automático	248
Configurar o prompt automático	249
Controlar a saída do comando	249
Output Format	250
Paginação	260
Filtragem da saída	266
Códigos de retorno	290
Assistentes	292
Como funcionam	292
Aliases	293
Pré-requisitos	294
Etapa 1: Criação do arquivo de alias	294
Etapa 2: Criação de um alias	295
Passo 3: Como chamar um alias	298
Exemplos de repositório de alias	300
Recursos	302

Exemplos de código	303
Exemplos de comando guiado	303
DynamoDB	304
Amazon EC2	308
S3 Glacier	327
IAM	334
Amazon S3	339
Amazon SNS	358
Exemplos de script Bash	360
Ações e cenários	361
Segurança	526
Proteção de dados	527
Criptografia de dados	528
Identity and Access Management	528
Público	529
Autenticando com identidades	529
Gerenciamento do acesso usando políticas	533
Como Serviços da AWS funcionam com o IAM	536
Solução de problemas de identidade e acesso do AWS	536
Compliance Validation	538
Resilience	539
Infrastructure Security	540
Aplicar uma versão mínima do TLS	541
Solucionar erros	542
Solução geral de problemas para tentar primeiro	542
Verificar a formatação do comando AWS CLI	543
Confirme se você está executando uma versão recente da AWS CLI	543
Como usar a opção --debug	544
Habilitar e analisar logs de histórico de comandos da AWS CLI	549
Confirmar se a AWS CLI está configurada	550
Erros de comando não encontrado	550
O comando "aws --version" retorna uma versão diferente da que você instalou	553
O comando "aws --version" retorna uma versão após a desinstalação da AWS CLI	554
A AWS CLI processou um comando com um nome de parâmetro incompleto	556
Erros de acesso negado	557
Credenciais inválidas e erros de chave	558

Assinatura não corresponde aos erros	560
Erros de certificado SSL	561
Erros JSON inválidos	562
Recursos adicionais	564
Guia de migração	565
Novos recursos e alterações	565
Novos recursos da AWS CLI versão 2	565
Alterações de última hora entre a AWS CLI versão 1 e a AWS CLI versão 2	567
Instruções para a migração	575
Substituindo a versão 1 pela versão 2	575
Instalação lado a lado	576
Desinstalar	578
Solução de problemas de erros de instalação e desinstalação da AWS CLI	581
Histórico do documento	582
AWS Glossário	588

O que é o AWS Command Line Interface?

A AWS Command Line Interface (AWS CLI) é uma ferramenta de código aberto que permite interagir com os serviços da AWS usando comandos no shell da linha de comando. Com o mínimo de configuração, a AWS CLI permite começar a executar comandos que implementam uma funcionalidade equivalente àquela fornecida pelo AWS Management Console baseado em navegador do prompt de comando em seu programa de terminal:

- Shells do Linux: use programas comuns de shell, como [bash](#), [zsh](#) e [tcsh](#) para executar comandos no Linux ou macOS.
- Linha de comando do Windows: no Windows, execute comandos no prompt de comando do Windows ou no PowerShell.
- Remotamente: execute comandos em instâncias do Amazon Elastic Compute Cloud (Amazon EC2) por meio de um programa de terminal remoto, como PuTTY ou SSH, ou com o .AWS Systems Manager

Toda as funções administração, gerenciamento e acesso da AWS IaaS (Infraestrutura como um serviço) no AWS Management Console estão disponíveis na API da AWS e na AWS CLI. Os novos recursos e serviços da AWS IaaS fornecem funcionalidade completa do AWS Management Console por meio da API e da CLI no lançamento ou dentro de 180 dias após o lançamento.

A AWS CLI fornece acesso direto às APIs públicas de serviços da AWS. Você pode explorar os recursos de um serviço com a AWS CLI e desenvolver scripts de shell para gerenciar seus recursos. Além dos comandos de nível inferior equivalentes a API, vários serviços da AWS fornecem personalizações para a AWS CLI. As personalizações podem incluir comandos de nível mais elevado que simplificam o uso de um serviço com uma API complexa.

Sobre a AWS CLI versão 2

A AWS CLI versão 2 é a versão principal mais recente da AWS CLI e oferece suporte a todos os recursos mais recentes. Alguns recursos apresentados na versão 2 não são compatíveis com a versão 1, e você deve fazer a atualização para acessá-los. Há algumas alterações "radicais" em relação à versão 1 que podem exigir alterações nos scripts. Para obter uma lista de alterações desse tipo na versão 2, consulte [Migrar da AWS CLI versão 1 para a versão 2](#).

A AWS CLI versão 2 está disponível para instalação apenas como um pacote de instalador. Embora você possa encontrá-la em gerenciadores de pacotes, eles são pacotes sem suporte e não oficiais

que não são produzidos nem gerenciados pela AWS. Recomendamos que você instale a AWS CLI apenas dos pontos de distribuição oficiais da AWS, conforme documentado neste guia.

Para instalar a AWS CLI versão 2, consulte [the section called “Instalar/atualizar”](#).

Para verificar a versão atualmente instalada, use o seguinte comando:

```
$ aws --version  
aws-cli/2.10.0 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

Para obter o histórico de versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Manutenção e suporte para as versões principais do SDK

Para obter informações sobre manutenção e suporte para versões principais do SDK e suas dependências subjacentes, consulte o seguinte no [Guia de referência de AWS SDKs e ferramentas](#):

- [Política de manutenção de ferramentas e SDKs da AWS](#)
- [Matriz de suporte a versões de ferramentas e SDKs da AWS](#)

Sobre a Amazon Web Services

A Amazon Web Services (AWS) é um conjunto de serviços de infraestrutura digital que os desenvolvedores podem utilizar ao desenvolver suas aplicações. Os serviços incluem computação, armazenamento, banco de dados e sincronização de aplicações (sistema de mensagens e filas). A AWS usa um modelo de serviço de pagamento conforme o uso. Você será cobrado apenas pelos serviços que você ou suas aplicações usam. Além disso, para tornar AWS mais acessível como plataforma para criação de protótipos e experimentação, a AWS oferece um nível de uso gratuito. Neste nível, os serviços são gratuitos abaixo de um determinado nível de uso. Para obter mais informações sobre os custos e o nível gratuito da AWS, consulte [Nível gratuito da AWS](#). Para obter uma conta da AWS, abra a [página inicial da AWS](#) e selecione Criar uma conta da AWS.

Sobre os exemplos da AWS CLI

Os exemplos da AWS Command Line Interface (AWS CLI) neste guia são formatados de acordo com seguintes convenções:

- Prompt: o prompt de comando usa o prompt do Linux e é exibido como (\$). Para comandos específicos do Windows, C:\> é usado como prompt. Não inclua prompt quando você digitar comandos.
- Diretório: quando comandos devem ser executados de um diretório específico, o nome do diretório é mostrado antes do símbolo do comando.
- Entrada do usuário o texto do comando inserido na linha de comando é formatado como **user input**.
- Texto substituível: o texto variável, incluindo nomes de recursos que você escolher ou IDs gerados pelos serviços da AWS, que você deve incluir em comandos é formatado como *texto substituível*. Em comandos ou comandos de várias linhas, em que é necessária uma entrada específica do teclado, os comandos do teclado também podem ser exibidos como texto substituível.
- Saída: a saída retornada pelos serviços da AWS é mostrada sob a entrada do usuário e é formatada como **computer output**.

O exemplo de comando **aws configure** a seguir inclui entradas do usuário, texto substituível e saída:

1. Insira **aws configure** na linha de comando e pressione Enter.
2. A AWS CLI gera linhas de texto, solicitando que você insira informações adicionais.
3. Insira cada uma de suas chaves de acesso e pressione Enter.
4. Depois, insira um nome de região da AWS no formato mostrado, pressione Enter e, depois, Enter uma última vez para ignorar o formato de saída.
5. O comando final Enter é mostrado como texto substituível porque não há entradas do usuário para essa linha.

```
$ aws configure
```

```
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
```

```
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
```

```
Default region name [None]: us-west-2
```

```
Default output format [None]: ENTER
```

O exemplo a seguir mostra um comando simples com saída. Para usar este exemplo, insira o texto completo do comando (o texto destacado após o prompt) e pressione Enter. O nome do grupo de segurança, *my-sg*, pode ser substituído pelo nome do grupo de segurança desejado. O documento

JSON, incluindo as chaves, é saída. Se configurar o CLI para resultar em texto ou formato de tabela, a saída será formatada de forma diferente. [JSON](#) é o formato de saída padrão.

```
$ aws ec2 create-security-group --group-name my-sg --description "My security group"
{
  "GroupId": "sg-903004f8"
}
```

Documentação e recursos adicionais

Documentação e recursos da AWS CLI

Além deste guia, os recursos a seguir são fontes online importantes para a AWS CLI.

- [AWS CLIGuia de referência da versão 2](#)
- [Repositório de exemplos de código da AWS CLI](#)
- [Repositório da AWS CLI no GitHub](#): você pode visualizar e bifurcar o código-fonte para a AWS CLI no GitHub. Faça parte da comunidade de usuários no GitHub para fornecer feedback, solicitar recursos e enviar suas próprias contribuições.
- [Repositório de exemplos de alias da AWS CLI](#): você pode visualizar e bifurcar exemplos de alias da AWS CLI no GitHub.
- [Log de alterações da AWS CLI versão 2](#)

Outros AWS SDKs

Dependendo do seu caso de uso, talvez você queira escolher um dos AWS SDKs ou o AWS Tools for PowerShell:

- [AWS Tools for PowerShell](#)
- [AWS SDK for Java](#)
- [AWS SDK for .NET](#)
- [AWS SDK for JavaScript](#)
- [AWS SDK for Ruby](#)
- [AWS SDK for Python \(Boto\)](#)
- [AWS SDK for PHP](#)

- [AWS SDK for Go](#)
- [AWS Mobile SDK for iOS](#)
- [AWS Mobile SDK for Android](#)

Conceitos básicos da AWS CLI

Este capítulo fornece as etapas para começar a usar a versão 2 do AWS Command Line Interface (AWS CLI) e traz os links de instruções relevantes.

1. [Preencha todos os pré-requisitos](#): para acessar os serviços da AWS com a AWS CLI, você precisa de, no mínimo, uma Conta da AWS e das credenciais IAM. Para aumentar a segurança de sua conta da AWS, recomendamos não usar as credenciais de sua conta raiz. Você deve criar um usuário com privilégio mínimo para conceder credenciais de acesso às tarefas que serão executadas na AWS.
2. Instale ou obtenha acesso à AWS CLI usando um dos seguintes métodos:
 - (Recomendado) [the section called “Instalar/atualizar”](#).
 - [the section called “Versões anteriores”](#). A instalação de uma versão específica é usada principalmente se sua equipe alinha suas ferramentas a essa versão.
 - [the section called “Compilar e instalar usando o código-fonte”](#). Criar a AWS CLI com base na origem do GitHub é um método mais aprofundado, usado principalmente por clientes que trabalham em plataformas às quais não oferecemos suporte direto com nossos instaladores pré-integrados.
 - [the section called “Amazon ECR Public/Docker”](#).
 - Acessar a AWS CLI versão 2 no AWS Console em seu navegador usando AWS CloudShell. Para obter mais informações, consulte o [AWS CloudShell Guia do Usuário](#).
3. [Depois de obter acesso à AWS CLI, configure a AWS CLI com suas credenciais do IAM para usá-la pela primeira vez](#).

Solucionar problemas de instalação ou configuração

Se você encontrar problemas após instalar, desinstalar ou configurar a AWS CLI, consulte [Solucionar erros](#) para obter as etapas de solução de problemas.

Tópicos

- [Pré-requisitos para usar a AWS CLI versão 2](#)
- [Instalar ou atualizar para a versão mais recente da AWS CLI](#)
- [Instalar versões anteriores à AWS CLI versão 2](#)

- [Compilar e instalar a AWS CLI da origem](#)
- [Execute a AWS CLI pelas imagens oficiais do Amazon ECR Public ou do Docker](#)
- [Configurar a AWS CLI](#)

Pré-requisitos para usar a AWS CLI versão 2

Para acessar os serviços da AWS com a AWS CLI, você precisa de uma Conta da AWS e as credenciais do IAM. Ao executar comandos da AWS CLI, a AWS CLI precisa ter acesso às credenciais da AWS. Para aumentar a segurança de sua conta da AWS, recomendamos não usar as credenciais de sua conta raiz. Você deve criar um usuário com privilégio mínimo para conceder credenciais de acesso às tarefas que serão executadas na AWS.

Tópicos

- [Crie uma conta administrativa do IAM ou do IAM Identity Center](#)
- [Próximas etapas](#)

Crie uma conta administrativa do IAM ou do IAM Identity Center

Antes que você possa configurar

Para criar um usuário administrador, selecione uma das opções a seguir.

Selecionar uma forma de gerenciar o administrador	Para	Por	Você também pode
Centro de Identidade do IAM	Use credenciais de curto prazo para acessar a AWS.	Seguindo as instruções em Conceitos básicos no Guia do usuário do AWS IAM Identity Center.	Para configurar o acesso programático, consulte Configurar a AWS CLI para usar o AWS IAM Identity Center no Guia do usuário da

Selecione uma forma de gerenciar o administrador	Para	Por	Você também pode
(Recomendado)	Isso está de acordo com as práticas recomendadas de segurança. Para obter informações sobre as práticas recomendadas, consulte Práticas recomendadas de segurança no IAM no Guia do usuário do IAM.		AWS Command Line Interface .
No IAM (Não recomendado)	Use credenciais de curto prazo para acessar a AWS.	Seguindo as instruções em Criar o seu primeiro usuário administrador e um grupo de usuários do IAM no Guia do usuário do IAM para obter instruções.	Para configurar o acesso programático, consulte o Gerenciamento de chaves de acesso de usuários do IAM no Guia do usuário do IAM.

Próximas etapas

Depois de criar uma Conta da AWS e credenciais do IAM para usar a AWS CLI, você poderá executar uma das seguintes ações:

- [Instalar a versão mais recente](#) da AWS CLI versão 2 em seu computador.
- [Instalar uma versão anterior](#) à AWS CLI versão 2 em seu computador.
- Acessar a AWS CLI versão 2 de seu computador [usando uma imagem do Docker](#).

- Acessar a AWS CLI versão 2 no AWS Console em seu navegador usando AWS CloudShell. Para obter mais informações, consulte o [Guia do usuário do AWS CloudShell](#).

Instalar ou atualizar para a versão mais recente da AWS CLI

Este tópico descreve como instalar ou atualizar a versão mais recente da AWS Command Line Interface (AWS CLI) em sistemas operacionais compatíveis. Para obter informações sobre as versões mais recentes da AWS CLI, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Para instalar uma versão anterior da AWS CLI, consulte [the section called “Versões anteriores”](#). Para obter instruções de desinstalação, consulte [Desinstalar](#).

Important

As versões 1 e 2 da AWS CLI usam o mesmo nome de comando da aws. Se você instalou a AWS CLI versão 1 anteriormente, consulte [Migrar da AWS CLI versão 1 para a versão 2](#).

Tópicos

- [Instruções de instalação e atualização da AWS CLI](#)
- [Solução de problemas de erros de instalação e desinstalação da AWS CLI](#)
- [Próximas etapas](#)

Instruções de instalação e atualização da AWS CLI

Para obter as instruções de instalação, expanda a seção do sistema operacional.

Linux

Requisitos de instalação e atualização

- Você deve ser capaz de extrair ou “descompactar” o pacote baixado. Se o sistema operacional não tiver o comando unzip integrado, use um equivalente.
- A AWS CLI usa glibc, groff e less. Estes são incluídos por padrão na maioria das principais distribuições do Linux.
- A AWS CLI pode ser utilizada em versões de 64 bits das distribuições recentes do CentOS, Fedora, Ubuntu, Amazon Linux 1, Amazon Linux 2, Amazon Linux 2023 e Linux ARM.

- Como a AWS não mantém repositórios de terceiros, não podemos garantir que eles contenham a versão mais recente da AWS CLI.

Instalar ou atualizar a AWS CLI

Warning

Se esta é a primeira vez que você realiza uma atualização no Amazon Linux, para instalar a versão mais recente da AWS CLI, você deve desinstalar a versão pré-instalada do yum usando o seguinte comando:

```
$ sudo yum remove awscli
```

Depois de remover a instalação do yum da AWS CLI, siga as instruções de instalação do Linux abaixo.

Para atualizar a instalação atual da AWS CLI, baixe um novo instalador sempre que atualizar para substituir as versões anteriores. Siga estes passos na linha de comando para instalar a AWS CLI no Linux.

Nós fornecemos as etapas em um grupo fácil de copiar e colar que leva em conta se você usa o Linux de 64 bits ou o Linux ARM. Consulte as descrições de cada linha nas etapas a seguir.

Linux x86 (64-bit)

Note

(Opcional) O bloco de comando a seguir baixa e instala a AWS CLI sem verificar a integridade do download. Para verificar a integridade do download, use as instruções passo a passo abaixo.

Para instalar a AWS CLI, execute os comandos a seguir.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"  
unzip awscliv2.zip  
sudo ./aws/install
```

Para atualizar a instalação atual da AWS CLI, adicione as informações do symlink e instalador existentes para criar o comando `install` com os parâmetros `--bin-dir`, `--install-dir` e `--update`. O bloco de comando a seguir usa um exemplo de symlink de `/usr/local/bin` e um exemplo de localização do instalador de `/usr/local/aws-cli`.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install --bin-dir /usr/local/bin --install-dir /usr/local/aws-cli --
update
```

Linux ARM

Note

(Opcional) O bloco de comando a seguir baixa e instala a AWS CLI sem verificar a integridade do download. Para verificar a integridade do download, use as instruções passo a passo abaixo.

Para instalar a AWS CLI, execute os comandos a seguir.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Para atualizar a instalação atual da AWS CLI, adicione as informações do symlink e instalador existentes para criar o comando `install` com os parâmetros `--bin-dir`, `--install-dir` e `--update`. O bloco de comando a seguir usa um exemplo de symlink de `/usr/local/bin` e um exemplo de localização do instalador de `/usr/local/aws-cli`.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install --bin-dir /usr/local/bin --install-dir /usr/local/aws-cli --
update
```

1. Baixe o arquivo de instalação de uma das seguintes maneiras:

Linux x86 (64-bit)

- Usar o comando **curl**: a opção **-o** especifica o nome do arquivo no qual o pacote obtido por download foi gravado. As opções no comando de exemplo a seguir gravam o arquivo obtido por download no diretório atual com o nome local `awscliv2.zip`.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o  
"awscliv2.zip"
```

- Baixe usando o URL: para baixar o instalador com o navegador, use o seguinte URL:
https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip

Linux ARM

- Usar o comando **curl**: a opção **-o** especifica o nome do arquivo no qual o pacote obtido por download foi gravado. As opções no comando de exemplo a seguir gravam o arquivo obtido por download no diretório atual com o nome local `awscliv2.zip`.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip" -o  
"awscliv2.zip"
```

- Baixe usando o URL: para baixar o instalador com o navegador, use o seguinte URL:
<https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip>

2. (Opcional) Verificar a integridade do arquivo zip baixado

Se você tiver optado por baixar manualmente o pacote do instalador `.zip` da AWS CLI nas etapas acima, poderá executar as etapas a seguir para verificar as assinaturas por meio da ferramenta GnuPG.

Os arquivos `.zip` do pacote do instalador da AWS CLI são assinados criptograficamente com assinaturas PGP. Se houver qualquer dano ou alteração dos arquivos, ocorrerá uma falha nessa verificação e você não deverá prosseguir com a instalação.

- a. Baixe e instale o comando `gpg` usando o gerenciador de pacotes. Para obter mais informações sobre a GnuPG, consulte o [site da GnuPG](#).
- b. Para criar o arquivo de chave pública, crie um arquivo de texto e cole o texto a seguir.

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
```

```
mQINBF2Cr7UBEADJZHcgus0Jl7ENSyUmXh85z0TRV0xJorM2B/JL0kH0yigQluUG
ZMLhENaG0bYatdrKP+3H911vK050pXwn0/R7fB/FSTouki4ciIx50uLlnJZIxSzx
PqG10mkxImLnbGWOi6Lto0LYxqHN2iQtzlwTVmq9733zd3XfcXrZ3+Lb1HAGet5G
TfnxEKJ8soPLyWmwDH6HWCnjZ/aIQRBTIQ05uVeEoYxSh6w0ai7ss/KveoSNBbYz
gbdzoqI2Y8cgH2nbfgp3DSasaLZEdCSsIsK1u05CinE7k2qZ7KgKAUIcT/cR/grk
C6VwsnDU00UCideXcQ8WeHutqvgZH1JgKDbznoIzeQHJD238GEu+eKhRHcz8/jeG
94zkcgJ0z3KbZGYMiTh277Fvj9zzvZsbMBCedV1BTg3TqgvdX4bdkhf5cH+7NtW0
lrFj6UwAsGukBTA0xC01/dnSmZhJ7Z1KmEWilro/g0rjt0xqRQut1IqG22TaqoPG
fYVN+en3ZwbT97kcgZDwqbuykNt64oZwc4XKCa3mprEGC3IbJTBFqglXmZ719ywG
EEUJY01b2XrSuPwm139beWdKM8kzr10jn10m6+lpTRCBfo0wa9F8YZRhHPAkWkKX
XDe0GpWRj4oh0x0d2GWkyV5xyN14p2tQ0Cd00Dmz80yUTgRpPVQUt0EHXQARAQAB
tCFBV1MgQ0xJIFRlYW0gPGF3cy1jbG1AYW1hem9uLmNvbT6JAlQEEwEiAD4CGwMF
CwkIBWIGFQoJCA5CBByCAwECHgECF4AWIQT7Xbd/1cEYuAURraimMQrMRnJHXAUC
ZMKcEgUJCSEf3QAKCRCmMQrMRnJHXCiLD/4vior9J5tB+icri5WbDudS3ak/ve4q
XS6ZLm5S8l+CBxy5aLQUlyFhuaaEHDC11fG780duxatzeHENASYVo3mmKNwrCBza
NJaeaWKLGT0MKwBSP5aa3dva8P/4oUP9GsQn0uWoXwNDWfrMbNI8gn+jC/3MigW
vD3fu6zC0WWLITNv2SJoQlwILmb/uGfha68o4iTB0vcftVRua06DyqF+CrHX/0j0
kLEDQFMY9M4tsYT7X8NwfI8Vmc89nzpvL9fwd44WwpKIw1FBZP8S0sgDx2xDsxv
L8kM2Gt0iH0cHqF0+V7xtTKZylo1iDbJKhu80Kc+YC/TmozD8oeGU2rEFXfLegwS
zT9N+jB38+dqaP9pRDsi45iGqyA8yavVBabpL0IQ9jU6eIV+kmcjIjcun/Uo8SjJ
0xQAsm41rxPaKV6vJUn10wVNuhSkKk8mzN01SZwu7Hua6rdcCaGeB8uJ44AP3QzW
BNnrjtoN6A1N0D2wFmfE/YL/rHPxU1XwPntubYB/t3rXFL7ENQ00QH0KVXgRC1ey
sHMglg46c+nQLRzVTshjDjmtzvvh9rcV9RKRoPetEggzCoD89veDA9jPR2Kw6RYkS
XzYm2fEv16/HRNYt7hJzneFqRIjHW5qAgSs/bcaRWpAU/QQzzJPVKCQNr4y0weyg
B8HctGjfoD0p1A==
=gdMc
-----END PGP PUBLIC KEY BLOCK-----
```

Para referência, veja a seguir os detalhes da chave pública.

```
Key ID:          A6310ACC4672
Type:            RSA
Size:            4096/4096
Created:         2019-09-18
Expires:         2024-07-26
User ID:         AWS CLI Team <aws-cli@amazon.com>
Key fingerprint: FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672 475C
```

- c. Importe a chave pública da AWS CLI com o comando a seguir, substituindo *public-key-file-name* pelo nome do arquivo da chave pública que você criou.

```
$ gpg --import public-key-file-name
```

```
gpg: /home/username/.gnupg/trustdb.gpg: trustdb created
gpg: key A6310ACC4672475C: public key "AWS CLI Team <aws-cli@amazon.com>"
imported
gpg: Total number processed: 1
gpg:             imported: 1
```

- d. Baixe o arquivo de assinatura da AWS CLI para o pacote baixado. Ele tem o mesmo caminho e nome do arquivo .zip ao qual ele corresponde, mas tem a extensão .sig. Nos exemplos a seguir, ele foi salvo no diretório atual como um arquivo chamado `awscliv2.sig`.

Linux x86 (64-bit)

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-
x86_64.zip.sig
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `awscli-exe-linux-x86_64-2.0.30.zip.sig`, o que resultaria no seguinte comando:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-
x86_64-2.0.30.zip.sig
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Linux ARM

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-
aarch64.zip.sig
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `awscli-exe-linux-aarch64-2.0.30.zip.sig`, o que resultaria no seguinte comando:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-aarch64-2.0.30.zip.sig
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

- e. Verifique a assinatura, passando os nomes dos arquivos .sig e .zip baixados como parâmetros para o comando gpg.

```
$ gpg --verify awscliv2.sig awscliv2.zip
```

A saída deve ser semelhante à seguinte.

```
gpg: Signature made Mon Nov  4 19:00:01 2019 PST
gpg:                using RSA key FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672
475C
gpg: Good signature from "AWS CLI Team <aws-cli@amazon.com>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:                There is no indication that the signature belongs to the owner.
Primary key fingerprint: FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672 475C
```

Important

O aviso na saída é esperado e não indica um problema. Isso ocorre porque não há uma cadeia de confiança entre a chave PGP pessoal (se você tiver uma) e a chave PGP da AWS CLI. Para obter mais informações, consulte [Web of trust](#).

3. Descompacte o instalador. Se a distribuição do Linux não tiver um comando unzip integrado, use um equivalente para descompactá-lo. O comando de exemplo a seguir descompacta o pacote e cria um diretório chamado aws no diretório atual.

```
$ unzip awscliv2.zip
```

Note

Ao atualizar de uma versão anterior, o comando unzip solicita a substituição dos arquivos existentes. Para ignorar essas solicitações, como com a automação de

scripts, use o sinalizador de atualização `-u` para `unzip`. Esse sinalizador atualiza automaticamente os arquivos existentes e cria outros conforme necessário.

```
$ unzip -u awscliv2.zip
```

4. Execute o programa de instalação. O comando de instalação usa um arquivo chamado `install` no diretório `aws` recém-descompactado. Por padrão, os arquivos são todos instalados em `/usr/local/aws-cli`, e um link simbólico é criado em `/usr/local/bin`. O comando inclui `sudo` para conceder permissões de gravação para esses diretórios.

```
$ sudo ./aws/install
```

Você poderá instalar sem `sudo` se especificar diretórios para os quais já tenha permissões de gravação. Use as seguintes instruções para o comando `install` para especificar o local de instalação:

- Os caminhos fornecidos para os parâmetros `-i` e `-b` não devem conter nomes de diretório nem nomes de volume com caracteres de espaço ou outros caracteres de espaço em branco. Se houver um espaço, a instalação falhará.
- `--install-dir` ou `-i`: essa opção especifica o diretório para o todos os arquivos serão copiados.

O valor padrão é `/usr/local/aws-cli`.

- `--bin-dir` ou `-b`: essa opção especifica que o programa `aws` principal no diretório de instalação está simbolicamente vinculado ao arquivo `aws` no caminho especificado. É necessário ter permissões de gravação no diretório especificado. Criar um symlink para um diretório que já está em seu caminho elimina a necessidade de adicionar o diretório de instalação à variável `$PATH` do usuário.

O valor padrão é `/usr/local/bin`.

```
$ ./aws/install -i /usr/local/aws-cli -b /usr/local/bin
```

 Note

Para atualizar a instalação atual da AWS CLI, adicione as informações do symlink e instalador existentes para criar o comando `install` com o parâmetro `--update`.

```
$ sudo ./aws/install --bin-dir /usr/local/bin --install-dir /usr/local/aws-  
cli --update
```

Para localizar o diretório de instalação e o symlink existentes, execute as seguintes etapas:

1. Use o comando `which` para encontrar o symlink. Isso fornece o caminho que deve ser usado com o parâmetro `--bin-dir`.

```
$ which aws  
/usr/local/bin/aws
```

2. Use o comando `ls` para encontrar o diretório para o qual o symlink aponta. Isso fornece o caminho que deve ser usado com o parâmetro `--install-dir`.

```
$ ls -l /usr/local/bin/aws  
lrwxrwxrwx 1 ec2-user ec2-user 49 Oct 22 09:49 /usr/local/bin/aws -> /usr/  
local/aws-cli/v2/current/bin/aws
```

5. Confirme a instalação com o comando a seguir.

```
$ aws --version  
aws-cli/2.10.0 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/2.4.5
```

Se não for possível encontrar o comando `aws`, talvez seja necessário reiniciar o terminal ou seguir a solução de problemas em [Solucionar erros](#).

macOS

Requisitos de instalação e atualização

- A AWS CLI pode ser utilizada no macOS versões 10.9 e posterior. Para obter mais informações, consulte as [atualizações da política de suporte do macOS para a AWS CLI v2](#) no blog de ferramentas para desenvolvedores da AWS.
- Como a AWS não mantém repositórios de terceiros, não podemos garantir que eles contenham a versão mais recente da AWS CLI.

Instalar ou atualizar a AWS CLI

Se você estiver atualizando para a versão mais recente, use o mesmo método de instalação usado na versão atual. É possível instalar a AWS CLI de uma das maneiras a seguir.

GUI installer

As etapas a seguir mostram como instalar a versão mais recente da AWS CLI usando a interface de usuário padrão do macOS e o navegador.

1. No navegador, baixe o arquivo pkg do macOS: <https://awscli.amazonaws.com/AWSCLIV2.pkg>
2. Execute o arquivo baixado e siga as instruções na tela. Você pode optar por instalar a AWS CLI das seguintes maneiras:
 - Para todos os usuários no computador (requer **sudo**)
 - Você pode instalar em qualquer pasta ou escolher a padrão recomendada `/usr/local/aws-cli`.
 - O instalador cria automaticamente um symlink em `/usr/local/bin/aws` que vincula o programa principal na pasta de instalação que você escolheu.
 - Apenas para o usuário atual (não requer **sudo**)
 - Você pode instalar em qualquer pasta para a qual tenha permissão de gravação.
 - Devido a permissões de usuário padrão, após a conclusão do instalador, é necessário criar manualmente um arquivo de symlink no `$PATH` que aponta para os programas `aws` e `aws_completer` usando os comandos a seguir no prompt de comando. Se `$PATH` incluir uma pasta na qual você pode gravar, será possível executar o comando a seguir sem `sudo` se você especificar essa pasta como o caminho de destino. Se você não

tiver uma pasta gravável no \$PATH, será necessário usar sudo nos comandos a fim de obter permissões para gravar na pasta de destino especificada. O local padrão para um symlink é /usr/local/bin/.

```
$ sudo ln -s /folder/installed/aws-cli/aws /usr/local/bin/aws
$ sudo ln -s /folder/installed/aws-cli/aws_completer /usr/local/bin/aws_completer
```

Note

É possível visualizar logs de depuração para a instalação pressionando Cmd+L em qualquer lugar do instalador. Essa ação abre um painel de log que permite filtrar e salvar o log. O arquivo de log também é salvo automaticamente em /var/log/install.log.

3. Para verificar se o shell pode encontrar e executar o comando aws no \$PATH, use os comandos a seguir.

```
$ which aws
/usr/local/bin/aws
$ aws --version
aws-cli/2.10.0 Python/3.11.2 Darwin/18.7.0 botocore/2.4.5
```

Se não for possível encontrar o comando aws, talvez seja necessário reiniciar o terminal ou seguir a solução de problemas em [Solucionar erros](#).

Command line installer - All users

Se você tiver permissões sudo, poderá instalar a AWS CLI para todos os usuários no computador. Nós fornecemos as etapas em um grupo fácil de copiar e colar. Consulte as descrições de cada linha nas etapas a seguir.

```
$ curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
$ sudo installer -pkg AWSCLIV2.pkg -target /
```

1. Baixe o arquivo usando o comando `curl`. A opção `-o` especifica o nome do arquivo no qual o pacote obtido por download foi gravado. No exemplo anterior, o arquivo é gravado como `AWSCLIV2.pkg` na pasta atual.

```
$ curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
```

2. Execute o programa `installer` padrão do macOS, especificando o arquivo `.pkg` baixado como a origem. Use o parâmetro `-pkg` para especificar o nome do pacote a ser instalado e o parâmetro `-target /` para especificar em qual unidade instalar o pacote. Os arquivos são instalados no `/usr/local/aws-cli`, e um symlink é criado automaticamente em `/usr/local/bin`. Você deve incluir `sudo` no comando para conceder permissões de gravação para essas pastas.

```
$ sudo installer -pkg ./AWSCLIV2.pkg -target /
```

Após a conclusão da instalação, os logs de depuração são gravados em `/var/log/install.log`.

3. Para verificar se o shell pode encontrar e executar o comando `aws` no `$PATH`, use os comandos a seguir.

```
$ which aws
/usr/local/bin/aws
$ aws --version
aws-cli/2.10.0 Python/3.11.2 Darwin/18.7.0 botocore/2.4.5
```

Se não for possível encontrar o comando `aws`, talvez seja necessário reiniciar o terminal ou seguir a solução de problemas em [Solucionar erros](#).

Command line - Current user

1. Para especificar em qual pasta a AWS CLI está instalada, é necessário criar um arquivo XML com qualquer nome. Esse é um arquivo no formato XML que se parece com o exemplo a seguir. Deixe todo o valor como mostrado e substitua o caminho `/Users/myusername` na linha 9 pelo caminho até a pasta em que deseja instalar a AWS CLI. A pasta já deve existir, caso contrário, o comando falhará. O exemplo de XML a seguir, chamado `choices.xml`, especifica o instalador para instalar a AWS CLI na pasta `/Users/myusername`, onde ele cria uma pasta chamada `aws-cli`.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
  <array>
    <dict>
      <key>choiceAttribute</key>
      <string>customLocation</string>
      <key>attributeSetting</key>
      <string>/Users/myusername</string>
      <key>choiceIdentifier</key>
      <string>default</string>
    </dict>
  </array>
</plist>
```

2. Baixe o instalador pkg usando o comando `curl`. A opção `-o` especifica o nome do arquivo no qual o pacote obtido por download foi gravado. No exemplo anterior, o arquivo é gravado como `AWSCLIV2.pkg` na pasta atual.

```
$ curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
```

3. Execute o programa `installer` padrão do macOS com as seguintes opções:
 - Especifique o nome do pacote a ser instalado usando o parâmetro `-pkg`.
 - Especifique uma instalação somente para o usuário atual definindo o parâmetro `-target` como `CurrentUserHomeDirectory`.
 - Especifique o caminho (relativo à pasta atual) e o nome do arquivo XML criado no parâmetro `-applyChoiceChangesXML` *choices.xml*.

O exemplo a seguir instala a AWS CLI na pasta `/Users/myusername/aws-cli`.

```
$ installer -pkg AWSCLIV2.pkg \
            -target CurrentUserHomeDirectory \
            -applyChoiceChangesXML choices.xml
```

4. Como as permissões de usuário padrão geralmente não permitem gravar em pastas no `$PATH`, o instalador nesse modo não tenta adicionar os symlinks aos programas `aws` e `aws_completer`. Para que a AWS CLI seja executada corretamente, é necessário criar

manualmente os symlinks após a conclusão do instalador. Se \$PATH incluir uma pasta na qual você pode gravar e você especificar a pasta como o caminho de destino, é possível executar o comando a seguir sem sudo. Se você não tiver uma pasta gravável no \$PATH, será necessário usar sudo para permissões para gravar na pasta de destino especificada. O local padrão para um symlink é /usr/local/bin/. Substitua o folder/installed pelo caminho da instalação da AWS CLI.

```
$ sudo ln -s /folder/installed/aws-cli/aws /usr/local/bin/aws
$ sudo ln -s /folder/installed/aws-cli/aws_completer /usr/local/bin/
aws_completer
```

Após a conclusão da instalação, os logs de depuração são gravados em /var/log/install.log.

5. Para verificar se o shell pode encontrar e executar o comando aws no \$PATH, use os comandos a seguir.

```
$ which aws
/usr/local/bin/aws
$ aws --version
aws-cli/2.10.0 Python/3.11.2 Darwin/18.7.0 botocore/2.4.5
```

Se não for possível encontrar o comando aws, talvez seja necessário reiniciar o terminal ou seguir a solução de problemas em [Solucionar erros](#).

Windows

Requisitos de instalação e atualização

- A AWS CLI pode ser utilizada em versões compatíveis com o Microsoft Windows de 64 bits.
- Direitos de administrador para instalar software

Instalar ou atualizar a AWS CLI

Para atualizar a instalação atual da AWS CLI no Windows, baixe um novo instalador sempre que atualizar para substituir as versões anteriores. A AWS CLI é atualizada regularmente. Para ver quando a versão mais recente foi lançada, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

1. Baixar e executar o instalador MSI da AWS CLI para Windows (64 bits)

<https://awscli.amazonaws.com/AWSCLIV2.msi>

Você também pode executar o comando `msiexec` para executar o instalador MSI.

```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```

Com relação a vários parâmetros que podem ser usados com `msiexec`, consulte [msiexec](#) no site de documentação da Microsoft. Por exemplo, você pode usar a sinalização `/qn` para uma instalação silenciosa.

```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi /qn
```

2. Para confirmar a instalação, abra o menu Início, procure cmd para abrir uma janela do prompt de comando e, no prompt de comando, use o comando `aws --version`.

```
C:\> aws --version
aws-cli/2.10.0 Python/3.11.2 Windows/10 exe/AMD64 prompt/off
```

Se o Windows não puder localizar o programa, talvez seja necessário fechar e abrir a janela do prompt de comando novamente para atualizar o caminho ou seguir a solução de problemas em [Solucionar erros](#).

Solução de problemas de erros de instalação e desinstalação da AWS CLI

Se você encontrar problemas após instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas. Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).

Próximas etapas

Após a instalação bem-sucedida da AWS CLI, você poderá excluir com segurança os arquivos do instalador baixados. Depois de concluir as etapas em [the section called “Pré-requisitos”](#) e instalar a AWS CLI, você deve utilizar [the section called “Configuração”](#).

Instalar versões anteriores à AWS CLI versão 2

Este tópico descreve como instalar ou atualizar versões anteriores à AWS Command Line Interface versão 2 (AWS CLI) em sistemas operacionais compatíveis. Para obter informações sobre os lançamentos da AWS CLI versão 2, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Instruções de instalação da AWS CLI versão 2:

Linux

Requisitos de instalação

- Você sabe qual lançamento da AWS CLI versão 2 gostaria de instalar. Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.
- Você deve ser capaz de extrair ou “descompactar” o pacote baixado. Se o sistema operacional não tiver o comando unzip integrado, use um equivalente.
- A AWS CLI versão 2 usa glibc, groff e less. Estes são incluídos por padrão na maioria das principais distribuições do Linux.
- A AWS CLI versão 2 é compatível com as versões de 64 bits das distribuições recentes do CentOS, Fedora, Ubuntu, Amazon Linux 1, Amazon Linux 2 e Linux ARM.
- Como a AWS não mantém repositórios de terceiros, não podemos garantir que eles contenham a versão mais recente da AWS CLI.

Instruções de instalação

Siga estes passos na linha de comando para instalar a AWS CLI no Linux.

Nós fornecemos as etapas em um grupo fácil de copiar e colar que leva em conta se você usa o Linux de 64 bits ou o Linux ARM. Consulte as descrições de cada linha nas etapas a seguir.

Linux x86 (64-bit)

Note

(Opcional) O bloco de comando a seguir baixa e instala a AWS CLI sem verificar a integridade do download. Para verificar a integridade do download, use as instruções passo a passo abaixo.

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Para instalar a AWS CLI, execute os comandos a seguir.

Para especificar uma versão, acrescente um hífen e o número da versão ao nome do arquivo.

Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `awscli-exe-linux-x86_64-2.0.30.zip`, o que resultaria no seguinte comando:

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.0.30.zip" -o  
"awscliv2.zip"  
unzip awscliv2.zip  
sudo ./aws/install
```

Para atualizar a instalação atual da AWS CLI, adicione as informações do symlink e instalador existentes para criar o comando `install` com os parâmetros `--bin-dir`, `--install-dir` e `--update`. O bloco de comando a seguir usa um exemplo de symlink de `/usr/local/bin` e um exemplo de localização do instalador de `/usr/local/aws-cli`.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.0.30.zip" -o  
"awscliv2.zip"  
unzip awscliv2.zip  
sudo ./aws/install --bin-dir /usr/local/bin --install-dir /usr/local/aws-cli --  
update
```

Linux ARM

Note

(Opcional) O bloco de comando a seguir baixa e instala a AWS CLI sem verificar a integridade do download. Para verificar a integridade do download, use as instruções passo a passo abaixo.

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Para instalar a AWS CLI, execute os comandos a seguir.

Para especificar uma versão, acrescente um hífen e o número da versão ao nome do arquivo.

Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `awscli-exe-linux-aarch64-2.0.30.zip`, o que resultaria no seguinte comando:

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-aarch64-2.0.30.zip" -o  
"awscliv2.zip"  
unzip awscliv2.zip  
sudo ./aws/install
```

Para atualizar a instalação atual da AWS CLI, adicione as informações do symlink e instalador existentes para criar o comando `install` com os parâmetros `--bin-dir`, `--install-dir` e `--update`. O bloco de comando a seguir usa um exemplo de symlink de */usr/local/bin* e um exemplo de localização do instalador de */usr/local/aws-cli*.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-aarch64-2.0.30.zip" -o  
"awscliv2.zip"  
unzip awscliv2.zip  
sudo ./aws/install --bin-dir /usr/local/bin --install-dir /usr/local/aws-cli --  
update
```

1. Baixe o arquivo de instalação de uma das seguintes maneiras:

Linux x86 (64-bit)

- Usar o comando **curl**: a opção `-o` especifica o nome do arquivo no qual o pacote obtido por download foi gravado. As opções no comando de exemplo a seguir gravam o arquivo obtido por download no diretório atual com o nome local `awscliv2.zip`.

Para especificar uma versão, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão *2.0.30* seria `awscli-exe-linux-x86_64-2.0.30.zip`, o que resultaria no seguinte comando:

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.0.30.zip" -o  
"awscliv2.zip"
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

- Baixe usando o URL:

Em seu navegador, baixe a versão específica da AWS CLI acrescentando um hífen e o número da versão ao nome do arquivo.

```
https://awscli.amazonaws.com/awscli-exe-linux-x86_64-version.number.zip
```

Nesse exemplo, o nome do arquivo da versão **2.0.30** seria awscli-exe-linux-x86_64-2.0.30.zip, resultando no seguinte link: https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.0.30.zip

Linux ARM

- Usar o comando **curl**: a opção **-o** especifica o nome do arquivo no qual o pacote obtido por download foi gravado. As opções no comando de exemplo a seguir gravam o arquivo obtido por download no diretório atual com o nome local awscliv2.zip.

Para especificar uma versão, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **2.0.30** seria awscli-exe-linux-aarch64-2.0.30.zip, o que resultaria no seguinte comando:

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-aarch64-2.0.30.zip" -o  
  "awscliv2.zip"  
unzip awscliv2.zip  
sudo ./aws/install
```

- Baixe usando o URL:

Em seu navegador, baixe a versão específica da AWS CLI acrescentando um hífen e o número da versão ao nome do arquivo.

```
https://awscli.amazonaws.com/awscli-exe-linux-aarch64-version.number.zip
```

Neste exemplo, o nome do arquivo para a versão **2.0.30** seria awscli-exe-linux-aarch64-2.0.30.zip, o que resultaria no seguinte link: <https://awscli.amazonaws.com/awscli-exe-linux-aarch64-2.0.30.zip>

2. (Opcional) Verificar a integridade do arquivo zip baixado

Se você tiver optado por baixar manualmente o pacote do instalador .zip da AWS CLI nas etapas acima, poderá executar as etapas a seguir para verificar as assinaturas por meio da ferramenta GnuPG.

Os arquivos .zip do pacote do instalador da AWS CLI são assinados criptograficamente com assinaturas PGP. Se houver qualquer dano ou alteração dos arquivos, ocorrerá uma falha nessa verificação e você não deverá prosseguir com a instalação.

- Baixe e instale o comando gpg usando o gerenciador de pacotes. Para obter mais informações sobre a GnuPG, consulte o [site da GnuPG](#).
- Para criar o arquivo de chave pública, crie um arquivo de texto e cole o texto a seguir.

```
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBF2Cr7UBEADJZHcgus0Jl7ENSyumXh85z0TRV0xJorM2B/JL0kH0yigQ1uUG
ZMLhENaG0bYatdrKP+3H911vK050pXwn0/R7fB/FSTouki4ciIx50uLlnJZIxSzx
PqG10mkxImLnBgWoi6Lto0LYxqHN2iQtzlwTVmq9733zd3XfcXrZ3+Lb1HAgEt5G
TfNxEKJ8soPLyWmwDH6HWCnjZ/aIQRBTIQ05uVeEoYxSh6w0ai7ss/KveoSNBbYz
gbdzoqI2Y8cgH2nbfpgp3DSasaLZEdCSsIsK1u05CinE7k2qZ7KgKAUIcT/cR/grk
C6VwsnDU00UCideXcQ8WeHutqvgZH1JgKDbznoIzeQHJD238GEu+eKhRHcz8/jeG
94zkcgJ0z3KbZGYMiTh277Fvj9zzvZsbMBCedV1BTg3Tqgvdx4bdkhf5cH+7NtW0
lrFj6UwAsGukBTA0xC01/dnSmZhJ7Z1KmEWilro/g0rjt0xqRQutlIqG22TaqoPG
fYVN+en3ZwbT97kcgZDwbqbuykNt64oZWc4XKCa3mpREGC3IbJTBfQglXmZ719ywG
EEUJY01b2XrSuPwml39beWdKM8kzr10jn10m6+lpTRCBfo0wa9F8YZRhHPAkWkX
XDe0GpWRj4oh0x0d2GWkyV5xyN14p2tQ0Cd00Dmz80yUTgRpPVQUt0EhXQARAQAB
tCFBV1MgQ0xJIFRlYW0gPGF3cy1jbG1AYW1hem9uLmNvbT6JAlQEEwEIAD4CGwMF
CwkIBwIGFQoJCA5CBByCAwECHgECF4AWIQT7Xbd/1cEYuAURraimMQrMRnJHXAUC
ZMKcEgUJCSEf3QAKCRCmMQrMRnJHXCiLD/4vior9J5tB+icri5WbDudS3ak/ve4q
XS6ZLm5S81+CBxy5aLQUlyFhuaaEHDC11fG780duxatzeHENASYVo3mmKNwrCBza
NJaeaWKLGT0MKwBSP5aa3dva8P/4oUP9GsQn0uWoXwNDWfrMbNI8gn+jC/3MigW
vD3fu6zC0WWLITNv2SJoQ1wILmb/uGfha68o4iTB0vcftVRua06DyqF+CrHX/0j0
k1EDQFMY9M4tsYT7X8NwfI8Vmc89nzpvL9fwd44WwpKIw1FBZP8S0sgDx2xDsxv
L8kM2Gt0iH0cHqF0+V7xtTKZylo1iDbJKhu80Kc+YC/TmozD8oeGU2rEFXfLegwS
zT9N+jB38+dqaP9pRDsi45iGqyA8yavVBabpL0IQ9jU6eIV+kmcjIjcun/Uo8SjJ
0xQAsm41rxPaKV6vJUn10wVnuhSkKk8mzN01SZwu7Hua6rdcCaGeB8uJ44AP3QzW
BNnrjtoN6A1N0D2wFmFE/YL/rHPxU1XwPntubYB/t3rXFL7ENQ00QH0KVXgRC1ey
sHMglg46c+nQLRzVTshjDjmtzvvh9rcV9RKRoPetEggzCoD89veDA9jPR2Kw6RYkS
XzYm2fEv16/HRNYt7hJzneFqRIjHW5qAgSs/bcaRWpAU/QQzzJPVKCQNr4y0weyg
B8HctGjfod0p1A==
=gdMc
-----END PGP PUBLIC KEY BLOCK-----
```

Para referência, veja a seguir os detalhes da chave pública.

Key ID:	A6310ACC4672
---------	--------------

```
Type:          RSA
Size:          4096/4096
Created:       2019-09-18
Expires:       2024-07-26
User ID:       AWS CLI Team <aws-cli@amazon.com>
Key fingerprint: FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672 475C
```

- c. Importe a chave pública da AWS CLI com o comando a seguir, substituindo *public-key-file-name* pelo nome do arquivo da chave pública que você criou.

```
$ gpg --import public-key-file-name
gpg: /home/username/.gnupg/trustdb.gpg: trustdb created
gpg: key A6310ACC4672475C: public key "AWS CLI Team <aws-cli@amazon.com>"
imported
gpg: Total number processed: 1
gpg:             imported: 1
```

- d. Baixe o arquivo de assinatura da AWS CLI para o pacote baixado. Ele tem o mesmo caminho e nome do arquivo .zip ao qual ele corresponde, mas tem a extensão .sig. Nos exemplos a seguir, ele foi salvo no diretório atual como um arquivo chamado awscliv2.sig.

Linux x86 (64-bit)

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip.sig
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão *2.0.30* seria awscli-exe-linux-x86_64-2.0.30.zip.sig, o que resultaria no seguinte comando:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.0.30.zip.sig
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Linux ARM

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip.sig
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `awscli-exe-linux-aarch64-2.0.30.zip.sig`, o que resultaria no seguinte comando:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-aarch64-2.0.30.zip.sig
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

- e. Verifique a assinatura, passando os nomes dos arquivos `.sig` e `.zip` baixados como parâmetros para o comando `gpg`.

```
$ gpg --verify awscliv2.sig awscliv2.zip
```

A saída deve ser semelhante à seguinte.

```
gpg: Signature made Mon Nov  4 19:00:01 2019 PST
gpg:                using RSA key FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672
475C
gpg: Good signature from "AWS CLI Team <aws-cli@amazon.com>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:                There is no indication that the signature belongs to the owner.
Primary key fingerprint: FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672 475C
```

Important

O aviso na saída é esperado e não indica um problema. Isso ocorre porque não há uma cadeia de confiança entre a chave PGP pessoal (se você tiver uma) e a chave PGP da AWS CLI. Para obter mais informações, consulte [Web of trust](#).

3. Descompacte o instalador. Se a distribuição do Linux não tiver um comando `unzip` integrado, use um equivalente para descompactá-lo. O comando de exemplo a seguir descompacta o pacote e cria um diretório chamado `aws` no diretório atual.

```
$ unzip awscliv2.zip
```

4. Execute o programa de instalação. O comando de instalação usa um arquivo chamado `install` no diretório `aws` recém-descompactado. Por padrão, os arquivos são todos instalados em `/usr/local/aws-cli`, e um link simbólico é criado em `/usr/local/bin`. O comando inclui `sudo` para conceder permissões de gravação para esses diretórios.

```
$ sudo ./aws/install
```

Você poderá instalar sem `sudo` se especificar diretórios para os quais já tenha permissões de gravação. Use as seguintes instruções para o comando `install` para especificar o local de instalação:

- Os caminhos fornecidos para os parâmetros `-i` e `-b` não devem conter nomes de diretório nem nomes de volume com caracteres de espaço ou outros caracteres de espaço em branco. Se houver um espaço, a instalação falhará.
- `--install-dir` ou `-i`: essa opção especifica o diretório para o todos os arquivos serão copiados.

O valor padrão é `/usr/local/aws-cli`.

- `--bin-dir` ou `-b`: essa opção especifica que o programa `aws` principal no diretório de instalação está simbolicamente vinculado ao arquivo `aws` no caminho especificado. É necessário ter permissões de gravação no diretório especificado. Criar um symlink para um diretório que já está em seu caminho elimina a necessidade de adicionar o diretório de instalação à variável `$PATH` do usuário.

O valor padrão é `/usr/local/bin`.

```
$ ./aws/install -i /usr/local/aws-cli -b /usr/local/bin
```


 Note

Para atualizar a instalação atual da AWS CLI versão 2 para uma versão mais recente, adicione as informações do symlink e instalador existentes para construir o comando `install` com o parâmetro `--update`.

```
$ sudo ./aws/install --bin-dir /usr/local/bin --install-dir /usr/local/aws-  
cli --update
```

Para localizar o diretório de instalação e o symlink existentes, execute as seguintes etapas:

1. Use o comando `which` para encontrar o symlink. Isso fornece o caminho que deve ser usado com o parâmetro `--bin-dir`.

```
$ which aws  
/usr/local/bin/aws
```

2. Use o comando `ls` para encontrar o diretório para o qual o symlink aponta. Isso fornece o caminho que deve ser usado com o parâmetro `--install-dir`.

```
$ ls -l /usr/local/bin/aws  
lrwxrwxrwx 1 ec2-user ec2-user 49 Oct 22 09:49 /usr/local/bin/aws -> /usr/  
local/aws-cli/v2/current/bin/aws
```

5. Confirme a instalação com o comando a seguir.

```
$ aws --version  
aws-cli/2.10.0 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/2.4.5
```

Se não for possível encontrar o comando `aws`, talvez seja necessário reiniciar o terminal ou seguir a solução de problemas em [Solucionar erros](#).

(Opcional) Verificar a integridade do arquivo zip baixado

Se você optar por baixar manualmente o pacote do instalador `.zip` da AWS CLI versão 2 nas etapas acima, poderá executar as etapas a seguir para verificar as assinaturas por meio da ferramenta GnuPG.

Os arquivos .zip do pacote do instalador da AWS CLI versão 2 são assinados criptograficamente com assinaturas PGP. Se houver qualquer dano ou alteração dos arquivos, ocorrerá uma falha nessa verificação e você não deverá prosseguir com a instalação.

1. Baixe e instale o comando gpg usando o gerenciador de pacotes. Para obter mais informações sobre a GnuPG, consulte o [site da GnuPG](#).
2. Para criar o arquivo de chave pública, crie um arquivo de texto e cole o texto a seguir.

```
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBF2Cr7UBEADJZHcgus0J17ENSyumXh85z0TRV0xJorM2B/JL0kH0yigQ1uUG
ZMLhENaG0bYatdrKP+3H911vK050pXwn0/R7fB/FSTouki4ciIx50uLlnJZIXSzx
PqG10mkxImLnbGwoi6Lto0LYxqHN2iQtzlwTVmq9733zd3XfcXrZ3+Lb1HAGEt5G
TfNxEKJ8soPLyWmwDH6HWCnjZ/aIQRBTIQ05uVeEoYxSh6w0ai7ss/KveoSNBbYz
gbdzoqI2Y8cgH2nbfpg3DSasaLZEdCSsIsK1u05CinE7k2qZ7KgKAUIcT/cR/grk
C6VwsnDU00UCideXcQ8WeHutqvgZH1JgKDbznoIzeQHJD238GEu+eKhRHcz8/jeG
94zkcgJ0z3KbZGYMiTh277Fvj9zzvZsbMCedV1BTg3Tqgvdx4bdkhf5cH+7NtW0
lrFj6UwAsGukBTA0xC01/dnSmZhJ7Z1KmEWilro/g0rjt0xqRQut1IqG22TaqoPG
fYVN+en3ZwbT97kcgZDwqbuykNt64oZWc4XKCa3mprEGC3IbJTBfqq1XmZ719ywG
EEUJY01b2XrSuPwml39beWdKM8kzr10jn10m6+lpTRCBfo0wa9F8YZRhHPAkWkKX
XDe0GpWRj4oh0x0d2GWkyV5xyN14p2tQ0Cd00Dmz80yUTgRpPVQUt0EhXQARAQAB
tCFBV1MgQ0xJIFRlYW0gPGF3cy1jbG1AYW1hem9uLmNvbT6JAlQEEwEiAD4CGwMF
CwkIBwIGFQoJCA5CBByCAwECHgECF4AWIQT7Xbd/1cEYuAURraimMQrMRnJHXAUC
ZMKcEgUJCSEf3QAKCRCmMQrMRnJHXCiLD/4vior9J5tB+icri5WbDudS3ak/ve4q
XS6ZLm5S81+CBxy5aLQUlyFhuuaEHDC11fG780duxatzeHENASYVo3mmKNwrCBza
NJaeaWKLGT0MKwBSP5aa3dva8P/4oUP9GsQn0uWoXwNDWfrMbNI8gn+jC/3MigW
vD3fu6zC0WWLITNv2SJoQ1wILmb/uGfha68o4iTB0vcftVRua06DyqF+CrHX/0j0
k1EDQFMY9M4tsYT7X8NWfI8Vmc89nzpVL9fwd44WwpKIw1FBZP8S0sgDx2xDsxv
L8kM2Gt0iH0cHqF0+V7xtTKZy1o1iDbJKhu80Kc+YC/TmozD8oeGU2rEFXfLegwS
zT9N+jB38+dqaP9pRDsi45iGqyA8yavVBabpL0IQ9jU6eIV+kmcjIjcun/Uo8SjJ
0xQAsm41rxPaKV6vJUn10wVNUhSkKk8mzN01SZwu7Hua6rdcCaGeB8uJ44AP3QzW
BNnrjtoN6A1N0D2wFmFE/YL/rHPxU1XwPntubYB/t3rXFL7ENQ00QH0KVXgRCley
sHMglg46c+nQLRzVTshjDjmtzvH9rcV9RKRoPetEggzCoD89veDA9jPR2Kw6RYkS
XzYm2fEv16/HRNYt7hJzneFqRIjHW5qAgSs/bcaRWpAU/QQzzJPVKCQNr4y0weyg
B8HCtGjfod0p1A==
=gdMc
-----END PGP PUBLIC KEY BLOCK-----
```

Para referência, veja a seguir os detalhes da chave pública.

Key ID:	A6310ACC4672
Type:	RSA

```
Size:                4096/4096
Created:             2019-09-18
Expires:             2024-07-26
User ID:             AWS CLI Team <aws-cli@amazon.com>
Key fingerprint:    FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672 475C
```

3. Importe a chave pública da AWS CLI com o comando a seguir, substituindo *public-key-file-name* pelo nome do arquivo da chave pública que você criou.

```
$ gpg --import public-key-file-name
gpg: /home/username/.gnupg/trustdb.gpg: trustdb created
gpg: key A6310ACC4672475C: public key "AWS CLI Team <aws-cli@amazon.com>" imported
gpg: Total number processed: 1
gpg:                imported: 1
```

4. Baixe o arquivo de assinatura da AWS CLI para o pacote baixado. Ele tem o mesmo caminho e nome do arquivo .zip ao qual ele corresponde, mas tem a extensão .sig. Nos exemplos a seguir, ele foi salvo no diretório atual como um arquivo chamado awscliv2.sig.

Linux x86 (64-bit)

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip.sig
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão *2.0.30* seria awscli-exe-linux-x86_64-2.0.30.zip.sig, o que resultaria no seguinte comando:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.0.30.zip.sig
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Linux ARM

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip.sig
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `awscli-exe-linux-aarch64-2.0.30.zip.sig`, o que resultaria no seguinte comando:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli-exe-linux-aarch64-2.0.30.zip.sig
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

5. Verifique a assinatura, passando os nomes dos arquivos `.sig` e `.zip` baixados como parâmetros para o comando `gpg`.

```
$ gpg --verify awscliv2.sig awscliv2.zip
```

A saída deve ser semelhante à seguinte.

```
gpg: Signature made Mon Nov  4 19:00:01 2019 PST
gpg:                using RSA key FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672 475C
gpg: Good signature from "AWS CLI Team <aws-cli@amazon.com>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:                There is no indication that the signature belongs to the owner.
Primary key fingerprint: FB5D B77F D5C1 18B8 0511  ADA8 A631 0ACC 4672 475C
```

Important

O aviso na saída é esperado e não indica um problema. Isso ocorre porque não há uma cadeia de confiança entre a chave PGP pessoal (se você tiver uma) e a chave PGP da AWS CLI. Para obter mais informações, consulte [Web of trust](#).

macOS

Requisitos de instalação

- Você sabe qual lançamento da AWS CLI versão 2 gostaria de instalar. Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.
- Oferecemos suporte à AWS CLI versão 2 nas versões do macOS de 64 bits com suporte da Apple.
- Como a AWS não mantém repositórios de terceiros, não podemos garantir que eles contenham a versão mais recente da AWS CLI.

Instruções de instalação

Você pode instalar a AWS CLI versão 2 no macOS usando as opções a seguir.

GUI installer

As etapas a seguir mostram como instalar ou atualizar para o lançamento mais recente da AWS CLI versão 2 usando a interface de usuário padrão do macOS e o navegador. Se você estiver atualizando para a versão mais recente, use o mesmo método de instalação usado para a versão atual.

1. Em seu navegador, baixe a versão específica da AWS CLI acrescentando um hífen e o número da versão ao nome do arquivo.

```
https://awscli.amazonaws.com/AWSCLIV2-version.number.pkg
```

Neste exemplo, o nome do arquivo para a versão **2.0.30** seria AWSCLIV2-2.0.30.pkg, o que resultaria no seguinte link: <https://awscli.amazonaws.com/AWSCLIV2-2.0.30.pkg>.

2. Execute o arquivo baixado e siga as instruções na tela. Você pode optar por instalar a AWS CLI versão 2 das seguintes maneiras:
 - Para todos os usuários no computador (requer **sudo**)
 - Você pode instalar em qualquer pasta ou escolher a padrão recomendada `/usr/local/aws-cli`.
 - O instalador cria automaticamente um symlink em `/usr/local/bin/aws` que vincula o programa principal na pasta de instalação que você escolheu.
 - Apenas para o usuário atual (não requer **sudo**)

- Você pode instalar em qualquer pasta para a qual tenha permissão de gravação.
- Devido a permissões de usuário padrão, após a conclusão do instalador, é necessário criar manualmente um arquivo de symlink no \$PATH que aponta para os programas aws e aws_completer usando os comandos a seguir no prompt de comando. Se \$PATH incluir uma pasta na qual você pode gravar, será possível executar o comando a seguir sem sudo se você especificar essa pasta como o caminho de destino. Se você não tiver uma pasta gravável no \$PATH, será necessário usar sudo nos comandos a fim de obter permissões para gravar na pasta de destino especificada. O local padrão para um symlink é /usr/local/bin/.

```
$ sudo ln -s /folder/installed/aws-cli/aws /usr/local/bin/aws  
$ sudo ln -s /folder/installed/aws-cli/aws_completer /usr/local/bin/  
aws_completer
```

 Note

É possível visualizar logs de depuração para a instalação pressionando Cmd+L em qualquer lugar do instalador. Essa ação abre um painel de log que permite filtrar e salvar o log. O arquivo de log também é salvo automaticamente em /var/log/install.log.

3. Para verificar se o shell pode encontrar e executar o comando aws no \$PATH, use os comandos a seguir.

```
$ which aws  
/usr/local/bin/aws  
$ aws --version  
aws-cli/2.10.0 Python/3.11.2 Darwin/18.7.0 botocore/2.4.5
```

Se não for possível encontrar o comando aws, talvez seja necessário reiniciar o terminal ou seguir a solução de problemas em [Solucionar erros](#).

Command line installer - All users

Se você tiver permissões `sudo`, poderá instalar a AWS CLI versão 2 para todos os usuários no computador. Nós fornecemos as etapas em um grupo fácil de copiar e colar. Consulte as descrições de cada linha nas etapas a seguir.

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `AWSCLI2-2.0.30.pkg`, o que resultaria no seguinte comando:

```
$ curl "https://awscli.amazonaws.com/AWSCLI2-2.0.30.pkg" -o "AWSCLI2.pkg"
$ sudo installer -pkg AWSCLI2.pkg -target /
```

1. Baixe o arquivo usando o comando `curl`. A opção `-o` especifica o nome do arquivo no qual o pacote obtido por download foi gravado. No exemplo anterior, o arquivo é gravado como `AWSCLI2.pkg` na pasta atual.

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `AWSCLI2-2.0.30.pkg`, o que resultaria no seguinte comando:

```
$ curl "https://awscli.amazonaws.com/AWSCLI2-2.0.30.pkg" -o "AWSCLI2.pkg"
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

2. Execute o programa `installer` padrão do macOS, especificando o arquivo `.pkg` baixado como a origem. Use o parâmetro `-pkg` para especificar o nome do pacote a ser instalado e o parâmetro `-target /` para especificar em qual unidade instalar o pacote. Os arquivos são instalados no `/usr/local/aws-cli`, e um symlink é criado automaticamente em `/usr/local/bin`. Você deve incluir `sudo` no comando para conceder permissões de gravação para essas pastas.

```
$ sudo installer -pkg ./AWSCLI2.pkg -target /
```

Após a conclusão da instalação, os logs de depuração são gravados em `/var/log/install.log`.

3. Para verificar se o shell pode encontrar e executar o comando `aws` no `$PATH`, use os comandos a seguir.

```
$ which aws
/usr/local/bin/aws
$ aws --version
aws-cli/2.10.0 Python/3.11.2 Darwin/18.7.0 botocore/2.4.5
```

Se não for possível encontrar o comando `aws`, talvez seja necessário reiniciar o terminal ou seguir a solução de problemas em [Solucionar erros](#).

Command line - Current user

1. Para especificar em qual pasta a AWS CLI está instalada, é necessário criar um arquivo XML. Esse é um arquivo no formato XML que se parece com o exemplo a seguir. Deixe todos os valores como mostrado e substitua o caminho `/Users/myusername` na linha 9 pelo caminho até a pasta em que deseja instalar a AWS CLI versão 2. A pasta já deve existir, caso contrário, o comando falhará. Esse exemplo XML especifica que o instalador instalará a AWS CLI na pasta `/Users/myusername`, em que ele criará uma pasta chamada `aws-cli`.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
  <array>
    <dict>
      <key>choiceAttribute</key>
      <string>customLocation</string>
      <key>attributeSetting</key>
      <string>/Users/myusername</string>
      <key>choiceIdentifier</key>
      <string>default</string>
    </dict>
  </array>
</plist>
```

2. Baixe o instalador `pkg` usando o comando `curl`. A opção `-o` especifica o nome do arquivo no qual o pacote obtido por download foi gravado. No exemplo anterior, o arquivo é gravado como `AWSCLIV2.pkg` na pasta atual.

Para a versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `AWSCLIV2-2.0.30.pkg`, o que resultaria no seguinte comando:

```
$ curl "https://awscli.amazonaws.com/AWSCLIV2-2.0.30.pkg" -o "AWSCLIV2.pkg"
```

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

3. Execute o programa `installer` padrão do macOS com as seguintes opções:
 - Especifique o nome do pacote a ser instalado usando o parâmetro `-pkg`.
 - Especifique uma instalação somente para o usuário atual definindo o parâmetro `-target` como `CurrentUserHomeDirectory`.
 - Especifique o caminho (relativo à pasta atual) e o nome do arquivo XML criado no parâmetro `-applyChoiceChangesXML`.

O exemplo a seguir instala a AWS CLI na pasta `/Users/myusername/aws-cli`.

```
$ installer -pkg AWSCLIV2.pkg \
            -target CurrentUserHomeDirectory \
            -applyChoiceChangesXML choices.xml
```

4. Como as permissões de usuário padrão geralmente não permitem gravar em pastas no `$PATH`, o instalador nesse modo não tenta adicionar os symlinks aos programas `aws` e `aws_completer`. Para que a AWS CLI seja executada corretamente, é necessário criar manualmente os symlinks após a conclusão do instalador. Se `$PATH` incluir uma pasta na qual você pode gravar e você especificar a pasta como o caminho de destino, é possível executar o comando a seguir sem `sudo`. Se você não tiver uma pasta gravável no `$PATH`, será necessário usar `sudo` para permissões para gravar na pasta de destino especificada. O local padrão para um symlink é `/usr/local/bin/`.

```
$ sudo ln -s /folder/installed/aws-cli/aws /usr/local/bin/aws
$ sudo ln -s /folder/installed/aws-cli/aws_completer /usr/local/bin/aws_completer
```

Após a conclusão da instalação, os logs de depuração são gravados em `/var/log/install.log`.

5. Para verificar se o shell pode encontrar e executar o comando `aws` no `$PATH`, use os comandos a seguir.

```
$ which aws
/usr/local/bin/aws
$ aws --version
aws-cli/2.10.0 Python/3.11.2 Darwin/18.7.0 botocore/2.4.5
```

Se não for possível encontrar o comando `aws`, talvez seja necessário reiniciar o terminal ou seguir a solução de problemas em [Solucionar erros](#).

Windows

Requisitos de instalação

- Você sabe qual lançamento da AWS CLI versão 2 gostaria de instalar. Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.
- A AWS CLI pode ser utilizada em versões compatíveis com o Microsoft Windows de 64 bits.
- Direitos de administrador para instalar software

Instruções de instalação

Para atualizar a instalação atual da AWS CLI versão 2 no Windows, baixe um novo instalador sempre que atualizar para substituir as versões anteriores. A AWS CLI é atualizada regularmente. Para ver quando a versão mais recente foi lançada, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

1. Baixe e execute o instalador MSI da AWS CLI para Windows (64 bits) de uma das seguintes maneiras:
 - Baixando e executando o instalador MSI: para criar seu link de download para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo.

```
https://awscli.amazonaws.com/AWSCLIV2-version.number.msi
```

Neste exemplo, o nome do arquivo da versão **2.0.30** seria `AWSCLI2-2.0.30.msi`, o que resultaria no seguinte link: <https://awscli.amazonaws.com/AWSCLI2-2.0.30.msi>.

- Usando o comando `msiexec`: você também pode usar o instalador MSI adicionando o link ao comando `msiexec`. Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo.

```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLI2-version.number.msi
```

Neste exemplo, o nome do arquivo para a versão **2.0.30** seria `AWSCLI2-2.0.30.msi`, o que resultaria no seguinte link: <https://awscli.amazonaws.com/AWSCLI2-2.0.30.msi>

```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLI2-2.0.30.msi
```

Com relação a vários parâmetros que podem ser usados com `msiexec`, consulte [msiexec](#) no site de documentação da Microsoft.

Para obter uma lista das versões, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

2. Para confirmar a instalação, abra o menu Início, procure `cmd` para abrir uma janela do prompt de comando e, no prompt de comando, use o comando `aws --version`.

```
C:\> aws --version  
aws-cli/2.10.0 Python/3.11.2 Windows/10 exe/AMD64 prompt/off
```

Se o Windows não puder localizar o programa, talvez seja necessário fechar e abrir a janela do prompt de comando novamente para atualizar o caminho ou seguir a solução de problemas em [Solucionar erros](#).

Solução de problemas de erros de instalação e desinstalação da AWS CLI

Se você encontrar problemas após instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas. Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).

Próximas etapas

Depois de concluir as etapas em [the section called “Pré-requisitos”](#) e instalar a AWS CLI, você deve utilizar [the section called “Configuração”](#).

Compilar e instalar a AWS CLI da origem

Este tópico descreve como usar o código-fonte para instalar ou atualizar para a versão mais recente da AWS Command Line Interface (AWS CLI) em sistemas operacionais compatíveis.

Para obter informações sobre as versões mais recentes da AWS CLI, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

Important

As versões 1 e 2 da AWS CLI usam o mesmo nome de comando da aws. Se você instalou a AWS CLI versão 1 anteriormente, consulte [Migrar da AWS CLI versão 1 para a versão 2](#).

Tópicos

- [Por que compilar usando o código-fonte?](#)
- [Etapas rápidas](#)
- [Etapa 1: Configurar todos os requisitos](#)
- [Etapa 2: Configurar a instalação da AWS CLI pelo código-fonte](#)
- [Etapa 3: Compilar a AWS CLI](#)
- [Etapa 4: Instalar a AWS CLI](#)
- [Etapa 5: Verificar a instalação da AWS CLI](#)
- [Exemplos de fluxo de trabalho](#)
- [Solução de problemas de erros de instalação e desinstalação da AWS CLI](#)
- [Próximas etapas](#)

Por que compilar usando o código-fonte?

A AWS CLI está [disponível na forma de instaladores predefinidos](#) para a maioria das plataformas e ambientes, bem como na forma de imagem do Docker.

Geralmente, esses instaladores oferecem cobertura para a maioria dos casos de uso. As instruções de instalação usando o código-fonte servem para ajudar nos casos de uso que nossos instaladores não cobrem. Alguns dos casos de uso incluem o seguinte:

- Os instaladores predefinidos não oferecem suporte ao seu ambiente. Por exemplo, o ARM de 32 bits não é compatível com instaladores predefinidos.
- Os instaladores predefinidos têm dependências que seu ambiente não tem. Por exemplo, o Alpine Linux usa [musl](#), mas os instaladores atuais exigem `glibc`, o que impede que os instaladores predefinidos funcionem imediatamente.
- Os instaladores predefinidos exigem recursos aos quais seu ambiente restringe o acesso. Por exemplo, sistemas com segurança reforçada podem não conceder permissões à memória compartilhada. Isso é necessário para o instalador congelado `aws`.
- Os instaladores predefinidos geralmente são bloqueadores para mantenedores em gerenciadores de pacotes, já que há preferência pelo controle total sobre o processo de compilação de código e pacotes. A compilação usando o código-fonte permite que os mantenedores da distribuição tenham um processo mais simplificado para manter a AWS CLI atualizada. A habilitação de mantenedores fornece aos clientes versões mais atualizadas da AWS CLI ao instalar usando um gerenciador de pacotes de terceiros, como `brew`, `yum` e `apt`.
- Os clientes que aplicam patches na funcionalidade da AWS CLI precisam compilar e instalar a AWS CLI usando o código-fonte. Isso é especialmente importante para membros da comunidade que desejam testar as alterações feitas no código-fonte antes de contribuir para a alteração no repositório GitHub da AWS CLI.

Etapas rápidas

Note

Assume-se que todos os exemplos de código são executados da raiz do diretório do código-fonte.

Para compilar e instalar a AWS CLI usando o código-fonte, siga as etapas nesta seção. A AWS CLI utiliza o [GNU Autotools](#) para instalar pelo código-fonte. No caso mais simples, a AWS CLI pode ser instalada usando o código-fonte ao executar os comandos de exemplo padrão na raiz do repositório GitHub da AWS CLI.

1. [Configure todos os requisitos para o seu ambiente](#). Isso inclui ser capaz de executar arquivos gerados pelo [GNU Autotools](#) e instalar o Python 3.8 ou posterior.
2. No seu terminal, navegue até o nível superior da pasta do código-fonte da AWS CLI e execute o comando `./configure`. Esse comando verifica o sistema em busca de todas as dependências necessárias e gera um `Makefile` para compilar e instalar a AWS CLI com base nas configurações detectadas e especificadas.

Linux and macOS

O exemplo de comando `./configure` a seguir define a configuração de compilação para a AWS CLI usando as configurações padrão.

```
$ ./configure
```

Windows PowerShell

Antes de executar qualquer comando que chame MSYS2, você deve preservar seu diretório de trabalho atual:

```
PS C:\> $env:CHERE_INVOKING = 'yes'
```

Depois, use o exemplo de comando `./configure` a seguir para definir a configuração de compilação para a AWS CLI usando seu caminho local para o executável do Python, instalando em `C:\Program Files\AWSCLI` e baixando todas as dependências.

```
PS C:\> C:\msys64\usr\bin\bash -lc " PYTHON='C:\path\to\python.exe' ./configure --prefix='C:\Program Files\AWSCLI' --with-download-deps "
```

Para obter detalhes, opções de configuração disponíveis e informações de configuração padrão, consulte a seção [the section called “Etapa 2: Configurar a instalação da AWS CLI pelo código-fonte”](#).

3. Execute o comando `make`. Esse comando compila a AWS CLI de acordo com suas definições de configuração.

O exemplo de comando `make` a seguir é compilado com opções padrão usando suas configurações de `./configure` existentes.

Linux and macOS

```
$ make
```

Windows PowerShell

```
PS C:\> C:\msys64\usr\bin\bash -lc "make"
```

Para obter detalhes e opções de compilação disponíveis, consulte a seção [the section called “Etapa 3: Compilar a AWS CLI”](#).

4. Execute o comando `make install`. Esse comando instala a AWS CLI compilada no local configurado em seu sistema.

O exemplo de comando `make install` a seguir instala a AWS CLI compilada e cria symlinks em seus locais configurados usando as configurações de comando padrão.

Linux and macOS

```
$ make install
```

Windows PowerShell

```
PS C:\> C:\msys64\usr\bin\bash -lc "make install"
```

Após a instalação, adicione o caminho à AWS CLI usando o seguinte:

```
PS C:\> $Env: PATH += ";C:\Program Files\AWSCLI\bin\"
```

Para obter detalhes e opções de instalação disponíveis, consulte a seção [the section called “Etapa 4: Instalar a AWS CLI”](#).

5. Confirme se a instalação da AWS CLI foi bem-sucedida usando o seguinte comando:

```
$ aws --version  
aws-cli/2.10.0 Python/3.11.2 Windows/10 exe/AMD64 prompt/off
```

Para obter as etapas de solução de problemas para erros de instalação, consulte a seção [the section called “Solução de problemas de erros de instalação e desinstalação da AWS CLI”](#).

Etapa 1: Configurar todos os requisitos

Para compilar a AWS CLI usando o código-fonte, antes você precisa concluir o seguinte:

Note

Assume-se que todos os exemplos de código são executados da raiz do diretório do código-fonte.

1. Faça download do código-fonte da AWS CLI bifurcando o repositório GitHub da AWS CLI ou baixando o tarball do código-fonte. As instruções são uma das seguintes opções:
 - Bifurcar e clonar o [repositório da AWS CLI](#) do GitHub. Para obter mais informações, consulte [Fork a repo](#) (Bifurcar um repositório) no GitHub Docs.
 - Baixe o tarball mais recente da fonte em <https://awscli.amazonaws.com/awscli.tar.gz> e extraia o conteúdo usando os seguintes comandos:

```
$ curl -o awscli.tar.gz https://awscli.amazonaws.com/awscli.tar.gz
$ tar -xzf awscli.tar.gz
```

Note

Para baixar uma versão específica, use o seguinte formato de link: <https://awscli.amazonaws.com/awscli-númerodaversão.tar.gz>

Por exemplo, para a versão 2.10.0, o link é o seguinte: <https://awscli.amazonaws.com/awscli-2.10.0.tar.gz>

As versões da fonte estão disponíveis a partir da versão 2.10.0 da AWS CLI.

(Opcional) Verificar a integridade do arquivo zip baixado ao realizar as seguintes etapas:

1. É possível usar as etapas a seguir para verificar as assinaturas usando a ferramenta GnuPG.

Os arquivos .zip do pacote do instalador da AWS CLI são assinados criptograficamente com assinaturas PGP. Se houver qualquer dano ou alteração dos arquivos, ocorrerá uma falha nessa verificação e você não deverá prosseguir com a instalação.

2. Baixe e instale o comando gpg usando o gerenciador de pacotes. Para obter mais informações sobre a GnuPG, consulte o [site da GnuPG](#).
3. Para criar o arquivo de chave pública, crie um arquivo de texto e cole o texto a seguir.

```
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBF2Cr7UBEADJZHcgusOJl7ENSyumXh85z0TRV0xJorM2B/JL0kH0yigQluUG
ZMLhENAG0bYatdrKP+3H911vK050pXwn0/R7fB/FSTouki4ciIx50uLlnJZIXszx
PqG10mkxImLNBGWoi6Lto0LYxqHN2iQtz1wTVmq9733zd3XfcXrZ3+Lb1HAgEt5G
TfNxEKJ8soPLYWmwDH6HWCnjZ/aIQRBTIQ05uVeEoYxSh6w0ai7ss/KveoSNBbYz
gbdzoqI2Y8cgH2nbfpg3DSasaLZECSsIsK1u05CinE7k2qZ7KgKAUIcT/cR/grk
C6VwsnDU00UCideXcQ8WeHutqvgZH1JgKDbznoIzeQHJD238GEu+eKhRHcz8/jeG
94zkcqJ0z3KbZGYMiTh277Fvj9zzvZsbMBCedV1BTg3Tqgvdx4bdkhf5cH+7NtW0
lrFj6UwAsGukBTA0xC01/dnSmZhJ7Z1KmEWilro/g0rjt0xqRQut1IqG22TaqoPG
fYVN+en3ZwbT97kcgZDwqbubuykNt64oZWc4XKCa3mprEGC3IbJTBfqq1XmZ719yWG
EEUJY01b2XrSuPWm139beWdKM8kzr10jn10m6+1pTRCBfo0wa9F8YZRhHPAkwKkX
XDe0GpWRj4oh0x0d2GWkyV5xyN14p2tQ0Cd00Dmz80yUTgRpPVQUt0EhXQARAQAB
tCFBV1MgQ0xJIFR1YW0gPGF3cy1jbG1AYW1hem9uLmNvbT6JAlQEEwEIAD4WIQT7
Xbd/1cEYUauraimMQrMRnJHXAUCXYKvtQIbAwUJB4TOAAULCQgHAgYVCgkICwIE
FgIDAQIeAQIXgAAKCRcmMQrMRnJHXJIXEACHUIkg80uPUkGjE3jejvQSA1aWuAM
yzy6fdpd1RUz6M6nmsUh0ExjVivibEJpzK5mhuSZ41b0vJ2ZUPgCv4zs2nBd7BGJ
MxKiWgBREgVtdqZ0SzyYH4PYCJSE732x/Fw9hfnh1dMTXNcrQXzw0mmFNNegG00x
au+Vnpr5Kz3smiTrIwZbRudo1ijhCYPQ7t5Cmp9kjC6b0bvy1hSIg2xNbMAN/Do
ikebAl36uA6Y/Uczjj3GxZW4ZWeFirMidKbtqvUz2y0UFszobjiBSqZZHCreC34B
hw9bFNpuWC/0SrXgohdsc6vK50pDGDv5kM2qo9tMQ/izsAwTh/d/GzZv8H41V9e0
tEis+EpR497PaxKKh9tJf0N6Q1YLRHof5xePZt0I1S3gfvSH5hXA3HJ9yIxb8T0H
QYmVr3aIUes20i6meI3fuV36VFupwfrTKaL7VXnsrK2fq5cRvyJLNzXucg0WAjPF
RrAGLzY7nPlxeg1a0aeP+pdsqjqlPJom80CWc1+6DWbg0jsC74WoesAqgBItODMB
rsal1y/q+bPzpsnWjzHV8+1/EtZmSc8ZUGSJOPkfc7h0bnfk118h+1QtKTjZme4d
H17gsBJr+opwJw/Zio2LMjQB0qlm3K1A4zFTh7wBC7He6KPQea1p2XAMgtvAtNe
YLZATHZKTJyiqA==
=vY0k
-----END PGP PUBLIC KEY BLOCK-----
```

Para referência, veja a seguir os detalhes da chave pública.

Key ID:	A6310ACC4672
---------	--------------

```
Type:                RSA
Size:                4096/4096
Created:             2019-09-18
Expires:             2023-09-17
User ID:             AWS CLI Team <aws-cli@amazon.com>
Key fingerprint:    FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672 475C
```

4. Importe a chave pública da AWS CLI com o comando a seguir, substituindo *public-key-file-name* pelo nome do arquivo da chave pública que você criou.

```
$ gpg --import public-key-file-name
gpg: /home/username/.gnupg/trustdb.gpg: trustdb created
gpg: key A6310ACC4672475C: public key "AWS CLI Team <aws-cli@amazon.com>"
imported
gpg: Total number processed: 1
gpg:             imported: 1
```

5. Baixe o arquivo de assinatura da AWS CLI para o pacote baixado em <https://awscli.amazonaws.com/awscli.tar.gz.sig>. Ele tem o mesmo caminho e nome do arquivo tarball ao qual ele corresponde, mas tem a extensão .sig. Salve-o no mesmo caminho do arquivo tarball. Ou use o bloco de comandos a seguir:

```
$ curl -o awscliv2.sig https://awscli.amazonaws.com/awscli.tar.gz.sig
```

6. Verifique a assinatura, passando os nomes dos arquivos .sig e .zip baixados como parâmetros para o comando gpg.

```
$ gpg --verify awscliv2.sig awscli.tar.gz
```

A saída deve ser semelhante à seguinte.

```
gpg: Signature made Mon Nov  4 19:00:01 2019 PST
gpg:             using RSA key FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672
475C
gpg: Good signature from "AWS CLI Team <aws-cli@amazon.com>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:             There is no indication that the signature belongs to the owner.
Primary key fingerprint: FB5D B77F D5C1 18B8 0511 ADA8 A631 0ACC 4672 475C
```

 Important

O aviso na saída é esperado e não indica um problema. Isso ocorre porque não há uma cadeia de confiança entre a chave PGP pessoal (se você tiver uma) e a chave PGP da AWS CLI. Para obter mais informações, consulte [Web of trust](#).

2. Você tem um ambiente que pode executar arquivos gerados pelo [GNU Autotools](#), como configure e Makefile. Esses arquivos são amplamente transportáveis entre as plataformas POSIX.

Linux and macOS

Se o Autotools ainda não estiver instalado em seu ambiente ou se você precisar atualizá-lo, siga as instruções de instalação em [How do I install the Autotools \(as user\)?](#) [Como instalar o Autotools (como usuário)?] ou [Basic Installation](#) (Instalação básica) na documentação da GNU.

Windows PowerShell

 Warning

Se você estiver em um ambiente Windows, sugerimos que use os instaladores predefinidos. Para obter instruções de instalação sobre os instaladores predefinidos, consulte [the section called “Instalar/atualizar”](#)

Como o Windows não vem com um shell compatível com POSIX, você precisa instalar um software adicional para instalar a AWS CLI usando o código-fonte. O [MSYS2](#) fornece um conjunto de ferramentas e bibliotecas para ajudar a compilar e instalar software do Windows, especialmente para os scripts baseados em POSIX que o Autotools usa.

1. Instale o MSYS2. Para obter informações sobre como instalar e usar o MSYS2, consulte as [instruções de instalação e uso](#) na documentação do MSYS2.
2. Abra o terminal MSYS2 e instale o Autotools usando o comando a seguir.

```
$ pacman -S autotools
```

Note

Ao usar os exemplos de código de configuração, compilação e instalação neste guia para Windows, assume-se o caminho de instalação padrão do MSYS2 de C:\msys64\usr\bin\bash. Ao chamar o MSYS2 dentro do PowerShell, você usará o seguinte formato, com o comando bash entre aspas:

```
PS C:\> C:\msys64\usr\bin\bash -lc "command example"
```

O exemplo de comando a seguir chama o comando `./configure`.

```
PS C:\> C:\msys64\usr\bin\bash -lc "./configure"
```

- Um intérprete Python 3.8 ou posterior é instalado. A versão mínima necessária do Python segue os mesmos cronogramas da [política oficial de suporte do Python para AWS SDKs e ferramentas](#). O intérprete recebe suporte somente até seis meses após a data de término do suporte.
- (Opcional) Instale todas as dependências da biblioteca Python de compilação e execução da AWS CLI. O comando `./configure` informa se está faltando alguma dependência e como instalá-la.

Você pode instalar e usar automaticamente essas dependências durante o processo de configuração. Consulte [the section called “Download de dependências”](#) para obter mais informações.

Etapa 2: Configurar a instalação da AWS CLI pelo código-fonte

A configuração para compilar e instalar a AWS CLI é especificada usando o script `configure`. Para obter a documentação de todas as opções de configuração, execute o script `configure` com a opção `--help`:

Linux and macOS

```
$ ./configure --help
```

Windows PowerShell

```
PS C:\> C:\msys64\usr\bin\bash -lc "./configure --help"
```

As opções mais importantes são as seguintes:

- [Local de instalação](#)
- [Intérprete Python](#)
- [Download de dependências](#)
- [Tipo de instalação](#)

Local de instalação

A instalação da AWS CLI pelo código-fonte usa dois diretórios configuráveis para instalar a AWS CLI:

- `libdir`: diretório pai onde a AWS CLI será instalada. O caminho para a instalação da AWS CLI é `<libdir-value>/aws-cli`. O valor padrão de `libdir` para Linux e macOS é `/usr/local/lib`, o que faz com que o diretório de instalação padrão seja `/usr/local/lib/aws-cli`.
- `bindir`: diretório em que os executáveis da AWS CLI são instalados. O local padrão é `/usr/local/bin`.

As seguintes opções de `configure` controlam os diretórios usados:

- `--prefix`: define o prefixo do diretório a ser usado na instalação. O valor padrão para Linux e macOS é `/usr/local`.
- `--libdir`: define o `libdir` a ser usado para instalar a AWS CLI. O valor padrão é `<prefix-value>/lib`. Se `--libdir` e `--prefix` não forem especificados, o padrão para Linux e macOS será `/usr/local/lib/`.
- `--bindir`: define o `bindir` a ser usado para instalar os executáveis `aws` e `aws_completer` da AWS CLI. O valor padrão é `<prefix-value>/bin`. Se `bindir` e `--prefix` não forem especificados, o padrão para Linux e macOS será `/usr/local/bin/`.

Linux and macOS

O exemplo de comando a seguir usa a opção `--prefix` para fazer uma instalação de usuário local da AWS CLI. Esse comando instala a AWS CLI em `$HOME/.local/lib/aws-cli` e os executáveis em `$HOME/.local/bin`:

```
$ ./configure --prefix=$HOME/.local
```

O exemplo de comando a seguir usa a opção `--libdir` para instalar a AWS CLI como uma aplicação complementar no diretório `/opt`. Esse comando instala a AWS CLI em `/opt/aws-cli` e os executáveis em seus locais padrão de `/usr/local/bin`.

```
$ ./configure --libdir=/opt
```

Windows PowerShell

O exemplo de comando a seguir usa a opção `--prefix` para fazer uma instalação de usuário local da AWS CLI. Esse comando instala a AWS CLI em `$HOME/.local/lib/aws-cli` e os executáveis em `$HOME/.local/bin`:

```
$ C:\msys64\usr\bin\bash -lc ".\configure --prefix='C:\Program Files\AWSCLI'"
```

O exemplo de comando a seguir usa a opção `--libdir` para instalar a AWS CLI como uma aplicação complementar no diretório `/opt`. Esse comando instala a AWS CLI em `C:\Program Files\AWSCLI\opt\aws-cli`.

Intérprete Python

Note

É altamente recomendável especificar o intérprete Python durante a instalação para Windows.

O script `./configure` seleciona automaticamente um intérprete instalado do Python 3.8 ou posterior para usar na compilação e execução da AWS CLI utilizando a macro [AM_PATH_PYTHON](#) em Autoconf.

O intérprete Python a ser usado pode ser definido explicitamente usando a variável de ambiente PYTHON ao executar o script configure:

Linux and macOS

```
$ PYTHON=/path/to/python ./configure
```

Windows PowerShell

```
PS C:\> C:\msys64\usr\bin\bash -lc "PYTHON='C:\path\to\python' ./configure"
```

Download de dependências

Por padrão, todas as dependências de compilação e de tempo de execução da AWS CLI já devem estar instaladas no sistema. Isso inclui todas as dependências da biblioteca Python. Todas as dependências são verificadas quando o script configure é executado; se o sistema não tiver uma das dependências do Python, o script configure retornará um erro.

O seguinte exemplo de código retorna um erro quando o sistema não tem uma das dependências:

Linux and macOS

```
$ ./configure
checking for a Python interpreter with version >= 3.8... python
checking for python... /Users/username/.envs/env3.11/bin/python
checking for python version... 3.11
checking for python platform... darwin
checking for GNU default python prefix... ${prefix}
checking for GNU default python exec_prefix... ${exec_prefix}
checking for python script directory (pythondir)... ${PYTHON_PREFIX}/lib/python3.11/
site-packages
checking for python extension module directory (pyexecdir)... ${PYTHON_EXEC_PREFIX}/
lib/python3.11/site-packages
checking for --with-install-type... system-sandbox
checking for --with-download-deps... Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "/Users/username/aws-code/aws-cli/./backends/build_system/__main__.py", line
125, in <module>
    main()
```

```

File "/Users/username/aws-code/aws-cli/./backends/build_system/__main__.py", line
121, in main
    parsed_args.func(parsed_args)
File "/Users/username/aws-code/aws-cli/./backends/build_system/__main__.py", line
49, in validate
    validate_env(parsed_args.artifact)
File "/Users/username/aws-code/aws-cli/./backends/build_system/validate_env.py",
line 68, in validate_env
    raise UnmetDependenciesException(unmet_deps, in_venv)
validate_env.UnmetDependenciesException: Environment requires following Python
dependencies:

awscli (required: ('>=0.12.4', '<0.17.0')) (version installed: None)

We recommend using --with-download-deps flag to automatically create a virtualenv
and download the dependencies.

If you want to manage the dependencies yourself instead, run the following pip
command:
/Users/username/.envs/env3.11/bin/python -m pip install --prefer-binary
'awscli>=0.12.4,<0.17.0'

configure: error: "Python dependencies not met."

```

Windows PowerShell

```

PS C:\> C:\msys64\usr\bin\bash -lc "./configure"
checking for a Python interpreter with version >= 3.8... python
checking for python... /Users/username/.envs/env3.11/bin/python
checking for python version... 3.11
checking for python platform... darwin
checking for GNU default python prefix... ${prefix}
checking for GNU default python exec_prefix... ${exec_prefix}
checking for python script directory (pythondir)... ${PYTHON_PREFIX}/lib/python3.11/
site-packages
checking for python extension module directory (pyexecdir)... ${PYTHON_EXEC_PREFIX}/
lib/python3.11/site-packages
checking for --with-install-type... system-sandbox
checking for --with-download-deps... Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "/Users/username/aws-code/aws-cli/./backends/build_system/__main__.py", line
125, in <module>

```



```
main()
File "/Users/username/aws-code/aws-cli/./backends/build_system/__main__.py", line
121, in main
    parsed_args.func(parsed_args)
File "/Users/username/aws-code/aws-cli/./backends/build_system/__main__.py", line
49, in validate
    validate_env(parsed_args.artifact)
File "/Users/username/aws-code/aws-cli/./backends/build_system/validate_env.py",
line 68, in validate_env
    raise UnmetDependenciesException(unmet_deps, in_venv)
validate_env.UnmetDependenciesException: Environment requires following Python
dependencies:

awsCRT (required: ('>=0.12.4', '<0.17.0')) (version installed: None)

We recommend using --with-download-deps flag to automatically create a virtualenv
and download the dependencies.

If you want to manage the dependencies yourself instead, run the following pip
command:
/Users/username/.envs/env3.11/bin/python -m pip install --prefer-binary
'awsCRT>=0.12.4,<0.17.0'

configure: error: "Python dependencies not met."
```

Para instalar automaticamente as dependências necessárias do Python, use a opção `--with-download-deps`. Ao usar esse sinalizador, o processo de compilação faz o seguinte:

- Ignora a verificação de dependências da biblioteca Python.
- Define as configurações para fazer download de todas as dependências necessárias do Python e usar somente as dependências baixadas para compilar a AWS CLI durante a compilação de make.

O seguinte exemplo de comando de configuração usa a opção `--with-download-deps` para fazer download e usar as dependências do Python:

Linux and macOS

```
$ ./configure --with-download-deps
```

Windows PowerShell

```
PS C:\> C:\msys64\usr\bin\bash -lc "./configure --with-download-deps"
```

Tipo de instalação

O processo de instalação pelo código-fonte é compatível com os seguintes tipos de instalação:

- **system-sandbox:** (padrão) cria um ambiente virtual Python isolado, instala a AWS CLI no ambiente virtual e symlinks nos executáveis `aws` e `aws_completer` no ambiente virtual. Essa instalação da AWS CLI depende diretamente do intérprete Python selecionado para o respectivo tempo de execução.

Esse é um mecanismo de instalação leve para instalar a AWS CLI em um sistema e segue as práticas recomendadas do Python ao colocar a instalação em um ambiente virtual de área restrita para testes. Essa instalação é destinada a clientes que desejam instalar a AWS CLI pelo código-fonte da maneira mais simples possível com a instalação acoplada à sua instalação do Python.

- **portable-exe:** congela a AWS CLI em um executável independente que pode ser distribuído para ambientes de arquiteturas semelhantes. Esse é o mesmo processo usado para gerar os executáveis oficiais predefinidos da AWS CLI. O `portable-exe` congela em uma cópia do intérprete Python escolhido na etapa `configure` a ser usada para o tempo de execução da AWS CLI. Isso permite que ela seja movida para outras máquinas que podem não ter um intérprete Python.

Esse tipo de compilação é útil porque você pode garantir que a instalação da AWS CLI não esteja acoplada à versão do Python instalada no ambiente e pode distribuir uma compilação para outro sistema que talvez ainda não tenha o Python instalado. Isso permite que você controle as dependências e a segurança dos executáveis da AWS CLI usados.

Para configurar o tipo de instalação, use a opção `--with-install-type` e especifique um valor de `portable-exe` ou `system-sandbox`.

O seguinte exemplo de comando `./configure` especifica um valor de `portable-exe`:

Linux and macOS

```
$ ./configure --with-install-type=portable-exe
```

Windows PowerShell

```
PS C:\> C:\msys64\usr\bin\bash -lc "./configure --with-install-type=portable-exe"
```

Etapa 3: Compilar a AWS CLI

Use o comando `make` para compilar a AWS CLI usando suas definições de configuração:

Linux and macOS

```
$ make
```

Windows PowerShell

```
PS C:\> C:\msys64\usr\bin\bash -lc "make"
```

Note

Ao usar o comando **make**, as seguintes etapas são concluídas em segundo plano:

1. Um ambiente virtual é criado no diretório de compilação usando o módulo [venv](#) do Python. O ambiente virtual é inicializado com uma [versão do pip que é fornecida na biblioteca padrão do Python](#).
2. Copia as dependências da biblioteca Python. Dependendo de o sinalizador `--with-download-deps` ter sido especificado ou não no comando `configure`, esta etapa executa uma das seguintes ações:
 - O `--with-download-deps` é especificado. As dependências do Python são instaladas por `pip`. Isso inclui `wheel`, `setuptools` e todas as dependências de tempo de execução da AWS CLI. Se você estiver compilando o `portable-exe`, `pyinstaller` será instalado. Esses requisitos são todos especificados em arquivos de bloqueio gerados de [pip-compile](#).
 - O `--with-download-deps` não é especificado. As bibliotecas Python do pacote do site do intérprete Python e quaisquer scripts (por exemplo, `pyinstaller`) são copiados para o ambiente virtual que está sendo usado para a compilação.

3. Executa `pip install` diretamente na base de código da AWS CLI para realizar uma compilação e instalação da AWS CLI off-line e em árvore no ambiente virtual de compilação. Essa instalação usa os sinalizadores `pip --no-build-isolation`, `--use-feature=in-tree-build`, `--no-cache-dir` e `--no-index`.
4. (Opcional) Se o `--install-type` estiver definido como `portable-exe` no comando `configure`, um executável independente é compilado usando `pyinstaller`.

Etapa 4: Instalar a AWS CLI

O comando `make install` instala a AWS CLI compilada no local configurado em seu sistema.

Linux and macOS

O seguinte exemplo de comando instala a AWS CLI usando suas definições de configuração e compilação:

```
$ make install
```

Windows PowerShell

O seguinte exemplo de comando instala a AWS CLI usando suas definições de configuração e compilação, depois adiciona uma variável de ambiente com o caminho para a AWS CLI:

```
PS C:\> C:\msys64\usr\bin\bash -lc " make install "
```

```
PS C:\> $Env: PATH += ";C:\Program Files\AWSCLI\bin\"
```

A regra `make install` é compatível com a variável `DESTDIR`. Quando especificada, essa variável prefixa o caminho indicado no caminho de instalação já configurado ao instalar a AWS CLI. Por padrão, nenhum valor é definido para essa variável.

Linux and macOS

O seguinte exemplo de código usa um sinalizador `--prefix=/usr/local` para configurar um local de instalação, depois altera esse destino usando `DESTDIR=/tmp/stage` para o comando `make install`. Esses comandos resultam na instalação da AWS CLI em `/tmp/stage/usr/local/lib/aws-cli` e na localização dos executáveis em `/tmp/stage/usr/local/bin`.

```
$ ./configure --prefix=/usr/local
$ make
$ make DESTDIR=/tmp/stage install
```

Windows PowerShell

O seguinte exemplo de código usa um sinalizador `--prefix=\awscli` para configurar um local de instalação, depois altera esse destino usando `DESTDIR=C:\Program Files` para o comando `make install`. Esses comandos resultam na instalação da AWS CLI em `C:\Program Files\awscli`.

```
$ ./configure --prefix=\awscli
$ make
$ make DESTDIR='C:\Program Files' install
```

Note

Ao executar `make install`, as seguintes etapas são concluídas em segundo plano:

1. Move um dos seguintes para o diretório de instalação configurado:
 - Se o tipo de instalação for `system-sandbox`, moverá o ambiente virtual compilado.
 - Se o tipo de instalação for `portable-exe`, moverá o executável independente compilado.
2. Cria symlinks para os executáveis `aws` e `aws_completer` no diretório `bin` configurado.

Etapa 5: Verificar a instalação da AWS CLI

Confirme se a instalação da AWS CLI foi bem-sucedida usando o seguinte comando:

```
$ aws --version
aws-cli/2.10.0 Python/3.11.2 Windows/10 exe/AMD64 prompt/off
```

Se o comando `aws` não for reconhecido, poderá ser necessário reiniciar o terminal para que novos symlinks sejam atualizados. Se você encontrar outros problemas depois de instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas comuns.

Exemplos de fluxo de trabalho

Esta seção fornece alguns exemplos básicos de fluxo de trabalho para instalação usando o código-fonte.

Instalação básica em Linux e macOS

O exemplo a seguir é um fluxo de trabalho de instalação básica em que a AWS CLI é instalada no local padrão de `/usr/local/lib/aws-cli`.

```
$ cd path/to/cli/respository/
$ ./configure
$ make
$ make install
```

Instalação automatizada em Windows

Note

Você deve executar o PowerShell como administrador para usar esse fluxo de trabalho.

O MSYS2 pode ser usado de forma automatizada em uma configuração de CI. Consulte [Using MSYS2 in CI](#) (Como usar o MSYS2 em CI) na documentação do MSYS2.

Downloaded Tarball

Faça download do arquivo `awscli.tar.gz`, depois extraia e instale a AWS CLI. Ao usar os comandos abaixo, substitua os seguintes caminhos:

- `C:\msys64\usr\bin\bash` pelo caminho do MSYS2.
- `.\awscli-2.x.x\` pelo nome da pasta de extração de `awscli.tar.gz`.
- `PYTHON='C:\path\to\python.exe'` pelo caminho local do Python.

O seguinte exemplo de código automatiza a compilação e a instalação da AWS CLI pelo PowerShell usando o MSYS2 e especifica qual instalação local do Python deve ser usada:

```
PS C:\> curl -o awscli.tar.gz https://awscli.amazonaws.com/awscli.tar.gz #
Download the awscli.tar.gz file in the current working directory
PS C:\> tar -xvzf .\awscli.tar.gz # Extract awscli.tar.gz file
```

```
PS C:\> cd .\awscli-2.x.x\ # Navigate to the root of the extracted files
PS C:\> $env:CHERE_INVOKING = 'yes' # Preserve the current working directory
PS C:\> C:\msys64\usr\bin\bash -lc " PYTHON='C:\path\to\python.exe' ./configure --
prefix='C:\Program Files\AWSCLI' --with-download-deps "
PS C:\> C:\msys64\usr\bin\bash -lc "make"
PS C:\> C:\msys64\usr\bin\bash -lc "make install"
PS C:\> $Env:PATH += ";C:\Program Files\AWSCLI\bin\"
PS C:\> aws --version
aws-cli/2.10.0 Python/3.11.2 Windows/10 source-sandbox/AMD64 prompt/off
```

GitHub Repository

Faça download do arquivo `awscli.tar.gz`, depois extraia e instale a AWS CLI. Ao usar os comandos abaixo, substitua os seguintes caminhos:

- `C:\msys64\usr\bin\bash` pelo caminho do MSYS2.
- `C:\path\to\cli\repository\` pelo caminho para o [repositório da AWS CLI](#) clonado do GitHub. Para obter mais informações, consulte [Fork a repo](#) (Bifurcar um repositório) no GitHub Docs.
- `PYTHON='C:\path\to\python.exe'` pelo caminho local do Python.

O seguinte exemplo de código automatiza a compilação e a instalação da AWS CLI pelo PowerShell usando o MSYS2 e especifica qual instalação local do Python deve ser usada:

```
PS C:\> cd C:\path\to\cli\repository\
PS C:\> $env:CHERE_INVOKING = 'yes' # Preserve the current working directory
PS C:\> C:\msys64\usr\bin\bash -lc " PYTHON='C:\path\to\python.exe' ./configure --
prefix='C:\Program Files\AWSCLI' --with-download-deps "
PS C:\> C:\msys64\usr\bin\bash -lc "make"
PS C:\> C:\msys64\usr\bin\bash -lc "make install"
PS C:\> $Env:PATH += ";C:\Program Files\AWSCLI\bin\"
PS C:\> aws --version
```

Contêiner do Alpine Linux

Veja a seguir um exemplo de Dockerfile que pode ser usado para obter uma instalação funcional da AWS CLI em um contêiner do Alpine Linux como uma [alternativa aos binários predefinidos para Alpine](#). Ao usar esse exemplo, substitua `AWSCLI_VERSION` pelo número de versão da AWS CLI desejado:

```
FROM python:3.8-alpine AS builder

ENV AWSCLI_VERSION=2.10.1

RUN apk add --no-cache \
    curl \
    make \
    cmake \
    gcc \
    g++ \
    libc-dev \
    libffi-dev \
    openssl-dev \
    && curl https://awscli.amazonaws.com/awscli-${AWSCLI_VERSION}.tar.gz | tar -xz \
    && cd awscli-${AWSCLI_VERSION} \
    && ./configure --prefix=/opt/aws-cli/ --with-download-deps \
    && make \
    && make install

FROM python:3.8-alpine

RUN apk --no-cache add groff

COPY --from=builder /opt/aws-cli/ /opt/aws-cli/

ENTRYPOINT ["/opt/aws-cli/bin/aws"]
```

Essa imagem é compilada e a AWS CLI é invocada de um contêiner semelhante ao que é compilado no Amazon Linux 2:

```
$ docker build --tag awscli-alpine .
$ docker run --rm -it awscli-alpine --version
aws-cli/2.2.1 Python/3.8.11 Linux/5.10.25-linuxkit source-sandbox/x86_64.alpine.3
prompt/off
```

O tamanho final dessa imagem é menor do que o tamanho da imagem do Docker oficial da AWS CLI. Para obter informações sobre a imagem do Docker oficial, consulte [the section called “Amazon ECR Public/Docker”](#).

Solução de problemas de erros de instalação e desinstalação da AWS CLI

Para obter as etapas de solução de problemas para erros de instalação comuns, consulte [Solucionar erros](#). Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).

Para quaisquer problemas não abordados nos guias de solução, pesquise os problemas com o rótulo source-distribution no [repositório da AWS CLI](#) no GitHub. Se nenhum problema existente abordar seus erros, [crie um problema](#) para receber ajuda dos mantenedores da AWS CLI.

Próximas etapas

Depois de instalar a AWS CLI, execute um [the section called “Configuração”](#).

Execute a AWS CLI pelas imagens oficiais do Amazon ECR Public ou do Docker

Este tópico descreve como executar, controlar a versão e configurar a AWS CLI versão 2 no Docker usando a imagem do Amazon Elastic Container Registry Public (Amazon ECR Public) ou do Docker Hub. Para obter mais informações sobre como usar o Docker, consulte a [documentação do Docker](#).

As imagens oficiais fornecem isolamento, portabilidade e segurança aos quais a AWS oferece suporte e mantém diretamente. Isso permite usar a AWS CLI versão 2 em um ambiente baseado em contêiner sem precisar gerenciar a instalação sozinho.

Tópicos

- [Pré-requisitos](#)
- [Decidir entre o Amazon ECR Public e o Docker Hub](#)
- [Executar as imagens oficiais da AWS CLI versão 2](#)
- [Observações sobre interfaces e compatibilidade com versões anteriores das imagens oficiais](#)
- [Usar versões e tags específicas](#)
- [Atualizar para a imagem oficial mais recente](#)
- [Compartilhar arquivos de host, credenciais, variáveis de ambiente e configuração](#)
- [Reduzir o comando de execução do Docker](#)

Pré-requisitos

É necessário ter o Docker instalado. Para obter instruções de instalação, consulte o [site do Docker](#).

Para verificar a instalação do Docker, execute o seguinte comando e confirme se há uma saída.

```
$ docker --version
Docker version 19.03.1
```

Decidir entre o Amazon ECR Public e o Docker Hub

Recomendamos usar o Amazon ECR Public em vez do Docker Hub para imagens da AWS CLI. O Docker Hub tem um limite de taxa mais rígido para consumidores públicos, o que pode causar problemas de controle de utilização. Além disso, o Amazon ECR Public replica imagens em mais de uma região para fornecer sólida disponibilidade e lidar com problemas de interrupção da região.

Para obter mais informações sobre os limites de taxa do Docker Hub, consulte [Understanding Docker Hub Rate Limiting](#) (Noções básicas sobre a limitação de taxa do Docker Hub) no site do Docker.

Executar as imagens oficiais da AWS CLI versão 2

Na primeira vez que você usar o comando `docker run`, a imagem mais recente será baixada no computador. Cada uso subsequente do comando `docker run` é executado de sua cópia local.

Para executar as imagens do Docker da AWS CLI versão 2, use o comando `docker run`.

Amazon ECR Public

A imagem oficial do Amazon ECR Public da AWS CLI versão 2 está hospedada no Amazon ECR Public no [repositório aws-cli/aws-cli](#).

```
$ docker run --rm -it public.ecr.aws/aws-cli/aws-cli command
```

Docker Hub

A imagem do Docker oficial da AWS CLI versão 2 está hospedada no Docker Hub no repositório `amazon/aws-cli`.

```
$ docker run --rm -it amazon/aws-cli command
```

É assim que o comando funciona:

- `docker run --rm -it repository/name`: o equivalente ao executável `aws`. Sempre que você executar esse comando, o Docker ativará um contêiner da imagem baixada e executará o comando `aws`. Por padrão, a imagem usa a versão mais recente da AWS CLI versão 2.

Por exemplo, para chamar o comando `aws --version` no Docker, execute o seguinte.

Amazon ECR Public

```
$ docker run --rm -it public.ecr.aws/aws-cli/aws-cli --version
aws-cli/2.10.0 Python/3.7.3 Linux/4.9.184-linuxkit botocore/2.4.5dev10
```

Docker Hub

```
$ docker run --rm -it amazon/aws-cli --version
aws-cli/2.10.0 Python/3.7.3 Linux/4.9.184-linuxkit botocore/2.4.5dev10
```

- `--rm`: especifica a limpeza do contêiner após a saída do comando.
- `-it`: especifica a abertura de um pseudo-TTY com `stdin`. Isso permite fornecer uma entrada na AWS CLI versão 2 enquanto ela está sendo executada em um contêiner, por exemplo, usando os comandos `aws configure` e `aws help`. Ao escolher se deseja omitir `-it`, considere o seguinte:
 - Se você estiver executando scripts, não será necessário usar `-it`.
 - Se você estiver enfrentando erros em seus scripts, omitir `-it` de sua chamada do Docker poderá resolver o problema.
 - Se você estiver tentando canalizar a saída, `-it` poderá causar erros e omitir `-it` de sua chamada do Docker poderá resolver esse problema. Se quiser manter o sinalizador `-it`, mas ainda desejar canalizar a saída, desabilitar a [paginação do lado do cliente](#) que a AWS CLI usa por padrão deve resolver o problema.

Para obter mais informações sobre o comando `docker run`, consulte o [Docker reference guide](#).

Observações sobre interfaces e compatibilidade com versões anteriores das imagens oficiais

- A única ferramenta compatível na imagem é a AWS CLI. Somente o executável `aws` deve ser executado diretamente. Por exemplo, mesmo que `less` e `groff` forem explicitamente instalados na imagem, eles não deverão ser executados diretamente fora de um comando da AWS CLI.

- O diretório de trabalho `/aws` é controlado pelo usuário. A imagem não será gravada nesse diretório, a menos que seja instruído pelo usuário na execução de um comando da AWS CLI.
- Não há garantias de compatibilidade com versões anteriores quando se utiliza a etiqueta mais recente. Para garantir a compatibilidade com versões anteriores, é necessário fixar uma tag `<major.minor.patch>` específica, pois essas tags são imutáveis. Elas só serão enviadas uma vez.

Usar versões e tags específicas

A imagem oficial da AWS CLI versão 2 oferece várias versões que podem ser usadas, começando pela versão `2.0.6`. Para executar uma versão específica da AWS CLI versão 2, anexe a etiqueta apropriada ao seu comando `docker run`. Na primeira vez que você usar o comando `docker run` com uma tag, a imagem mais recente com essa tag será baixada no computador. Cada uso subsequente do comando `docker run` com essa etiqueta é executado de sua cópia local.

É possível usar dois tipos de etiqueta:

- `latest`: define a versão mais recente da AWS CLI versão 2 para a imagem. Recomendamos usar a etiqueta `latest` quando quiser a versão mais recente da AWS CLI versão 2. No entanto, não há garantias de compatibilidade com versões anteriores ao depender dessa etiqueta. A etiqueta `latest` é usada por padrão no comando `docker run`. Para usar explicitamente a etiqueta `latest`, anexe a etiqueta ao nome da imagem do contêiner.

Amazon ECR Public

```
$ docker run --rm -it public.ecr.aws/aws-cli/aws-cli:latest command
```

Docker Hub

```
$ docker run --rm -it amazon/aws-cli:latest command
```

- `<major.minor.patch>`: define uma versão específica da AWS CLI versão 2 para a imagem. Se você planeja usar a imagem oficial na produção, recomendamos usar uma versão específica da AWS CLI versão 2 para garantir a compatibilidade com versões anteriores. Por exemplo, para executar a versão `2.0.6`, anexe a versão ao nome da imagem do contêiner.

Amazon ECR Public

```
$ docker run --rm -it public.ecr.aws/aws-cli/aws-cli:2.0.6 command
```

Docker Hub

```
$ docker run --rm -it amazon/aws-cli:2.0.6 command
```

Atualizar para a imagem oficial mais recente

Como a imagem mais recente é baixada no computador somente na primeira vez que você usa o comando `docker run`, é necessário extrair manualmente uma imagem atualizada. Para atualizar manualmente para a versão mais recente, recomendamos extrair a imagem marcada com a etiqueta `latest`. Ao extrair a imagem, você baixa a versão mais recente no computador.

Amazon ECR Public

```
$ docker pull public.ecr.aws/aws-cli/aws-cli:latest
```

Docker Hub

```
$ docker pull amazon/aws-cli:latest
```

Compartilhar arquivos de host, credenciais, variáveis de ambiente e configuração

Como a AWS CLI versão 2 é executada em um contêiner, por padrão, a CLI não pode acessar o sistema de arquivos de host, que inclui a configuração e as credenciais. Para compartilhar o sistema de arquivos de host, as credenciais e a configuração com o contêiner, monte o diretório `~/ .aws` do sistema de host no contêiner em `/root/.aws` com o sinalizador `-v` para o comando `docker run`. Isso permite que a AWS CLI versão 2 em execução no contêiner localize informações do arquivo de host.

Amazon ECR Public

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws public.ecr.aws/aws-cli/aws-cli command
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%.aws:/root/.aws public.ecr.aws/aws-cli/aws-cli command
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\.aws:/root/.aws public.ecr.aws/aws-cli/aws-cli command
```

Docker Hub

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws amazon/aws-cli command
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%.aws:/root/.aws amazon/aws-cli command
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\.aws:/root/.aws amazon/aws-cli command
```

Para obter mais informações sobre o sinalizador `-v` e a montagem, consulte o [Docker reference guide](#).

Note

Para obter mais informações sobre os arquivos `config` e `credentials`, consulte [the section called “Configurações do arquivo de configuração e credenciais”](#).

Exemplo 1: Fornecer credenciais e configuração

Neste exemplo, estamos fornecendo a configuração e as credenciais de host ao executar o comando `s3 ls` para listar os buckets no Amazon Simple Storage Service (Amazon S3). Os exemplos abaixo usam o local padrão para credenciais e arquivos de configuração da AWS CLI. Para usar um local diferente, altere o caminho do arquivo.

Amazon ECR Public

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws public.ecr.aws/aws-cli/aws-cli s3 ls
2020-03-25 00:30:48 aws-cli-docker-demo
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%\aws:/root/.aws public.ecr.aws/aws-cli/aws-cli s3 ls
2020-03-25 00:30:48 aws-cli-docker-demo
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\aws:/root/.aws public.ecr.aws/aws-cli/aws-cli s3 ls
```

Docker Hub

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws amazon/aws-cli s3 ls
2020-03-25 00:30:48 aws-cli-docker-demo
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%\aws:/root/.aws amazon/aws-cli s3 ls
2020-03-25 00:30:48 aws-cli-docker-demo
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\aws:/root/.aws amazon/aws-cli s3 ls
```

Você pode chamar variáveis de ambiente do sistema específicas usando o sinalizador `-e`. Para usar uma variável de ambiente, chame-a pelo nome.

Amazon ECR Public

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws -e ENVVAR_NAME public.ecr.aws/aws-cli/  
aws-cli s3 ls  
2020-03-25 00:30:48 aws-cli-docker-demo
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%\aws:/root/.aws -e ENVVAR_NAME  
public.ecr.aws/aws-cli/aws-cli s3 ls  
2020-03-25 00:30:48 aws-cli-docker-demo
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\aws:/root/.aws -e ENVVAR_NAME  
public.ecr.aws/aws-cli/aws-cli s3 ls
```

Docker Hub

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws -e ENVVAR_NAME amazon/aws-cli s3 ls  
2020-03-25 00:30:48 aws-cli-docker-demo
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%\aws:/root/.aws -e ENVVAR_NAME amazon/aws-cli  
s3 ls  
2020-03-25 00:30:48 aws-cli-docker-demo
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\aws:/root/.aws -e ENVVAR_NAME amazon/  
aws-cli s3 ls
```

Exemplo 2: Baixar um arquivo do Amazon S3 no sistema de host

Para alguns comandos da AWS CLI versão 2, é possível ler arquivos do sistema de host no contêiner ou gravar arquivos do contêiner no sistema de host.

Neste exemplo, baixamos o objeto do S3 `s3://aws-cli-docker-demo/hello` no sistema de arquivos local, montando o diretório de trabalho atual no diretório `/aws` do contêiner. Ao baixar o objeto `hello` no diretório `/aws` do contêiner, o arquivo também é salvo no diretório de trabalho atual do sistema de host.

Amazon ECR Public

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws -v $(pwd):/aws public.ecr.aws/aws-cli/
aws-cli s3 cp s3://aws-cli-docker-demo/hello .
download: s3://aws-cli-docker-demo/hello to ./hello
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%.aws:/root/.aws -v %cd%:/aws public.ecr.aws/
aws-cli/aws-cli s3 cp s3://aws-cli-docker-demo/hello .
download: s3://aws-cli-docker-demo/hello to ./hello
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\.aws:/root/.aws -v $pwd\aws:/aws
public.ecr.aws/aws-cli/aws-cli s3 cp s3://aws-cli-docker-demo/hello .
```

Docker Hub

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws -v $(pwd):/aws amazon/aws-cli s3 cp s3://
aws-cli-docker-demo/hello .
download: s3://aws-cli-docker-demo/hello to ./hello
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%.aws:/root/.aws -v %cd%:/aws amazon/aws-cli
s3 cp s3://aws-cli-docker-demo/hello .
download: s3://aws-cli-docker-demo/hello to ./hello
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\.aws:/root/.aws -v $pwd\aws:/aws
amazon/aws-cli s3 cp s3://aws-cli-docker-demo/hello .
```

Para confirmar que o arquivo baixado existe no sistema de arquivos local, execute o seguinte.

Linux e macOS

```
$ cat hello
Hello from Docker!
```

Windows PowerShell

```
$ type hello
Hello from Docker!
```

Exemplo 3: Usar sua variável de ambiente AWS_PROFILE

Você pode chamar variáveis de ambiente do sistema específicas usando o sinalizador `-e`. Chame cada variável de ambiente que gostaria de usar. Neste exemplo, estamos fornecendo credenciais de host, a configuração e a variável de ambiente `AWS_PROFILE` ao executar o comando `s3 ls` para listar os buckets no Amazon Simple Storage Service (Amazon S3).

Amazon ECR Public

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws -e AWS_PROFILE public.ecr.aws/aws-cli/
aws-cli s3 ls
2020-03-25 00:30:48 aws-cli-docker-demo
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%\aws:/root/.aws -e AWS_PROFILE
public.ecr.aws/aws-cli/aws-cli s3 ls
2020-03-25 00:30:48 aws-cli-docker-demo
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\.aws:/root/.aws -e AWS_PROFILE  
public.ecr.aws/aws-cli/aws-cli s3 ls
```

Docker Hub

Linux e macOS

```
$ docker run --rm -it -v ~/.aws:/root/.aws -e AWS_PROFILE amazon/aws-cli s3 ls  
2020-03-25 00:30:48 aws-cli-docker-demo
```

Prompt de comando do Windows

```
$ docker run --rm -it -v %userprofile%.aws:/root/.aws -e AWS_PROFILE amazon/aws-cli  
s3 ls  
2020-03-25 00:30:48 aws-cli-docker-demo
```

Windows PowerShell

```
C:\> docker run --rm -it -v $env:userprofile\.aws:/root/.aws -e AWS_PROFILE amazon/  
aws-cli s3 ls
```

Reduzir o comando de execução do Docker

Para reduzir o comando `docker run`, sugerimos usar a capacidade do sistema operacional para criar um [symbolic link](#) (symlink) ou [alias](#) no Linux e no macOS ou [doskey](#) no Windows. Para definir o alias `aws`, é possível executar um dos comandos a seguir.

- Para obter acesso básico a comandos `aws`, execute o seguinte.

Amazon ECR Public

Linux e macOS

```
$ alias aws='docker run --rm -it public.ecr.aws/aws-cli/aws-cli'
```

Prompt de comando do Windows

```
C:\> doskey aws=docker run --rm -it public.ecr.aws/aws-cli/aws-cli $*
```

Windows PowerShell

```
C:\> Function AWSCLI {docker run --rm -it public.ecr.aws/aws-cli/aws-cli $args}
Set-Alias -Name aws -Value AWSCLI
```

Docker Hub

Linux e macOS

```
$ alias aws='docker run --rm -it amazon/aws-cli'
```

Prompt de comando do Windows

```
C:\> doskey aws=docker run --rm -it amazon/aws-cli $*
```

Windows PowerShell

```
C:\> Function AWSCLI {docker run --rm -it amazon/aws-cli $args}
Set-Alias -Name aws -Value AWSCLI
```

- Para obter acesso ao sistema de arquivos de host e às definições de configuração ao usar comandos aws, execute o indicado a seguir.

Amazon ECR Public

Linux e macOS

```
$ alias aws='docker run --rm -it -v ~/.aws:/root/.aws -v $(pwd):/aws
public.ecr.aws/aws-cli/aws-cli'
```

Prompt de comando do Windows

```
C:\> doskey aws=docker run --rm -it -v %userprofile%.aws:/root/.aws -v %cd%:/aws
public.ecr.aws/aws-cli/aws-cli $*
```

Windows PowerShell

```
C:\> Function AWSCLI {docker run --rm -it -v $env:userprofile\.aws:/root/.aws -v
$pwd\aws:/aws public.ecr.aws/aws-cli/aws-cli $args}
```

```
Set-Alias -Name aws -Value AWSCLI
```

Docker Hub

Linux e macOS

```
$ alias aws='docker run --rm -it -v ~/.aws:/root/.aws -v $(pwd):/aws amazon/aws-cli'
```

Prompt de comando do Windows

```
C:\> doskey aws=docker run --rm -it -v %userprofile%\aws:/root/.aws -v %cd%:/aws amazon/aws-cli %*
```

Windows PowerShell

```
C:\> Function AWSCLI {docker run --rm -it -v $env:userprofile\aws:/root/.aws -v $pwd\aws:/aws amazon/aws-cli $args}
Set-Alias -Name aws -Value AWSCLI
```

- Para atribuir uma versão específica para usar no alias aws, anexe a etiqueta de versão.

Amazon ECR Public

Linux e macOS

```
$ alias aws='docker run --rm -it -v ~/.aws:/root/.aws -v $(pwd):/aws public.ecr.aws/aws-cli/aws-cli:2.0.6'
```

Prompt de comando do Windows

```
C:\> doskey aws=docker run --rm -it -v %userprofile%\aws:/root/.aws -v %cd%:/aws public.ecr.aws/aws-cli/aws-cli:2.0.6 %*
```

Windows PowerShell

```
C:\> Function AWSCLI {docker run --rm -it -v $env:userprofile\aws:/root/.aws -v $pwd\aws:/aws public.ecr.aws/aws-cli/aws-cli:2.0.6 $args}
Set-Alias -Name aws -Value AWSCLI
```

Docker Hub

Linux e macOS

```
$ alias aws='docker run --rm -it -v ~/.aws:/root/.aws -v $(pwd):/aws amazon/aws-cli:2.0.6'
```

Prompt de comando do Windows

```
C:\> doskey aws=docker run --rm -it -v %userprofile%.aws:/root/.aws -v %cd%:/aws amazon/aws-cli:2.0.6 $*
```

Windows PowerShell

```
C:\> Function AWSCLI {docker run --rm -it -v $env:userprofile\.aws:/root/.aws -v $pwd\aws:/aws amazon/aws-cli:2.0.6 $args}  
Set-Alias -Name aws -Value AWSCLI
```

Depois de definir o alias, é possível executar a AWS CLI versão 2 de dentro de um contêiner como se ela estivesse instalada no sistema do host.

```
$ aws --version  
aws-cli/2.10.0 Python/3.7.3 Linux/4.9.184-linuxkit botocore/2.4.5dev10
```

Configurar a AWS CLI

Este tópico explica como definir rapidamente as configurações básicas que a AWS Command Line Interface (AWS CLI) usa para interagir com a AWS. Elas incluem as credenciais de segurança, o formato de saída padrão e a região padrão da AWS.

Tópicos

- [Reunir suas informações de credencial para acesso programático](#)
- [Definir credenciais e configurações novas](#)
- [Usar arquivos de credenciais e configurações existentes](#)

Reunir suas informações de credencial para acesso programático

Você precisará de acesso programático se quiser interagir com a AWS de fora do AWS Management Console. Para obter instruções de autenticação e credenciais, selecione uma das seguintes opções:

Qual usuário precisa de acesso programático?	Finalidade	Instruções
Identidade do quadro de funcionários (usuários do AWS IAM Identity Center)	(Recomendado) Use credenciais de curto prazo.	the section called “Autenticação do IAM Identity Center”
IAM	Use credenciais de curto prazo.	the section called “Credenciais de curto prazo”
IAM ou Identidade do quadro de funcionários (usuários do AWS IAM Identity Center)	Use metadados da instância do Amazon EC2 para credenciais.	the section called “Uso de credenciais para metadados de instância do Amazon EC2.”
IAM ou Identidade do quadro de funcionários (usuários do AWS IAM Identity Center)	Combine outro método de credencial e assuma um perfil para permissões.	the section called “Perfis do IAM”
IAM	(Não recomendado) Use credenciais de longo prazo.	the section called “IAM users”
IAM ou Identidade do quadro de funcionários (usuários do AWS IAM Identity Center)	(Não recomendado) Combine outro método de credencial, mas use valores de credencial armazenados em um local fora da AWS CLI.	the section called “Credenciais externas”

Definir credenciais e configurações novas

A AWS CLI armazena suas informações de configuração e credencial em um perfil (uma coleção de configurações) nos arquivos `credentials` e `config`.

Existem basicamente dois métodos para configurar rapidamente:

- [Configurar usando os comandos da AWS CLI](#)
- [Editar manualmente as credenciais e os arquivos de configuração](#)

Os exemplos a seguir usam valores de amostra para cada um dos métodos de autenticação. Substitua os valores de amostra por seus próprios valores.

Configurar usando os comandos da AWS CLI

Para uso geral, os comandos `aws configure` ou `aws configure sso` em seu terminal de sua preferência são a maneira mais rápida de configurar sua instalação da AWS CLI. Com base no método de credencial de sua preferência, a AWS CLI solicita a você as informações relevantes. Por padrão, as informações neste perfil são usadas quando você executa um comando da AWS CLI que não especifica explicitamente um perfil a ser usado.

Para obter mais informações sobre os arquivos `credentials` e `config`, consulte [Configurações do arquivo de configuração e credenciais](#).

IAM Identity Center (SSO)

Este exemplo é para o AWS IAM Identity Center usando o assistente `aws configure sso`. Para obter mais informações, consulte [the section called “Configurar a atualização automática de tokens”](#).

```
$ aws configure sso
SSO session name (Recommended): my-sso
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]: us-east-1

Attempting to automatically open the SSO authorization page in your default browser.

There are 2 AWS accounts available to you.
> DeveloperAccount, developer-account-admin@example.com (111122223333)
   ProductionAccount, production-account-admin@example.com (444455556666)
```



```
Using the account ID 111122223333

There are 2 roles available to you.
> ReadOnly
  FullAccess

Using the role name "ReadOnly"

CLI default client Region [None]: us-west-2
CLI default output format [None]: json
CLI profile name [123456789011_ReadOnly]: user1
```

IAM Identity Center (Legacy SSO)

Este exemplo é para o método legado do AWS IAM Identity Center usando o assistente `aws configure sso`. Para usar o SSO legado, deixe o nome da sessão em branco. Para obter mais informações, consulte [the section called “Configuração herdada não atualizável”](#).

```
$ aws configure sso
SSO session name (Recommended):
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]:us-east-1

SSO authorization page has automatically been opened in your default browser.
Follow the instructions in the browser to complete this authorization request.

There are 2 AWS accounts available to you.
> DeveloperAccount, developer-account-admin@example.com (111122223333)
  ProductionAccount, production-account-admin@example.com (444455556666)

Using the account ID 111122223333

There are 2 roles available to you.
> ReadOnly
  FullAccess

Using the role name "ReadOnly"

CLI default client Region [None]: us-west-2
CLI default output format [None]: json
CLI profile name [123456789011_ReadOnly]: user1
```

Short-term credentials

Este exemplo é para as credenciais de curto prazo do AWS Identity and Access Management. O assistente de configuração aws é usado para definir os valores iniciais e, depois, o comando `aws configure set` atribui o último valor necessário. Para obter mais informações, consulte [the section called “Credenciais de curto prazo”](#).

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJa1rXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json

$ aws configure set
aws_session_token fcZib3JpZ2LuX2IQoJb3JpZ2LuX2IQoJb3JpZ2LuX2IQoJb3JpZ2LuX2IQoJb3JpZ2LuX2IQoJb3JpZVERYLONG
```

IAM role

Este exemplo serve para assumir um perfil do IAM. Os perfis que usam perfis do IAM extraem credenciais de outro perfil e, depois, aplicam as permissões de perfil do IAM. Nos exemplos a seguir, `default` é o perfil de origem das credenciais, e `user1` empresta as mesmas credenciais e assume um novo perfil. Não há assistente para esse processo, portanto, cada valor é definido usando o comando `aws configure set`. Para obter mais informações, consulte [the section called “Perfis do IAM”](#).

```
$ aws configure set role_arn arn:aws:iam::123456789012:role/defaultrole
$ aws configure set source_profile default
$ aws configure set role_session_name session_user1
$ aws configure set region us-west-2
$ aws configure set output json
```

Amazon EC2 instance metadata credentials

Este exemplo é para as credenciais obtidas dos metadados de instância do Amazon EC2 de `host`. Não há assistente para esse processo, portanto, cada valor é definido usando o comando `aws configure set`. Para obter mais informações, consulte [the section called “Uso de credenciais para metadados de instância do Amazon EC2.”](#).

```
$ aws configure set role_arn arn:aws:iam::123456789012:role/defaultrole
$ aws configure set credential_source Ec2InstanceMetadata
$ aws configure set region us-west-2
```

```
$ aws configure set output json
```

Long-term credentials

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

Este exemplo é para as credenciais de longo prazo do AWS Identity and Access Management. Para obter mais informações, consulte [the section called “IAM users”](#).

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

Para obter mais informações mais detalhadas sobre os métodos de credenciais e autenticação, consulte [Autenticação e credenciais de acesso](#).

Editar manualmente as credenciais e os arquivos de configuração

Ao copiar e colar as informações, sugerimos editar manualmente o arquivo `config` e `credentials`. Com base no método de credencial de sua preferência, os arquivos são configurados de uma forma diferente.

Os arquivos são armazenados no diretório inicial, na pasta `.aws`. O local do diretório inicial varia de acordo com o sistema operacional, mas é acessado usando as variáveis de ambiente `%UserProfile%` no Windows e `$HOME` ou `~` (til) em sistemas baseados em Unix. Para obter mais informações sobre onde essas configurações estão armazenadas, consulte [the section called “Onde as definições de configuração ficam armazenadas?”](#).

Os exemplos a seguir mostram um perfil `default` e um perfil chamado `user1` e usam valores de amostra. Substitua os valores de amostra por seus próprios valores. Para obter mais informações sobre os arquivos `credentials` e `config`, consulte [Configurações do arquivo de configuração e credenciais](#).

IAM Identity Center (SSO)

Este exemplo é para AWS IAM Identity Center. Para obter mais informações, consulte [the section called “Configurar a atualização automática de tokens”](#).

Arquivo de credenciais

O arquivo `credentials` não é usado para esse método de autenticação.

Arquivo de configuração

```
[default]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = readOnly
region = us-west-2
output = text

[profile user1]
sso_session = my-sso
sso_account_id = 444455556666
sso_role_name = readOnly
region = us-east-1
output = json

[sso-session my-sso]
sso_region = us-east-1
sso_start_url = https://my-sso-portal.awsapps.com/start
sso_registration_scopes = sso:account:access
```

IAM Identity Center (Legacy SSO)

Este exemplo é para o método legado de AWS IAM Identity Center. Para obter mais informações, consulte [the section called “Configuração herdada não atualizável”](#).

Arquivo de credenciais

O arquivo `credentials` não é usado para esse método de autenticação.

Arquivo de configuração

```
[default]
sso_start_url = https://my-sso-portal.awsapps.com/start
```


Arquivo de configuração

```
[default]
role_arn=arn:aws:iam::123456789012:role/defaultrole
credential_source=Ec2InstanceMetadata
region=us-west-2
output=json

[profile user1]
role_arn=arn:aws:iam::777788889999:role/user1role
credential_source=Ec2InstanceMetadata
region=us-east-1
output=text
```

Long-term credentials

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

Este exemplo é para as credenciais de longo prazo do AWS Identity and Access Management. Para obter mais informações, consulte [the section called “IAM users”](#).

Arquivo de credenciais

```
[default]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

[user1]
aws_access_key_id=AKIAI44QH8DHBEXAMPLE
aws_secret_access_key=je7MtGbClwBF/2Zp9Utk/h3yCo8nvbEXAMPLEKEY
```

Arquivo de configuração

```
[default]
region=us-west-2
output=json
```

```
[profile user1]  
region=us-east-1  
output=text
```

Para obter mais informações mais detalhadas sobre os métodos de credenciais e autenticação, consulte [Autenticação e credenciais de acesso](#).

Usar arquivos de credenciais e configurações existentes

Se você tiver arquivos de configuração e credenciais existentes, eles poderão ser usados para a AWS CLI.

Para usar os arquivos `config` e `credentials`, mova-os para a pasta chamada `.aws` em seu diretório inicial. O local do diretório inicial varia de acordo com o sistema operacional, mas é acessado usando as variáveis de ambiente `%UserProfile%` no Windows e `$HOME` ou `~` (til) em sistemas baseados em Unix.

Você pode especificar um local não padrão para os arquivos `config` e `credentials` definindo as variáveis de ambiente `AWS_CONFIG_FILE` e `AWS_SHARED_CREDENTIALS_FILE` para outro caminho local. Para mais detalhes, consulte [Variáveis de ambiente para configurar a AWS CLI](#).

Para obter informações mais detalhadas sobre arquivos de configuração e credenciais, consulte [the section called “Configurações do arquivo de configuração e credenciais”](#).

Configurar o AWS CLI

Esta seção explica como definir as configurações que a AWS Command Line Interface (AWS CLI) usa para interagir com a AWS. Incluindo o seguinte:

- As credenciais identificam quem está chamando a API. As credenciais de acesso são usadas para criptografar a solicitação aos servidores da AWS para confirmar sua identidade e recuperar as políticas de permissões associadas. Essas permissões determinam as ações que você pode realizar. Para obter informações sobre como configurar suas credenciais, consulte [Autenticação e credenciais de acesso](#).
- Outros detalhes de configuração para informar à AWS CLI como processar solicitações, como o formato de saída padrão e a região da AWS padrão.

Note

A AWS exige que todas as solicitações recebidas sejam assinadas criptograficamente. O AWS CLI faz isso para você. A “assinatura” inclui uma data/time stamp. Portanto, você deve garantir que a data e a hora do seu computador estejam definidas corretamente. Se não fizer isso, e a data/hora na assinatura estiver muito longe da data/hora reconhecida pelo serviço da AWS, a AWS rejeitará a solicitação.

Precedência de credenciais e configurações

As credenciais e as configurações estão localizadas em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. Certos locais têm precedência sobre outros. As credenciais da AWS CLI e as definições de configuração têm precedência na seguinte ordem:

1. [Opções da linha de comando](#): substituem as configurações em qualquer outro local, como nos parâmetros `--region`, `--output` e `--profile`.
2. [Variáveis de ambiente](#): você pode armazenar valores nas variáveis de ambiente do sistema.
3. [Assumir perfil](#): assuma as permissões de um perfil do IAM por meio da configuração ou do comando `aws sts assume-role`.

4. [Assumir perfil com identidade da web](#): assuma as permissões de um perfil do IAM usando uma identidade da web por meio da configuração ou do comando [aws sts assume-role](#).
5. [AWS IAM Identity Center](#): as configurações do Centro de Identidade do IAM são armazenadas no arquivo config. As credenciais são autenticadas quando você executa o comando `aws configure sso`. O arquivo config está localizado em `~/.aws/config` no Linux ou MacOS ou em `C:\Users\USERNAME\.aws\config` no Windows.
6. [Arquivo de credenciais](#): os arquivos `credentials` e `config` são atualizados quando você executa o comando `aws configure`. O arquivo `credentials` está localizado em `~/.aws/credentials` no Linux ou MacOS ou em `C:\Users\USERNAME\.aws\credentials` no Windows.
7. [Processo personalizado](#): obtenha suas credenciais de uma fonte externa.
8. [Arquivo de configuração](#): os arquivos `credentials` e `config` são atualizados quando você executa o comando `aws configure`. O arquivo config está localizado em `~/.aws/config` no Linux ou MacOS ou em `C:\Users\USERNAME\.aws\config` no Windows.
9. [Credenciais de perfil de instância](#): você pode associar um perfil do IAM a cada uma das suas instâncias do Amazon Elastic Compute Cloud (Amazon EC2). As credenciais temporárias para essa função estão disponíveis para o código em execução na instância. As credenciais são fornecidas por meio do serviço de metadados do Amazon EC2. Para obter mais informações, consulte [Funções do IAM para o Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux e [Uso de perfis de instância](#) no Manual do usuário do IAM.
10. [Credenciais de container](#): você pode associar uma função do IAM a cada uma das suas definições de tarefa do Amazon Elastic Container Service (Amazon ECS). As credenciais temporárias para essa função estão disponíveis para os contêineres dessa tarefa. Para mais informações, consulte [Funções do IAM para Tarefas](#) no Guia de Desenvolvedor Amazon Elastic Container Service.

Tópicos adicionais nesta seção

- [the section called “Configurações do arquivo de configuração e credenciais”](#)
- [the section called “Variáveis de ambiente”](#)
- [the section called “Opções de linha de comando”](#)
- [the section called “Conclusão de comando”](#)
- [the section called “Repetições”](#)
- [the section called “Usar um proxy HTTP”](#)

Configurações do arquivo de configuração e credenciais

Você pode salvar as definições de configuração usadas com frequência e credenciais em arquivos que são mantidos pela AWS CLI.

Os arquivos são divididos em `profiles`. Por padrão, a AWS CLI usa as configurações encontradas no perfil chamado `default`. Para usar outras configurações, você pode criar e fazer referência a perfis adicionais.

Você pode substituir uma configuração definindo uma das variáveis de ambiente com suporte ou usando um parâmetro de linha de comando. Para obter mais informações sobre a precedência de configuração, consulte [Configurar o AWS CLI](#).

Note

Para obter informações sobre como configurar suas credenciais, consulte [Autenticação e credenciais de acesso](#).

Tópicos

- [Formato dos arquivos de credenciais e configuração](#)
- [Onde as definições de configuração ficam armazenadas?](#)
- [Usar perfis nomeados](#)
- [Definir e visualizar as configurações usando comandos](#)
- [Definir novos exemplos de comandos de configuração e credenciais](#)
- [Configurações de arquivo config compatíveis](#)

Formato dos arquivos de credenciais e configuração

Os arquivos `config` e `credentials` são organizados em seções. As seções incluem perfis, `sso-sessions`, e serviços. Uma seção é um conjunto nomeado de configurações e continua até que outra linha de definição de seção seja encontrada. Vários perfis e seções podem ser armazenados nos arquivos `config` e `credentials`.

Esses arquivos são arquivos de texto simples que usam o seguinte formato:

- Os nomes das seções estão entre colchetes [], como [default], [profile *user1*] e [sso-session].
- Todas as entradas em uma seção assumem a forma geral de setting_name=value.
- As linhas podem ser comentadas iniciando-as com um caractere de hashtag (#).

Os arquivos config e credentials contêm os seguintes tipos de seção:

- [Tipo de seção: profile](#)
- [Tipo de seção: sso-session](#)
- [Tipo de seção: services](#)

Tipo de seção: **profile**

A AWS CLI armazena

Dependendo do arquivo, os nomes de seção de perfil usam o seguinte formato:

- Arquivo de configuração: [default] [profile *user1*]
- Arquivo de credenciais: [default] [*user1*]

Não use a palavra `profile` ao criar uma entrada no arquivo `credentials`.

Cada perfil especifica credenciais diferentes e também pode especificar regiões da AWS e formatos de saída diferentes. Ao nomear o perfil em um arquivo `config`, inclua a palavra de prefixo “`profile`”, mas não a inclua no arquivo `credentials`.

Os exemplos a seguir mostram um arquivo `credentials` e `config` com dois perfis, região e saída especificados. O primeiro [padrão] é usado quando você executa um comando da AWS CLI sem perfil especificado. O segundo é usado quando você executa um comando da AWS CLI com o parâmetro `--profile user1`.

IAM Identity Center (SSO)

Este exemplo é para AWS IAM Identity Center. Para obter mais informações, consulte [the section called “Configurar a atualização automática de tokens”](#).

Arquivo de credenciais

O arquivo `credentials` não é usado para esse método de autenticação.

Arquivo de configuração

```
[default]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = readOnly
region = us-west-2
output = text

[profile user1]
sso_session = my-sso
sso_account_id = 444455556666
sso_role_name = readOnly
region = us-east-1
output = json

[sso-session my-sso]
sso_region = us-east-1
sso_start_url = https://my-sso-portal.awsapps.com/start
sso_registration_scopes = sso:account:access
```

IAM Identity Center (Legacy SSO)

Este exemplo é para o método legado de AWS IAM Identity Center. Para obter mais informações, consulte [the section called “Configuração herdada não atualizável”](#).

Arquivo de credenciais

O arquivo `credentials` não é usado para esse método de autenticação.

Arquivo de configuração

```
[default]
sso_start_url = https://my-sso-portal.awsapps.com/start
sso_region = us-east-1
sso_account_id = 111122223333
sso_role_name = readOnly
region = us-west-2
output = text

[profile user1]
```



```
credential_source=Ec2InstanceMetadata
region=us-west-2
output=json

[profile user1]
role_arn=arn:aws:iam::777788889999:role/user1role
credential_source=Ec2InstanceMetadata
region=us-east-1
output=text
```

Long-term credentials

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

Este exemplo é para as credenciais de longo prazo do AWS Identity and Access Management. Para obter mais informações, consulte [the section called “IAM users”](#).

Arquivo de credenciais

```
[default]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

[user1]
aws_access_key_id=AKIAI44QH8DHBEXAMPLE
aws_secret_access_key=je7MtGbClwBF/2Zp9Utk/h3yCo8nvbEXAMPLEKEY
```

Arquivo de configuração

```
[default]
region=us-west-2
output=json

[profile user1]
region=us-east-1
output=text
```


Para obter mais informações e métodos adicionais de autorização e credencial, consulte [the section called “IAM users”](#).

Tipo de seção: **sso-session**

A seção do `sso-session` do arquivo `config` é usada para agrupar variáveis de configuração para adquirir tokens de acesso do SSO, que podem então ser usados para adquirir credenciais da AWS. As seguintes configurações são usadas:

- (Obrigatório) [sso_start_url](#)
- (Obrigatório) [sso_region](#)
- [sso_account_id](#)
- [sso_role_name](#)
- [sso_registration_scopes](#)

Defina uma seção de `sso-session` e associe-a a um perfil. `sso_region` e `sso_start_url` devem ser definidos na seção `sso-session`. Normalmente, `sso_account_id` e `sso_role_name` devem ser definidos na seção `profile` para que o SDK possa solicitar credenciais do SSO.

O exemplo a seguir configura o SDK para solicitar credenciais do SSO e é compatível com a atualização automática de tokens:

```
[profile dev]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = SampleRole

[sso-session my-sso]
sso_region = us-east-1
sso_start_url = https://my-sso-portal.awsapps.com/start
```

Isso também permite que as configurações de `sso-session` sejam reutilizadas em vários perfis:

```
[profile dev]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = SampleRole

[profile prod]
```

```
sso_session = my-ssosso_account_id = 111122223333sso_role_name = SampleRole2[sso-session my-ssosso_region = us-east-1sso_start_url = https://my-ssosso-portal.awsapps.com/start
```

No entanto, `sso_account_id` e `sso_role_name` não são necessários para todos os cenários de configuração do token do SSO. Se a aplicação usa apenas serviços da AWS compatíveis com a autenticação do portador, as credenciais tradicionais da AWS não são necessárias. A autenticação do portador é um esquema de autenticação HTTP que usa tokens de segurança chamados tokens de portador. Nesse cenário, `sso_account_id` e `sso_role_name` não são obrigatórios. Consulte o guia individual do serviço da AWS para determinar se ele é compatível com a autorização do token do portador.

Além disso, os escopos de registro podem ser configurados como parte de uma `sso-session`. O escopo é um mecanismo no OAuth 2.0 para limitar o acesso de uma aplicação à conta de um usuário. Uma aplicação pode solicitar um ou mais escopos, e o token de acesso emitido para a aplicação será limitado aos escopos concedidos. Esses escopos definem as permissões solicitadas para serem autorizadas para o cliente OIDC registrado e os tokens de acesso recuperados pelo cliente. O exemplo a seguir define `sso_registration_scopes` para fornecer acesso a fim de listar contas/perfis:

```
[sso-session my-ssosso_region = us-east-1sso_start_url = https://my-ssosso-portal.awsapps.com/startsso_registration_scopes = sso:account:access
```

O token de autenticação é armazenado em cache no disco sob o diretório `~/.aws/sso/cache` com um nome de arquivo baseado no nome da sessão.

Para obter mais informações sobre esse tipo de configuração, consulte [the section called “Configurar a atualização automática de tokens”](#).

Tipo de seção: **services**

A seção `services` é um grupo de configurações que configura endpoints personalizados para solicitações AWS service (Serviço da AWS). Depois, um perfil é vinculado a uma seção `services`.

```
[profile dev]
services = my-services
```

A seção `services` é separada em subseções por linhas `<SERVICE> =`, onde `<SERVICE>` é a chave de identificação do AWS service (Serviço da AWS). O identificador AWS service (Serviço da AWS) é baseado no `serviceId` do modelo de API, substituindo todos os espaços por sublinhados e colocando todas as letras em minúsculas. Para obter uma lista de todas as chaves de identificação de serviço a serem usadas na seção `services`, consulte [Usar endpoints na AWS CLI](#). A chave de identificação de serviço é seguida por configurações aninhadas, cada uma em sua própria linha e recuada por dois espaços.

O exemplo a seguir configura o endpoint a ser usado para solicitações feitas ao serviço Amazon DynamoDB na seção `my-services` que é usada no perfil `dev`. Todas as linhas imediatamente seguintes que estejam recuadas são incluídas nessa subseção e se aplicam a esse serviço.

```
[profile dev]
services = my-services

[services my-services]
dynamodb =
    endpoint_url = http://localhost:8000
```

Para obter mais informações sobre endpoints específicos de serviço, consulte [Usar endpoints na AWS CLI](#).

Se o seu perfil tiver credenciais baseadas em perfis configurados por meio de um parâmetro `source_profile` para a funcionalidade “assumir função” do IAM, o SDK usará somente configurações de serviço para o perfil especificado. Ele não usa perfis com funções vinculadas a ele. Por exemplo, usando o seguinte arquivo compartilhado `config`:

```
[profile A]
credential_source = Ec2InstanceMetadata
endpoint_url = https://profile-a-endpoint.aws/

[profile B]
source_profile = A
role_arn = arn:aws:iam::123456789012:role/roleB
services = profileB

[services profileB]
```

```
ec2 =  
    endpoint_url = https://profile-b-ec2-endpoint.aws
```

Se você usar o perfil B e fizer uma chamada em seu código para o Amazon EC2, o endpoint será resolvido como `https://profile-b-ec2-endpoint.aws`. Se o seu código fizer uma solicitação para qualquer outro serviço, a resolução do endpoint não seguirá nenhuma lógica personalizada. O endpoint não é resolvido para o endpoint global definido no perfil A. Para que um endpoint global tenha efeito para o perfil B, você precisaria configurar `endpoint_url` diretamente no perfil B.

Onde as definições de configuração ficam armazenadas?

A AWS CLI armazena informações confidenciais das credenciais com `aws configure` em um arquivo local chamado `credentials`, em uma pasta chamada `.aws`, no seu diretório inicial. As opções menos confidenciais de configuração especificadas com `aws configure` são armazenadas em um arquivo local chamado `config`, também armazenado na pasta `.aws`, no seu diretório inicial.

Armazenar credenciais no arquivo de configuração

Você pode manter todas as configurações de perfil em um único arquivo, já que a AWS CLI pode ler as credenciais do arquivo `config`. Se houver credenciais nos dois arquivos para um perfil que compartilhe o mesmo nome, as chaves no arquivo de credenciais terão precedência. Sugerimos manter as credenciais nos arquivos `credentials`. Esses arquivos também são usados pelos vários kits de desenvolvimento de software de linguagem (SDKs). Se você usar um dos SDKs além da AWS CLI, confirme se as credenciais precisarão ser armazenadas no próprio arquivo.

O local do diretório inicial varia de acordo com o sistema operacional, mas é acessado usando as variáveis de ambiente `%UserProfile%` no Windows e `$HOME` ou `~` (til) em sistemas baseados em Unix. Você pode especificar um local não padrão para os arquivos configurando as variáveis de ambiente `AWS_CONFIG_FILE` e `AWS_SHARED_CREDENTIALS_FILE` para outro caminho local. Para mais detalhes, consulte [Variáveis de ambiente para configurar a AWS CLI](#).

Quando você usa um perfil compartilhado que especifica uma função do AWS Identity and Access Management (IAM), a AWS CLI chama a operação `AssumeRole` do AWS STS para recuperar as credenciais temporárias. Essas credenciais são armazenadas (no `~/.aws/cli/cache`). Os comandos subsequentes da AWS CLI usam as credenciais temporárias armazenadas em cache até que expirem e, nesse ponto, a AWS CLI atualiza automaticamente as credenciais.

Usar perfis nomeados

Se nenhum perfil for definido explicitamente, será usado o perfil `default`.

Para usar um perfil designado, adicione a opção `--profile` *profile-name* para o comando. O exemplo a seguir lista todas as instâncias do Amazon EC2 que usam as credenciais e as configurações definidas no perfil `user1`.

```
$ aws ec2 describe-instances --profile user1
```

Para usar um perfil designado para vários comandos, evite especificar o perfil ao configurar o comando em cada variável de ambiente `AWS_PROFILE` como o perfil padrão. É possível substituir essa configuração usando o parâmetro `--profile`.

Linux or macOS

```
$ export AWS_PROFILE=user1
```

Windows

```
C:\> setx AWS_PROFILE user1
```

O uso de [set](#) para definir uma variável de ambiente altera o valor usado até o final da sessão de prompt de comando atual ou até que você defina a variável como um valor diferente.

Usar [setx](#) para definir uma variável de ambiente altera o valor em todos os shells de comando criados depois de executar o comando. Isso não afeta nenhum shell de comando que já esteja em execução no momento em que você executa o comando. Feche e reinicie o shell de comando para ver os efeitos da alteração.

Configurar a variável de ambiente altera o perfil padrão até o final do seu shell sessão ou até que você defina a variável a um valor diferente. Você pode tornar as variáveis de ambiente persistentes em sessões futuras colocando-as no script de inicialização do shell. Para obter mais informações, consulte [Variáveis de ambiente para configurar a AWS CLI](#).

Definir e visualizar as configurações usando comandos

Existem várias formas de visualizar e definir as configurações usando comandos.

aws configure

Execute esse comando para definir e visualizar rapidamente as credenciais do , a região e o formato de saída. O exemplo a seguir mostra valores de amostra.

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

aws configure set

Você pode definir quaisquer credenciais ou configurações usando `aws configure set`. Especifique o perfil que você deseja visualizar ou modificar com a configuração `--profile`.

Por exemplo, o comando a seguir define a `region` no perfil chamado `integ`.

```
$ aws configure set region us-west-2 --profile integ
```

Para remover uma configuração, use uma string vazia como o valor ou exclua manualmente a configuração nos arquivos `config` e `credentials` em um editor de texto.

```
$ aws configure set cli_pager "" --profile integ
```

aws configure get

Você pode recuperar quaisquer credenciais ou configurações definidas usando `aws configure get`. Especifique o perfil que você deseja visualizar ou modificar com a configuração `--profile`.

Por exemplo, o comando a seguir recupera a configuração `region` no perfil chamado `integ`.

```
$ aws configure get region --profile integ
us-west-2
```

Se a saída estiver vazia, a configuração não estará explicitamente definida e usará o valor padrão.

aws configure import

Importe as credenciais CSV geradas do console da web do IAM. Isso não vale para credenciais geradas pelo IAM Identity Center; os clientes que usam o IAM Identity Center devem usar `aws`

configure sso. Um arquivo CSV é importado com o nome do perfil correspondente ao nome do usuário. O arquivo CSV deve conter os cabeçalhos a seguir.

- Nome de usuário
- Access key ID (ID da chave de acesso)
- Secret access key (Chave de acesso secreta)

Note

Durante a criação inicial do par de chaves, depois de fechar a caixa de diálogo Download .csv file (Baixar arquivo .csv), você não poderá acessar a chave de acesso secreta depois de fechar a caixa de diálogo. Se você precisar de um arquivo .csv, será necessário criá-lo por conta própria com os cabeçalhos necessários e as informações do par de chaves armazenado. Se você não tiver acesso às informações do par de chaves, será necessário criar outro par de chaves.

```
$ aws configure import --csv file://credentials.csv
```

aws configure list

Para listar os dados de configuração, use o comando `aws configure list`. Esse comando lista as informações de perfil, chave de acesso, chave secreta e configuração da região usadas para o perfil especificado. Para cada item de configuração, ele mostra o valor, onde o valor da configuração foi recuperado e o nome da variável de configuração.

Por exemplo, se você fornecer a Região da AWS em uma variável de ambiente, esse comando mostrará o nome da região que você configurou, que esse valor veio de uma variável de ambiente e o nome da variável de ambiente.

No caso de métodos de credenciais temporárias, como perfis e o Centro de Identidade do IAM, esse comando exibe a chave de acesso armazenada temporariamente em cache e a chave de acesso secreta é exibida.

```
$ aws configure list
```

Name	Value	Type	Location
----	-----	----	-----
profile	<not set>	None	None

```
access_key      *****ABCD  shared-credentials-file
secret_key      *****ABCD  shared-credentials-file
region          us-west-2      env    AWS_DEFAULT_REGION
```

aws configure list-profiles

Para listar todos os nomes de perfil, use o comando `aws configure list-profiles`.

```
$ aws configure list-profiles
default
test
```

aws configure sso

Execute esse comando para definir e visualizar rapidamente as credenciais do AWS IAM Identity Center, a região e o formato de saída. O exemplo a seguir mostra valores de amostra.

```
$ aws configure sso
SSO session name (Recommended): my-sso
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]: us-east-1
SSO registration scopes [None]: sso:account:access
```

aws configure sso-session

Execute esse comando para definir e visualizar rapidamente as credenciais do AWS IAM Identity Center, a região e o formato de saída na seção `sso-session` dos arquivos `credentials` e `config`. O exemplo a seguir mostra valores de amostra.

```
$ aws configure sso-session
SSO session name: my-sso
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]: us-east-1
SSO registration scopes [None]: sso:account:access
```

Definir novos exemplos de comandos de configuração e credenciais

Os exemplos a seguir mostram a configuração de um perfil padrão com credenciais, região e saída especificadas para diferentes métodos de autenticação.

IAM Identity Center (SSO)

Este exemplo é para o AWS IAM Identity Center usando o assistente `aws configure sso`. Para obter mais informações, consulte [the section called “Configurar a atualização automática de tokens”](#).

```
$ aws configure sso
SSO session name (Recommended): my-sso
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]:us-east-1

Attempting to automatically open the SSO authorization page in your default browser.

There are 2 AWS accounts available to you.
> DeveloperAccount, developer-account-admin@example.com (111122223333)
  ProductionAccount, production-account-admin@example.com (444455556666)

Using the account ID 111122223333

There are 2 roles available to you.
> ReadOnly
  FullAccess

Using the role name "ReadOnly"

CLI default client Region [None]: us-west-2
CLI default output format [None]: json
CLI profile name [123456789011_ReadOnly]: user1
```

IAM Identity Center (Legacy SSO)

Este exemplo é para o método legado do AWS IAM Identity Center usando o assistente `aws configure sso`. Para usar o SSO legado, deixe o nome da sessão em branco. Para obter mais informações, consulte [the section called “Configuração herdada não atualizável”](#).

```
$ aws configure sso
SSO session name (Recommended):
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]:us-east-1

SSO authorization page has automatically been opened in your default browser.
Follow the instructions in the browser to complete this authorization request.
```



```
$ aws configure set source_profile default
$ aws configure set role_session_name session_user1
$ aws configure set region us-west-2
$ aws configure set output json
```

Amazon EC2 instance metadata credentials

Este exemplo é para as credenciais obtidas dos metadados de instância do Amazon EC2 de host. Não há assistente para esse processo, portanto, cada valor é definido usando o comando `aws configure set`. Para obter mais informações, consulte [the section called “Uso de credenciais para metadados de instância do Amazon EC2.”](#).

```
$ aws configure set role_arn arn:aws:iam::123456789012:role/defaultrole
$ aws configure set credential_source Ec2InstanceMetadata
$ aws configure set region us-west-2
$ aws configure set output json
```

Long-term credentials

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

Este exemplo é para as credenciais de longo prazo do AWS Identity and Access Management. Para obter mais informações, consulte [the section called “IAM users”](#).

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFcIYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

Configurações de arquivo **config** compatíveis

Tópicos

- [Configurações globais](#)

- [Configurações de comando personalizadas do S3](#)

As configurações a seguir têm suporte no arquivo `config`. Os valores listados no perfil especificado (ou padrão) serão usados, a menos que eles sejam ignorados pela presença de uma variável de ambiente ou uma opção de linha de comando com o mesmo nome. Para obter informações sobre a ordem de precedência das configurações, consulte [Configurar o AWS CLI](#)

Configurações globais

aws_access_key_id

Especifica a chave de acesso da AWS usada como parte das credenciais para autenticar a solicitação do comando. Embora ela possa ser armazenada no arquivo `config`, recomendamos armazená-la no arquivo `credentials`.

Pode ser substituída pela variável de ambiente `AWS_ACCESS_KEY_ID`. Você não pode especificar o ID de chave de acesso como uma opção de linha de comando.

```
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
```

aws_secret_access_key

Especifica a chave secreta da AWS usada como parte das credenciais para autenticar a solicitação do comando. Embora ela possa ser armazenada no arquivo `config`, recomendamos armazená-la no arquivo `credentials`.

Pode ser substituída pela variável de ambiente `AWS_SECRET_ACCESS_KEY`. Você não pode especificar a chave de acesso secreta como uma opção de linha de comando.

```
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

aws_session_token

Especifica um token de sessão da AWS. Um token de sessão só será necessário se você especificar manualmente credenciais de segurança temporárias. Embora ela possa ser armazenada no arquivo `config`, recomendamos armazená-la no arquivo `credentials`.

Pode ser substituída pela variável de ambiente `AWS_SESSION_TOKEN`. Você não pode especificar o token de sessão como uma opção de linha de comando.

```
aws_session_token = AQoEXAMPLEH4aoAH0gNCAPyJxz4BlCFFxWNE1OPTgk5TthT
+FvwnKwRc0IfrrRh3c/LTo6UDdyJw00vEVPvLXCrrrUtdnniCEXAMPLE/
IvU1dYUg2RVAJBanLiHb4IgRmpRV3zrkuWJ0gQs8IZZaIv2BXIa2R40lgk
```

ca_bundle

Especifica um pacote de certificado CA (um arquivo com a extensão .pem) que é usado para verificar certificados SSL.

Pode ser substituído pela variável de ambiente [AWS_CA_BUNDLE](#) ou pela opção de linha de comando [--ca-bundle](#).

```
ca_bundle = dev/apps/ca-certs/cabundle-2019mar05.pem
```

cli_auto_prompt

Habilita o prompt automático para a AWS CLI versão 2. Há duas configurações que podem ser usadas:

- **on** usa o modo de prompt automático completo cada vez que você tenta executar um comando da aws. Isso inclui pressionar ENTER após um comando completo ou um comando incompleto.

```
cli_auto_prompt = on
```

- **on-partial** usa o modo de prompt automático parcial. Se um comando estiver incompleto ou não puder ser executado devido a erros de validação do lado do cliente, o prompt automático será usado. Esse modo é particularmente útil se você tem scripts pré-existentis, runbooks ou se deseja receber o prompt automático somente para comandos com os quais você não está familiarizado, em vez de ver o prompt para todos os comandos.

```
cli_auto_prompt = on-partial
```

Você pode sobrescrever essa configuração usando a variável de ambiente [aws_cli_auto_prompt](#) ou os parâmetros de linha de comando [--cli-auto-prompt](#) e [--no-cli-auto-prompt](#).

Para obter informações sobre o recurso de prompt automático da AWS CLI versão 2, consulte [Fazer a AWS CLI solicitar comandos](#).

cli_binary_format

Especifica como a AWS CLI versão 2 interpreta parâmetros de entrada binários. Pode ter um dos valores a seguir:

- **base64**: o valor padrão. Um parâmetro de entrada que é digitado como um objeto grande binário (BLOB) aceita uma string codificada em base64. Para passar conteúdo binário verdadeiro, coloque o conteúdo em um arquivo e forneça o caminho e o nome do arquivo com o prefixo `fileb://` como o valor do parâmetro. Para passar um texto codificado em base64 contido em um arquivo, forneça o caminho e o nome do arquivo com o prefixo `file://` como o valor do parâmetro.
- **raw-in-base64-out**: padrão para a AWS CLI versão 1. Se o valor da configuração for `raw-in-base64-out`, os arquivos referenciados usando o prefixo `file://` serão lidos como texto e, então, a AWS CLI tenta codificá-lo em binário.

Esta entrada não tem uma variável de ambiente equivalente. Você pode especificar o valor em um único comando usando o parâmetro `--cli-binary-format raw-in-base64-out`.

```
cli_binary_format = raw-in-base64-out
```

Se você fizer referência a um valor binário em um arquivo usando a notação de prefixo `fileb://`, a AWS CLI sempre esperará que o arquivo tenha um conteúdo binário bruto e não tentará converter o valor.

Se você fizer referência a um valor binário em um arquivo usando a notação de prefixo `file://`, a AWS CLI tratará o arquivo de acordo com a configuração `cli_binary_format` atual. Se o valor dessa configuração for `base64` (o padrão quando não é definido explicitamente), a AWS CLI esperará que o arquivo contenha texto codificado em base64. Se o valor dessa configuração for `raw-in-base64-out`, a AWS CLI esperará que o arquivo tenha conteúdo binário bruto.

cli_history

Desabilitado por padrão. Essa configuração ativa o histórico de comandos para a AWS CLI. Após habilitar essa configuração, a AWS CLI registra o histórico de comandos da aws.

```
cli_history = enabled
```

Você pode listar seu histórico usando o comando `aws history list` e usar os `command_ids` resultantes no comando `aws history show` para obter detalhes. Para obter mais informações, consulte [aws history](#) no guia de referência da AWS CLI.

cli_pager

Especifica o programa de paginação usado para saída. Por padrão, a AWS CLI versão 2 retorna toda a saída pelo programa de paginação padrão do sistema operacional.

Pode ser substituído pela variável de ambiente `AWS_PAGER`.

```
cli_pager=less
```

Para desativar o uso total de um programa de paginação externo, defina a variável como uma string vazia, conforme o exemplo a seguir.

```
cli_pager=
```

cli_timestamp_format

Especifica o formato dos valores de timestamp incluídos no resultado. Você pode especificar qualquer um dos seguintes valores:

- `iso8601`: o valor padrão para a AWS CLI versão 2. Se especificado, a AWS CLI reformata todos os time stamps de acordo com a [ISO 8601](#).

Os time stamps formatados como ISO 8601 são parecidos com os exemplos a seguir. O primeiro exemplo mostra a hora no [Tempo Universal Coordenado \(UTC\)](#) incluindo um Z após o horário. A data e a hora são separadas por T.

```
2019-10-31T22:21:41Z
```

Para especificar um fuso horário diferente, em vez do Z, especifique + ou - e o número de horas em que o fuso horário desejado está à frente ou atrás do UTC, como um valor de dois dígitos. O exemplo a seguir mostra o mesmo tempo que o exemplo anterior, mas ajustado para o horário padrão do Pacífico, que está oito horas atrás do UTC:

```
2019-10-31T14:21:41-08
```

- `wire`: o valor padrão para a AWS CLI versão 1. Se especificado, a AWS CLI exibe todos os valores de time stamp exatamente como recebidos na resposta de consulta HTTP.

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
cli_timestamp_format = iso8601
```

credential_process

Especifica um comando externo que a AWS CLI executa para gerar ou recuperar credenciais de autenticação a serem usadas para esse comando. O comando deve retornar as credenciais em um formato específico. Para obter mais informações sobre como usar essa configuração, consulte [Credenciais de origem com um processo externo](#).

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
credential_process = /opt/bin/awscreds-retriever --username susan
```

credential_source

Usado em instâncias ou contêineres do Amazon EC2 para especificar onde a AWS CLI pode encontrar credenciais a serem usadas para assumir a função especificada com o parâmetro `role_arn`. Não é possível especificar `source_profile` e `credential_source` no mesmo perfil.

Esse parâmetro pode ter um dos três valores:

- `Environment`: especifica que a AWS CLI deve recuperar credenciais de origem de variáveis de ambiente.
- `Ec2InstanceMetadata`: especifica que a AWS CLI deve usar a função do IAM associada ao [perfil de instância do EC2](#) para obter credenciais de origem.
- `EcsContainer`: especifica que a AWS CLI deve usar a função do IAM associada ao contêiner do ECS como credenciais de origem.

```
credential_source = Ec2InstanceMetadata
```

duration_seconds

Especifica a duração máxima da sessão da função, em segundos. Esse valor pode variar de 900 segundos (15 minutos) até o valor configurado de duração máxima da sessão para a função (que pode ser até 43200). Esse é um parâmetro opcional e, por padrão, o valor é definido como 3600 segundos.

endpoint_url

Especifica o endpoint usado para todas as solicitações de serviço. Se essa configuração for usada na seção [services](#) do arquivo config, o endpoint será usado somente para o serviço especificado.

O exemplo a seguir usa o endpoint global `http://localhost:1234` e um endpoint específico de serviço de `http://localhost:4567` para o Amazon S3.

```
[profile dev]
endpoint_url = http://localhost:1234
services = s3-specific

[services s3-specific]
s3 =
    endpoint_url = http://localhost:4567
```

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como `AWS_ENDPOINT_URL_DYNAMODB`.
4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

ignore_configure_endpoint_urls

Se habilitada, a AWS CLI ignorará todas as configurações de endpoint personalizadas especificadas no arquivo config. Os valores válidos são **true** e **false**.

```
ignore_configure_endpoint_urls = true
```

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como [AWS_ENDPOINT_URL_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado config.
6. O valor fornecido pela configuração [endpoint_url](#) em um profile do arquivo compartilhado config.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

[external_id](#)

Especifica um identificador exclusivo que é usado por terceiros para assumir uma função em suas contas de clientes. Isso é mapeado para o parâmetro `ExternalId` na operação `AssumeRole`. Esse parâmetro será necessário somente se a política de confiança para a função especificar um valor para `ExternalId`. Para obter mais informações, consulte [Como usar um ID externo ao conceder acesso aos seus recursos da AWS a terceiros](#) no Guia do usuário do IAM.

max_attempts

Especifica um valor para o máximo de novas tentativas utilizadas pelo manipulador de novas tentativas da AWS CLI, onde a chamada inicial conta para o valor de `max_attempts` fornecido por você.

Você pode substituir esse valor usando a variável de ambiente `AWS_MAX_ATTEMPTS`.

```
max_attempts = 3
```

mfa_serial

O número de identificação de um dispositivo MFA a ser usado ao assumir uma função. Isso será obrigatório apenas se a política de confiança da função que está sendo assumida incluir uma condição que exija a autenticação MFA. O valor pode ser um número de série de um dispositivo de hardware (como GAHT12345678) ou um nome de recurso da Amazon (ARN) de um dispositivo MFA virtual (como `arn:aws:iam::123456789012:mfa/user`).

output

Especifica o formato de saída padrão para comandos solicitados usando esse perfil. Você pode especificar qualquer um dos seguintes valores:

- **json**: a saída é formatada como uma string [JSON](#).
- **yaml**: a saída é formatada como uma string [YAML](#).
- **yaml-stream**: a saída é transmitida e formatada como uma string [YAML](#). A transmissão possibilita um manuseio mais rápido de tipos de dados grandes.
- **text**: a saída é formatada como várias linhas de valores de string separados por tabulação. Isso pode ser útil para passar a saída para um processador de texto, como `grep`, `sed` ou `awk`.
- **table**: a saída é formatada como uma tabela usando os caracteres `+` e `|` para formar as bordas da célula. Geralmente, a informação é apresentada em um formato "amigável", que é muito mais fácil de ler do que outros, mas não tão útil programaticamente.

Pode ser substituído pela variável de ambiente `AWS_DEFAULT_OUTPUT` ou pela opção de linha de comando `--output`.

```
output = table
```

parameter_validation

Especifica se o cliente da AWS CLI tentará validar os parâmetros antes de enviá-los para o endpoint de serviço da AWS.

- `true`: o valor padrão. Se especificado, a AWS CLI executa a validação local de parâmetros da linha de comando.
- `false`: se especificado, a AWS CLI não valida parâmetros da linha de comando antes de enviá-los para o endpoint de serviço da AWS.

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
parameter_validation = false
```

region

Especifica a Região da AWS para a qual enviar solicitações para os comandos solicitados usando esse perfil.

- É possível especificar qualquer um dos códigos de região disponíveis para o serviço escolhido, conforme listado em [Regiões e endpoints da AWS](#) na Referência geral da Amazon Web Services.
- `aws_global` permite especificar o endpoint global para serviços compatíveis com um endpoint global, além de endpoints regionais, como AWS Security Token Service (AWS STS) e o Amazon Simple Storage Service (Amazon S3).

Você pode substituir esse valor usando a variável de ambiente `AWS_REGION`, a variável de ambiente `AWS_DEFAULT_REGION` ou a opção da linha de comando `--region`.

```
region = us-west-2
```

retry_mode

Especifica qual modo de repetição a AWS CLI utiliza. Existem três modos de repetição disponíveis: herdado (o modo usado por padrão), padrão e adaptivo. Para obter mais informações sobre novas tentativas, consulte [Novas tentativas da AWS CLI](#).

Você pode substituir esse valor usando a variável de ambiente `AWS_RETRY_MODE`.

```
retry_mode = standard
```

role_arn

Especifica o nome do recurso da Amazon (ARN) de uma função do IAM que você deseja usar para executar os comandos da AWS CLI. Também é necessário especificar um dos seguintes parâmetros para identificar as credenciais que têm permissão para assumir essa função:

- `source_profile`
- `credential_source`

```
role_arn = arn:aws:iam::123456789012:role/role-name
```

A variável de ambiente [AWS_ROLE_ARN](#) sobrescreve essa configuração.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

role_session_name

Especifica o nome a ser associado à sessão da função. Esse valor é fornecido ao parâmetro `RoleSessionName` quando a AWS CLI chama a operação `AssumeRole` e se torna parte do ARN do usuário da função assumida: `arn:aws:sts::123456789012:assumed-role/role_name/role_session_name`. Esse parâmetro é opcional. Se você não fornecer esse valor, um nome de sessão será gerado automaticamente. Esse nome aparece nos logs do AWS CloudTrail para entradas associadas a essa sessão.

```
role_session_name = maria_garcia_role
```

A variável de ambiente [AWS_ROLE_SESSION_NAME](#) sobrescreve essa configuração.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

services

Especifica a configuração de serviço a ser usada em seu perfil.

```
[profile dev-s3-specific-and-global]  
endpoint_url = http://localhost:1234  
services = s3-specific  
  
[services s3-specific]
```

```
s3 =  
    endpoint_url = http://localhost:4567
```

Para obter mais informações sobre a seção `services`, consulte [the section called “services”](#).

A variável de ambiente [AWS_ROLE_SESSION_NAME](#) sobrescreve essa configuração.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

[source_profile](#)

Especifica um perfil nomeado com credenciais de longo prazo que a AWS CLI pode usar para assumir uma função que você especificou com o parâmetro `role_arn`. Não é possível especificar `source_profile` e `credential_source` no mesmo perfil.

```
source_profile = production-profile
```

[sso_account_id](#)

Especifica o ID da conta da AWS que contém o perfil do IAM com a permissão que você deseja conceder ao usuário associado do IAM Identity Center.

Essa configuração não tem uma variável de ambiente nem uma opção de linha de comando.

```
sso_account_id = 123456789012
```

[sso_region](#)

Especifica a região da AWS que contém o host do portal de acesso da AWS. Isso é separado e pode ser uma região diferente do parâmetro padrão `region` da CLI.

Essa configuração não tem uma variável de ambiente nem uma opção de linha de comando.

```
sso_region = us-west-2
```

[sso_registration_scopes](#)

Uma lista delimitada por vírgulas de escopos a serem autorizados para `sso-session`. Os escopos autorizam o acesso aos endpoints autorizados portadores do token do IAM Identity

Center. Um escopo válido é uma string, como `sso:account:access`. Essa configuração não é aplicável à configuração herdada não atualizável.

```
sso_registration_scopes = sso:account:access
```

[sso_role_name](#)

Especifica o nome amigável da função do IAM que define as permissões do usuário ao usar esse perfil.

Essa configuração não tem uma variável de ambiente nem uma opção de linha de comando.

```
sso_role_name = ReadAccess
```

[sso_start_url](#)

Especifica o URL que aponta para o portal de acesso da AWS da organização. A AWS CLI usa esse URL a fim de estabelecer uma sessão no serviço do IAM Identity Center para autenticar os respectivos usuários. Para encontrar o URL do portal de acesso da AWS, use um dos seguintes procedimentos:

- Abra seu e-mail de convite, onde o URL do portal de acesso da AWS está listado.
- Abra o console do AWS IAM Identity Center em <https://console.aws.amazon.com/singlesignon/>. O URL do portal de acesso da AWS está listado em suas configurações.

Essa configuração não tem uma variável de ambiente nem uma opção de linha de comando.

```
sso_start_url = https://my-sso-portal.awsapps.com/start
```

[use_dualstack_endpoint](#)

Permite o uso de endpoints de pilha dupla para enviar solicitações da AWS. Para saber mais sobre endpoints de pilha dupla, que suportam tráfego IPv4 e IPv6, consulte [Como usar endpoints de pilha dupla do Amazon S3](#) no Guia do usuário do Amazon Simple Storage Service. Endpoints de pilha dupla estão disponíveis para alguns serviços em algumas regiões. Se não existir um endpoint de pilha dupla para o serviço ou Região da AWS, a solicitação falhará. Ela fica desabilitada por padrão.

Isso é mutuamente exclusivo com a configuração `use_accelerate_endpoint`.

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando `--endpoint-url`.
2. Se habilitada, a variável de ambiente global `AWS_IGNORE_CONFIGURED_ENDPOINT_URLS` ou a configuração do perfil `ignore_configure_endpoint_urls` para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço `AWS_ENDPOINT_URL_<SERVICE>`, como `AWS_ENDPOINT_URL_DYNAMODB`.
4. Os valores fornecidos pelas variáveis de ambiente `AWS_USE_DUALSTACK_ENDPOINT`, `AWS_USE_FIPS_ENDPOINT` e `AWS_ENDPOINT_URL`.
5. O valor do endpoint específico de serviço fornecido pela configuração `endpoint_url` em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração `endpoint_url` em um `profile` do arquivo compartilhado `config`.
7. Configurações `use_dualstack_endpoint`, `use_fips_endpoint` e `endpoint_url`.
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

use_fips_endpoint

Alguns serviços da AWS oferecem endpoints compatíveis com o [Federal Information Processing Standard \(FIPS\) 140-2](#) em algumas Regiões da AWS. Quando o serviço da AWS é compatível com o FIPS, essa configuração especifica qual endpoint do FIPS a AWS CLI deve usar. Ao contrário dos endpoints-padrão da AWS, os endpoints do FIPS usam uma biblioteca de software TLS compatível com o FIPS 140-2. Esses endpoints podem ser necessários por empresas que interagem com o governo dos Estados Unidos.

Se essa configuração estiver ativada, mas não existir um endpoint FIPS para o serviço em sua Região da AWS, o comando AWS poderá falhar. Nesse caso, especifique manualmente o endpoint a ser usado no comando usando a opção `--endpoint-url` ou use [endpoints específicos do serviço](#).

Para saber mais sobre como especificar endpoints FIPS por Região da AWS, consulte [Endpoints FIPS por serviço](#).

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como [AWS_ENDPOINT_URL_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

[web_identity_token_file](#)

Especifica o caminho para um arquivo que contém um token de acesso OAuth 2.0 ou token de ID OpenID Connect fornecido por um provedor de identidade. A AWS CLI carrega o conteúdo desse arquivo e o transmite como o argumento `WebIdentityToken` para a operação `AssumeRoleWithWebIdentity`.

A variável de ambiente [AWS_WEB_IDENTITY_TOKEN_FILE](#) sobreescreve essa configuração.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

tcp_keepalive

Especifica se o cliente de AWS CLI usará pacotes keep-alive de TCP.

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
tcp_keepalive = false
```

Configurações de comando personalizadas do S3

O Amazon S3 oferece suporte a várias configurações que definem como a AWS CLI executa operações do Amazon S3. Algumas se aplicam a todos os comandos do S3 nos namespaces `s3api` e `s3`. Outras são especificamente para os comandos “personalizados” do S3 que abstraem operações comuns e fazem mais do que um mapeamento individual para uma operação de API. Os comandos de transferência `aws s3 cp`, `sync`, `mv` e `rm` têm configurações adicionais que você pode usar para controlar as transferências do S3.

Todas essas opções podem ser configuradas especificando a configuração `s3` aninhada em seu arquivo `config`. Depois, cada configuração é recuada em sua própria linha.

Note

Essas configurações são totalmente opcionais. Você deve ser capaz de usar com êxito os comandos de transferência `aws s3` sem definir nenhuma dessas configurações. Essas configurações são fornecidas para permitir que você ajuste a performance ou considere o ambiente específico no qual está executando esses comandos `aws s3`.

Essas configurações são todas definidas em uma chave `s3` de nível superior no arquivo `config`, conforme mostrado no exemplo a seguir para o perfil `development`.

```
[profile development]
s3 =
    max_concurrent_requests = 20
    max_queue_size = 10000
    multipart_threshold = 64MB
    multipart_chunksize = 16MB
    max_bandwidth = 50MB/s
    use_accelerate_endpoint = true
    addressing_style = path
```

As configurações a seguir se aplicam a qualquer comando do S3 nos namespaces `s3` ou `s3api`.

addressing_style

Especifica qual estilo de endereçamento usar. Isso controla se o nome do bucket está no nome do host ou em parte do URL. Os valores válidos são: `path`, `virtual` e `auto`. O valor padrão é `auto`.

Há dois estilos de criação de um endpoint do Amazon S3. O primeiro é chamado `virtual` e inclui o nome do bucket como parte do nome do host. Por exemplo: `https://bucketname.s3.amazonaws.com`. Como alternativa, com o estilo `path`, você trata o nome do bucket como se fosse um caminho no URI. Por exemplo, `https://s3.amazonaws.com/bucketname`. O valor padrão na CLI é usar `auto`, que tentará empregar o estilo `virtual` onde puder, mas voltará para o estilo `path` quando necessário. Por exemplo, se o nome do bucket não for compatível com DNS, ele não poderá fazer parte do nome do host e deverá estar no caminho. Com `auto`, a CLI detectará essa condição e alternará automaticamente para o estilo `path`. Se você definir o estilo de endereçamento como `path`, garanta que a região da AWS que você configurou na AWS CLI corresponda à região do bucket.

payload_signing_enabled

Especifica se o SHA256 deverá assinar cargas sigv4. Por padrão, ele fica desativado para uploads de streaming (`UploadPart` e `PutObject`) ao usar HTTPS. Por padrão, esse valor é definido como `false` para uploads de streaming (`UploadPart` e `PutObject`), mas somente se um `ContentMD5` estiver presente (é gerado por padrão) e o endpoint usar HTTPS.

Se definido como `true`, as solicitações do S3 receberão validação de conteúdo adicional na forma de uma soma de verificação SHA256 que é calculada para você e será incluída na assinatura da solicitação. Se for definido como `false`, a soma de verificação não será calculada. Desativar essa opção poderá ser útil para reduzir a sobrecarga de performance criada pela soma da soma de verificação.

use_accelerate_endpoint

Use o endpoint Amazon S3 Accelerate para todos os comandos `s3` e `s3api`. O valor padrão é falso. Isso é mutuamente exclusivo com a configuração `use_dualstack_endpoint`.

Se definido como `true`, a AWS CLI direcionará todas as solicitações do Amazon S3 para o endpoint `s3-accelerate.amazonaws.com` em S3 Accelerate. Para usar esse endpoint, é necessário ativar o bucket para usar o S3 Accelerate. Todas as solicitações são enviadas usando o estilo virtual de endereçamento de bucket: `my-bucket.s3-accelerate.amazonaws.com`. As solicitações `ListBuckets`, `CreateBucket`

e `DeleteBucket` não são enviadas ao endpoint do S3 Accelerate porque esse endpoint não oferece suporte a essas operações. Esse comportamento também poderá ser definido se o parâmetro `--endpoint-url` estiver definido como `https://s3-accelerate.amazonaws.com` ou `http://s3-accelerate.amazonaws.com` para qualquer comando `s3` ou `s3api`.

As seguintes configurações se aplicam somente aos comandos do conjunto de comandos do namespace `s3`.

max_bandwidth

Especifica a largura de banda máxima que pode ser consumida para carregar e baixar dados de e para o Amazon S3. O padrão é sem limite.

Isso limita a largura de banda máxima que os comandos do S3 podem usar para transferir dados de e para o Amazon S3. Esse valor se aplica apenas a uploads e downloads; ele não se aplica a cópias nem as exclui. O valor é expresso em bytes por segundo. O valor pode ser especificado como:

- Um valor inteiro. Por exemplo, `1048576` define o uso máximo da largura de banda como 1 megabyte por segundo.
- Um valor inteiro seguido por um sufixo de taxa. Você pode especificar sufixos de taxa usando: `KB/s`, `MB/s` ou `GB/s`. Por exemplo, `300KB/s`, `10MB/s`.

Em geral, recomendamos que você primeiro tente reduzir o consumo de largura de banda diminuindo `max_concurrent_requests`. Se isso não adequar o consumo de largura de banda limite para a taxa desejada, você poderá usar a configuração `max_bandwidth` para limitar ainda mais o consumo de largura de banda. Isso ocorre porque `max_concurrent_requests` controla o número de threads em execução no momento. Se, em vez disso, você baixar primeiro, `max_bandwidth` mas deixar uma configuração `max_concurrent_requests` alta, isso pode fazer com que threads tenham que esperar desnecessariamente. Isso pode levar ao excesso de consumo de recursos e tempos limite de conexão.

max_concurrent_requests

Especifica o número máximo de solicitações simultâneas. O valor padrão é 10.

Os comandos de transferência `aws s3` são multithread. Em um determinado momento, várias solicitações do Amazon S3 podem estar em execução. Por exemplo, quando

you use the `aws s3 cp localdir s3://bucket/ --recursive` command to upload files to a bucket in S3, the AWS CLI can upload the files `localdir/file1`, `localdir/file2`, and `localdir/file3` in parallel. The `max_concurrent_requests` configuration specifies the maximum number of transfer operations that can be executed at the same time.

Maybe you need to change this value for some reasons:

- **Diminuir esse valor:** in some environments, the default of 10 simultaneous requests can overload a system. This can cause connection timeout errors or decrease the system's response capacity. Reducing this value means that the S3 transfer commands use fewer resources. The disadvantage is that S3 transmissions can take more time to complete. Reducing this value may be necessary if you use a tool to limit bandwidth.
- **Aumentar esse valor:** in some scenarios, maybe you want the Amazon S3 transfers to be as fast as possible, using as much bandwidth as necessary. In this scenario, the default number of simultaneous requests may not be sufficient to use all the available bandwidth. Increasing this value can improve the time needed to complete an Amazon S3 transfer.

max_queue_size

Specifies the maximum number of tasks in the task queue. The default value is 1000.

The AWS CLI uses internally a model that places tasks in the queue. The tasks, in turn, are executed by consumers whose numbers are limited by `max_concurrent_requests`. In general, a task is mapped to a single operation in Amazon S3. For example, a task can be `PutObjectTask`, `GetObjectTask`, or `UploadPartTask`. The rate at which tasks are added to the queue can be much faster than the rate at which consumers complete tasks. To avoid unbounded growth, the task queue size is limited to a specific value. This configuration changes the value of this maximum.

In general, you don't need to change this configuration. This configuration also corresponds to the number of tasks that the AWS CLI is aware of that need to be executed. This means that, by default, the AWS CLI can only see 1000 tasks at a time. Increasing this value means that the AWS CLI can know the total number of necessary tasks faster, assuming that the enqueueing rate is faster than the completion rate of tasks. The disadvantage is that a larger `max_queue_size` requires more memory.

multipart_chunksize

Especifica o tamanho de bloco que a AWS CLI usa para transferências multipart de arquivos individuais. O valor padrão é de 8 MB, com um mínimo de 5 MB.

Quando uma transferência de arquivos excede o `multipart_threshold`, a AWS CLI divide o arquivo em blocos desse tamanho. Esse valor pode ser especificado usando a mesma sintaxe que `multipart_threshold`, como o número de bytes como um número inteiro ou usando um tamanho e um sufixo.

multipart_threshold

Especifica o limite de tamanho que a AWS CLI usa para transferências multipart de arquivos individuais. O valor padrão é de 8 MB.

Ao carregar, baixar ou copiar um arquivo, os comandos do Amazon S3 alternarão para operações multipart se o arquivo exceder esse tamanho. É possível especificar esse valor de duas formas:

- O tamanho do arquivo em bytes. Por exemplo, 1048576.
- O tamanho do arquivo com um sufixo de tamanho. Você pode usar KB, MB, GB ou TB. Por exemplo: 10MB, 1GB.

Note

O S3 pode impor restrições quanto a valores válidos que podem ser usados para operações multipart. Para obter mais informações, consulte a [documentação do Carregamento fracionado do S3](#) no Manual do usuário do Amazon Simple Storage Service.

Variáveis de ambiente para configurar a AWS CLI

Variáveis de ambiente fornecem outra maneira de especificar opções de configuração e credenciais e podem ser úteis para criação de scripts ou configuração temporária de um perfil nomeado como o padrão.

Precedência de opções

- Se você especificar uma opção usando uma das variáveis de ambiente descritas nesse tópico, ela substituirá qualquer valor carregado de um perfil no arquivo de configuração.

- Se você especificar uma opção usando um parâmetro na linha de comando da AWS CLI, ela sobrescreverá qualquer valor da variável de ambiente correspondente ou de um perfil no arquivo de configuração.

Para obter mais informações sobre precedência e como a AWS CLI determina quais credenciais usar, consulte [Configurar o AWS CLI](#).

Tópicos

- [Como definir variáveis de ambiente](#)
- [Variáveis de ambiente compatíveis da AWS CLI](#)

Como definir variáveis de ambiente

Os exemplos a seguir mostram como configurar variáveis de ambiente para o usuário padrão.

Linux or macOS

```
$ export AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE  
$ export AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY  
$ export AWS_DEFAULT_REGION=us-west-2
```

Configurar a variável de ambiente altera o valor usado até o final da sua sessão de shell ou até que você defina a variável como um valor diferente. Você pode tornar as variáveis persistentes em sessões futuras definindo-as no script de inicialização do shell.

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_ACCESS_KEY_ID AKIAIOSFODNN7EXAMPLE  
C:\> setx AWS_SECRET_ACCESS_KEY wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY  
C:\> setx AWS_DEFAULT_REGION us-west-2
```

O uso de [setx](#) para definir uma variável de ambiente altera o valor usado na sessão de prompt de comando atual e todas as sessões de prompt de comando que você criar após a execução do comando. Não afeta outros shells de comando que já estejam em execução no momento em que você executar o comando. Talvez seja necessário reiniciar o terminal para que as configurações sejam carregadas.

Como definir somente para a sessão atual

O uso de [set](#) para definir uma variável de ambiente altera o valor usado até o final da sessão de prompt de comando atual ou até que você defina a variável como um valor diferente.

```
C:\> set AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
C:\> set AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
C:\> set AWS_DEFAULT_REGION=us-west-2
```

PowerShell

```
PS C:\> $Env:AWS_ACCESS_KEY_ID="AKIAIOSFODNN7EXAMPLE"
PS C:\> $Env:AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
PS C:\> $Env:AWS_DEFAULT_REGION="us-west-2"
```

Se você definir uma variável de ambiente no prompt do PowerShell conforme mostrado nos exemplos anteriores, ela salvará o valor somente pela duração da sessão atual. Para fazer com que a configuração da variável de ambiente seja persistente em todas as sessões do prompt de comando e do PowerShell, armazene-a usando o aplicativo System (Sistema) no Control Panel (Painel de controle). Como alternativa, você pode definir a variável para todas as futuras sessões do PowerShell adicionando-a ao seu perfil do PowerShell. Consulte a documentação do [PowerShell](#) para obter mais informações sobre como armazenar variáveis de ambiente ou como persisti-las nas sessões.

Variáveis de ambiente compatíveis da AWS CLI

A AWS CLI é compatível com as seguintes variáveis de ambiente.

AWS_ACCESS_KEY_ID

Especifica uma chave de acesso da AWS associada a uma conta do IAM.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `aws_access_key_id`. Você não pode especificar o ID de chave de acesso com uma opção de linha de comando.

AWS_CA_BUNDLE

Especifica o caminho para um pacote de certificado a ser usado para a validação de certificado HTTPS.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil [ca_bundle](#). Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando [--ca-bundle](#).

AWS_CLI_AUTO_PROMPT

Habilita o prompt automático para a AWS CLI versão 2. Há duas configurações que podem ser usadas:

- **on** usa o modo de prompt automático completo cada vez que você tenta executar um comando da aws. Isso inclui pressionar ENTER após um comando completo ou um comando incompleto.
- **on-partial** usa o modo de prompt automático parcial. Se um comando estiver incompleto ou não puder ser executado devido a erros de validação do lado do cliente, o prompt automático será usado. Esse modo é particularmente útil se você tem scripts pré-existentes, runbooks ou se deseja receber o prompt automático somente para comandos com os quais você não está familiarizado, em vez de ver o prompt para todos os comandos.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil [cli_auto_prompt](#). Você pode sobrescrever essa variável de ambiente usando os parâmetros de linha de comando [--cli-auto-prompt](#) e [--no-cli-auto-prompt](#).

Para obter informações sobre o recurso de prompt automático da AWS CLI versão 2, consulte [Fazer a AWS CLI solicitar comandos](#).

AWS_CLI_FILE_ENCODING

Especifica a codificação usada para arquivos de texto. Por padrão, a codificação corresponde à sua localidade. Para definir uma codificação diferente da localidade, use a variável de ambiente `aws_cli_file_encoding`. Por exemplo, se você usar o Windows com a codificação padrão CP1252, a configuração de `aws_cli_file_encoding=UTF-8` definirá a CLI para abrir arquivos de texto usando UTF-8.

AWS_CONFIG_FILE

Especifica o local do arquivo que a AWS CLI usa para armazenar perfis de configuração. O caminho padrão é `~/.aws/config`.

Você não pode especificar esse valor em uma configuração de perfil nomeado nem usando um parâmetro de linha de comando.

AWS_DATA_PATH

Uma lista de diretórios adicionais a serem verificados fora do caminho de pesquisa interno do `~/ .aws/models` ao carregar dados da AWS CLI. A definição dessa variável de ambiente indica os diretórios adicionais a serem verificados primeiro antes de voltar para os caminhos de pesquisa internos. Várias entradas devem ser separadas com o caractere `os.pathsep`, que é : no Linux ou no macOS e ; no Windows.

AWS_DEFAULT_OUTPUT

Especifica o [formato de saída](#) a ser usado.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `output`. Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando `--output`.

AWS_DEFAULT_REGION

O `Default region name` identifica a região da AWS dos servidores para os quais você deseja enviar suas solicitações por padrão. Normalmente, é a região mais próxima de você, mas pode ser qualquer região. Por exemplo, você pode digitar `us-west-2` para usar a região Oeste dos EUA (Oregon). Essa é a região para a qual todas as solicitações posteriores são enviadas, a menos que você especifique o contrário em um comando individual.

Note

Você deve especificar uma região da AWS ao usar a AWS CLI, explicitamente ou definindo uma região padrão. Para obter uma lista das regiões disponíveis, consulte [Regiões e endpoints](#). Os designadores de região usados pela AWS CLI têm os mesmos nomes que você vê nos URLs nos endpoints de serviço do AWS Management Console.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `region`. Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando `--region` e a variável de ambiente compatível com o AWS SDK `AWS_REGION`.

AWS_EC2_METADATA_DISABLED

Desabilita o uso do serviço de metadados da instância do Amazon EC2 (IMDS).

Se definido como `true`, as credenciais do usuário ou a configuração (como a região) não são solicitadas do IMDS.

AWS_ENDPOINT_URL

Especifica o endpoint usado para todas as solicitações de serviço.

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como [AWS_ENDPOINT_URL_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

AWS_ENDPOINT_URL_<SERVICE>

Especifica um endpoint personalizado usado para um serviço específico, onde <SERVICE> é substituído pelo identificador AWS service (Serviço da AWS). Por exemplo, Amazon DynamoDB tem um `serviceId` do [DynamoDB](#). Para esse serviço, a variável de ambiente do URL do endpoint é [AWS_ENDPOINT_URL_DYNAMODB](#).

Para obter uma lista de todas as variáveis de ambiente específicas do serviço, consulte [Lista de identificadores específicos de serviço](#).

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de

comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como [AWS_ENDPOINT_URL_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

AWS_IGNORE_CONFIGURED_ENDPOINT_URLS

Se habilitada, a AWS CLI ignora todas as configurações personalizadas de endpoint. Os valores válidos são **true** e **false**.

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como [AWS_ENDPOINT_URL_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).

5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

AWS_MAX_ATTEMPTS

Especifica um valor para o máximo de novas tentativas utilizadas pelo manipulador de novas tentativas da AWS CLI, onde a chamada inicial conta para o valor fornecido por você. Para obter mais informações sobre novas tentativas, consulte [Novas tentativas da AWS CLI](#).

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `max_attempts`.

AWS_METADATA_SERVICE_NUM_ATTEMPTS

Ao tentar recuperar credenciais em uma instância do Amazon EC2 que foi configurada com uma função do IAM, a AWS CLI fará apenas uma tentativa para recuperar as credenciais do serviço de metadados da instância antes de interromper. Se souber que seus comandos serão executados em uma instância do Amazon EC2, você poderá aumentar esse valor para possibilitar que a AWS CLI faça várias tentativas antes de desistir.

AWS_METADATA_SERVICE_TIMEOUT

O número de segundos antes de uma conexão ao serviço de metadados da instância atingir o tempo limite. Ao tentar recuperar credenciais em uma instância do Amazon EC2 que foi configurada com uma função do IAM, por padrão uma conexão ao serviço de metadados da instância atingirá o tempo limite depois de um segundo. Se souber que está executando em uma instância do Amazon EC2 com uma função do IAM configurada, você poderá aumentar esse valor, se necessário.

AWS_PAGER

Especifica o programa de paginação usado para saída. Por padrão, a AWS CLI versão 2 retorna toda a saída pelo programa de paginação padrão do sistema operacional.

Para desabilitar todos os usos de um programa de paginação externo, defina a variável como uma string vazia.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `cli_pager`.

AWS_PROFILE

Especifica o nome do perfil da AWS CLI com as credenciais e as opções a serem usadas. Esse pode ser o nome de um perfil armazenado em um arquivo `credentials` ou `config` ou o valor `default` para usar o perfil padrão.

Se você especificar essa variável de ambiente, ela substituirá o comportamento de usar o perfil nomeado `[default]` no arquivo de configuração. Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando `--profile`.

AWS_REGION

A variável de ambiente compatível com o AWS SDK que especifica a região da AWS para a qual enviar a solicitação.

Se definida, essa variável de ambiente substituirá os valores da variável de ambiente `AWS_DEFAULT_REGION` e a configuração do perfil `region`. Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando `--region`.

AWS_RETRY_MODE

Especifica qual modo de repetição a AWS CLI utiliza. Existem três modos de repetição disponíveis: herdado (o modo usado por padrão), padrão e `adaptive`. Para obter mais informações sobre novas tentativas, consulte [Novas tentativas da AWS CLI](#).

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `retry_mode`.

AWS_ROLE_ARN

Especifica o nome do recurso da Amazon (ARN) de uma função do IAM com um provedor de identidade da Web que você deseja usar para executar os comandos da AWS CLI.

Usado com as variáveis de ambiente `AWS_WEB_IDENTITY_TOKEN_FILE` e `AWS_ROLE_SESSION_NAME`.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil [`role\_arn`](#). Você não pode especificar um nome de sessão de função como um parâmetro de linha de comando.

 Note

Essa variável de ambiente se aplica somente a uma função assumida com o provedor de identidade da Web, mas não se aplica à configuração do provedor de função assumida geral.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

AWS_ROLE_SESSION_NAME

Especifica o nome a ser associado à sessão da função. Esse valor é fornecido ao parâmetro RoleSessionName quando a AWS CLI chama a operação AssumeRole e se torna parte do ARN do usuário da função assumida: `arn:aws:sts::123456789012:assumed-role/role_name/role_session_name`. Esse parâmetro é opcional. Se você não fornecer esse valor, um nome de sessão será gerado automaticamente. Esse nome aparece nos logs do AWS CloudTrail para entradas associadas a essa sessão.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil [role_session_name](#).

Usado com as variáveis de ambiente AWS_ROLE_ARN e AWS_WEB_IDENTITY_TOKEN_FILE.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

 Note

Essa variável de ambiente se aplica somente a uma função assumida com o provedor de identidade da Web, mas não se aplica à configuração do provedor de função assumida geral.

AWS_SECRET_ACCESS_KEY

Especifica a chave secreta associada à chave de acesso. Essencialmente, essa é a “senha” para a chave de acesso.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `aws_secret_access_key`. Você não pode especificar o ID chave de acesso secreta como uma opção de linha de comando.

AWS_SESSION_TOKEN

Especifica o valor de token de sessão que é necessário se você estiver usando credenciais de segurança temporárias recuperadas diretamente das operações do AWS STS. Para obter mais informações, consulte a [Seção de saída do comando `assume-role`](#) na Referência de comandos da AWS CLI.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `aws_session_token`.

AWS_SHARED_CREDENTIALS_FILE

Especifica o local do arquivo que a AWS CLI usa para armazenar chaves de acesso. O caminho padrão é `~/.aws/credentials`.

Você não pode especificar esse valor em uma configuração de perfil nomeado nem usando um parâmetro de linha de comando.

AWS_USE_DUALSTACK_ENDPOINT

Permite o uso de endpoints de pilha dupla para enviar solicitações da AWS. Para saber mais sobre endpoints de pilha dupla, que suportam tráfego IPv4 e IPv6, consulte [Como usar endpoints de pilha dupla do Amazon S3](#) no Guia do usuário do Amazon Simple Storage Service. Endpoints de pilha dupla estão disponíveis para alguns serviços em algumas regiões. Se não existir um endpoint de pilha dupla para o serviço ou Região da AWS, a solicitação falhará. Ela fica desabilitada por padrão.

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando `--endpoint-url`.
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como `AWS_ENDPOINT_URL_DYNAMODB`.

4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

AWS_USE_FIPS_ENDPOINT

Alguns serviços da AWS oferecem endpoints compatíveis com o [Federal Information Processing Standard \(FIPS\) 140-2](#) em algumas Regiões da AWS. Quando o serviço da AWS é compatível com o FIPS, essa configuração especifica qual endpoint do FIPS a AWS CLI deve usar. Ao contrário dos endpoints-padrão da AWS, os endpoints do FIPS usam uma biblioteca de software TLS compatível com o FIPS 140-2. Esses endpoints podem ser necessários por empresas que interagem com o governo dos Estados Unidos.

Se essa configuração estiver ativada, mas não existir um endpoint FIPS para o serviço em sua Região da AWS, o comando AWS poderá falhar. Nesse caso, especifique manualmente o endpoint a ser usado no comando usando a opção [--endpoint-url](#) ou use [endpoints específicos do serviço](#).

Para saber mais sobre como especificar endpoints FIPS por Região da AWS, consulte [Endpoints FIPS por serviço](#).

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como [AWS_ENDPOINT_URL_DYNAMODB](#).

4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

[AWS_WEB_IDENTITY_TOKEN_FILE](#)

Especifica o caminho para um arquivo que contém um token de acesso OAuth 2.0 ou token de ID OpenID Connect fornecido por um provedor de identidade. A AWS CLI carrega o conteúdo desse arquivo e o transmite como o argumento `WebIdentityToken` para a operação `AssumeRoleWithWebIdentity`.

Usado com as variáveis de ambiente `AWS_ROLE_ARN` e `AWS_ROLE_SESSION_NAME`.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `web_identity_token_file`.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

Note

Essa variável de ambiente se aplica somente a uma função assumida com o provedor de identidade da Web, mas não se aplica à configuração do provedor de função assumida geral.

Opções de linha de comando

Na AWS CLI, as opções de linha de comando são parâmetros globais que você pode usar para substituir as configurações padrão, qualquer configuração de perfil correspondente ou a configuração de variável de ambiente para esse único comando. Você não pode usar opções de linha de comando para especificar diretamente credenciais, embora seja possível especificar qual perfil usar.

Tópicos

- [Como usar as opções de linha de comando](#)
- [A AWS CLI comporta opções de linha de comando globais](#)
- [Usos comuns das opções de linha de comando](#)

Como usar as opções de linha de comando

As opções de linha de comando são, na maioria, strings simples, como o nome do perfil `profile1` no exemplo a seguir:

```
$ aws s3 ls --profile profile1
example-bucket-1
example-bucket-2
...
```

Cada opção que obtém um argumento requer com um espaço ou sinal de igual (=) separando o argumento do nome da opção. Se o valor do argumento for uma string que contenha um espaço, coloque o argumento entre aspas. Para obter detalhes sobre tipos de argumento e formatação de parâmetros, consulte [Especificar valores de parâmetro para a AWS CLI](#).

A AWS CLI comporta opções de linha de comando globais

Na AWS CLI, você pode usar as opções de linha de comando a seguir para substituir as configurações padrão, qualquer configuração de perfil correspondente ou a configuração de variável de ambiente para esse único comando.

`--ca-bundle` **<string>**

Especifica o pacote de certificados da autoridade de certificação (CA) a ser usado ao verificar certificados SSL.

Se definida, essa opção substituirá o valor da configuração de perfil [ca_bundle](#) e [AWS_CA_BUNDLE](#) da variável de ambiente.

`--cli-auto-prompt`

Habilita o modo de prompt automático para um único comando. Conforme mostrado nos exemplos a seguir, você pode especificá-lo a qualquer momento.

```
$ aws --cli-auto-prompt
$ aws dynamodb --cli-auto-prompt
$ aws dynamodb describe-table --cli-auto-prompt
```

Essa opção substituirá o valor da variável de ambiente [aws_cli_auto_prompt](#) e da configuração de perfil [cli_auto_prompt](#).

Para obter informações sobre o recurso de prompt automático da AWS CLI versão 2, consulte [Fazer a AWS CLI solicitar comandos](#).

--cli-binary-format

Especifica como a AWS CLI versão 2 interpreta parâmetros de entrada binários. Pode ter um dos valores a seguir:

- **base64**: o valor padrão. Um parâmetro de entrada que é digitado como um objeto grande binário (BLOB) aceita uma string codificada em base64. Para passar conteúdo binário verdadeiro, coloque o conteúdo em um arquivo e forneça o caminho e o nome do arquivo com o prefixo `fileb://` como o valor do parâmetro. Para passar um texto codificado em base64 contido em um arquivo, forneça o caminho e o nome do arquivo com o prefixo `file://` como o valor do parâmetro.
- **raw-in-base64-out**: padrão para a AWS CLI versão 1. Se o valor da configuração for `raw-in-base64-out`, os arquivos referenciados usando o prefixo `file://` serão lidos como texto e, então, a AWS CLI tenta codificá-lo em binário.

Isso substitui o arquivo de configuração [cli_binary_format](#).

```
$ aws lambda invoke \
  --cli-binary-format raw-in-base64-out \
  --function-name my-function \
  --invocation-type Event \
  --payload '{ "name": "Bob" }' \
  response.json
```

Se você fizer referência a um valor binário em um arquivo usando a notação de prefixo `fileb://`, a AWS CLI sempre esperará que o arquivo tenha um conteúdo binário bruto e não tentará converter o valor.

Se você fizer referência a um valor binário em um arquivo usando a notação de prefixo `file://`, a AWS CLI tratará o arquivo de acordo com a configuração `cli_binary_format` atual. Se o

valor dessa configuração for base64 (o padrão quando não é definido explicitamente), a AWS CLI esperará que o arquivo contenha texto codificado em base64. Se o valor dessa configuração for raw-in-base64-out, a AWS CLI esperará que o arquivo tenha conteúdo binário bruto.

`--cli-connect-timeout` **<integer>**

Especifica o tempo de conexão de soquete máximo em segundos. Se o valor for definido como zero (0), a conexão de soquete aguardará indefinidamente (será bloqueada) e não atingirá o tempo limite.

`--cli-read-timeout` **<integer>**

Especifica o tempo de leitura de soquete máximo em segundos. Se o valor for definido como zero (0), a leitura do soquete aguardará indefinidamente (será bloqueada) e não atingirá o tempo limite.

`--color` **<string>**

Especifica o suporte para saída de cores. Os valores válidos são on, off e auto. O valor padrão é auto.

`--debug`

Uma operação booliana que ativa o registro em log de depuração. Por padrão, a AWS CLI fornece informações limpas sobre quaisquer sucessos ou falhas em relação aos resultados do comando na saída do comando. A opção `--debug` fornece os logs completos do Python. Isso inclui informações adicionais de diagnóstico de `stderr` sobre a operação do comando que podem ser úteis para a solução de problemas de um comando que gera resultados inesperados. Para visualizar facilmente os logs de depuração, sugerimos enviar os logs para um arquivo para que seja possível pesquisar as informações de forma mais fácil. Isso pode ser feito de uma das formas a seguir.

Para enviar somente as informações de diagnóstico de `stderr`, anexe `2> debug.txt`, onde `debug.txt` é o nome que você deseja usar para o seu arquivo de depuração:

```
$ aws servicename commandname options --debug 2> debug.txt
```

Para enviar ambas a saída e as informações de diagnóstico de `stderr`, anexe `&> debug.txt`, onde `debug.txt` é o nome que você deseja usar para o seu arquivo de depuração:

```
$ aws servicename commandname options --debug &> debug.txt
```

--endpoint-url <string>

Especifica o URL para o qual enviar a solicitação. Para a maioria dos comandos, a AWS CLI determina automaticamente o URL com base no serviço selecionado e na região da AWS especificada. No entanto, alguns comandos exigem que você especifique um URL específico para a conta. Você também pode configurar alguns serviços da AWS para [hospedar um endpoint diretamente dentro de sua VPC privada](#), que talvez precise ser especificada.

O exemplo de comando a seguir usa um URL personalizado do endpoint do Amazon S3.

```
$ aws s3 ls --endpoint-url http://localhost:4567
```

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como [AWS_ENDPOINT_URL_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

--no-cli-auto-prompt

Desabilita o modo de prompt automático para um único comando.

```
$ aws dynamodb describe-table --table-name Table1 --no-cli-auto-prompt
```

Essa opção substituirá o valor da variável de ambiente [aws_cli_auto_prompt](#) e da configuração de perfil [cli_auto_prompt](#).

Para obter informações sobre o recurso de prompt automático da AWS CLI versão 2, consulte [Fazer a AWS CLI solicitar comandos](#).

--no-cli-pager

Uma opção booleana que desabilita o uso de uma paginação para a saída do comando.

--no-paginate

Uma opção booleana que desabilita as chamadas múltiplas que a AWS CLI faz automaticamente para receber todos os resultados de comandos que criam a paginação da saída. Isso significa que apenas a primeira página da sua saída é exibida.

--no-sign-request

Uma operação booleana que desativa a assinatura de solicitações HTTP para o endpoint de serviço da AWS. Isso evita que as credenciais sejam carregadas.

--no-verify-ssl

Por padrão, a AWS CLI usa SSL ao se comunicar com serviços da AWS. Para cada chamada e conexão SSL, a AWS CLI verifica os certificados SSL. O uso dessa opção substitui o comportamento padrão de verificação de certificados SSL.

 Warning

Essa opção não é uma prática recomendada. Se você usa `--no-verify-ssl`, o tráfego entre seu cliente e os serviços da AWS não está mais protegido. Isso significa que o tráfego é um risco de segurança e está vulnerável a explorações man-in-the-middle. Se estiver tendo problemas com certificados, é melhor resolvê-los. Para ver as etapas de resolução de problemas de certificados, consulte [the section called “Erros de certificado SSL”](#).

--output *<string>*

Especifica o formato de saída a ser usado para este comando. Você pode especificar qualquer um dos seguintes valores:

- **json**: a saída é formatada como uma string [JSON](#).
- **yaml**: a saída é formatada como uma string [YAML](#).
- **yaml-stream**: a saída é transmitida e formatada como uma string [YAML](#). A transmissão possibilita um manuseio mais rápido de tipos de dados grandes.
- **text**: a saída é formatada como várias linhas de valores de string separados por tabulação. Isso pode ser útil para passar a saída para um processador de texto, como grep, sed ou awk.
- **table**: a saída é formatada como uma tabela usando os caracteres +|- para formar as bordas da célula. Geralmente, a informação é apresentada em um formato "amigável", que é muito mais fácil de ler do que outros, mas não tão útil programaticamente.

--profile **<string>**

Especifica o [perfil nomeado](#) para usar esse comando. Para configurar perfis nomeados adicionais, você pode usar o comando `aws configure` com a opção `--profile`.

```
$ aws configure --profile <profilename>
```

--query **<string>**

Especifica uma [consulta JMESPath](#) para uso na filtragem dos dados de resposta. Para obter mais informações, consulte [Filtragem da saída da AWS CLI](#).

--region **<string>**

Especifica para qual região da AWS enviar a solicitação da AWS desse comando. Para obter uma lista de todas as regiões que você pode especificar, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

--version

Uma operação booliana que exibe a versão atual do programa de AWS CLI que está em execução.

Usos comuns das opções de linha de comando

Os usos comuns das opções de linha de comando incluem a verificação de seus recursos em várias regiões da AWS e a alteração do formato de saída para legibilidade ou a facilidade de uso ao criar scripts. Nos exemplos a seguir, executamos o comando `describe-instances` em cada região até descobrirmos em qual delas nossa instância está.


```
$ aws ec2 describe-instances --output table --region us-west-1
-----
|DescribeInstances|
+-----+
$ aws ec2 describe-instances --output table --region us-west-2
-----
|                               DescribeInstances                               |
+-----+
||                               Reservations                               ||
|+-----+|
||   OwnerId                       | 012345678901                             ||
||   ReservationId                 | r-abcdefgh                             ||
|+-----+|
||                               Instances                               || | | |
||+-----+||
|||   AmiLaunchIndex               | 0                                     |||
|||   Architecture                 | x86_64                             |||
...

```

Conclusão de comando

A AWS Command Line Interface (AWS CLI) inclui um recurso de preenchimento automático de comandos que permite usar a tecla Tab para concluir o preenchimento um comando inserido parcialmente. Na maioria dos sistemas, esse recurso deve ser configurado manualmente.

Para obter informações sobre o recurso de prompt automático da AWS CLI versão 2, consulte [Fazer a AWS CLI solicitar comandos](#).

Tópicos

- [Como funciona](#)
- [Configuração do preenchimento automático de comando no Linux ou no macOS](#)
- [Configuração do preenchimento de comandos no Windows](#)

Como funciona

Ao inserir parcialmente um comando, um parâmetro ou uma opção, o recurso de conclusão de comando conclui automaticamente o comando ou exibe uma lista sugerida de comandos. Para solicitar a conclusão de comandos, insira parcialmente um comando e pressione a tecla de conclusão, que normalmente é **Tab** na maioria dos shells.

Os exemplos a seguir mostram diferentes maneiras de usar a conclusão de comando:

- Insira parcialmente um comando e pressione **Tab** para exibir uma lista sugerida de comandos.

```
$ aws dynamodb dTAB
delete-backup                describe-global-table
delete-item                  describe-global-table-settings
delete-table                 describe-limits
describe-backup              describe-table
describe-continuous-backups  describe-table-replica-auto-scaling
describe-contributor-insights describe-time-to-live
describe-endpoints
```

- Insira parcialmente um parâmetro e pressione **Tab** para exibir uma lista sugerida de parâmetros.

```
$ aws dynamodb delete-table --TAB
--ca-bundle           --endpoint-url           --profile
--cli-connect-timeout --generate-cli-skeleton --query
--cli-input-json      --no-paginate             --region
--cli-read-timeout    --no-sign-request        --table-name
--color               --no-verify-ssl          --version
--debug               --output
```

- Insira um parâmetro e pressione **Tab** para exibir uma lista sugerida de valores de recursos. Esse recurso só está disponível na AWS CLI versão 2.

```
$ aws dynamodb db delete-table --table-name TAB
Table 1                Table 2                Table 3
```

Configuração do preenchimento automático de comando no Linux ou no macOS

Para configurar o preenchimento automático de comandos no Linux ou macOS, você deve conhecer o nome do shell que está usando e a localização do script `aws_completer`.

Note

O preenchimento do comando é configurado automaticamente e habilitado por padrão nas instâncias do Amazon EC2 que executam o Amazon Linux.

Tópicos

- [Confirme se a pasta do completer está na variável path](#)
- [Habilitar a conclusão de comando](#)
- [Verifique o preenchimento de comandos](#)

Confirme se a pasta do completer está na variável path

Para o completer da AWS funcionar corretamente, o `aws_completer` deverá estar na variável `path` do seu shell. O comando `which` pode verificar se o completer está na variável `path`.

```
$ which aws_completer
/usr/local/bin/aws_completer
```

Se o comando não conseguir encontrar o completer, use as etapas a seguir para adicionar a pasta do completer à variável `PATH`.

Etapas 1: Localizar o completer da AWS

A localização do completer da AWS pode variar, dependendo do método de instalação usado.

- Gerenciador de pacotes: programas como `pip`, `yum`, `brew` e `apt-get` normalmente instalam o completer da AWS (ou um symlink para ele) em um local que faz parte do caminho padrão.
 - Se você usou o `pip` sem o parâmetro `--user`, o caminho padrão é `/usr/local/bin/aws_completer`.
 - Se você usou o `pip` com o parâmetro `--user`, o caminho padrão é `home/username/.local/bin/aws_completer`.
- Instalador empacotado: se você usou o instalador empacotado, o caminho padrão é `/usr/local/bin/aws_completer`.

Se tudo falhar, você poderá usar o comando `find` para pesquisar o completer da AWS em todo o seu sistema de arquivos.

```
$ find / -name aws_completer
/usr/local/bin/aws_completer
```

Etapa 2: Identificar o shell

Para identificar qual shell você está usando, é possível usar um dos seguintes comandos.

- `echo $SHELL`: exibe o nome do arquivo do programa de shell. Isso geralmente corresponde ao nome do shell em uso, a menos que você tenha iniciado um shell diferente após fazer o login.

```
$ echo $SHELL
/bin/bash
```

- `ps`: exibe os processos em execução para o usuário atual. Um deles é o shell.

```
$ ps
  PID TTY          TIME CMD
 2148 pts/1    00:00:00 bash
 8756 pts/1    00:00:00 ps
```

Etapa 3: Adicionar o completer ao caminho

1. Encontre o script de perfil do shell em sua pasta de usuário.

```
$ ls -a ~/
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash: `.bash_profile`, `.profile` ou `.bash_login`
 - Zsh: `.zshrc`
 - Tcsh: `.tcshrc`, `.cshrc` ou `.login`
2. Adicione um comando de exportação ao final do script de perfil que é semelhante ao exemplo a seguir. Substitua `/usr/local/bin/` pela pasta que você descobriu na seção anterior.

```
export PATH=/usr/local/bin/:$PATH
```

3. Recarregue o perfil na sessão atual para colocar essas alterações em vigor. Substitua `.bash_profile` pelo nome do script de shell que você descobriu na primeira seção.

```
$ source ~/.bash_profile
```

Habilitar a conclusão de comando

Depois de confirmar que o `completer` está no caminho, habilite a conclusão de comando executando o comando apropriado para o shell que você está usando. Você pode adicionar o comando ao perfil do shell para executá-lo cada vez que você abrir um novo shell. Em cada comando, substitua o caminho `/usr/local/bin/` pelo encontrado no seu sistema em [Confirme se a pasta do completer está na variável path](#).

- **bash**: usa o comando interno `complete`.

```
$ complete -C '/usr/local/bin/aws_completer' aws
```

Adicione o comando anterior ao `~/.bashrc` para executá-lo cada vez que você abrir um novo shell. O `~/.bash_profile` deve se basear em `~/.bashrc` para garantir que o comando também seja executado em shells de login.

- **zsh**: para executar o preenchimento de comandos, é necessário executar `bashcompinit` adicionando a seguinte linha de carregamento automático ao final do script do perfil `~/.zshrc`.

```
$ autoload bashcompinit && bashcompinit
$ autoload -Uz compinit && compinit
```

Para habilitar a conclusão de comando, use o comando interno `complete`.

```
$ complete -C '/usr/local/bin/aws_completer' aws
```

Adicione os comandos anteriores ao `~/.zshrc` para executá-los cada vez que você abrir um novo shell.

- **tcsh**: o preenchimento para `tcsh` utiliza um tipo de palavra e padrão para definir o comportamento do preenchimento.

```
> complete aws 'p/*/'`aws_completer`/'
```

Adicione o comando anterior ao `~/.tcshrc` para executá-lo cada vez que você abrir um novo shell.

Depois de habilitar o preenchimento automático de comandos, [Verifique o preenchimento de comandos](#) está funcionando.

Verifique o preenchimento de comandos

Após habilitar o preenchimento de comandos, insira um comando parcial e pressione Tab para ver os comandos disponíveis.

```
$ aws sTAB
s3          ses          sqs          sts          swf
s3api       sns          storagegateway support
```

Configuração do preenchimento de comandos no Windows

Note

Para obter informações sobre como o PowerShell lida com a conclusão, incluindo as várias teclas de conclusão, consulte [about_tab_Expansion](#) na documentação do Microsoft PowerShell.

Para habilitar o preenchimento de comandos para o PowerShell no Windows, conclua as etapas a seguir no PowerShell.

1. Abra o arquivo \$PROFILE com o comando a seguir.

```
PS C:\> Notepad $PROFILE
```

Se você ainda não tiver um \$PROFILE, use o procedimento a seguir para criar um perfil de usuário.

```
PS C:\> if (!(Test-Path -Path $PROFILE ))
{ New-Item -Type File -Path $PROFILE -Force }
```

Para obter mais informações sobre os perfis do PowerShell, consulte [Como usar perfis no Windows PowerShell ISE](#) no site de Documentação da Microsoft.

2. Para ativar o preenchimento de comandos, adicione o seguinte bloco de código ao seu perfil, salve e, em seguida, feche o arquivo.

```
Register-ArgumentCompleter -Native -CommandName aws -ScriptBlock {
    param($commandName, $wordToComplete, $cursorPosition)
```

```

$env:COMP_LINE=$wordToComplete
if ($env:COMP_LINE.Length -lt $cursorPosition){
    $env:COMP_LINE=$env:COMP_LINE + " "
}
$env:COMP_POINT=$cursorPosition
aws_completer.exe | ForEach-Object {
    [System.Management.Automation.CompletionResult]::new($_, $_,
'ParameterValue', $_)
}
Remove-Item Env:\COMP_LINE
Remove-Item Env:\COMP_POINT
}

```

3. Depois de habilitar a conclusão dos comandos, recarregue o shell, insira um comando parcial e pressione Tab para percorrer os comandos disponíveis.

```
$ aws sTab
```

```
$ aws s3
```

Para ver todos os comandos disponíveis para conclusão, insira um comando parcial e pressione Ctrl + Espaço.

```
$ aws sCtrl + Space
```

s3	ses	sqs	sts	swf
s3api	sns	storagegateway	support	

Novas tentativas da AWS CLI

Este tópico descreve como a AWS CLI pode observar chamadas para AWS falharem devido a problemas inesperados. Esses problemas podem ocorrer no lado do servidor ou podem falhar devido à limitação de taxa do serviço da AWS que você está tentando chamar. Esses tipos de falhas geralmente não exigem tratamento especial e a chamada é feita automaticamente novamente, em geral após um breve período de espera. A AWS CLI oferece muitos recursos para ajudar a repetir chamadas de cliente para os serviços da AWS quando esses tipos de erros ou exceções ocorrem.

Tópicos

- [Modos de novas tentativas disponíveis](#)

- [Configuração um modo de nova tentativa](#)
- [Visualização de logs de novas tentativas](#)

Modos de novas tentativas disponíveis

A AWS CLI oferece vários modos para escolha dependendo da sua versão:

- [Modo de novas tentativas herdado](#)
- [Modo de nova tentativa padrão](#)
- [Modo de nova tentativa adaptável](#)

Modo de novas tentativas herdado

O modo herdado usa um manipulador de novas tentativas mais antigo e com funcionalidade limitada que inclui:

- Um valor padrão de 4 para o máximo de novas tentativas, totalizando 5 tentativas de chamada. Esse valor pode ser sobrescrito por meio do parâmetro de configuração `max_attempts`.
- No DynamoDB, o valor padrão máximo de novas tentativas é nove, totalizando dez tentativas de chamada. Esse valor pode ser sobrescrito por meio do parâmetro de configuração `max_attempts`.
- Novas tentativas para o seguinte número limitado de erros e exceções:
 - Erros gerais de soquete e conexão:
 - `ConnectionError`
 - `ConnectionClosedError`
 - `ReadTimeoutError`
 - `EndpointConnectionError`
 - Erros e exceções de controle de utilização ou limitação no lado do serviço:
 - `Throttling`
 - `ThrottlingException`
 - `ThrottledException`
 - `RequestThrottledException`
 - `ProvisionedThroughputExceededException`
- Novas tentativas em vários códigos de status HTTP, incluindo 429, 500, 502, 503, 504 e 509.

- Qualquer tentativa de repetição incluirá um recuo exponencial por um fator de base 2.

Modo de nova tentativa padrão

O modo padrão é um conjunto padrão de regras de novas tentativas nos AWS SDKs com mais funcionalidade do que herdado. Esse é o modo padrão para a AWS CLI versão 2. O modo padrão foi criado para a AWS CLI versão 2 e é compatível com a AWS CLI versão 1. A funcionalidade do modo padrão inclui:

- Um valor padrão de 2 para o máximo de novas tentativas, totalizando 3 tentativas de chamada. Esse valor pode ser sobrescrito por meio do parâmetro de configuração `max_attempts`.
- Novas tentativas para a seguinte lista estendida de erros/exceções:
 - Erros e exceções transientes
 - `RequestTimeout`
 - `RequestTimeoutException`
 - `PriorRequestNotComplete`
 - `ConnectionError`
 - `HTTPClientError`
 - Erros e exceções de controle de utilização ou limitação no lado do serviço:
 - `Throttling`
 - `ThrottlingException`
 - `ThrottledException`
 - `RequestThrottledException`
 - `TooManyRequestsException`
 - `ProvisionedThroughputExceededException`
 - `TransactionInProgressException`
 - `RequestLimitExceeded`
 - `BandwidthLimitExceeded`
 - `LimitExceededException`
 - `RequestThrottled`
 - `SlowDown`
 - `EC2ThrottledException`

- Novas tentativas em códigos de erro não descritivos transientes. Especificamente, estes códigos de status HTTP: 500, 502, 503, 504.
- Qualquer nova tentativa incluirá um recuo exponencial por um fator de base 2 para um tempo máximo de recuo de 20 segundos.

Modo de nova tentativa adaptável

Warning

O modo adaptativo é um experimental e está sujeito a modificações, tanto em suas características quanto em seu comportamento.

O modo de nova tentativa adaptativo é um modo de repetição experimental que inclui todos os recursos do modo padrão. Além dos recursos do modo padrão, o modo adaptativo também introduz limitação de taxa no lado do cliente através do uso de um bucket de token e variáveis de limite de taxa que são atualizadas dinamicamente com cada tentativa de repetição. Esse modo oferece flexibilidade de novas tentativas no lado do cliente que se adapta à resposta de estado de erro e exceção de um serviço da AWS.

Com cada nova tentativa, o modo adaptativo modifica as variáveis de limite de taxa com base no erro, exceção ou código de status HTTP apresentado na resposta do serviço da AWS. Essas variáveis de limite de taxa são usadas para calcular uma nova taxa de chamada para o cliente. Cada resposta HTTP de exceção/erro ou não bem-sucedida (fornecida na lista acima) de um serviço da AWS atualiza as variáveis de limite de taxa à medida que as tentativas ocorrerem até alcançarem êxito, o bucket de token se esgotar ou o valor máximo de tentativas configurado ser atingido.

Configuração um modo de nova tentativa

A AWS CLI inclui uma variedade de configurações de novas tentativas, bem como métodos de configuração a serem considerados ao criar seu objeto cliente.

Métodos de configuração disponíveis

Na AWS CLI, os usuários podem configurar novas tentativas das seguintes formas:

- Variáveis de ambiente
- Arquivo de configuração da AWS CLI

Os usuários podem personalizar as seguintes opções de novas tentativas:

- **Modo de repetição:** especifica qual modo de repetição será usado pela AWS CLI. Conforme descrito anteriormente existem três modos de repetição disponíveis: herdado (o modo usado por padrão), padrão e adaptativo. O valor padrão para a AWS CLI e para a versão 2 é padrão.
- **Máximo de tentativas:** especifica um valor para o máximo de novas tentativas utilizadas pelo manipulador de novas tentativas da AWS CLI, onde a chamada inicial conta para o valor fornecido por você. O valor padrão é 5.

Definição de uma configuração de novas tentativas em suas variáveis de ambiente

Para definir a configuração de novas tentativas para o AWS CLI, atualize as variáveis de ambiente do seu sistema operacional.

As variáveis de ambiente de novas tentativas são:

- `AWS_RETRY_MODE`
- `AWS_MAX_ATTEMPTS`

Para obter mais informações sobre variáveis de ambiente, consulte [Variáveis de ambiente para configurar a AWS CLI](#).

Definição de uma configuração de nova tentativa em seu arquivo de configuração da AWS

Para alterar a configuração de novas tentativas, atualize o arquivo de configuração global da AWS. O local padrão do seu arquivo de configuração da AWS é `~/.aws/config`.

Exemplo de um arquivo de configuração da AWS:

```
[default]
retry_mode = standard
max_attempts = 6
```

Para obter mais informações sobre arquivos de configuração, consulte [Configurações do arquivo de configuração e credenciais](#).

Visualização de logs de novas tentativas

A AWS CLI usa a metodologia de novas tentativas e log do Boto3. Você pode usar a opção `--debug` em qualquer comando para receber logs de depuração. Para obter informações sobre como usar a opção `--debug`, consulte [Opções de linha de comando](#).

Se você procurar “retry” em seus logs de depuração, encontrará as informações de repetição de que necessita. As entradas de log do cliente para tentativas de repetição dependem do modo de repetição ativado.

Modo herdado:

As mensagens de novas tentativas são geradas por `botocore.retryhandler`. Você verá uma de três mensagens:

- `No retry needed`
- `Retry needed, action of: <action_name>`
- `Reached the maximum number of retry attempts: <attempt_number>`

Modo padrão ou adaptativo:

As mensagens de novas tentativas são geradas por `botocore.retries.standard`. Você verá uma de três mensagens:

- `No retrying request`
- `Retry needed, retrying request after delay of: <delay_value>`
- `Retry needed but retry quota reached, not retrying request`

Para obter o arquivo de definição completa de novas tentativas do `botocore`, consulte [_retry.json](#) no Repositório do `botocore` no GitHub.

Usar um proxy HTTP

Para acessar a AWS por meio de servidores de proxy, é possível configurar as variáveis de ambiente `HTTP_PROXY` e `HTTPS_PROXY` com os nomes de domínio DNS ou endereços IP e números de porta usados pelos servidores de proxy.

Tópicos

- [Como usar os exemplos da](#)
- [Autenticar para um proxy](#)
- [Uso de proxy em instâncias do Amazon EC2](#)
- [Solução de problemas](#)

Como usar os exemplos da

Note

Os exemplos a seguir mostram o nome da variável de ambiente com todas as letras maiúsculas. No entanto, se você especificar uma variável duas vezes usando letras maiúsculas e minúsculas, as minúsculas terão precedência. Recomendamos que você defina cada variável somente uma vez para evitar confusão e comportamento inesperado do sistema.

Os exemplos a seguir mostram como você pode usar o endereço IP explícito do proxy ou um nome de DNS que seja resolvido para o endereço IP do proxy. Também pode ser seguido por uma vírgula e o número da porta para a qual as consultas devem ser enviadas.

Linux or macOS

```
$ export HTTP_PROXY=http://10.15.20.25:1234
$ export HTTP_PROXY=http://proxy.example.com:1234
$ export HTTPS_PROXY=http://10.15.20.25:5678
$ export HTTPS_PROXY=http://proxy.example.com:5678
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx HTTP_PROXY http://10.15.20.25:1234
C:\> setx HTTP_PROXY http://proxy.example.com:1234
C:\> setx HTTPS_PROXY http://10.15.20.25:5678
C:\> setx HTTPS_PROXY http://proxy.example.com:5678
```

O uso de [setx](#) para definir uma variável de ambiente altera o valor usado na sessão de prompt de comando atual e todas as sessões de prompt de comando que você criar após a execução do

comando. Não afeta outros shells de comando que já estejam em execução no momento em que você executar o comando.

Como definir somente para a sessão atual

O uso de [set](#) para definir uma variável de ambiente altera o valor usado até o final da sessão de prompt de comando atual ou até que você defina a variável como um valor diferente.

```
C:\> set HTTP_PROXY=http://10.15.20.25:1234
C:\> set HTTP_PROXY=http://proxy.example.com:1234
C:\> set HTTPS_PROXY=http://10.15.20.25:5678
C:\> set HTTPS_PROXY=http://proxy.example.com:5678
```

Autenticar para um proxy

Note

A AWS CLI não é compatível com proxies NTLM. Se você usa um proxy de protocolo NTLM ou Kerberos, talvez seja possível se conectar por meio de um proxy de autenticação, como [Cntlm](#).

O AWS CLI é compatível com a autenticação básica HTTP. Especifique o nome do usuário e uma senha no URL de proxy da forma a seguir.

Linux or macOS

```
$ export HTTP_PROXY=http://username:password@proxy.example.com:1234
$ export HTTPS_PROXY=http://username:password@proxy.example.com:5678
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx HTTP_PROXY http://username:password@proxy.example.com:1234
C:\> setx HTTPS_PROXY http://username:password@proxy.example.com:5678
```

Como definir somente para a sessão atual

```
C:\> set HTTP_PROXY=http://username:password@proxy.example.com:1234  
C:\> set HTTPS_PROXY=http://username:password@proxy.example.com:5678
```

Uso de proxy em instâncias do Amazon EC2

Se você configurar um proxy em uma instância do Amazon EC2 iniciada com uma função do IAM anexada, certifique-se de isentar o endereço usado do acesso aos [metadados da instância](#). Para fazer isso, defina a variável de ambiente NO_PROXY como o endereço IP do serviço de metadados da instância 169.254.169.254. Esse endereço não varia.

Linux or macOS

```
$ export NO_PROXY=169.254.169.254
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx NO_PROXY 169.254.169.254
```

Como definir somente para a sessão atual

```
C:\> set NO_PROXY=169.254.169.254
```

Solução de problemas

Se você encontrar problemas com a AWS CLI, consulte [Solucionar erros](#) para obter as etapas de solução de problemas. Para obter as etapas mais relevantes de solução de problemas, consulte [the section called “Erros de certificado SSL”](#).

Usar endpoints na AWS CLI

Para se conectar a um AWS service (Serviço da AWS) de forma programática, use um endpoint. Um endpoint é o URL do ponto de entrada para um serviço da Web da AWS. A AWS Command Line Interface (AWS CLI) usa automaticamente o endpoint padrão para cada serviço em uma Região da AWS, mas você pode especificar um endpoint alternativo para suas solicitações de API.

Tópicos de endpoint

- [Definir o endpoint para um único comando](#)
- [Definir um endpoint global para todos os Serviços da AWS](#)
- [Definido para usar endpoints FIPs para todos os Serviços da AWS](#)
- [Configurado para usar os endpoints de pilha dupla para todos os Serviços da AWS](#)
- [Definir endpoints específicos de serviço](#)
 - [Endpoints específicos de serviço: variáveis de ambiente](#)
 - [Endpoints específicos de serviço: arquivo compartilhado config](#)
 - [Endpoints específicos de serviço: lista de identificadores específicos de serviço](#)
- [Precedência de configurações e definições do endpoint](#)

Definir o endpoint para um único comando

Para substituir qualquer configuração de endpoint ou variável de ambiente referente a um único comando, use a opção de linha de comando [--endpoint-url](#). O exemplo de comando a seguir usa um URL personalizado do endpoint do Amazon S3.

```
$ aws s3 ls --endpoint-url http://localhost:4567
```

Definir um endpoint global para todos os Serviços da AWS

Para rotear solicitações de todos os serviços para um URL de endpoint personalizado, use uma das seguintes configurações:

- Variáveis de ambiente:
 - [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#): ignore os URLs de endpoints configurados.

Linux or macOS

```
$ export AWS_IGNORE_CONFIGURED_ENDPOINT_URLS=true
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_IGNORE_CONFIGURED_ENDPOINT_URLS true
```


Como definir somente para a sessão atual

```
C:\> set AWS_IGNORE_CONFIGURED_ENDPOINT_URLS=true
```

PowerShell

```
PS C:\> $Env:AWS_IGNORE_CONFIGURED_ENDPOINT_URLS="true"
```

- [AWS_ENDPOINT_URL](#): defina o URL do endpoint global.

Linux or macOS

```
$ export AWS_ENDPOINT_URL=http://localhost:4567
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_ENDPOINT_URL http://localhost:4567
```

Como definir somente para a sessão atual

```
C:\> set AWS_ENDPOINT_URL=http://localhost:4567
```

PowerShell

```
PS C:\> $Env:AWS_ENDPOINT_URL="http://localhost:4567"
```

- O arquivo config:
 - [ignore_configure_endpoint_urls](#): ignore os URLs de endpoints configurados.

```
ignore_configure_endpoint_urls = true
```

- [endpoint_url](#): defina o URL do endpoint global.

```
endpoint_url = http://localhost:4567
```

Os endpoints específicos de serviço e a opção de linha de comando `--endpoint-url` substituem qualquer endpoint global.

Definido para usar endpoints FIPs para todos os Serviços da AWS

Para rotear solicitações para todos os serviços usarem endpoints FIPS, use uma das seguintes opções:

- variável de ambiente [AWS_USE_FIPS_ENDPOINT](#).

Linux or macOS

```
$ export AWS_USE_FIPS_ENDPOINT=true
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_USE_FIPS_ENDPOINT true
```

Como definir somente para a sessão atual

```
C:\> set AWS_USE_FIPS_ENDPOINT=true
```

PowerShell

```
PS C:\> $Env:AWS_USE_FIPS_ENDPOINT="true"
```

- configuração do arquivo [use_fips_endpoint](#).

```
use_fips_endpoint = true
```

Alguns serviços da AWS oferecem endpoints compatíveis com o [Federal Information Processing Standard \(FIPS\) 140-2](#) em algumas Regiões da AWS. Quando o serviço da AWS é compatível com o FIPS, essa configuração especifica qual endpoint do FIPS a AWS CLI deve usar. Ao contrário dos endpoints-padrão da AWS, os endpoints do FIPS usam uma biblioteca de software TLS compatível com o FIPS 140-2. Esses endpoints podem ser necessários por empresas que interagem com o governo dos Estados Unidos.

Se essa configuração estiver ativada, mas não existir um endpoint FIPS para o serviço em sua Região da AWS, o comando AWS poderá falhar. Nesse caso, especifique manualmente o endpoint a ser usado no comando usando a opção [--endpoint-url](#) ou use [endpoints específicos do serviço](#).

Para saber mais sobre como especificar endpoints FIPS por Região da AWS, consulte [Endpoints FIPS por serviço](#).

Configurado para usar os endpoints de pilha dupla para todos os Serviços da AWS

Para rotear solicitações para todos os serviços usarem endpoints de pilha dupla, use uma das seguintes configurações:

- variável de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#).

Linux or macOS

```
$ export AWS_USE_DUALSTACK_ENDPOINT=true
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_USE_DUALSTACK_ENDPOINT true
```

Como definir somente para a sessão atual

```
C:\> set AWS_USE_DUALSTACK_ENDPOINT=true
```

PowerShell

```
PS C:\> $Env:AWS_USE_DUALSTACK_ENDPOINT="true"
```

- configuração do arquivo [use_dualstack_endpoint](#).

```
use_dualstack_endpoint = true
```

Permite o uso de endpoints de pilha dupla para enviar solicitações da AWS. Para saber mais sobre endpoints de pilha dupla, que suportam tráfego IPv4 e IPv6, consulte [Como usar endpoints de pilha](#)

[dupla do Amazon S3](#) no Guia do usuário do Amazon Simple Storage Service. Endpoints de pilha dupla estão disponíveis para alguns serviços em algumas regiões. Se não existir um endpoint de pilha dupla para o serviço ou Região da AWS, a solicitação falhará. Ela fica desabilitada por padrão.

Definir endpoints específicos de serviço

A configuração de endpoint específico de serviço oferece a opção de usar um endpoint persistente de sua escolha para solicitações da AWS CLI. Essas configurações oferecem flexibilidade para permitir endpoints locais, endpoints da VPC e ambientes de desenvolvimento da AWS local de terceiros. Diferentes endpoints podem ser usados para ambientes de teste e produção. Você pode especificar um URL de endpoint para Serviços da AWS individuais.

Os endpoints específicos de serviço podem ser designados das seguintes maneiras:

- A opção de linha de comando [--endpoint-url](#) para um único comando.
- Variáveis de ambiente:
 - [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#): ignora todos os URLs de endpoint configurados, a menos que especificado na linha de comando.
 - [AWS_ENDPOINT_URL_<SERVICE>](#): especifica um endpoint personalizado usado para um serviço específico, onde <SERVICE> é substituído pelo identificador AWS service (Serviço da AWS). Para todas as variáveis específicas de serviço, consulte [the section called “Lista de identificadores específicos de serviço”](#).
- Arquivo config:
 - [ignore_configure_endpoint_urls](#): ignora todos os URLs de endpoint configurados, a menos que especificado por meio de variáveis de ambiente ou na linha de comando.
 - A seção [services](#) do arquivo config combinada com a configuração do arquivo [endpoint_url](#).

Tópicos de endpoints específicos de serviço:

- [Endpoints específicos de serviço: variáveis de ambiente](#)
- [Endpoints específicos de serviço: arquivo compartilhado config](#)
- [Endpoints específicos de serviço: lista de identificadores específicos de serviço](#)

Endpoints específicos de serviço: variáveis de ambiente

As variáveis de ambiente substituem as configurações no arquivo de configuração, mas não substituem as opções especificadas na linha de comando. Use variáveis de ambiente se quiser que todos os perfis usem os mesmos endpoints no dispositivo.

Veja a seguir as variáveis de ambiente específicas de serviço:

- [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#): ignora todos os URLs de endpoint configurados, a menos que especificado na linha de comando.

Linux or macOS

```
$ export AWS_IGNORE_CONFIGURED_ENDPOINT_URLS=true
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_IGNORE_CONFIGURED_ENDPOINT_URLS true
```

Como definir somente para a sessão atual

```
C:\> set AWS_IGNORE_CONFIGURED_ENDPOINT_URLS=true
```

PowerShell

```
PS C:\> $Env:AWS_IGNORE_CONFIGURED_ENDPOINT_URLS="true"
```

- [AWS_ENDPOINT_URL_<SERVICE>](#): especifica um endpoint personalizado usado para um serviço específico, onde <SERVICE> é substituído pelo identificador AWS service (Serviço da AWS). Para todas as variáveis específicas de serviço, consulte [the section called “Lista de identificadores específicos de serviço”](#).

O exemplo a seguir de variável de ambiente define um endpoint para o AWS Elastic Beanstalk:

Linux or macOS

```
$ export AWS_ENDPOINT_URL_ELASTIC_BEANSTALK=http://localhost:4567
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_ENDPOINT_URL_ELASTIC_BEANSTALK http://localhost:4567
```

Como definir somente para a sessão atual

```
C:\> set AWS_ENDPOINT_URL_ELASTIC_BEANSTALK=http://localhost:4567
```

PowerShell

```
PS C:\> $Env:AWS_ENDPOINT_URL_ELASTIC_BEANSTALK="http://localhost:4567"
```

Para obter mais informações sobre como definir variáveis de ambiente, consulte [the section called “Variáveis de ambiente”](#).

Endpoints específicos de serviço: arquivo compartilhado **config**

No arquivo compartilhado `config`, `endpoint_url` é usado em várias seções. Para definir um endpoint específico de serviço, use a configuração `endpoint_url` aninhada em uma chave de identificação de serviço em uma seção `services`. Para obter detalhes sobre como definir uma seção `services` no arquivo compartilhado [the section called “services”](#), consulte `config`.

O exemplo a seguir usa uma seção `services` para configurar um URL de endpoint específico do serviço para o Amazon S3 e um endpoint global personalizado usado para todos os outros serviços:

```
[profile dev1]
endpoint_url = http://localhost:1234
services = s3-specific

[services testing-s3]
s3 =
    endpoint_url = http://localhost:4567
```

Um único perfil pode configurar endpoints para vários serviços. O exemplo a seguir define os URLs de endpoint específicos de serviço para o Amazon S3 e o AWS Elastic Beanstalk no mesmo perfil.

Para obter uma lista de todas as chaves de identificação de serviço a serem usadas na seção `services`, consulte [Lista de identificadores específicos de serviço](#).

```
[profile dev1]
services = testing-s3-and-eb

[services testing-s3-and-eb]
s3 =
    endpoint_url = http://localhost:4567
elastic_beanstalk =
    endpoint_url = http://localhost:8000
```

A seção de configuração de serviço pode ser usada em vários perfis. O exemplo a seguir tem dois perfis que usam a mesma definição de `services`:

```
[profile dev1]
output = json
services = testing-s3

[profile dev2]
output = text
services = testing-s3

[services testing-s3]
s3 =
    endpoint_url = https://localhost:4567
```

Endpoints específicos de serviço: lista de identificadores específicos de serviço

O identificador AWS service (Serviço da AWS) é baseado no `serviceId` do modelo de API, substituindo todos os espaços por sublinhados e colocando todas as letras em minúsculas.

O exemplo de identificador de serviço a seguir usa o AWS Elastic Beanstalk. O AWS Elastic Beanstalk tem um `serviceId` de [Elastic Beanstalk](#); portanto, a chave de identificação de serviço é `elastic_beanstalk`.

A tabela a seguir lista todos os identificadores, chaves do arquivo `config` e variáveis de ambiente específicos de serviço.

Precedência de configurações e definições do endpoint

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS_IGNORE_CONFIGURED_ENDPOINT_URLS](#) ou a configuração do perfil [ignore_configure_endpoint_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS_ENDPOINT_URL_<SERVICE>](#), como [AWS_ENDPOINT_URL_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS_USE_DUALSTACK_ENDPOINT](#), [AWS_USE_FIPS_ENDPOINT](#) e [AWS_ENDPOINT_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use_dualstack_endpoint](#), [use_fips_endpoint](#) e [endpoint_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

Autenticação e credenciais de acesso

É necessário estabelecer como a AWS CLI faz a autenticação com a AWS quando você desenvolve com serviços da AWS. Para configurar credenciais para acesso programático para a AWS CLI, escolha uma das opções a seguir. As opções estão em ordem de recomendação.

Qual usuário precisa de acesso programático?	Finalidade	Instruções
Identidade do quadro de funcionários (usuários do AWS IAM Identity Center)	(Recomendado) Use credenciais de curto prazo.	the section called “Autenticação do IAM Identity Center”
IAM	Use credenciais de curto prazo.	the section called “Credenciais de curto prazo”
IAM ou Identidade do quadro de funcionários (usuários do AWS IAM Identity Center)	Use metadados da instância do Amazon EC2 para credenciais.	the section called “Uso de credenciais para metadados de instância do Amazon EC2.”
IAM ou Identidade do quadro de funcionários (usuários do AWS IAM Identity Center)	Combine outro método de credencial e assuma um perfil para permissões.	the section called “Perfis do IAM”
IAM	(Não recomendado) Use credenciais de longo prazo.	the section called “IAM users”
IAM ou Identidade do quadro de funcionários (usuários do AWS IAM Identity Center)	(Não recomendado) Combine outro método de credencial, mas use valores de credencial armazenados em um local fora da AWS CLI.	the section called “Credenciais externas”

Precedência de credenciais e configurações

As credenciais e as configurações estão localizadas em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. Certas autenticações têm precedência sobre outras. As definições de configuração da AWS CLI têm precedência na seguinte ordem:

1. [Opções da linha de comando](#): substituem as configurações em qualquer outro local, como nos parâmetros `--region`, `--output` e `--profile`.
2. [Variáveis de ambiente](#): você pode armazenar valores nas variáveis de ambiente do sistema.
3. [Assumir perfil](#): assuma as permissões de um perfil do IAM por meio da configuração ou do comando `aws sts assume-role`.
4. [Assumir perfil com identidade da web](#): assuma as permissões de um perfil do IAM usando uma identidade da web por meio da configuração ou do comando `aws sts assume-role`.
5. [AWS IAM Identity Center](#): as configurações do Centro de Identidade do IAM são armazenadas no arquivo `config`. As credenciais são autenticadas quando você executa o comando `aws configure sso`. O arquivo `config` está localizado em `~/.aws/config` no Linux ou MacOS ou em `C:\Users\USERNAME\.aws\config` no Windows.
6. [Arquivo de credenciais](#): os arquivos `credentials` e `config` são atualizados quando você executa o comando `aws configure`. O arquivo `credentials` está localizado em `~/.aws/credentials` no Linux ou MacOS ou em `C:\Users\USERNAME\.aws\credentials` no Windows.
7. [Processo personalizado](#): obtenha suas credenciais de uma fonte externa.
8. [Arquivo de configuração](#): os arquivos `credentials` e `config` são atualizados quando você executa o comando `aws configure`. O arquivo `config` está localizado em `~/.aws/config` no Linux ou MacOS ou em `C:\Users\USERNAME\.aws\config` no Windows.
9. [Credenciais de perfil de instância](#): você pode associar um perfil do IAM a cada uma das suas instâncias do Amazon Elastic Compute Cloud (Amazon EC2). As credenciais temporárias para essa função estão disponíveis para o código em execução na instância. As credenciais são fornecidas por meio do serviço de metadados do Amazon EC2. Para obter mais informações, consulte [Funções do IAM para o Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux e [Uso de perfis de instância](#) no Manual do usuário do IAM.
10. [Credenciais de container](#): você pode associar uma função do IAM a cada uma das suas definições de tarefa do Amazon Elastic Container Service (Amazon ECS). As credenciais temporárias para

essa função estão disponíveis para os contêineres dessa tarefa. Para mais informações, consulte [Funções do IAM para Tarefas](#) no Guia de Desenvolvedor Amazon Elastic Container Service.

Tópicos adicionais nesta seção

- [the section called “Autenticação do IAM Identity Center”](#)
- [the section called “Credenciais de curto prazo”](#)
- [the section called “Perfis do IAM”](#)
- [the section called “IAM users”](#)
- [the section called “Uso de credenciais para metadados de instância do Amazon EC2.”](#)
- [the section called “Credenciais externas”](#)

Configurar a AWS CLI para usar o AWS IAM Identity Center

Esta seção dá instruções para configurar a AWS CLI para autenticar usuários com o AWS IAM Identity Center (Centro de Identidade do IAM) e conseguir credenciais para executar comandos da AWS CLI. Há duas formas principais de configurar o SSO por meio do arquivo `config`:

- (Recomendado) [Configuração do provedor de token do SSO](#). A configuração do provedor de token do SSO, o AWS SDK ou ferramenta podem recuperar automaticamente tokens de autenticação atualizados.
- [Configuração herdada não atualizável](#). Ao usar a configuração herdada não atualizável, você precisa atualizar o token de forma manual, pois ele expira periodicamente.

Ao usar o IAM Identity Center, é possível fazer login no Active Directory, um diretório integrado do IAM Identity Center, ou em [outro IdP conectado ao IAM Identity Center](#). É possível mapear essas credenciais para um perfil do AWS Identity and Access Management (IAM) para executar comandos da AWS CLI.

Independentemente do IdP que você usa, o IAM Identity Center abstrai essas distinções. Por exemplo, é possível conectar o Microsoft Azure AD conforme descrito no artigo de blog [The Next Evolution in IAM Identity Center](#) (O que há de mais novo no IAM Identity Center).

 Note

Para obter informações sobre como usar o bearer auth, que não usa ID de conta e perfil, consulte [Configuração para usar a AWS CLI com o CodeCatalyst](#) no Guia do usuário do Amazon CodeCatalyst.

Tópicos nesta seção

- [Configuração do provedor de tokens com atualização automática de autenticação para o Centro de Identidade do IAM](#)
- [Configuração herdada não atualizável para AWS IAM Identity Center](#)
- [Usar um perfil nomeado do Centro de Identidade do IAM](#)

Configuração do provedor de tokens com atualização automática de autenticação para o Centro de Identidade do IAM

Este tópico descreve como configurar a AWS CLI para autenticar usuários com a configuração do provedor de token AWS IAM Identity Center (Centro de Identidade do IAM). Usando a configuração do provedor de token do SSO, o AWS SDK ou a ferramenta podem recuperar automaticamente tokens de autenticação atualizados.

Ao usar o IAM Identity Center, é possível fazer login no Active Directory, um diretório integrado do IAM Identity Center, ou em [outro IdP conectado ao IAM Identity Center](#). É possível mapear essas credenciais para um perfil do AWS Identity and Access Management (IAM) para executar comandos da AWS CLI.

Independentemente do IdP que você usa, o IAM Identity Center abstrai essas distinções. Por exemplo, é possível conectar o Microsoft Azure AD conforme descrito no artigo de blog [The Next Evolution in IAM Identity Center](#) (O que há de mais novo no IAM Identity Center).

 Note

Para obter informações sobre como usar o bearer auth, que não usa ID de conta e perfil, consulte [Configuração para usar a AWS CLI com o CodeCatalyst](#) no Guia do usuário do Amazon CodeCatalyst.

É possível usar a configuração do provedor de token do SSO para atualizar automaticamente os tokens de autenticação conforme necessário para sua aplicação e usar opções de duração de sessão estendida. É possível configurar isso das seguintes maneiras:

- Automaticamente, usando os comandos `aws configure sso` e `aws configure sso-session`. Os comandos a seguir são assistentes que orientam você na configuração do perfil e as informações de `sso-session` são as seguintes:
 - Use [aws configure sso](#) para criar ou editar seus perfis de config e seções de `sso-session`.
 - Use [aws configure sso-session](#) para criar ou editar somente as seções de `sso-session`.
- [Manualmente](#), editando o arquivo `config` que armazena os perfis nomeados.

Pré-requisitos

- Instale o AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#).
- É necessário primeiro ter acesso à autenticação do SSO no IAM Identity Center. Escolha um dos métodos a seguir para acessar as credenciais da AWS.

Não estabeleci acesso por meio do IAM Identity Center

Siga as instruções em [Conceitos básicos](#) no Guia do usuário do AWS IAM Identity Center. Esse processo ativa o Centro de Identidade do IAM, cria um usuário administrativo e adiciona um conjunto de permissões apropriado com privilégio mínimo.

Note

Para a Etapa 6: crie um conjunto de permissões que aplique permissões de privilégio mínimo. Recomendamos usar o conjunto de permissões predefinido `PowerUserAccess`, a menos que seu empregador tenha criado um conjunto de permissões personalizado para essa finalidade.

Saia do portal e faça login novamente para ver as Contas da AWS e as opções para `Administrator` ou `PowerUserAccess`. Selecione `PowerUserAccess` ao trabalhar com o SDK. Isso também ajuda você a encontrar detalhes sobre o acesso programático.

Eu já tenho acesso à AWS por meio de um provedor de identidade federado gerenciado pelo meu empregador (como Azure AD ou Okta)

Faça login na AWS por meio do portal do seu provedor de identidade. Se o seu administrador de nuvem concedeu permissões `PowerUserAccess` (de desenvolvedor) a você, serão exibidas as Contas da AWS às quais você tem acesso e seu conjunto de permissões. Ao lado do nome do seu conjunto de permissões, você vê opções para acessar as contas manual ou programaticamente usando esse conjunto de permissões.

Implementações personalizadas podem resultar em experiências diferentes, como nomes de conjuntos de permissões diferentes. Se não tiver certeza sobre qual conjunto de permissões usar, entre em contato com a equipe de TI para obter ajuda.

Eu já tenho acesso à AWS por meio do portal de acesso da AWS gerenciado pelo meu empregador

Faça login na AWS pelo portal de acesso da AWS. Se o seu administrador de nuvem concedeu permissões `PowerUserAccess` (de desenvolvedor) a você, serão exibidas as Contas da AWS às quais você tem acesso e seu conjunto de permissões. Ao lado do nome do seu conjunto de permissões, você vê opções para acessar as contas manual ou programaticamente usando esse conjunto de permissões.

Eu já tenho acesso à AWS por meio de um provedor de identidades federadas personalizado gerenciado pelo meu empregador

Entre em contato com a equipe de TI para obter ajuda.

Configurar seu perfil com o assistente **aws configure sso**

Para configurar um perfil do Centro de Identidade do IAM e **sso-session** para a AWS CLI

1. Reúna as informações do Centro de Identidade do IAM fazendo o seguinte:
 1. No portal de acesso da AWS, selecione o conjunto de permissões utilizado para desenvolvimento e escolha o link Linha de comando ou acesso programático.
 2. Na caixa de diálogo Obter credenciais, selecione MacOS e Linux ou Windows, dependendo do sistema operacional.
 3. Selecione o método Credenciais do Centro de Identidade do IAM para obter os valores SSO Start URL e SSO Region necessários para executar `aws configure sso`.
 4. Para obter informações sobre qual valor de escopo registrar, consulte [Escopos de acesso do OAuth 2.0](#) no Guia do usuário do Centro de Identidade do IAM.

2. No terminal de sua preferência, execute o comando `aws configure sso` e forneça o URL de início do Centro de Identidade do IAM e a região da AWS que hospeda o diretório do Centro de Identidade.

```
$ aws configure sso
SSO session name (Recommended): my-sso
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]: us-east-1
SSO registration scopes [None]: sso:account:access
```

3. A AWS CLI tenta abrir seu navegador padrão e iniciar o processo de login para sua conta do IAM Identity Center.

Attempting to automatically open the SSO authorization page in your default browser.

Se a AWS CLI não puder abrir o navegador, a seguinte mensagem será exibida com instruções sobre como iniciar manualmente o processo de login.

If the browser does not open or you wish to use a different device to authorize this request, open the following URL:

<https://device.sso.us-west-2.amazonaws.com/>

Then enter the code:

QCFK-N451

O IAM Identity Center usa o código para associar a sessão do IAM Identity Center à sessão atual da AWS CLI. A página do navegador do IAM Identity Center solicita que você faça login com suas credenciais do IAM Identity Center. Isso permite que a AWS CLI recupere e exiba as contas e os perfis da AWS que você está autorizado a usar com o IAM Identity Center.

Note

O processo de login pode solicitar que você permita que a AWS CLI acesse seus dados. Como a AWS CLI é criada sobre o SDK para Python, as mensagens de permissão podem conter variações do nome `botocore`.

4. A AWS CLI exibe as contas da AWS disponíveis para uso. Se você estiver autorizado a usar apenas uma conta, a AWS CLI selecionará essa conta para você automaticamente e ignorará o prompt. As contas da AWS disponíveis para uso são determinadas pela configuração do usuário no IAM Identity Center.

```
There are 2 AWS accounts available to you.  
> DeveloperAccount, developer-account-admin@example.com (123456789011)  
   ProductionAccount, production-account-admin@example.com (123456789022)
```

Use as teclas de seta para selecionar a conta que você deseja usar. O caractere ">" à esquerda aponta para a escolha atual. Pressione ENTER para fazer sua seleção.

5. A AWS CLI confirma sua escolha de conta e exibe os perfis do IAM que estão disponíveis para você na conta selecionada. Se a conta selecionada listar apenas uma função, a AWS CLI selecionará essa função para você automaticamente e ignorará o prompt. Os perfis que estão disponíveis para uso são determinados pela configuração do usuário no IAM Identity Center.

```
Using the account ID 123456789011  
There are 2 roles available to you.  
> ReadOnly  
   FullAccess
```

Use as teclas de seta para selecionar o perfil do IAM que deseja usar e pressione <ENTER>.

6. Especifique o [formato de saída padrão](#), a [Região da AWS padrão](#) para enviar comandos e forneça um [nome para o perfil](#) para poder fazer referência a ele entre todos os definidos no computador local. No exemplo a seguir, o usuário insere uma região padrão, um formato de saída padrão e o nome do perfil. Como alternativa, se você tiver uma configuração prévia, poderá pressionar <ENTER> para selecionar os valores padrão mostrados entre colchetes. O nome do perfil sugerido é o número de ID da conta seguido de um sublinhado e pelo nome da função.

```
CLI default client Region [None]: us-west-2<ENTER>  
CLI default output format [None]: json<ENTER>  
CLI profile name [123456789011_ReadOnly]: my-dev-profile<ENTER>
```


 Note

Se você especificar `default` como o nome do perfil, esse perfil será usado sempre que você executar um comando da AWS CLI e não especificar um nome de perfil.

7. Uma mensagem final descreve a configuração do perfil concluída.

To use this profile, specify the profile name using `--profile`, as shown:

```
aws s3 ls --profile my-dev-profile
```

8. Isso resulta na criação da seção do `sso-session` e do perfil nomeado no `~/.aws/config` com a seguinte aparência:

```
[profile my-dev-profile]  
sso_session = my-sso  
sso_account_id = 123456789011  
sso_role_name = readOnly  
region = us-west-2  
output = json  
  
[sso-session my-sso]  
sso_region = us-east-1  
sso_start_url = https://my-sso-portal.awsapps.com/start  
sso_registration_scopes = sso:account:access
```

Agora você pode usar essa `sso-session` e o perfil para solicitar credenciais atualizadas. É necessário usar o comando `aws sso login` para solicitar e recuperar as credenciais necessárias para executar comandos. Para obter instruções, consulte [Usar um perfil nomeado do Centro de Identidade do IAM](#).

Configurar somente sua seção de **sso-session** com o assistente **aws configure sso-session**

O comando `aws configure sso-session` atualiza somente as seções do `sso-session` no arquivo `~/.aws/config`. Esse comando pode ser usado para criar ou atualizar suas sessões. Isso é útil se você já tiver configurações existentes e quiser criar uma ou editar a configuração de `sso-session` existente.

Execute o comando `aws configure sso-session` e forneça o URL de início do IAM Identity Center e a região da AWS que hospeda o diretório do Identity Center.

```
$ aws configure sso-session
SSO session name: my-sso
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]: us-east-1
SSO registration scopes [None]: sso:account:access
```

Depois de inserir suas informações, uma mensagem descreve a configuração completa do perfil.

```
Completed configuring SSO session: my-sso
Run the following to login and refresh access token for this session:
```

```
aws sso login --sso-session my-sso
```

Note

Se você estiver conectado à `sso-session` que está atualizando, atualize seu token executando o comando `aws sso login`.

Configuração manual usando o arquivo **config**

A seção do `sso-session` do arquivo `config` é usada para agrupar variáveis de configuração para adquirir tokens de acesso do SSO, que podem então ser usados para adquirir credenciais da AWS. As seguintes configurações são usadas:

- (Obrigatório) [`sso\_start\_url`](#)
- (Obrigatório) [`sso\_region`](#)
- [`sso\_account\_id`](#)
- [`sso\_role\_name`](#)
- [`sso\_registration\_scopes`](#)

Defina uma seção de `sso-session` e associe-a a um perfil. `sso_region` e `sso_start_url` devem ser definidos na seção `sso-session`. Normalmente, `sso_account_id` e `sso_role_name` devem ser definidos na seção `profile` para que o SDK possa solicitar credenciais do SSO.

O exemplo a seguir configura o SDK para solicitar credenciais do SSO e é compatível com a atualização automática de tokens:

```
[profile dev]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = SampleRole

[sso-session my-sso]
sso_region = us-east-1
sso_start_url = https://my-sso-portal.awsapps.com/start
```

Isso também permite que as configurações de sso-session sejam reutilizadas em vários perfis:

```
[profile dev]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = SampleRole

[profile prod]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = SampleRole2

[sso-session my-sso]
sso_region = us-east-1
sso_start_url = https://my-sso-portal.awsapps.com/start
```

No entanto, sso_account_id e sso_role_name não são necessários para todos os cenários de configuração do token do SSO. Se a aplicação usa apenas serviços da AWS compatíveis com a autenticação do portador, as credenciais tradicionais da AWS não são necessárias. A autenticação do portador é um esquema de autenticação HTTP que usa tokens de segurança chamados tokens de portador. Nesse cenário, sso_account_id e sso_role_name não são obrigatórios. Consulte o guia individual do serviço da AWS para determinar se ele é compatível com a autorização do token do portador.

Além disso, os escopos de registro podem ser configurados como parte de uma sso-session. O escopo é um mecanismo no OAuth 2.0 para limitar o acesso de uma aplicação à conta de um usuário. Uma aplicação pode solicitar um ou mais escopos, e o token de acesso emitido para a aplicação será limitado aos escopos concedidos. Esses escopos definem as permissões solicitadas

para serem autorizadas para o cliente OIDC registrado e os tokens de acesso recuperados pelo cliente. O exemplo a seguir define `sso_registration_scopes` para fornecer acesso a fim de listar contas/perfis:

```
[sso-session my-sso]
sso_region = us-east-1
sso_start_url = https://my-sso-portal.awsapps.com/start
sso_registration_scopes = sso:account:access
```

O token de autenticação é armazenado em cache no disco sob o diretório `~/.aws/sso/cache` com um nome de arquivo baseado no nome da sessão.

Configuração herdada não atualizável para AWS IAM Identity Center

Este tópico descreve como configurar a AWS CLI para autenticar usuários com o AWS IAM Identity Center (Centro de Identidade do IAM) e receber credenciais para executar comandos da AWS CLI usando o método legado. Ao usar a configuração herdada não atualizável, você precisa atualizar o token de forma manual, pois ele expira periodicamente.

Ao usar o IAM Identity Center, é possível fazer login no Active Directory, um diretório integrado do IAM Identity Center, ou em [outro IdP conectado ao IAM Identity Center](#). É possível mapear essas credenciais para um perfil do AWS Identity and Access Management (IAM) onde é possível executar comandos da AWS CLI.

Independentemente do IdP que você usa, o IAM Identity Center abstrai essas distinções. Por exemplo, é possível conectar o Microsoft Azure AD conforme descrito no artigo de blog [The Next Evolution in IAM Identity Center](#) (O que há de mais novo no IAM Identity Center).

Note

Para obter informações sobre como usar o bearer auth, que não usa ID de conta e perfil, consulte [Configuração para usar a AWS CLI com o CodeCatalyst](#) no Guia do usuário do Amazon CodeCatalyst.

É possível configurar um ou mais dos [perfis nomeados](#) da AWS CLI para usar um perfil do IAM Identity Center herdado das seguintes maneiras:

- [Automaticamente](#), usando o comando `aws configure sso`.

- [Manualmente](#), editando o arquivo `config` que armazena os perfis nomeados.

Pré-requisitos

- Instale o AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#).
- É necessário primeiro ter acesso à autenticação do SSO no IAM Identity Center. Escolha um dos métodos a seguir para acessar as credenciais da AWS.

Não estabeleci acesso por meio do IAM Identity Center

Siga as instruções em [Conceitos básicos](#) no Guia do usuário do AWS IAM Identity Center. Esse processo ativa o Centro de Identidade do IAM, cria um usuário administrativo e adiciona um conjunto de permissões apropriado com privilégio mínimo.

Note

Para a Etapa 6: crie um conjunto de permissões que aplique permissões de privilégio mínimo. Recomendamos usar o conjunto de permissões predefinido `PowerUserAccess`, a menos que seu empregador tenha criado um conjunto de permissões personalizado para essa finalidade.

Saia do portal e faça login novamente para ver as Contas da AWS e as opções para `Administrator` ou `PowerUserAccess`. Selecione `PowerUserAccess` ao trabalhar com o SDK. Isso também ajuda você a encontrar detalhes sobre o acesso programático.

Eu já tenho acesso à AWS por meio de um provedor de identidade federado gerenciado pelo meu empregador (como Azure AD ou Okta)

Faça login na AWS por meio do portal do seu provedor de identidade. Se o seu administrador de nuvem concedeu permissões `PowerUserAccess` (de desenvolvedor) a você, serão exibidas as Contas da AWS às quais você tem acesso e seu conjunto de permissões. Ao lado do nome do seu conjunto de permissões, você vê opções para acessar as contas manual ou programaticamente usando esse conjunto de permissões.

Implementações personalizadas podem resultar em experiências diferentes, como nomes de conjuntos de permissões diferentes. Se não tiver certeza sobre qual conjunto de permissões usar, entre em contato com a equipe de TI para obter ajuda.

Eu já tenho acesso à AWS por meio do portal de acesso da AWS gerenciado pelo meu empregador

Faça login na AWS pelo portal de acesso da AWS. Se o seu administrador de nuvem concedeu permissões `PowerUserAccess` (de desenvolvedor) a você, serão exibidas as Contas da AWS às quais você tem acesso e seu conjunto de permissões. Ao lado do nome do seu conjunto de permissões, você vê opções para acessar as contas manual ou programaticamente usando esse conjunto de permissões.

Eu já tenho acesso à AWS por meio de um provedor de identidades federadas personalizado gerenciado pelo meu empregador

Entre em contato com a equipe de TI para obter ajuda.

Configuração automática para configuração herdada

Para configurar um perfil do Centro de Identidade do IAM para a AWS CLI

1. Execute o comando `aws configure sso` e forneça o URL de início do IAM Identity Center e a região da AWS que hospeda o diretório do Identity Center.

```
$ aws configure sso
SSO session name (Recommended):
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]:us-east-1
```

2. A AWS CLI tenta abrir seu navegador padrão e iniciar o processo de login para sua conta do IAM Identity Center.

SSO authorization page has automatically been opened in your default browser.
Follow the instructions in the browser to complete this authorization request.

Se a AWS CLI não puder abrir o navegador, a seguinte mensagem será exibida com instruções sobre como iniciar manualmente o processo de login.

Using a browser, open the following URL:

[**https://device.sso.us-west-2.amazonaws.com/**](https://device.sso.us-west-2.amazonaws.com/)

and enter the following code:

QCFK-N451

O IAM Identity Center usa o código para associar a sessão do IAM Identity Center à sessão atual da AWS CLI. A página do navegador do IAM Identity Center solicita que você insira suas credenciais do IAM Identity Center. Isso permite que a AWS CLI recupere e exiba as contas e os perfis da AWS que você está autorizado a usar com o IAM Identity Center.

- Depois, a AWS CLI exibe as contas da AWS disponíveis para uso. Se você estiver autorizado a usar apenas uma conta, a AWS CLI selecionará essa conta para você automaticamente e ignorará o prompt. As contas da AWS disponíveis para uso são determinadas pela configuração do usuário no IAM Identity Center.

```
There are 2 AWS accounts available to you.  
> DeveloperAccount, developer-account-admin@example.com (123456789011)  
   ProductionAccount, production-account-admin@example.com (123456789022)
```

Use as teclas de seta para selecionar a conta que deseja usar com esse perfil. O caractere ">" à esquerda aponta para a escolha atual. Pressione ENTER para fazer sua seleção.

- Depois, a AWS CLI confirma sua escolha de conta e exibe as funções do IAM que estão disponíveis para você na conta selecionada. Se a conta selecionada listar apenas uma função, a AWS CLI selecionará essa função para você automaticamente e ignorará o prompt. Os perfis que estão disponíveis para uso são determinados pela configuração do usuário no IAM Identity Center.

```
Using the account ID 123456789011  
There are 2 roles available to you.  
> ReadOnly  
   FullAccess
```

Use as teclas de seta para selecionar o perfil do IAM que você deseja usar com esse perfil e pressione <ENTER>.

- A AWS CLI confirma sua seleção de função.

```
Using the role name "ReadOnly"
```

- Termine a configuração do seu perfil especificando o formato de saída padrão, a Região da AWS padrão para enviar comandos e fornecendo um [nome para o perfil](#) para poder fazer referência a ele entre todos os definidos no computador local. No exemplo a seguir, o usuário insere uma região padrão, um formato de saída padrão e o nome do perfil. Você pode

pressionar <ENTER> para selecionar os valores padrão mostrados entre colchetes. O nome do perfil sugerido é o número de ID da conta seguido de um sublinhado e pelo nome da função.

```
CLI default client Region [None]: us-west-2<ENTER>
CLI default output format [None]: json<ENTER>
CLI profile name [123456789011_ReadOnly]: my-dev-profile<ENTER>
```

Note

Se você especificar `default` como o nome do perfil, esse perfil será usado sempre que você executar um comando da AWS CLI e não especificar um nome de perfil.

7. Uma mensagem final descreve a configuração do perfil concluída.

Para usar este perfil, especifique o nome do perfil usando `--profile`, como mostrado:

```
aws s3 ls --profile my-dev-profile
```

8. As entradas de exemplo anteriores resultariam em um perfil nomeado em `~/.aws/config` que se parece com o exemplo a seguir.

```
[profile my-dev-profile]
sso_start_url = https://my-sso-portal.awsapps.com/start
sso_region = us-east-1
sso_account_id = 123456789011
sso_role_name = readOnly
region = us-west-2
output = json
```

Neste ponto, você tem um perfil que pode usar para solicitar credenciais temporárias. Você deve usar o comando `aws sso login` para realmente solicitar e recuperar as credenciais temporárias necessárias para executar comandos. Para obter instruções, consulte [Usar um perfil nomeado do Centro de Identidade do IAM](#).

Configuração manual para configuração herdada

A atualização automática de tokens não é compatível usando a configuração herdada não atualizável. Recomendamos usar a configuração do token do SSO.

Para adicionar suporte do IAM Identity Center manualmente a um perfil nomeado, você deve adicionar as chaves e os valores a seguir à definição de perfil no arquivo `~/.aws/config` (Linux ou macOS) ou `%USERPROFILE%/.aws/config` (Windows).

- [`sso\_start\_url`](#)
- [`sso\_region`](#)
- [`sso\_account\_id`](#)
- [`sso\_role\_name`](#)

É possível incluir outras chaves e valores válidos no arquivo `.aws/config`, como [`region`](#), [`output`](#) ou [`s3`](#). Para evitar erros, não inclua valores relacionados à credencial, como [`role\_arn`](#) ou [`aws\_secret\_access\_key`](#).

Veja a seguir um exemplo de perfil do IAM Identity Center em `.aws/config`:

```
[profile my-sso-profile]
sso_start_url = https://my-sso-portal.awsapps.com/start
sso_region = us-west-2
sso_account_id = 111122223333
sso_role_name = SSOReadOnlyRole
region = us-west-2
output = json
```

Seu perfil para credenciais temporárias está completo.

Para executar comandos, é necessário primeiro usar o comando `aws sso login` para solicitar e recuperar suas credenciais temporárias. Para obter instruções, consulte a próxima seção, [Usar um perfil nomeado do Centro de Identidade do IAM](#). O token de autenticação é armazenado em cache no disco sob o diretório `~/.aws/sso/cache` com um nome de arquivo baseado no `sso_start_url`.

Usar um perfil nomeado do Centro de Identidade do IAM

Este tópico descreve como usar a AWS CLI para autenticar usuários com o AWS IAM Identity Center (Centro de Identidade do IAM) e obter credenciais para executar comandos da AWS CLI.

 Note

Se suas credenciais são temporárias ou são atualizadas automaticamente, depende de como você configurou seu perfil anteriormente.

Tópicos

- [Pré-requisitos](#)
- [Fazer login e obter credenciais](#)
- [Executar um comando com o perfil do Centro de Identidade do IAM](#)
- [Sair de sessões do IAM Identity Center](#)

Pré-requisitos

Você configurou um perfil do Centro de Identidade do IAM. Consulte [the section called “Configurar a atualização automática de tokens”](#) e [the section called “Configuração herdada não atualizável”](#) para obter mais informações.

Fazer login e obter credenciais

 Note

O processo de login pode solicitar que você permita que a AWS CLI acesse seus dados. Como a AWS CLI é criada sobre o SDK para Python, as mensagens de permissão podem conter variações do nome `botocore`.

Depois de configurar um perfil nomeado, é possível invocá-lo para solicitar credenciais da AWS. Antes de executar um comando de serviço da AWS CLI, primeiro é necessário recuperar e armazenar em cache um conjunto de credenciais. Para obter essas credenciais, execute o comando a seguir.

```
$ aws sso login --profile my-dev-profile
```

A AWS CLI abre seu navegador padrão e verifica seu login no IAM Identity Center.

```
SSO authorization page has automatically been opened in your default browser.
```

Follow the instructions in the browser to complete this authorization request.
Successfully logged into Start URL: <https://my-sso-portal.awsapps.com/start>

Se você não estiver conectado ao IAM Identity Center, forneça suas credenciais do IAM Identity Center.

Se a AWS CLI não conseguir abrir o navegador, você será solicitado a abri-lo e inserir o código especificado.

```
$ aws sso login --profile my-dev-profile
```

Using a browser, open the following URL:

<https://device.sso.us-west-2.amazonaws.com/>

and enter the following code:

QCFF-N451

A AWS CLI abre o navegador padrão (ou você abre manualmente o navegador de sua escolha) para a página especificada e você insere o código fornecido. Em seguida, a página da Web solicita suas credenciais do IAM Identity Center.

Suas credenciais de sessão do IAM Identity Center são armazenadas em cache. Se essas credenciais forem temporárias, elas incluirão um carimbo de data/hora de validade e, quando expirarem, a AWS CLI solicitará que você faça login novamente no IAM Identity Center.

Se suas credenciais do IAM Identity Center forem válidas, a AWS CLI as usará a fim de recuperar com segurança as credenciais da AWS para o perfil do IAM especificado no perfil.

Welcome, you have successfully signed-in to the AWS-CLI.

Também é possível especificar qual perfil de sso-session usar ao registrar usando o `--sso-session` parâmetro do comando `aws sso login`.

```
$ aws sso login --sso-session my-dev-session
```

Attempting to automatically open the SSO authorization page in your default browser.
If the browser does not open or you wish to use a different device to authorize this request, open the following URL:

<https://device.sso.us-west-2.amazonaws.com/>

and enter the following code:

QCCK-N451

Successfully logged into Start URL: <https://cli-reinvent.awsapps.com/start>

Executar um comando com o perfil do Centro de Identidade do IAM

Você pode usar essas credenciais para invocar um comando da AWS CLI com o perfil nomeado associado. O exemplo a seguir mostra que o comando foi executado sob uma função assumida que faz parte da conta especificada.

```
$ aws sts get-caller-identity --profile my-dev-profile
{
  "UserId": "AROAI2345678901234567:test-user@example.com",
  "Account": "123456789011",
  "Arn": "arn:aws:sts::123456789011:assumed-role/
  AWSPeregrine_readOnly_12321abc454d123/test-user@example.com"
}
```

Contanto que você faça login no IAM Identity Center e essas credenciais armazenadas em cache não estejam expiradas, a AWS CLI renovará automaticamente as credenciais expiradas da AWS, quando necessário. No entanto, se suas credenciais do IAM Identity Center expirarem, você deverá renová-las explicitamente fazendo login em sua conta do IAM Identity Center novamente.

```
$ aws s3 ls --profile my-sso-profile
Your short-term credentials have expired. Please sign-in to renew your credentials
SSO authorization page has automatically been opened in your default browser.
Follow the instructions in the browser to complete this authorization request.
```

Sair de sessões do IAM Identity Center

Quando terminar de usar os perfis do Centro de Identidade do IAM, você poderá optar por não fazer nada e deixar as credenciais temporárias da AWS e as credenciais do Centro de Identidade do IAM expirarem. No entanto, você também pode optar por executar o comando a seguir para excluir imediatamente todas as credenciais armazenadas em cache na pasta de cache de credenciais do SSO e todas as credenciais temporárias da AWS baseadas nas credenciais do IAM Identity Center. Isso torna essas credenciais indisponíveis para serem usadas em comandos futuros.

```
$ aws sso logout
Successfully signed out of all SSO profiles.
```

Se você quiser executar comandos mais tarde com um de seus perfis do Centro de Identidade do IAM, execute novamente o comando `aws sso login` (consulte a seção anterior) e especifique o perfil que deve ser usado.

Autenticar com credenciais de curto prazo

Recomendamos configurar o SDK ou a ferramenta para usar a [autenticação do IAM Identity Center](#) com opções de duração de sessão estendida. No entanto, você pode copiar e usar credenciais temporárias que estão disponíveis no portal de acesso da AWS. As novas credenciais precisarão ser copiadas quando essas expirarem. É possível usar as credenciais temporárias em um perfil ou usá-las como valores para propriedades do sistema e variáveis de ambiente.

1. [Faça login no portal de acesso da AWS](#).
2. Siga [estas instruções](#) para copiar as credenciais de perfil do IAM do portal de acesso da AWS.
 1. Na etapa 2 das instruções vinculadas, escolha a conta da AWS e o nome do perfil do IAM que concede acesso para suas necessidades de desenvolvimento. Esse perfil normalmente tem um nome como PowerUserAccess ou Developer.
 2. Na etapa 4, selecione a opção Adicionar um perfil ao seu arquivo de credenciais da AWS e copie o conteúdo.
3. Crie ou abra o arquivo `credentials` compartilhado. Esse arquivo é `~/.aws/credentials` em sistemas Linux e macOS e `%USERPROFILE%\aws\credentials` no Windows. Para obter mais informações, consulte [the section called “Configurações do arquivo de configuração e credenciais”](#).
4. Adicione o texto a seguir ao arquivo `credentials` compartilhado. Substitua os valores da amostra pelas credenciais que você copiou.

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
aws_session_token =
    IQoJb3JpZ2luX2I0oJb3JpZ2luX2I0oJb3JpZ2luX2I0oJb3JpZ2luX2I0oJb3JpZVERYLONGSTRINGEXAMPLE
```

5. Adicione a região e o formato padrão preferidos ao arquivo `config` compartilhado.

```
[default]
region=us-west-2
output=json
```

```
[profile user1]
region=us-east-1
output=text
```

Quando o SDK cria um cliente de serviço, ele acessa essas credenciais temporárias e as usa para cada solicitação. As configurações do perfil do IAM escolhidas na etapa 2a determinam [por quanto tempo as credenciais temporárias são válidas](#). A duração máxima é de doze horas.

Repita essas etapas sempre que suas credenciais expirarem.

Uso de perfis do IAM na AWS CLI

Um [perfil do AWS Identity and Access Management \(IAM\)](#) é uma ferramenta de autorização que permite que um usuário obtenha permissões adicionais (ou diferentes) ou obtenha permissões para executar ações em uma conta diferente da AWS.

Tópicos

- [Pré-requisitos](#)
- [Visão geral do uso de funções do IAM](#)
- [Configurar e usar uma função](#)
- [Usar autenticação multifator](#)
- [Funções entre contas e ID externo](#)
- [Especificar um nome de sessão de função para facilitar a auditoria](#)
- [Assumir a função com a identidade da web](#)
- [Limpar as credenciais em cache](#)

Pré-requisitos

Para executar os comandos `iam`, você precisa instalar e configurar a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#).

Visão geral do uso de funções do IAM

Você pode configurar a AWS Command Line Interface (AWS CLI) para usar uma função do IAM definindo um perfil para a função no arquivo `~/.aws/config`.

O exemplo a seguir mostra um perfil de função chamado `marketingadmin`. Se você executar comandos com `--profile marketingadmin` (ou especificá-los com a [variável de ambiente `AWS\_PROFILE`](#)), a AWS CLI usará as credenciais definidas em um perfil do `user1` separado para assumir a função com o nome de recurso da Amazon (ARN) `arn:aws:iam::123456789012:role/marketingadminrole`. É possível executar quaisquer operações que forem permitidas pelas permissões atribuídas a essa função.

```
[profile marketingadmin]
role_arn = arn:aws:iam::123456789012:role/marketingadminrole
source_profile = user1
```

Então, é possível especificar um `source_profile` que aponte para um perfil nomeado separado que contenha credenciais de usuário com permissão para usar o perfil. No exemplo anterior, o perfil `marketingadmin` usa as credenciais no perfil `user1`. Quando você especifica que um comando da AWS CLI deve usar o perfil `marketingadmin`, a AWS CLI procura automaticamente as credenciais para o perfil `user1` vinculado e as utiliza para solicitar credenciais temporárias para a função especificada do IAM. A CLI usa a operação [sts:AssumeRole](#) em segundo plano para fazer isso. Essas credenciais temporárias são então usadas para executar o comando da AWS CLI solicitado. A função especificada deve ter políticas de permissão do IAM associadas que permitam a execução do comando da AWS CLI solicitado.

Para executar um comando da AWS CLI em uma instância do Amazon Elastic Compute Cloud (Amazon EC2) ou um contêiner do Amazon Elastic Container Service (Amazon ECS), você pode usar uma função do IAM anexada ao perfil de instância ou ao contêiner. Se você não especificar nenhum perfil ou não definir variáveis de ambiente, essa função será usada diretamente. Isso permite que você evite armazenar chaves de acesso de longa duração em suas instâncias. Também é possível usar essas funções de instância ou contêiner apenas para obter credenciais para outra função. Para isso, use `credential_source` (em vez de `source_profile`) para especificar como localizar as credenciais. O atributo `credential_source` oferece suporte aos seguintes valores:

- `Environment`: recupera as credenciais de origem de variáveis de ambiente.
- `Ec2InstanceMetadata`: usa a função do IAM anexada ao perfil de instância do Amazon EC2.
- `EcsContainer`: usa a função do IAM anexada ao contêiner do Amazon ECS.

O exemplo a seguir mostra a mesma função `marketingadminrole` usada fazendo referência a um perfil de instância do Amazon EC2.

```
[profile marketingadmin]
role_arn = arn:aws:iam::123456789012:role/marketingadminrole
credential_source = Ec2InstanceMetadata
```

Ao invocar uma função, você tem opções adicionais que podem ser exigidas, como o uso de autenticação multifator e um ID externo (usado por empresas de terceiros para acessar os recursos de seus clientes). Também é possível especificar nomes de sessão de função exclusivos que possam ser auditados com mais facilidade nos logs do AWS CloudTrail.

Configurar e usar uma função

Ao executar comandos usando um perfil que especifica uma função do IAM, a AWS CLI usa as credenciais do perfil de origem para chamar AWS Security Token Service (AWS STS) e solicitar credenciais temporárias para a função especificada. O usuário no perfil de origem deve ter permissão para chamar `sts:assume-role` para a função no perfil especificado. A função deve ter uma relação de confiança que permita que o usuário no perfil de origem use a função. Geralmente, o processo de recuperar e depois usar credenciais temporárias para uma função é chamado de assumir a função.

É possível criar um perfil no IAM com as permissões que você deseja que os usuários assumam seguindo o procedimento em [Criação de um perfil para conceder permissões a um usuário do IAM](#) no Manual do usuário do AWS Identity and Access Management. Se a função e o usuário do perfil de origem estão na mesma conta, é possível inserir seu próprio ID de conta ao configurar a relação de confiança da função.

Depois de criar a função, modifique a relação de confiança para permitir que o usuário assumir .

O exemplo a seguir mostra uma política de confiança que pode ser associada a uma função. Essa política permite que o perfil seja assumido por qualquer usuário na conta 123456789012, se o administrador dessa conta conceder explicitamente a permissão `sts:AssumeRole` ao usuário.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:root"
      },
    },
  ],
}
```



```
    "Action": "sts:AssumeRole"
  }
]
}
```

A política de confiança não concede permissões. O administrador da conta deve delegar a permissão para assumir a função para usuários individuais, anexando uma política com as permissões apropriadas. O exemplo a seguir mostra uma política que pode ser associada a um usuário, permitindo que ele assuma apenas o perfil `marketingadminrole`. Para obter mais informações sobre como conceder a um usuário acesso para assumir uma função, consulte [Como conceder a um usuário permissão para alternar funções no IAM](#) no Manual do usuário do IAM.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::123456789012:role/marketingadminrole"
    }
  ]
}
```

O usuário não precisa ter permissões adicionais para executar os comandos da AWS CLI usando o perfil. Em vez disso, as permissões para executar o comando vêm das associadas à função. Você pode associar políticas à função para especificar quais ações podem ser executadas em quais recursos da AWS. Para obter mais informações sobre como associar permissões a um perfil (que funciona de maneira idêntica a um usuário), consulte [Alteração de permissões de um usuário do IAM](#) no Manual do usuário do IAM.

Agora que o perfil de função, as permissões de função, a relação de confiança de função e as permissões de usuário estão configurados corretamente, é possível usar a função na linha de comando invocando a opção `--profile`. Por exemplo, o indicado a seguir chama o comando `ls` do Amazon S3 usando as permissões anexadas à função `marketingadmin`, conforme definido no exemplo no início desse tópico.

```
$ aws s3 ls --profile marketingadmin
```

Para usar a função para várias chamadas, defina a variável de ambiente `AWS_PROFILE` para a sessão atual na linha de comando. Enquanto essa variável de ambiente é definida, você não precisa especificar a opção `--profile` em cada comando.

Linux ou macOS

```
$ export AWS_PROFILE=marketingadmin
```

Windows

```
C:\> setx AWS_PROFILE marketingadmin
```

Para obter mais informações sobre como configurar usuários e perfis, consulte [Usuários e grupos](#) e [Perfis](#) no Manual do usuário do IAM.

Usar autenticação multifator

Para segurança adicional, você pode exigir que os usuários forneçam uma chave única gerada a partir de um dispositivo de autenticação multifator (MFA), um dispositivo U2F ou um aplicativo móvel quando eles tentarem fazer uma chamada usando o perfil de função.

Primeiro, você pode optar por modificar a relação de confiança na função do IAM para exigir MFA. Isso impede que alguém usando a função sem antes autenticar usando MFA. Por exemplo, veja a linha `Condition` no exemplo a seguir. Essa política permite que o usuário chamado `anika` assuma o perfil ao qual a política está anexada, mas somente se ele se autenticar usando MFA.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:user/anika" },
      "Action": "sts:AssumeRole",
      "Condition": { "Bool": { "aws:multifactorAuthPresent": true } }
    }
  ]
}
```

Depois, adicione uma linha à função que especifica o perfil do usuário ARN do dispositivo MFA. As entradas do exemplo de arquivo `config` a seguir mostram dois perfis que usam as chaves de

acesso para o usuário anika solicitar credenciais temporárias para o perfil `cli-role`. O usuário anika tem permissão para assumir a função, concedida pela política de confiança da função.

```
[profile role-without-mfa]
region = us-west-2
role_arn= arn:aws:iam::128716708097:role/cli-role
source_profile=cli-user

[profile role-with-mfa]
region = us-west-2
role_arn= arn:aws:iam::128716708097:role/cli-role
source_profile = cli-user
mfa_serial = arn:aws:iam::128716708097:mfa/cli-user

[profile cli-user]
region = us-west-2
output = json
```

A configuração `mfa_serial` pode usar um ARN, conforme mostrado, ou o número de série de um token MFA de hardware.

O primeiro perfil, `role-without-mfa`, não exige MFA. No entanto, como a política de confiança do exemplo anterior anexada à função requer MFA, qualquer tentativa de executar um comando com esse perfil vai falhar.

```
$ aws iam list-users --profile role-without-mfa
```

```
An error occurred (AccessDenied) when calling the AssumeRole operation: Access denied
```

A segunda entrada do perfil, `role-with-mfa`, identifica um dispositivo MFA para usar. Quando o usuário tenta executar um comando da AWS CLI com esse perfil, a AWS CLI solicita que o usuário insira a senha de uso único (OTP) fornecida pelo dispositivo MFA. Se a autenticação de MFA for bem-sucedida, o comando executará a operação solicitada. A OTP não é exibida na tela.

```
$ aws iam list-users --profile role-with-mfa
Enter MFA code for arn:aws:iam::123456789012:mfa/cli-user:
{
  "Users": [
    {
      ...
```

Funções entre contas e ID externo

É possível permitir que os usuários do usem funções que pertençam a diferentes contas ao configurar a função como uma função entre contas. Durante a criação da função, defina o tipo de função como Another AWS account (Outra conta da AWS), conforme descrito em [Criar uma função para delegar permissões a um usuário do IAM](#). Como opção, selecione Require MFA (Exigir MFA). Require MFA (Exigir MFA) configura a condição apropriada na relação de confiança, conforme descrito em [Usar autenticação multifator](#).

Se usar um [ID externo](#) para fornecer mais controle sobre quem pode usar uma função em todas as contas, você também deverá adicionar o parâmetro `external_id` ao perfil da função. Normalmente, isso é usado somente quando a outra conta é controlada por alguém de fora da sua empresa ou organização.

```
[profile crossaccountrole]
role_arn = arn:aws:iam::234567890123:role/SomeRole
source_profile = default
mfa_serial = arn:aws:iam::123456789012:mfa/saanvi
external_id = 123456
```

Especificar um nome de sessão de função para facilitar a auditoria

Quando muitos indivíduos compartilham uma função, a auditoria torna-se um desafio. É recomendável associar cada operação invocada ao indivíduo que invocou a ação. No entanto, quando o indivíduo usa uma função, a assunção da função por ele é uma ação separada da invocação de uma operação, e é necessário correlacionar manualmente as duas.

É possível simplificar isso especificando nomes de sessão de função exclusivos quando os usuários assumem uma função. Para fazer isso, adicione um parâmetro `role_session_name` a cada perfil nomeado no arquivo `config` que especifica uma função. O valor `role_session_name` é transmitido para a operação `AssumeRole` e se torna parte do ARN da sessão da função. Ele também é incluído nos logs do AWS CloudTrail de todas as operações registradas.

Por exemplo, é possível criar um perfil baseado em função da maneira a seguir.

```
[profile namedsessionrole]
role_arn = arn:aws:iam::234567890123:role/SomeRole
source_profile = default
role_session_name = Session_Maria_Garcia
```

Isso faz com que a sessão da função tenha o ARN a seguir.

```
arn:aws:iam::234567890123:assumed-role/SomeRole/Session_Maria_Garcia
```

Além disso, todos os logs do AWS CloudTrail incluem o nome da sessão da função nas informações capturadas para cada operação.

Assumir a função com a identidade da web

É possível configurar um perfil para indicar que a AWS CLI deve assumir uma função usando a [federação de identidades da web e o Open ID Connect \(OIDC\)](#). Quando isso é especificado em um perfil, a AWS CLI faz automaticamente a chamada AWS STS AssumeRoleWithWebIdentity correspondente para você.

Note

Quando você especifica um perfil que usa uma função do IAM, a AWS CLI faz as chamadas apropriadas para recuperar credenciais temporárias. Essas credenciais são armazenadas em `~/.aws/cli/cache`. Os comandos subsequentes da AWS CLI que especificam o mesmo perfil usam as credenciais temporárias armazenadas em cache até que elas expirem. Nesse ponto, a AWS CLI atualiza automaticamente as credenciais.

Para recuperar e usar credenciais temporárias usando a federação de identidades da web, é possível especificar os valores de configuração a seguir em um perfil compartilhado.

role_arn

Especifica o ARN da função a assumir.

web_identity_token_file

Especifica o caminho para um arquivo que contém um token de acesso OAuth 2.0 ou token de ID OpenID Connect fornecido pelo provedor de identidade. A AWS CLI carrega esse arquivo e transmite seu conteúdo como o argumento `WebIdentityToken` da operação `AssumeRoleWithWebIdentity`.

role_session_name

Especifica um nome opcional aplicado a essa sessão `assume-role`.

Veja a seguir um exemplo da configuração mínima necessária para uma função de admissão com o perfil de identidade da web:

```
# In ~/.aws/config

[profile web-identity]
role_arn=arn:aws:iam:123456789012:role/RoleNameToAssume
web_identity_token_file=/path/to/a/token
```

Também é possível fornecer essa configuração usando [variáveis de ambiente](#).

AWS_ROLE_ARN

O ARN da função a ser assumida

AWS_WEB_IDENTITY_TOKEN_FILE

O caminho para o arquivo de token de identidade da web.

AWS_ROLE_SESSION_NAME

O nome aplicado a essa sessão assume-role.

Note

No momento, essas variáveis de ambiente se aplicam somente à função de admissão com o provedor de identidade da web. Elas não se aplicam à configuração geral do provedor de função.

Limpar as credenciais em cache

Quando você usa uma função, a AWS CLI armazena em cache as credenciais temporárias localmente até que elas expirem. Na próxima vez que você tentar usá-las, a AWS CLI tentará renová-las em seu nome.

Se as credenciais temporárias da função forem [revogadas](#), elas não serão renovadas automaticamente, e as tentativas de usá-las falharão. No entanto, é possível excluir o cache para forçar a AWS CLI a recuperar novas credenciais.

Linux ou macOS

```
$ rm -r ~/.aws/cli/cache
```

Windows

```
C:\> del /s /q %UserProfile%\aws\cli\cache
```

Autenticar com credenciais de usuário do IAM

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

Essa seção explica como definir as configurações básicas com um usuário do IAM. Isso inclui suas credenciais de segurança usando os arquivos `config` e `credentials`. Em vez disso, veja as instruções de configuração do AWS IAM Identity Center; consulte [the section called “Autenticação do IAM Identity Center”](#).

Tópicos

- [Etapa 1: criar o usuário do IAM](#)
- [Etapa 2: obter as chaves de acesso](#)
- [Configurar o AWS CLI](#)
 - [Usar o `aws configure`](#)
 - [Importar chaves de acesso por meio do arquivo `.CSV`](#)
 - [Editar diretamente os arquivos `config` e `credentials`](#)

Etapa 1: criar o usuário do IAM

Crie o usuário do IAM seguindo o procedimento de [Criação de usuários do IAM \(console\)](#) no Guia do usuário do IAM.

- Em Opções de permissão, escolha Anexar políticas diretamente para como você deseja atribuir permissões a esse usuário.

- A maioria dos tutoriais de “Conceitos básicos” do SDK usa o serviço Amazon S3 como exemplo. Para fornecer à aplicação acesso total ao Amazon S3, selecione a política AmazonS3FullAccess para anexar a esse usuário.

Etapa 2: obter as chaves de acesso

1. Faça login no AWS Management Console e abra o console do IAM em <https://console.aws.amazon.com/iam/>.
2. No painel de navegação do console do IAM, selecione Usuários e escolha o usuário **User name** que você criou anteriormente.
3. Na página do usuário, selecione a página Credenciais de segurança. Depois, em Chaves de acesso, selecione Criar chave de acesso.
4. Em Criar chave de acesso: etapa 1, escolha Command Line Interface (CLI).
5. Em Criar chave de acesso: etapa 2, insira uma tag opcional e selecione Próximo.
6. Em Criar chave de acesso: etapa 3, selecione Baixar arquivo .csv para salvar um arquivo .csv com a chave de acesso e a chave de acesso secreta do usuário do IAM. Você precisará dessas informações posteriormente.
7. Selecione Concluído.

Configurar o AWS CLI

Para uso geral, a AWS CLI precisa das seguintes informações:

- Access key ID (ID da chave de acesso)
- Secret access key (Chave de acesso secreta)
- Região da AWS
- Formato de saída

A AWS CLI armazena essas informações em um perfil (um conjunto de configurações) chamado default no arquivo `credentials`. Por padrão, as informações neste perfil são usadas quando você executa um comando da AWS CLI que não especifica explicitamente um perfil a ser usado. Para obter mais informações sobre o arquivo `credentials`, consulte [Configurações do arquivo de configuração e credenciais](#).

Para configurar a AWS CLI, use um dos seguintes procedimentos:

Tópicos

- [Usar o aws configure](#)
- [Importar chaves de acesso por meio do arquivo .CSV](#)
- [Editar diretamente os arquivos config e credentials](#)

Usar o **aws configure**

Para uso geral, o comando `aws configure` é a maneira mais rápida de configurar sua instalação da AWS CLI. Esse assistente de configuração solicita cada informação necessária para começar. A menos que especificado de outra forma usando a opção `--profile`, a AWS CLI armazena essas informações no perfil `default`.

O exemplo a seguir configura um perfil `default` usando valores de amostra. Substitua-os por seus próprios valores, conforme descrito nas seções a seguir.

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

O exemplo a seguir configura um perfil chamado `userprod` usando valores de amostra. Substitua-os por seus próprios valores, conforme descrito nas seções a seguir.

```
$ aws configure --profile userprod
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

Importar chaves de acesso por meio do arquivo .CSV

Em vez de usar o `aws configure` para inserir chaves de acesso, você pode importar o arquivo `.csv` que baixou depois de criar as chaves de acesso.

O arquivo `.csv` deve conter os cabeçalhos a seguir.

- Nome de usuário: essa coluna deve ser adicionada ao `.csv`. Isso é usado para criar o nome do perfil ao importar.

- Access key ID (ID da chave de acesso)
- Secret access key (Chave de acesso secreta)

Note

Durante a criação inicial das chaves de acesso, depois de fechar a caixa de diálogo Baixar arquivo .csv, você não poderá acessar a chave de acesso secreta. Se você precisar de um arquivo .csv, será necessário criá-lo por conta própria com os cabeçalhos necessários e as informações das chaves de acesso armazenadas. Se você não tiver acesso às informações das chaves de acesso, será necessário criar outras chaves de acesso.

Para importar o arquivo .csv, use o comando `aws configure import` com a opção `--csv` da seguinte forma:

```
$ aws configure import --csv file://credentials.csv
```

Para obter mais informações, consulte [aws_configure_import](#).

Editar diretamente os arquivos **config** e **credentials**

Para editar diretamente os arquivos `config` e `credentials`, faça o seguinte.

1. Crie ou abra o arquivo `credentials` da AWS compartilhado. Esse arquivo é `~/.aws/credentials` em sistemas Linux e macOS e `%USERPROFILE%\aws\credentials` no Windows. Para obter mais informações, consulte [the section called “Configurações do arquivo de configuração e credenciais”](#).
2. Adicione o texto a seguir ao arquivo `credentials` compartilhado. Substitua os valores de amostra no arquivo .csv que você baixou anteriormente e salve o arquivo.

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

Uso de credenciais para metadados de instância do Amazon EC2.

Quando você executa a AWS CLI em uma instância do Amazon Elastic Compute Cloud (Amazon EC2), é possível simplificar o fornecimento de credenciais aos seus comandos. Cada instância do Amazon EC2 contém metadados que a AWS CLI pode consultar diretamente por credenciais temporárias. Quando uma função do IAM é associada à instância, a AWS CLI recupera de forma automática e segura as credenciais dos metadados da instância.

Para desabilitar esse serviço, use a variável de ambiente [AWS_EC2_METADATA_DISABLED](#).

Tópicos

- [Pré-requisitos](#)
- [Configuração de um perfil para metadados do Amazon EC2](#)

Pré-requisitos

Para usar credenciais do Amazon EC2 com a AWS CLI, é necessário concluir o seguinte:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- Conhecer os arquivos de configuração e os perfis nomeados. Para obter mais informações, consulte [Configurações do arquivo de configuração e credenciais](#).
- Você criou uma função do AWS Identity and Access Management (IAM) que tem acesso aos recursos necessários e associou essa função à instância do Amazon EC2 ao iniciá-la. Para obter mais informações, consulte [Políticas do IAM para Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux e [Concessão de acesso a recursos da AWS para aplicações executadas em instâncias do Amazon EC2](#) no Manual do usuário do IAM.

Configuração de um perfil para metadados do Amazon EC2

Para especificar que você deseja usar as credenciais disponíveis no perfil de instância de hospedagem do Amazon EC2, use a sintaxe a seguir no perfil nomeado em seu arquivo de configuração. Veja as etapas a seguir para obter mais instruções.

```
[profile profilename]  
role_arn = arn:aws:iam::123456789012:role/rolename  
credential_source = Ec2InstanceMetadata
```

```
region = region
```

1. Crie um perfil em seu arquivo de configuração.

```
[profile profilename]
```

2. Adicione a função arn do IAM que tem acesso aos recursos necessários.

```
role_arn = arn:aws:iam::123456789012:role/rolename
```

3. Especifique Ec2InstanceMetadata como sua fonte de credencial.

```
credential_source = Ec2InstanceMetadata
```

4. Defina sua região.

```
region = region
```

Exemplo

O exemplo a seguir assume o perfil *marketingadminrole* e usa a região *us-west-2* em um perfil de instância do Amazon EC2 chamado *marketingadmin*.

```
[profile marketingadmin]  
role_arn = arn:aws:iam::123456789012:role/marketingadminrole  
credential_source = Ec2InstanceMetadata  
region = us-west-2
```

Credenciais de origem com um processo externo

Warning

Este tópico discute as credenciais de fornecimento de um processo externo. Isso poderá ser um risco de segurança se o comando para gerar as credenciais se tornar acessível por processos ou usuários não aprovados. Recomendamos que você use as alternativas seguras compatíveis fornecidas pela AWS CLI e pela AWS para reduzir o risco de comprometer suas credenciais. Certifique-se de proteger o arquivo config e todos os arquivos e ferramentas com suporte para impedir a divulgação.

Certifique-se de que sua ferramenta de credenciais personalizada não grave informações secretas em `StdErr` porque os SDKs e a AWS CLI podem capturar e registrar essas informações, possivelmente expondo-as a usuários não autorizados.

Se você tiver um método para gerar ou procurar credenciais não compatíveis diretamente com a AWS CLI, poderá configurar a AWS CLI para usá-lo definindo a configuração `credential_process` no arquivo `config`.

Por exemplo, você pode incluir uma entrada semelhante à seguinte no arquivo `config`.

```
[profile developer]
credential_process = /opt/bin/awscreds-custom --username helen
```

Sintaxe

Para criar essa string de uma forma compatível com qualquer sistema operacional, siga estas regras:

- Se o caminho ou o nome do arquivo contiver um espaço, coloque o caminho completo e o nome do arquivo entre aspas duplas (" "). O caminho e o nome do arquivo podem ter somente os caracteres: A–Z a–z 0–9 - _ . espaço
- Se um nome de parâmetro ou um valor de parâmetro tiver um espaço, coloque esse elemento entre aspas duplas (" "). Coloque somente o nome ou o valor entre aspas, não o par.
- Não inclua variáveis de ambiente nas strings. Por exemplo, não inclua `$HOME` ou `%USERPROFILE`.
- Não especifique a pasta base como `~`. É necessário especificar o caminho completo.

Exemplo para Windows

```
credential_process = "C:\Path\To\credentials.cmd" parameterWithoutSpaces "parameter with spaces"
```

Exemplo para Linux ou macOS

```
credential_process = "/Users/Dave/path/to/credentials.sh" parameterWithoutSpaces "parameter with spaces"
```

Saída esperada do programa de credenciais

A AWS CLI executa o comando conforme especificado no perfil e lê os dados de STDOUT. O comando especificado deverá gerar a saída JSON em STDOUT que corresponde à sintaxe a seguir.

```
{
  "Version": 1,
  "AccessKeyId": "an AWS access key",
  "SecretAccessKey": "your AWS secret access key",
  "SessionToken": "the AWS session token for temporary credentials",
  "Expiration": "ISO8601 timestamp when the credentials expire"
}
```

 Note

No momento da elaboração deste documento, a chave `Version` deve ser definida como 1. Isso pode aumentar ao longo do tempo conforme a estrutura evolui.

A chave `Expiration` é um timestamp no formato [ISO8601](#). Se a chave `Expiration` não estiver presente na saída da ferramenta, a CLI vai supor que as credenciais são em longo prazo que não são atualizadas. Caso contrário, as credenciais serão consideradas temporárias e serão atualizadas automaticamente com a nova execução do comando `credential_process` antes de expirarem.

 Note

A AWS CLI não armazena em cache as credenciais do processo como faz com credenciais `assume-role`. Se o armazenamento em cache for obrigatório, implemente-o no processo externo.

O processo externo pode retornar um código de retorno diferente de zero para indicar que ocorreu um erro ao recuperar as credenciais.

Usar a AWS CLI

Esta seção fornece informações sobre uso geral, recursos comuns e opções disponíveis na AWS Command Line Interface (AWS CLI), além do que está escrito na seção Configuração [the section called “Endpoints”](#). As instruções incluem como escrever um comando, estrutura básica, formatação, filtragem e localização do conteúdo de ajuda ou da documentação de um comando.

Para obter exemplos específicos do AWS service (Serviço da AWS), consulte [Exemplos de código](#) ou [a versão 2 do guia de referência da AWS CLI](#).

Note

Por padrão, a AWS CLI envia solicitações para serviços do AWS usando HTTPS na porta TCP 443. Para usar a AWS CLI com êxito, você deve ser capaz de fazer as conexões de saída na porta TCP 443.

Tópicos neste guia

- [Obter ajuda com a AWS CLI](#)
- [Estrutura do comando na AWS CLI](#)
- [Especificar valores de parâmetro para a AWS CLI](#)
- [Fazer a AWS CLI solicitar comandos](#)
- [Controlar a saída do comando da AWS CLI](#)
- [Códigos de retorno da AWS CLI](#)
- [Uso dos assistentes da AWS CLI](#)
- [Criar e usar aliases da AWS CLI](#)

Obter ajuda com a AWS CLI

Este tópico descreve como acessar o conteúdo da ajuda da AWS Command Line Interface (AWS CLI).

Tópicos

- [O comando de ajuda integrado da AWS CLI](#)

- [Guia de referência da AWS CLI](#)
- [Documentação de API](#)
- [Solucionar de problemas de erros](#)
- [Ajuda adicional](#)

O comando de ajuda integrado da AWS CLI

Você pode obter ajuda com qualquer comando ao usar a AWS Command Line Interface (AWS CLI). Para fazer isso, basta digitar `help` no final de um nome do comando.

Por exemplo, o comando a seguir exibe a ajuda para as opções gerais da AWS CLI e os comandos de nível superior disponíveis.

```
$ aws help
```

O comando a seguir exibe os comandos específicos do Amazon Elastic Compute Cloud (Amazon EC2) disponíveis.

```
$ aws ec2 help
```

O exemplo a seguir exibe ajuda detalhada para a operação `DescribeInstances` do Amazon EC2. A ajuda inclui descrições de seus parâmetros de entrada, filtros disponíveis, e o que é incluído como saída. Ela também inclui exemplos que mostram como digitar as variações comuns do comando.

```
$ aws ec2 describe-instances help
```

A ajuda para cada comando é dividida em seis seções:

Nome

O nome do comando.

```
NAME
    describe-instances -
```

Descrição

Uma descrição da operação da API que o comando invoca.

DESCRIPTION

Describes one or more of your instances.

If you specify one or more instance IDs, Amazon EC2 returns information for those instances. If you do not specify instance IDs, Amazon EC2 returns information for all relevant instances. If you specify an instance ID that is not valid, an error is returned. If you specify an instance that you do not own, it is not included in the returned results.

...

Resumo

A sintaxe básica para usar o comando e suas opções. Se uma opção estiver entre colchetes, significa que ela é opcional, tem um valor padrão ou tem uma opção alternativa que você pode usar.

SYNOPSIS

```
describe-instances
[--dry-run | --no-dry-run]
[--instance-ids <value>]
[--filters <value>]
[--cli-input-json <value>]
[--starting-token <value>]
[--page-size <value>]
[--max-items <value>]
[--generate-cli-skeleton]
```

Por exemplo, `describe-instances` tem um comportamento padrão que descreve todas as instâncias na conta e na região atuais da AWS. Se preferir, você poderá especificar uma lista de `instance-ids` para descrever uma ou mais instâncias. O `dry-run` é um sinalizador booliano opcional que não tem um valor. Para usar um sinalizador booliano, especifique um valor apresentado, nesse caso `--dry-run` ou `--no-dry-run`. Da mesma forma, `--generate-cli-skeleton` não tem um valor. Se houver condições no uso de uma opção, elas serão descritas na seção **OPTIONS** ou mostradas nos exemplos.

Opções

A descrição de cada uma das opções mostradas na sinopse.

OPTIONS

```
--dry-run | --no-dry-run (boolean)
    Checks whether you have the required permissions for the action,
    without actually making the request, and provides an error response.
    If you have the required permissions, the error response is DryRun-
    Operation . Otherwise, it is UnauthorizedOperation .

--instance-ids (list)
    One or more instance IDs.

    Default: Describes all your instances.

...
```

Exemplos

Os exemplos que mostram o uso do comando e suas opções. Se nenhum exemplo estiver disponível para um comando ou caso de uso que você precisa, solicite um usando o link de comentários nesta página ou na referência de comandos da AWS CLI na página de ajuda para o comando.

EXAMPLES

To describe an Amazon EC2 instance

Command:

```
aws ec2 describe-instances --instance-ids i-5203422c
```

To describe all instances with the instance type m1.small

Command:

```
aws ec2 describe-instances --filters "Name=instance-type,Values=m1.small"
```

To describe all instances with an Owner tag

Command:

```
aws ec2 describe-instances --filters "Name=tag-key,Values=Owner"
```

...

Saída

As descrições de cada um dos campos e os tipos de dados incluídos na resposta da AWS.

Para `describe-instances`, a saída é uma lista de objetos de reserva, cada um com vários campos e objetos que contêm informações sobre as instâncias associadas a ele. Essas informações são provenientes da [documentação da API para o tipo de dados de reserva](#) usada pelo Amazon EC2.

OUTPUT

```
Reservations -> (list)
  One or more reservations.

  (structure)
    Describes a reservation.

    ReservationId -> (string)
      The ID of the reservation.

    OwnerId -> (string)
      The ID of the AWS account that owns the reservation.

    RequesterId -> (string)
      The ID of the requester that launched the instances on your
      behalf (for example, AWS Management Console or Auto Scaling).

    Groups -> (list)
      One or more security groups.

      (structure)
        Describes a security group.

        GroupName -> (string)
          The name of the security group.

        GroupId -> (string)
          The ID of the security group.

    Instances -> (list)
      One or more instances.

      (structure)
        Describes an instance.

        InstanceId -> (string)
          The ID of the instance.
```

```

ImageId -> (string)
    The ID of the AMI used to launch the instance.

State -> (structure)
    The current state of the instance.

Code -> (integer)
    The low byte represents the state. The high byte
    is an opaque internal value and should be ignored.

...

```

Quando a AWS CLI renderiza a saída em JSON, ela se torna uma matriz de objetos de reserva, semelhante ao exemplo a seguir.

```

{
  "Reservations": [
    {
      "OwnerId": "012345678901",
      "ReservationId": "r-4c58f8a0",
      "Groups": [],
      "RequesterId": "012345678901",
      "Instances": [
        {
          "Monitoring": {
            "State": "disabled"
          },
          "PublicDnsName": "ec2-52-74-16-12.us-
west-2.compute.amazonaws.com",
          "State": {
            "Code": 16,
            "Name": "running"
          },
        },
        ...
      ]
    },
    ...
  ]
}

```

Cada objeto de reserva contém campos que descrevem a reserva e uma série de objetos de instância, cada um com seus próprios campos (por exemplo, `PublicDnsName`) e objetos (por exemplo, `State`) que os descrevem.

Usuários do Windows

Você pode usar pipe (|) na saída do comando de ajuda do comando `more` para visualizar o arquivo de ajuda uma página por vez. Pressione a barra de espaço ou PgDn para visualizar mais detalhes do documento ou **q** se quiser sair.

```
C:\> aws ec2 describe-instances help | more
```

Guia de referência da AWS CLI

Os arquivos de ajuda contêm links que não podem ser visualizados a partir da linha de comando nem é possível navegar até eles a partir dela. Você pode visualizar e interagir com esses links usando o e o [Guia de referência da AWS CLI versão 2](#) online. A referência também inclui o conteúdo da ajuda para todos os comandos da AWS CLI. As descrições são apresentadas para facilitar a navegação e a visualização em dispositivos móveis, tablets e telas de desktop.

Documentação de API

Todos os comandos na AWS CLI correspondem a solicitações feitas para uma API pública de serviço do AWS. Cada serviço com uma API pública tem uma referência da API que pode ser encontrada na página inicial do serviço no [site de documentação da AWS](#). O conteúdo de uma referência da API varia de acordo com a forma como a API é criada e qual protocolo é usado. Normalmente, uma referência da API contém informações detalhadas sobre as operações compatíveis com a API, os dados enviados para e do serviço e qualquer condição de erro que o serviço pode relatar.

Seções da documentação de API

- **Ações:** informações detalhadas sobre cada operação e seus parâmetros (incluindo restrições de tamanho ou conteúdo e valores padrão). Ela lista os erros que podem ocorrer nessa operação. Cada operação corresponde a um subcomando na AWS CLI.
- **Tipos de dados:** informações detalhadas sobre estruturas que um comando pode exigir como um parâmetro ou retornar em resposta a uma solicitação.
- **Parâmetros comuns:** informações detalhadas sobre os parâmetros que são compartilhados por toda a ação para o serviço.

- Erros comuns: informações detalhadas sobre erros que podem ser retornados por todas as operações de um serviço.

O nome e a disponibilidade de cada seção podem variar de acordo com o serviço.

CLIs específicas para o serviço

Alguns serviços têm uma CLI separado desde antes da criação de uma única AWS CLI que funciona com todos os serviços. Essas CLIs específicas do serviço têm documentação separada vinculada a partir da página de documentação do serviço. A documentação para CLIs específicas de serviço não se aplica à AWS CLI.

Solucionar de problemas de erros

Para obter ajuda para diagnosticar e corrigir erros da AWS CLI, consulte [Solucionar erros](#).

Ajuda adicional

Para obter ajuda adicional para problemas da AWS CLI, visite a [Comunidade da AWS CLI](#) no GitHub.

Estrutura do comando na AWS CLI

Este tópico aborda como o comando da AWS Command Line Interface (AWS CLI) é estruturado e como usar comandos de espera.

Tópicos

- [Estrutura do comando](#)
- [Comandos de espera](#)

Estrutura do comando

A AWS CLI usa uma estrutura em várias partes na linha de comando que deve ser especificada nesta ordem:

1. A chamada básica para o programa aws.

2. O comando de nível superior que normalmente corresponde a um serviço do AWS compatível com a AWS CLI.
3. O subcomando que especifica a operação a ser realizada.
4. As opções gerais da AWS CLI ou os parâmetros necessários para a operação. Você pode especificá-los em qualquer ordem, desde que siga as três primeiras partes. Se um parâmetro exclusivo for especificado várias vezes, apenas o último valor se aplicará.

```
$ aws <command> <subcommand> [options and parameters]
```

Parâmetros pode levar vários tipos de valores de entrada, como números, sequências de caracteres, listas, mapas e estruturas de JSON. O que é compatível depende do comando e do subcomando que você especificar.

Exemplos

Amazon S3

O exemplo a seguir lista todos os seus buckets do Amazon S3.

```
$ aws s3 ls
2018-12-11 17:08:50 my-bucket
2018-12-14 14:55:44 my-bucket2
```

Para obter mais informações sobre os comandos do Amazon S3, consulte [aws s3](#) na Referência de comandos da AWS CLI.

AWS CloudFormation

O exemplo de comando [create-change-set](#) a seguir altera o nome da pilha cloudformation para *my-change-set*.

```
$ aws cloudformation create-change-set --stack-name my-stack --change-set-name my-change-set
```

Para obter mais informações sobre os comandos AWS CloudFormation, consulte [aws cloudformation](#) na Referência de comandos da AWS CLI.

Comandos de espera

Alguns serviços da AWS contam com comandos `wait`. Qualquer comando que usa `aws wait` normalmente espera até que um comando seja concluído antes de passar para a próxima etapa. Isso é especialmente útil para comandos em várias partes ou scripts, pois você pode usar um comando `wait` para impedir o avanço para etapas subsequentes se o comando `wait` falhar.

A AWS CLI usa uma estrutura em várias partes na linha de comando para o comando `wait` que deve ser especificada nesta ordem:

1. A chamada básica para o programa `aws`.
2. O comando de nível superior que normalmente corresponde a um serviço do AWS compatível com a AWS CLI.
3. O comando `wait`.
4. O subcomando que especifica a operação a ser realizada.
5. As opções gerais da CLI ou os parâmetros necessários para a operação. Você pode especificá-los em qualquer ordem, desde que siga as três primeiras partes. Se um parâmetro exclusivo for especificado várias vezes, apenas o último valor se aplicará.

```
$ aws <command> wait <subcommand> [options and parameters]
```

Parâmetros pode levar vários tipos de valores de entrada, como números, sequências de caracteres, listas, mapas e estruturas de JSON. O que é compatível depende do comando e do subcomando que você especificar.

Note

Nem todos os serviços da AWS comportam comandos `wait`. Consulte o [AWS CLI versão 2](#) para ver se seu serviço comporta comandos `wait`.

Exemplos

AWS CloudFormation

Os exemplos de comando [wait change-set-create-complete](#) a seguir pausam e continuam somente depois que o comando pode confirmar que o conjunto de alterações *my-change-set* na pilha *my-stack* está pronto para ser executado.

```
$ aws cloudformation wait change-set-create-complete --stack-name my-stack --change-set-name my-change-set
```

Para obter mais informações sobre os comandos da AWS CloudFormation wait, consulte [wait](#) na Referência de comandos da AWS CLI.

AWS CodeDeploy

Os exemplos de comando [wait deployment-successful](#) pausam até a implantação de *d-A1B2C3111* ser concluída com êxito.

```
$ aws deploy wait deployment-successful --deployment-id d-A1B2C3111
```

Para obter mais informações sobre os comandos da AWS CodeDeploy wait, consulte [wait](#) na Referência de comandos da AWS CLI.

Especificar valores de parâmetro para a AWS CLI

Muitos parâmetros usados na AWS Command Line Interface (AWS CLI) são strings ou valores numéricos simples, como o nome do par de chaves *my-key-pair* no exemplo a seguir.

```
$ aws ec2 create-key-pair --key-name my-key-pair
```

A formatação entre os terminais pode variar. Por exemplo, a maioria dos terminais faz distinção entre letras maiúsculas e minúsculas, mas não é o caso do Powershell. Isso significa que os dois exemplos de comando a seguir produziram resultados diferentes para terminais com distinção entre letras maiúsculas e minúsculas, pois eles veem *MyFile*.txt* e *myfile*.txt* como parâmetros diferentes.

No entanto, o PowerShell processaria essas solicitações da mesma forma, já que vê *MyFile*.txt* e *myfile*.txt* como os mesmos parâmetros.

```
$ aws s3 cp . s3://my-bucket/path --include "MyFile*.txt"
```

```
$ aws s3 cp . s3://my-bucket/path --include "myfile*.txt"
```

Para obter mais informações sobre a indistinção de letras maiúsculas e minúsculas do PowerShell, consulte [about_Case-Sensitivity](#) na documentação do PowerShell.

Às vezes, é necessário usar aspas ou literais em strings que incluem caracteres especiais ou de espaço. As regras sobre essa formatação também podem variar entre os terminais. Para obter mais informações sobre como usar aspas em parâmetros complexos, consulte [Aspas com strings na AWS CLI](#).

Tópicos de parâmetros

- [Tipos comuns de parâmetros da AWS CLI](#)
- [Aspas com strings na AWS CLI](#)
- [Carregar os parâmetros da AWS CLI de um arquivo](#)
- [Esqueletos e arquivos de entrada da AWS CLI](#)
- [Usar sintaxe simplificada com a AWS CLI](#)

Tipos comuns de parâmetros da AWS CLI

Esta seção descreve alguns dos tipos de parâmetros comuns e o formato típico necessário.

Caso haja problemas com a formatação de um parâmetro para um comando específico, verifique a ajuda inserindo **help** após o nome do comando. A ajuda de cada subcomando inclui o nome e a descrição de uma opção. O tipo de parâmetro da opção está listado entre parênteses. Para obter mais informações sobre a exibição da ajuda, consulte [the section called “Obter ajuda”](#).

Os tipos de parâmetro incluem:

- [String](#)
- [a função de hora](#)
- [Listar](#)
- [Boolean](#)
- [Inteiro](#)
- [Binário/blob \(objeto grande binário\) e blob de streaming](#)
- [Mapa](#)
- [Documento](#)

String

Os parâmetros de string podem conter caracteres alfanuméricos, símbolos e espaço em branco no conjunto de caracteres [ASCII](#). Strings com espaço em branco devem ser incluídas entre aspas. Recomendamos que você não use símbolos nem espaços em branco diferentes do caractere de espaço padrão e observe as [regras de aspas](#) para evitar resultados inesperados.

Alguns parâmetros de sequência de caracteres pode aceitar dados binários a partir de um arquivo. Consulte [Arquivos binários](#) para ver um exemplo.

a função de hora

As marcas de tempo são formatadas de acordo com a norma [ISO 8601](#). São geralmente chamados de parâmetros “DateTime” ou “Date”.

```
$ aws ec2 describe-spot-price-history --start-time 2014-10-13T19:00:00Z
```

Os formatos aceitos incluem:

- *AAAA-MM-DDThh:mm:ss.sssTZD (UTC)*, por exemplo, 2014-10-01T20:30:00.000Z
- *AAAA-MM-DDThh:mm:ss.sssTZD (com deslocamento)*, por exemplo, 2014-10-01T12:30:00.000-08:00
- *AAAA-MM-DD*, por exemplo, 2014-10-01
- Tempo do Unix em segundos, por exemplo, 1412195400. Isso às vezes é conhecido como [Tempo epoch Unix](#) e representa o número de segundos desde a meia-noite, 1º de janeiro de 1970 UTC.

Por padrão, a AWS CLI versão 2 converte todos os valores DateTime da resposta no formato ISO 8601.

Você pode definir o formato do carimbo de data/hora usando a configuração do arquivo [cli_timestamp_format](#).

Listar

Uma ou mais strings separadas por espaços. Se qualquer um dos itens de string contiver espaços, você deverá colocar esse item entre aspas. Observe as [regras de aspas](#) de seu terminal para evitar resultados inesperados

```
$ aws ec2 describe-spot-price-history --instance-types m1.xlarge m1.medium
```

Boolean

Sinalizador binário que ativa ou desativa uma opção. Por exemplo, `ec2 describe-spot-price-history` tem um parâmetro booleano `--dry-run` que, quando especificado, valida a consulta com o serviço sem realmente executá-la.

```
$ aws ec2 describe-spot-price-history --dry-run
```

A saída indica se o comando foi bem formado. Esse comando também inclui uma versão `--no-dry-run` do parâmetro que você pode usar para indicar explicitamente que o comando deve ser executado normalmente. A inclusão não é necessária, pois esse é o comportamento padrão.

Inteiro

Um número inteiro não assinado.

```
$ aws ec2 describe-spot-price-history --max-items 5
```

Binário/blob (objeto grande binário) e blob de streaming

Na AWS CLI, é possível transmitir um valor binário como uma string diretamente na linha de comando. Há dois tipos de blobs:

- [Blob](#)
- [Blob de streaming](#)

Blob

Para transmitir um valor para um parâmetro com o tipo blob, é necessário especificar um caminho para um arquivo local que contenha os dados binários usando o prefixo `fileb://`. Arquivos referenciados usando o prefixo `fileb://` são sempre tratados como um binário bruto não codificado. O caminho especificado é interpretado como sendo relativo ao diretório de trabalho atual. Por exemplo, o parâmetro `--plaintext` para `aws kms encrypt` é um blob.

```
$ aws kms encrypt \
```

```
--key-id 1234abcd-12ab-34cd-56ef-1234567890ab \  
--plaintext file://ExamplePlaintextFile \  
--output text \  
--query CiphertextBlob | base64 \  
--decode > ExampleEncryptedFile
```

Note

Para compatibilidade com versões anteriores, é possível usar o prefixo `file://`. Há dois formatos usados com base na opção da linha de comando [cli_binary_format](#) ou `--cli-binary-format` da configuração do arquivo:

- Padrão para a AWS CLI versão 2. Se o valor da configuração for `base64`, os arquivos referenciados com o prefixo `file://` serão tratados como texto codificado em base64.
- Padrão para a AWS CLI versão 1. Se o valor da configuração for `raw-in-base64-out`, os arquivos referenciados usando o prefixo `file://` serão lidos como texto e, então, a AWS CLI tenta codificá-lo em binário.

Para obter mais informações, consulte a configuração do arquivo [cli_binary_format](#) ou a opção `--cli-binary-format` da linha de comando.

Blob de streaming

Blobs de streaming como `aws cloudsearchdomain upload-documents` não usam prefixos. Em vez disso, os parâmetros de blob de streaming são formatados usando o caminho direto do arquivo. O seguinte exemplo usa o caminho direto do arquivo `document-batch.json` para o comando `aws cloudsearchdomain upload-documents`:

```
$ aws cloudsearchdomain upload-documents \  
  --endpoint-url https://doc-my-domain.us-west-1.cloudsearch.amazonaws.com \  
  --content-type application/json \  
  --documents document-batch.json
```

Mapa

Um conjunto de pares de chave-valor especificado em JSON ou com a [sintaxe abreviada](#) da CLI. O exemplo de JSON a seguir lê um item de uma tabela do Amazon DynamoDB chamada `my-table` com

um parâmetro de mapa, `--key`. O parâmetro especifica a chave primária chamada `id` com um valor numérico de 1 em uma estrutura JSON aninhada.

Para uso mais avançado de JSON em uma linha de comando, considere usar um processador JSON de linha de comando, como `jq`, para criar strings JSON. Para obter mais informações sobre o `jq`, consulte o [repositório do jq](#) no GitHub.

```
$ aws dynamodb get-item --table-name my-table --key '{"id": {"N": "1"}}'

{
  "Item": {
    "name": {
      "S": "John"
    },
    "id": {
      "N": "1"
    }
  }
}
```

Documento

Note

A [sintaxe simplificada](#) não é compatível com tipos de documento.

Os tipos de documento são usados para enviar dados sem a necessidade de incorporar JSON dentro de strings. O tipo de documento permite que os serviços forneçam esquemas arbitrários para você usar tipos de dados mais flexíveis.

Isso permite enviar dados JSON sem precisar usar caracteres escape nos valores. Por exemplo, em vez de usar a seguinte entrada JSON com escape:

```
{"document": "{\"key\": true}"}
```

Você pode usar o seguinte tipo de documento:

```
{"document": {"key": true}}
```

Valores válidos para tipos de documento

Devido à natureza flexível dos tipos de documento, existem vários tipos de valor válidos. Entre os valores válidos estão os seguintes:

String

```
--option "value"
```

Número

```
--option 123  
--option 123.456
```

Boolean

```
--option true
```

Nulo

```
--option null
```

Array

```
--option ["value1", "value2", "value3"]  
--option ["value", 1, true, null, ["key1", 2.34], {"key2": "value2"}]
```

Objeto

```
--option {"key": "value"}  
--option {"key1": "value1", "key2": 123, "key3": true, "key4": null, "key5":  
["value3", "value4"], "key6": {"value5": "value6"}}
```

Aspas com strings na AWS CLI

Há duas formas principais de usar aspas simples e duplas na AWS CLI.

- [Uso de aspas em torno de strings que contêm espaços em branco](#)
- [Uso de aspas dentro de strings](#)

Uso de aspas em torno de strings que contêm espaços em branco

Os nomes dos parâmetros e valores são separados por espaços na linha de comando. Se um valor de string contiver um espaço incorporado, você deve fechar a string inteira com aspas para evitar que a AWS CLI interprete mal o espaço como um divisor entre o valor e o próximo nome de parâmetro. O tipo de aspas depende do sistema operacional em que você executa a AWS CLI.

Linux and macOS

Uso de aspas simples ' '

```
$ aws ec2 create-key-pair --key-name 'my key pair'
```

Para obter mais informações sobre como usar aspas, consulte a documentação do usuário para o seu shell preferido.

PowerShell

Aspas simples (recomendado)

As aspas simples ' ' são chamadas de strings verbatim. A string é passada ao comando exatamente como você a digita, o que significa que as variáveis do PowerShell não serão passadas.

```
PS C:\> aws ec2 create-key-pair --key-name 'my key pair'
```

Aspas duplas

As aspas duplas " " são chamadas de string expandable. As variáveis podem ser passadas em strings expansíveis.

```
PS C:\> aws ec2 create-key-pair --key-name "my key pair"
```

Para obter mais informações sobre como usar aspas, consulte [Sobre regras de aspas](#) na Documentação do Microsoft PowerShell.

Windows command prompt

Uso de aspas duplas " ".

```
C:\> aws ec2 create-key-pair --key-name "my key pair"
```


Opcionalmente, você pode separar o nome de parâmetro do valor com um sinal de igual =, em vez de um espaço. Isso geralmente é necessário apenas se o valor do parâmetro começa com um hífen.

```
$ aws ec2 delete-key-pair --key-name=-mykey
```

Uso de aspas dentro de strings

As strings podem conter aspas, e seu shell pode exigir aspas de escape para que funcionem corretamente. Um dos tipos de valor de parâmetro comuns é uma string JSON. Isso é complexo, pois inclui espaços e aspas duplas " " em torno de cada nome de elemento e valor na estrutura JSON. A maneira como insere os parâmetros formatados pelo JSON na linha de comando difere dependendo de seu sistema operacional.

Para uso mais avançado de JSON na linha de comando, considere usar um processador JSON de linha de comando, como `jq`, para criar strings JSON. Para obter mais informações sobre o `jq`, consulte o [repositório do jq](#) no GitHub.

Linux and macOS

Para que o Linux e o macOS interpretem strings literalmente, use aspas simples ' ' para delimitar estrutura de dados JSON, como no exemplo a seguir. Você não precisa adicionar sequências de escape para as aspas duplas incorporadas na string JSON, pois elas são tratadas literalmente. Como o JSON é delimitado por aspas simples, quaisquer aspas simples na string precisarão utilizar sequências de escape. Isso geralmente é feito usando-se uma barra invertida antes da aspas simples \ '.

```
$ aws ec2 run-instances \
  --image-id ami-12345678 \
  --block-device-mappings '[{"DeviceName":"/dev/sdb","Ebs":
{"VolumeSize":20,"DeleteOnTermination":false,"VolumeType":"standard"}}]'
```

Para obter mais informações sobre como usar aspas, consulte a documentação do usuário para o seu shell preferido.

PowerShell

Use aspas simples ' ' ou aspas duplas " ".

Aspas simples (recomendado)

As aspas simples ' ' são chamadas de strings verbatim. A string é passada ao comando exatamente como você a digita, o que significa que as variáveis do PowerShell não serão passadas.

Como as estruturas de dados JSON incluem aspas duplas, sugerimos utilizar aspas simples ' ' para delimitá-las. Se você usar aspas simples, não será necessário usar uma sequência de escape para as aspas duplas incorporadas na string JSON. No entanto, será necessário usar a sequência de escape para cada uma das aspas simples utilizando um apóstrofe ` dentro da estrutura JSON.

```
PS C:\> aws ec2 run-instances `
  --image-id ami-12345678 `
  --block-device-mappings '[{"DeviceName":"/dev/sdb","Ebs":
{"VolumeSize":20,"DeleteOnTermination":false,"VolumeType":"standard"}}]'
```

Aspas duplas

As aspas duplas " " são chamadas de string expandable. As variáveis podem ser passadas em strings expansíveis.

Se você usar aspas duplas, não será necessário usar uma sequência de escape para as aspas simples incorporadas na string JSON. No entanto, será necessário usar a sequência de escape para cada uma das aspas duplas utilizando um apóstrofe ` dentro da estrutura JSON, conforme mostrado no exemplo a seguir.

```
PS C:\> aws ec2 run-instances `
  --image-id ami-12345678 `
  --block-device-mappings "[{\"DeviceName`\":`\"/dev/sdb`\",`\"Ebs`\":
{`\"VolumeSize`\":20,`\"DeleteOnTermination`\":false,`\"VolumeType`\":`\"standard`\"}}]"
```

Para obter mais informações sobre como usar aspas, consulte [Sobre regras de aspas](#) na Documentação do Microsoft PowerShell.

Warning

Antes que o PowerShell envie um comando para a AWS CLI, ele determina se seu comando é interpretado usando o PowerShell típico ou regras de aspas da CommandLineToArgvW. Quando o PowerShell usa CommandLineToArgvW no processamento, é necessário escapar dos caracteres com barra invertida \.

Para obter mais informações sobre CommandLineToArgvW no PowerShell, consulte [O que há com o estranho tratamento de aspas e barras invertidas por CommandLineToArgVW](#) no Blog de desenvolvedores da Microsoft, [Todos delimitam argumentos de linha de comando da maneira errada](#) no Blog de documentação da Microsoft e [Função CommandLineToArgVW](#) na Documentação da Microsoft.

Aspas simples

As aspas simples ' ' são chamadas de strings verbatim. A string é passada ao comando exatamente como você a digita, o que significa que as variáveis do PowerShell não serão passadas. Caracteres de escape com barra invertida \.

```
PS C:\> aws ec2 run-instances `
  --image-id ami-12345678 `
  --block-device-mappings '[{"DeviceName\":\"/dev/sdb\", \"Ebs\":
{\"VolumeSize\":20, \"DeleteOnTermination\":false, \"VolumeType\":\"standard\"}}]`
```

Aspas duplas

As aspas duplas " " são chamadas de string expandable. As variáveis podem ser passadas em strings expansíveis. Para strings entre aspas duplas, é necessário escapar duas vezes usando \" para cada aspa, em vez de usar um único backtick. O backtick escapa da barra invertida e, em seguida, a barra invertida é usada como um caractere de escape para o processo CommandLineToArgvW.

```
PS C:\> aws ec2 run-instances `
  --image-id ami-12345678 `
  --block-device-mappings "[{\"DeviceName\": \"\"/dev/sdb\", \"Ebs\":
{\"VolumeSize\":20, \"DeleteOnTermination\":false, \"VolumeType\": `
\"standard\"}}]"
```

Blobs (recomendados)

Para ignorar as regras de aspas do PowerShell para entrada de dados JSON, use Blobs para passar seus dados JSON diretamente para a AWS CLI. Para obter mais informações sobre o Blobs, consulte [Blob](#).

Windows command prompt

O prompt de comando do Windows exige aspas duplas " " para delimitar a estrutura de dados JSON. Além disso, para evitar que o processador de comando interprete mal as aspas duplas

incorporadas no JSON, você também deve usar um caractere de escape (preceder com um caractere de barra invertida \) cada uma das aspas duplas " dentro da própria estrutura de dados JSON, como no exemplo a seguir.

```
C:\> aws ec2 run-instances ^  
    --image-id ami-12345678 ^  
    --block-device-mappings "[{\\"DeviceName\\": \"/dev/sdb\\", \\"Ebs\\":  
{\\"VolumeSize\\": 20, \\"DeleteOnTermination\\": false, \\"VolumeType\\": \\"standard\\"} }]"
```

Somente as aspas duplas mais externas não são de escape.

Carregar os parâmetros da AWS CLI de um arquivo

Alguns parâmetros esperam nomes de arquivos como argumentos, dos quais a AWS CLI carrega os dados. Outros parâmetros permitem que você especifique o valor do parâmetro como texto digitado na linha de comando ou lido de um arquivo. Independentemente se um arquivo for obrigatório ou opcional, você deverá codificá-lo corretamente para que a AWS CLI possa entendê-lo. A codificação do arquivo deve corresponder à localidade padrão do sistema de leitura. É possível determinar isso usando o método `locale.getpreferredencoding()` do Python.

Note

Por padrão, o Windows PowerShell gera texto como UTF-16, o que entra em conflito com a codificação UTF-8 usada por arquivos JSON e muitos sistemas Linux. Recomendamos que você use `-Encoding ascii` com os comandos `Out-File` do PowerShell para garantir que a AWS CLI possa ler o arquivo resultante.

Tópicos

- [Como carregar os parâmetros de um arquivo](#)
- [Arquivos binários](#)

Como carregar os parâmetros de um arquivo

Às vezes é conveniente carregar um valor de parâmetro de um arquivo, em vez de tentar digitá-lo como um valor de parâmetro de linha de comando, por exemplo, quando o parâmetro é uma string

JSON complexa. Para especificar um arquivo que contém o valor, especifique um URL de arquivo no formato a seguir.

```
file://complete/path/to/file
```

- Os dois primeiros caracteres de barra “/” fazem parte da especificação. Se o caminho exigido começar com “/”, o resultado será três caracteres de barra: `file:///folder/file`.
- O URL fornece o caminho para o arquivo que apresenta o conteúdo real do parâmetro.
- Ao usar arquivos com espaços ou caracteres especiais, siga as [regras de aspas e escape](#) para seu terminal.

Os caminhos do arquivo no exemplo a seguir são interpretados como sendo relativos ao diretório de trabalho atual.

Linux or macOS

```
// Read from a file in the current directory
$ aws ec2 describe-instances --filters file://filter.json

// Read from a file in /tmp
$ aws ec2 describe-instances --filters file:///tmp/filter.json

// Read from a file with a filename with whitespaces
$ aws ec2 describe-instances --filters 'file://filter content.json'
```

Windows command prompt

```
// Read from a file in C:\temp
C:\> aws ec2 describe-instances --filters file://C:\temp\filter.json

// Read from a file with a filename with whitespaces
C:\> aws ec2 describe-instances --filters "file://C:\temp\filter content.json"
```

A opção de prefixo `file://` é compatível com as expansões de estilo Unix, inclusive `~/`, `./` e `../`. No Windows, a expressão `~/` se expande para o seu diretório de usuário, armazenado na variável de ambiente `%USERPROFILE%`. Por exemplo, no Windows 10, haveria normalmente um diretório de usuário em `C:\Users\UserName\`.

Ainda é necessário inserir um caractere de escape nos documentos JSON que são incorporados como o valor de outro documento JSON.

```
$ aws sqs create-queue --queue-name my-queue --attributes file://attributes.json
```

attributes.json

```
{
  "RedrivePolicy": "{\\"deadLetterTargetArn\\":\\"arn:aws:sqs:us-west-2:0123456789012:deadletter\\", \\"maxReceiveCount\\":\\"5\\"}"
}
```

Arquivos binários

Para comandos que incluem dados binários como um parâmetro, especifique que os dados são conteúdo binário usando o prefixo `fileb://`. Comandos que aceitam dados binários incluem:

- **aws ec2 run-instances**: parâmetro `--user-data`.
- **aws s3api put-object**: parâmetro `--sse-customer-key`.
- **aws kms decrypt**: parâmetro `--ciphertext-blob`.

O exemplo a seguir gera uma chave binária AES de 256 bits usando uma ferramenta de linha de comando do Linux e a fornece ao Amazon S3 para criptografar o arquivo enviado no lado do servidor.

```
$ dd if=/dev/urandom bs=1 count=32 > sse.key
32+0 records in
32+0 records out
32 bytes (32 B) copied, 0.000164441 s, 195 kB/s
$ aws s3api put-object \
  --bucket my-bucket \
  --key test.txt \
  --body test.txt \
  --sse-customer-key fileb://sse.key \
  --sse-customer-algorithm AES256
{
  "SSECustomerKeyMD5": "iVg8oWa8sy714+FjtesrJg==",
  "SSECustomerAlgorithm": "AES256",
  "ETag": "\"a6118e84b76cf98bf04bbe14b6045c6c\""
}
```

```
}
```

Para outro exemplo com referência a um arquivo que contém parâmetros formatados pelo JSON, consulte [Anexar uma política gerenciada do IAM a um usuário](#).

Esqueletos e arquivos de entrada da AWS CLI

A maioria dos comandos da AWS CLI aceita todas as entradas de parâmetros de um arquivo. Esses modelos podem ser gerados usando a opção `generate-cli-skeleton`.

Tópicos

- [Sobre esqueletos e arquivos de entrada da AWS CLI](#)
- [Gerar um esqueleto de comando](#)

Sobre esqueletos e arquivos de entrada da AWS CLI

A maioria dos comandos da AWS Command Line Interface (AWS CLI) aceita todas as entradas de parâmetros usando os parâmetros `--cli-input-json` e `--cli-input-yaml`.

Felizmente, esses mesmos comandos fornecem o parâmetro `--generate-cli-skeleton` para gerar um arquivo tanto no formato JSON ou YAML com todos os parâmetros que você pode editar e preencher. Depois, você pode executar o comando com o parâmetro relevante `--cli-input-json` ou `--cli-input-yaml` e apontar para o arquivo preenchido.

Important

Vários comandos da AWS CLI não são mapeados diretamente para as operações individuais de API da AWS, como os comandos [aws s3](#). Esses comandos não são compatíveis com os parâmetros `--generate-cli-skeleton` ou `--cli-input-json` e `--cli-input-yaml` descritos neste tópico. Se você souber se um comando específico é compatível com esses parâmetros, execute o comando a seguir, substituindo os nomes do *serviço* e do *comando* pelos quais você está interessado.

```
$ aws service command help
```

A saída inclui uma seção *Synopsis* que mostra os parâmetros compatíveis com o comando especificado.

```
$ aws iam list-users help
...
SYNOPSIS
    list-users
    ...
    [--cli-input-json | --cli-input-yaml]
    ...
    [--generate-cli-skeleton <value>]
...
```

O parâmetro `--generate-cli-skeleton` faz com que o comando não seja executado, mas, em vez disso, gere e exibe um modelo de parâmetro que você pode personalizar e usar como entrada em um comando posterior. O modelo gerado inclui todos os parâmetros compatíveis com o comando.

O parâmetro `--generate-cli-skeleton` aceita um dos seguintes valores:

- `input`: o modelo gerado inclui todos os parâmetros de entrada formatados como JSON. Este é o valor padrão.
- `yaml-input`: o modelo gerado inclui todos os parâmetros de entrada formatados como YAML.
- `output`: o modelo gerado inclui todos os parâmetros de saída formatados como JSON. No momento, não é possível solicitar os parâmetros de saída como YAML.

Como a AWS CLI é essencialmente um “wrapper” em torno da API de serviço, o arquivo de esqueleto espera que você faça referência a todos os parâmetros pelo nome do parâmetro da API subjacente. Isso provavelmente é diferente do nome do parâmetro da AWS CLI. Por exemplo, um parâmetro da AWS CLI chamado `user-name` pode ser mapeado para o parâmetro da API do serviço da AWS chamado `UserName` (observe a capitalização alterada e o traço ausente). Recomendamos usar a opção `--generate-cli-skeleton` para gerar o modelo com os nomes de parâmetro “corretos” a fim de evitar erros. Você também pode fazer referência ao Guia de referência da API do serviço para ver os nomes de parâmetro esperados. É possível excluir quaisquer parâmetros do modelo que não sejam necessários e para os quais você não deseja fornecer um valor.

Por exemplo, se você executar o seguinte comando, ele gerará o modelo de parâmetro para o comando `run-instances` do Amazon Elastic Compute Cloud (Amazon EC2).

JSON

O exemplo a seguir mostra como gerar um modelo formatado em JSON usando o valor padrão (input) para o `--generate-cli-skeleton` parâmetro.

```
$ aws ec2 run-instances --generate-cli-skeleton
```

```
{
  "DryRun": true,
  "ImageId": "",
  "MinCount": 0,
  "MaxCount": 0,
  "KeyName": "",
  "SecurityGroups": [
    ""
  ],
  "SecurityGroupIds": [
    ""
  ],
  "UserData": "",
  "InstanceType": "",
  "Placement": {
    "AvailabilityZone": "",
    "GroupName": "",
    "Tenancy": ""
  },
  "KernelId": "",
  "RamdiskId": "",
  "BlockDeviceMappings": [
    {
      "VirtualName": "",
      "DeviceName": "",
      "Ebs": {
        "SnapshotId": "",
        "VolumeSize": 0,
        "DeleteOnTermination": true,
        "VolumeType": "",
        "Iops": 0,
        "Encrypted": true
      },
      "NoDevice": ""
    }
  ],
}
```

```

    "Monitoring": {
        "Enabled": true
    },
    "SubnetId": "",
    "DisableApiTermination": true,
    "InstanceInitiatedShutdownBehavior": "",
    "PrivateIpAddress": "",
    "ClientToken": "",
    "AdditionalInfo": "",
    "NetworkInterfaces": [
        {
            "NetworkInterfaceId": "",
            "DeviceIndex": 0,
            "SubnetId": "",
            "Description": "",
            "PrivateIpAddress": "",
            "Groups": [
                ""
            ],
            "DeleteOnTermination": true,
            "PrivateIpAddresses": [
                {
                    "PrivateIpAddress": "",
                    "Primary": true
                }
            ],
            "SecondaryPrivateIpAddressCount": 0,
            "AssociatePublicIpAddress": true
        }
    ],
    "IamInstanceProfile": {
        "Arn": "",
        "Name": ""
    },
    "EbsOptimized": true
}

```

YAML

O exemplo a seguir mostra como gerar um modelo formatado em YAML usando o valor `yaml-input` para o parâmetro `--generate-cli-skeleton`.

```
$ aws ec2 run-instances --generate-cli-skeleton yaml-input
```

```

BlockDeviceMappings: # The block device mapping entries.
- DeviceName: '' # The device name (for example, /dev/sdh or xvdh).
  VirtualName: '' # The virtual device name (ephemeralN).
  Ebs: # Parameters used to automatically set up Amazon EBS volumes when the
instance is launched.
    DeleteOnTermination: true # Indicates whether the EBS volume is deleted on
instance termination.
    Iops: 0 # The number of I/O operations per second (IOPS) that the volume
supports.
    SnapshotId: '' # The ID of the snapshot.
    VolumeSize: 0 # The size of the volume, in GiB.
    VolumeType: st1 # The volume type. Valid values are: standard, io1, gp2, sc1,
st1.
    Encrypted: true # Indicates whether the encryption state of an EBS volume is
changed while being restored from a backing snapshot.
    KmsKeyId: '' # Identifier (key ID, key alias, ID ARN, or alias ARN) for a
customer managed KMS key under which the EBS volume is encrypted.
    NoDevice: '' # Suppresses the specified device included in the block device
mapping of the AMI.
ImageId: '' # The ID of the AMI.
InstanceType: c4.4xlarge # The instance type. Valid values are: t1.micro, t2.nano,
t2.micro, t2.small, t2.medium, t2.large, t2.xlarge, t2.2xlarge, t3.nano, t3.micro,
t3.small, t3.medium, t3.large, t3.xlarge, t3.2xlarge, t3a.nano, t3a.micro,
t3a.small, t3a.medium, t3a.large, t3a.xlarge, t3a.2xlarge, m1.small, m1.medium,
m1.large, m1.xlarge, m3.medium, m3.large, m3.xlarge, m3.2xlarge, m4.large,
m4.xlarge, m4.2xlarge, m4.4xlarge, m4.10xlarge, m4.16xlarge, m2.xlarge, m2.2xlarge,
m2.4xlarge, cr1.8xlarge, r3.large, r3.xlarge, r3.2xlarge, r3.4xlarge, r3.8xlarge,
r4.large, r4.xlarge, r4.2xlarge, r4.4xlarge, r4.8xlarge, r4.16xlarge, r5.large,
r5.xlarge, r5.2xlarge, r5.4xlarge, r5.8xlarge, r5.12xlarge, r5.16xlarge,
r5.24xlarge, r5.metal, r5a.large, r5a.xlarge, r5a.2xlarge, r5a.4xlarge,
r5a.8xlarge, r5a.12xlarge, r5a.16xlarge, r5a.24xlarge, r5d.large, r5d.xlarge,
r5d.2xlarge, r5d.4xlarge, r5d.8xlarge, r5d.12xlarge, r5d.16xlarge, r5d.24xlarge,
r5d.metal, r5ad.large, r5ad.xlarge, r5ad.2xlarge, r5ad.4xlarge, r5ad.8xlarge,
r5ad.12xlarge, r5ad.16xlarge, r5ad.24xlarge, x1.16xlarge, x1.32xlarge, x1e.xlarge,
x1e.2xlarge, x1e.4xlarge, x1e.8xlarge, x1e.16xlarge, x1e.32xlarge, i2.xlarge,
i2.2xlarge, i2.4xlarge, i2.8xlarge, i3.large, i3.xlarge, i3.2xlarge, i3.4xlarge,
i3.8xlarge, i3.16xlarge, i3.metal, i3en.large, i3en.xlarge, i3en.2xlarge,
i3en.3xlarge, i3en.6xlarge, i3en.12xlarge, i3en.24xlarge, i3en.metal, hi1.4xlarge,
hs1.8xlarge, c1.medium, c1.xlarge, c3.large, c3.xlarge, c3.2xlarge, c3.4xlarge,
c3.8xlarge, c4.large, c4.xlarge, c4.2xlarge, c4.4xlarge, c4.8xlarge, c5.large,
c5.xlarge, c5.2xlarge, c5.4xlarge, c5.9xlarge, c5.12xlarge, c5.18xlarge,
c5.24xlarge, c5.metal, c5d.large, c5d.xlarge, c5d.2xlarge, c5d.4xlarge,
c5d.9xlarge, c5d.18xlarge, c5n.large, c5n.xlarge, c5n.2xlarge, c5n.4xlarge,

```

c5n.9xlarge, c5n.18xlarge, cc1.4xlarge, cc2.8xlarge, g2.2xlarge, g2.8xlarge, g3.4xlarge, g3.8xlarge, g3.16xlarge, g3s.xlarge, g4dn.xlarge, g4dn.2xlarge, g4dn.4xlarge, g4dn.8xlarge, g4dn.12xlarge, g4dn.16xlarge, cg1.4xlarge, p2.xlarge, p2.8xlarge, p2.16xlarge, p3.2xlarge, p3.8xlarge, p3.16xlarge, p3dn.24xlarge, d2.xlarge, d2.2xlarge, d2.4xlarge, d2.8xlarge, f1.2xlarge, f1.4xlarge, f1.16xlarge, m5.large, m5.xlarge, m5.2xlarge, m5.4xlarge, m5.8xlarge, m5.12xlarge, m5.16xlarge, m5.24xlarge, m5.metal, m5a.large, m5a.xlarge, m5a.2xlarge, m5a.4xlarge, m5a.8xlarge, m5a.12xlarge, m5a.16xlarge, m5a.24xlarge, m5d.large, m5d.xlarge, m5d.2xlarge, m5d.4xlarge, m5d.8xlarge, m5d.12xlarge, m5d.16xlarge, m5d.24xlarge, m5d.metal, m5ad.large, m5ad.xlarge, m5ad.2xlarge, m5ad.4xlarge, m5ad.8xlarge, m5ad.12xlarge, m5ad.16xlarge, m5ad.24xlarge, h1.2xlarge, h1.4xlarge, h1.8xlarge, h1.16xlarge, z1d.large, z1d.xlarge, z1d.2xlarge, z1d.3xlarge, z1d.6xlarge, z1d.12xlarge, z1d.metal, u-6tb1.metal, u-9tb1.metal, u-12tb1.metal, u-18tb1.metal, u-24tb1.metal, a1.medium, a1.large, a1.xlarge, a1.2xlarge, a1.4xlarge, a1.metal, m5dn.large, m5dn.xlarge, m5dn.2xlarge, m5dn.4xlarge, m5dn.8xlarge, m5dn.12xlarge, m5dn.16xlarge, m5dn.24xlarge, m5n.large, m5n.xlarge, m5n.2xlarge, m5n.4xlarge, m5n.8xlarge, m5n.12xlarge, m5n.16xlarge, m5n.24xlarge, r5dn.large, r5dn.xlarge, r5dn.2xlarge, r5dn.4xlarge, r5dn.8xlarge, r5dn.12xlarge, r5dn.16xlarge, r5dn.24xlarge, r5n.large, r5n.xlarge, r5n.2xlarge, r5n.4xlarge, r5n.8xlarge, r5n.12xlarge, r5n.16xlarge, r5n.24xlarge.

Ipv6AddressCount: 0 # [EC2-VPC] The number of IPv6 addresses to associate with the primary network interface.

Ipv6Addresses: # [EC2-VPC] The IPv6 addresses from the range of the subnet to associate with the primary network interface.

- Ipv6Address: '' # The IPv6 address.

KernelId: '' # The ID of the kernel.

KeyName: '' # The name of the key pair.

MaxCount: 0 # [REQUIRED] The maximum number of instances to launch.

MinCount: 0 # [REQUIRED] The minimum number of instances to launch.

Monitoring: # Specifies whether detailed monitoring is enabled for the instance.

Enabled: true # [REQUIRED] Indicates whether detailed monitoring is enabled.

Placement: # The placement for the instance.

AvailabilityZone: '' # The Availability Zone of the instance.

Affinity: '' # The affinity setting for the instance on the Dedicated Host.

GroupName: '' # The name of the placement group the instance is in.

PartitionNumber: 0 # The number of the partition the instance is in.

HostId: '' # The ID of the Dedicated Host on which the instance resides.

Tenancy: dedicated # The tenancy of the instance (if the instance is running in a VPC). Valid values are: default, dedicated, host.

SpreadDomain: '' # Reserved for future use.

RamdiskId: '' # The ID of the RAM disk to select.

SecurityGroupIds: # The IDs of the security groups.

- ''

SecurityGroups: # [default VPC] The names of the security groups.

```

- ''
SubnetId: '' # [EC2-VPC] The ID of the subnet to launch the instance into.
UserData: '' # The user data to make available to the instance.
AdditionalInfo: '' # Reserved.
ClientToken: '' # Unique, case-sensitive identifier you provide to ensure the
    idempotency of the request.
DisableApiTermination: true # If you set this parameter to true, you can't terminate
    the instance using the Amazon EC2 console, CLI, or API; otherwise, you can.
DryRun: true # Checks whether you have the required permissions for the action,
    without actually making the request, and provides an error response.
EbsOptimized: true # Indicates whether the instance is optimized for Amazon EBS I/O.
IamInstanceProfile: # The IAM instance profile.
    Arn: '' # The Amazon Resource Name (ARN) of the instance profile.
    Name: '' # The name of the instance profile.
InstanceInitiatedShutdownBehavior: stop # Indicates whether an instance stops or
    terminates when you initiate shutdown from the instance (using the operating system
    command for system shutdown). Valid values are: stop, terminate.
NetworkInterfaces: # The network interfaces to associate with the instance.
- AssociatePublicIpAddress: true # Indicates whether to assign a public IPv4
    address to an instance you launch in a VPC.
    DeleteOnTermination: true # If set to true, the interface is deleted when the
    instance is terminated.
    Description: '' # The description of the network interface.
    DeviceIndex: 0 # The position of the network interface in the attachment order.
    Groups: # The IDs of the security groups for the network interface.
- ''
    Ipv6AddressCount: 0 # A number of IPv6 addresses to assign to the network
    interface.
    Ipv6Addresses: # One or more IPv6 addresses to assign to the network interface.
    - Ipv6Address: '' # The IPv6 address.
    NetworkInterfaceId: '' # The ID of the network interface.
    PrivateIpAddress: '' # The private IPv4 address of the network interface.
    PrivateIpAddresses: # One or more private IPv4 addresses to assign to the network
    interface.
    - Primary: true # Indicates whether the private IPv4 address is the primary
    private IPv4 address.
        PrivateIpAddress: '' # The private IPv4 addresses.
    SecondaryPrivateIpAddressCount: 0 # The number of secondary private IPv4
    addresses.
    SubnetId: '' # The ID of the subnet associated with the network interface.
    InterfaceType: '' # The type of network interface.
PrivateIpAddress: '' # [EC2-VPC] The primary IPv4 address.
ElasticGpuSpecification: # An elastic GPU to associate with the instance.
- Type: '' # [REQUIRED] The type of Elastic Graphics accelerator.

```

```

ElasticInferenceAccelerators: # An elastic inference accelerator to associate with
the instance.
- Type: '' # [REQUIRED] The type of elastic inference accelerator.
TagSpecifications: # The tags to apply to the resources during launch.
- ResourceType: network-interface # The type of resource to tag. Valid values
are: client-vpn-endpoint, customer-gateway, dedicated-host, dhcp-options, elastic-
ip, fleet, fpga-image, host-reservation, image, instance, internet-gateway,
launch-template, natgateway, network-acl, network-interface, reserved-instances,
route-table, security-group, snapshot, spot-instances-request, subnet, traffic-
mirror-filter, traffic-mirror-session, traffic-mirror-target, transit-gateway,
transit-gateway-attachment, transit-gateway-route-table, volume, vpc, vpc-peering-
connection, vpn-connection, vpn-gateway.
  Tags: # The tags to apply to the resource.
    - Key: '' # The key of the tag.
      Value: '' # The value of the tag.
LaunchTemplate: # The launch template to use to launch the instances.
  LaunchTemplateId: '' # The ID of the launch template.
  LaunchTemplateName: '' # The name of the launch template.
  Version: '' # The version number of the launch template.
InstanceMarketOptions: # The market (purchasing) option for the instances.
  MarketType: spot # The market type. Valid values are: spot.
  SpotOptions: # The options for Spot Instances.
    MaxPrice: '' # The maximum hourly price you're willing to pay for the Spot
Instances.
    SpotInstanceType: one-time # The Spot Instance request type. Valid values are:
one-time, persistent.
    BlockDurationMinutes: 0 # The required duration for the Spot Instances (also
known as Spot blocks), in minutes.
    ValidUntil: 1970-01-01 00:00:00 # The end date of the request.
    InstanceInterruptionBehavior: terminate # The behavior when a Spot Instance is
interrupted. Valid values are: hibernate, stop, terminate.
CreditSpecification: # The credit option for CPU usage of the T2 or T3 instance.
  CpuCredits: '' # [REQUIRED] The credit option for CPU usage of a T2 or T3
instance.
CpuOptions: # The CPU options for the instance.
  CoreCount: 0 # The number of CPU cores for the instance.
  ThreadsPerCore: 0 # The number of threads per CPU core.
CapacityReservationSpecification: # Information about the Capacity Reservation
targeting option.
  CapacityReservationPreference: none # Indicates the instance's Capacity
Reservation preferences. Valid values are: open, none.
  CapacityReservationTarget: # Information about the target Capacity Reservation.
    CapacityReservationId: '' # The ID of the Capacity Reservation.
HibernationOptions: # Indicates whether an instance is enabled for hibernation.

```

```
Configured: true # If you set this parameter to true, your instance is enabled
for hibernation.
LicenseSpecifications: # The license configurations.
- LicenseConfigurationArn: '' # The Amazon Resource Name (ARN) of the license
configuration.
```

Gerar um esqueleto de comando

Para gerar e usar um arquivo de esqueleto de parâmetro

1. Execute o comando com o parâmetro `--generate-cli-skeleton` para produzir tanto JSON quanto YAML e direcionar a saída a um arquivo para salvá-lo.

JSON

```
$ aws ec2 run-instances --generate-cli-skeleton input > ec2runinst.json
```

YAML

```
$ aws ec2 run-instances --generate-cli-skeleton yaml-input > ec2runinst.yaml
```

2. Abra o arquivo de esqueleto do parâmetro em seu editor de texto e remova os parâmetros que não são mais necessários. Por exemplo, você pode reduzir o modelo ao indicado a seguir. Verifique se o arquivo ainda é JSON ou YAML válido depois de remover os elementos que não são necessários.

JSON

```
{
  "DryRun": true,
  "ImageId": "",
  "KeyName": "",
  "SecurityGroups": [
    ""
  ],
  "InstanceType": "",
  "Monitoring": {
    "Enabled": true
  }
}
```

YAML

```
DryRun: true
ImageId: ''
KeyName: ''
SecurityGroups:
- ''
InstanceType:
Monitoring:
  Enabled: true
```

Neste exemplo, deixamos o parâmetro `DryRun` definido como `true` para usar o recurso de simulação do Amazon EC2. Esse recurso permite que você teste com segurança o comando sem realmente criar ou modificar nenhum recursos.

3. Preencha os valores restantes com valores apropriados para seu cenário. Neste exemplo, fornecemos o tipo de instância, o nome da chave, o grupo de segurança e o identificador da imagem de máquina da Amazon (AMI) que devem ser usados. Este exemplo assume a região padrão da AWS. A AMI `ami-dfc39aef` é uma imagem de 64 bits do hospedada na região `us-west-2`. Se você usar uma região diferente, será necessário [encontrar o ID da AMI correta para usar](#).

JSON

```
{
  "DryRun": true,
  "ImageId": "ami-dfc39aef",
  "KeyName": "mykey",
  "SecurityGroups": [
    "my-sg"
  ],
  "InstanceType": "t2.micro",
  "Monitoring": {
    "Enabled": true
  }
}
```

YAML

```
DryRun: true
```



```
ImageId: 'ami-dfc39aef'
KeyName: 'mykey'
SecurityGroups:
- 'my-sg'
InstanceType: 't2.micro'
Monitoring:
  Enabled: true
```

4. Execute o comando com os parâmetros concluídos passando o arquivo do modelo concluído para o parâmetro `--cli-input-json` ou `--cli-input-yaml` usando o prefixo `file://`. A AWS CLI interpreta o caminho a ser associado ao diretório de trabalho atual, de forma que o exemplo a seguir que exibe somente o nome do arquivo sem nenhum caminho seja procurado pelo arquivo diretamente no diretório de trabalho atual.

JSON

```
$ aws ec2 run-instances --cli-input-json file://ec2runinst.json
```

```
A client error (DryRunOperation) occurred when calling the RunInstances
operation: Request would have succeeded, but DryRun flag is set.
```

YAML

```
$ aws ec2 run-instances --cli-input-yaml file://ec2runinst.yaml
```

```
A client error (DryRunOperation) occurred when calling the RunInstances
operation: Request would have succeeded, but DryRun flag is set.
```

O erro de simulação indica que JSON ou YAML está formado corretamente e os valores de parâmetro são válidos. Se outros problemas forem relatados na saída, corrija-os e repita a etapa anterior até que a mensagem “Request would have succeeded” seja exibida.

5. Agora você pode definir o parâmetro `DryRun` como `false` para desativar a simulação.

JSON

```
{
  "DryRun": false,
  "ImageId": "ami-dfc39aef",
```

```
"KeyName": "mykey",
"SecurityGroups": [
    "my-sg"
],
"InstanceType": "t2.micro",
"Monitoring": {
    "Enabled": true
}
}
```

YAML

```
DryRun: false
ImageId: 'ami-dfc39aef'
KeyName: 'mykey'
SecurityGroups:
- 'my-sg'
InstanceType: 't2.micro'
Monitoring:
  Enabled: true
```

6. Execute o comando, e `run-instances` realmente iniciará uma instância do Amazon EC2 e exibirá os detalhes gerados pelo início bem-sucedido. O formato da saída é controlado pelo parâmetro `--output`, separadamente do formato do modelo de parâmetro de entrada.

JSON

```
$ aws ec2 run-instances --cli-input-json file://ec2runinst.json --output json
```

```
{
  "OwnerId": "123456789012",
  "ReservationId": "r-d94a2b1",
  "Groups": [],
  "Instances": [
    ...
  ]
}
```

YAML

```
$ aws ec2 run-instances --cli-input-yaml file://ec2runinst.yaml --output yaml
```

```
OwnerId: '123456789012'  
ReservationId: 'r-d94a2b1',  
Groups":  
- ''  
Instances:  
...
```

Usar sintaxe simplificada com a AWS CLI

A AWS Command Line Interface (AWS CLI) pode aceitar muitos de seus parâmetros de opção no formato JSON. No entanto, pode ser entediante inserir grandes estruturas ou listas de JSON na linha de comando. Para tornar isso mais fácil, a AWS CLI também oferece suporte a uma sintaxe abreviada mais simples que permite a representação de seus parâmetros de opção em vez de utilizar o formato JSON completo.

Tópicos

- [Parâmetros de estrutura](#)
- [Usar sintaxe simplificada com a AWS Command Line Interface](#)

Parâmetros de estrutura

A sintaxe abreviada no AWS CLI facilita a inserção de parâmetros simples de entrada pelos usuários (estruturas não aninhadas). O formato é uma lista de pares de chave/valor separados por vírgula. Use as regras de [aspas](#) e escape apropriadas para seu terminal, pois a sintaxe abreviada são strings.

Linux or macOS

```
--option key1=value1,key2=value2,key3=value3
```

PowerShell

```
--option "key1=value1,key2=value2,key3=value3"
```

Ambos são equivalentes ao exemplo a seguir formatado em JSON.

```
--option '{"key1":"value1","key2":"value2","key3":"value3"}'
```

Não pode haver nenhum espaço em branco entre cada par de chave/valor separado por vírgula. Aqui está um exemplo do comando `update-table` do Amazon DynamoDB com a opção `--provisioned-throughput` especificada no formato simplificado.

```
$ aws dynamodb update-table \  
  --provisioned-throughput ReadCapacityUnits=15,WriteCapacityUnits=10 \  
  --table-name MyDDDBTable
```

Isso é equivalente ao exemplo a seguir formatado em JSON.

```
$ aws dynamodb update-table \  
  --provisioned-throughput '{"ReadCapacityUnits":15,"WriteCapacityUnits":10}' \  
  --table-name MyDDDBTable
```

Usar sintaxe simplificada com a AWS Command Line Interface

Você pode especificar os parâmetros de entrada em um formulário de lista de duas formas: JSON ou abreviada. A sintaxe abreviada da AWS CLI é projetada para facilitar a inserção de listas com número, sequência de caracteres, estruturas aninhadas ou não.

O formato básico é mostrada aqui, onde os valores na lista são separados por um único espaço.

```
--option value1 value2 value3
```

Isso é equivalente ao exemplo a seguir, formatado em JSON.

```
--option '[value1,value2,value3]'
```

Como mencionado anteriormente, é possível especificar uma lista de números, uma lista de strings ou uma lista de estruturas de dados não aninhadas em formato abreviado. Veja a seguir um exemplo do comando `stop-instances` do Amazon Elastic Compute Cloud (Amazon EC2), onde o parâmetro de entrada (lista de strings) para a opção `--instance-ids` é especificado no formato simplificado.

```
$ aws ec2 stop-instances \  
  --instance-ids i-1234567890
```

```
--instance-ids i-1486157a i-1286157c i-ec3a7e87
```

Isso é equivalente ao exemplo a seguir formatado em JSON.

```
$ aws ec2 stop-instances \  
  --instance-ids '["i-1486157a","i-1286157c","i-ec3a7e87"]'
```

A seguir está um exemplo do comando `create-tags` do Amazon EC2, que leva uma lista de estruturas não aninhadas para a opção `--tags`. A opção `--resources` especifica o ID da instância a ser marcada.

```
$ aws ec2 create-tags \  
  --resources i-1286157c \  
  --tags Key=My1stTag,Value=Value1 Key=My2ndTag,Value=Value2  
Key=My3rdTag,Value=Value3
```

Isso é equivalente ao exemplo a seguir, formatado em JSON. O parâmetro JSON é escrito em várias linhas para melhor leitura.

```
$ aws ec2 create-tags \  
  --resources i-1286157c \  
  --tags '[  
    { "Key": "My1stTag", "Value": "Value1" },  
    { "Key": "My2ndTag", "Value": "Value2" },  
    { "Key": "My3rdTag", "Value": "Value3" }  
]'
```

Fazer a AWS CLI solicitar comandos

Você pode fazer com que a AWS CLI versão 2 solicite comandos, parâmetros e recursos quando você executa um comando da `aws`.

Tópicos

- [Como funcionam](#)
- [Recursos do prompt automático](#)
- [Modos de prompt automático](#)
- [Configurar o prompt automático](#)

Como funcionam

Se estiver habilitado, o prompt automático permitirá usar a tecla ENTER para concluir um comando inserido parcialmente. Após pressionar a tecla ENTER, comandos, parâmetros e recursos serão sugeridos com base no que você digitar. As sugestões listam o nome do comando, parâmetro ou recurso à esquerda e uma descrição dele à direita. Para selecionar e usar uma sugestão, use as teclas de setas para realçar uma linha e pressione a Barra de espaço. Após terminar de inserir seu comando, pressione ENTER para executá-lo. O exemplo a seguir demonstra como é a aparência de uma lista sugerida do prompt automático.

```
$ aws
> aws a
    accessanalyzer      Access Analyzer
    acm                  AWS Certificate Manager
    acm-pca              AWS Certificate Manager Private Certificate
Authority
    alexaforbusiness     Alexa For Business
    amplify              AWS Amplify
```

Recursos do prompt automático

O prompt automático contém os seguintes recursos úteis:

Painel de documentação

Fornece a documentação de ajuda para o comando atual. Para abrir a documentação, pressione a tecla F3.

Conclusão de comando

Sugere comandos da aws que podem ser usados. Para ver uma lista, digite parcialmente o comando. O exemplo a seguir está procurando um serviço começando com a letra a.

```
$ aws
> aws a
    accessanalyzer      Access Analyzer
    acm                  AWS Certificate Manager
    acm-pca              AWS Certificate Manager Private Certificate
Authority
    alexaforbusiness     Alexa For Business
    amplify              AWS Amplify
```

Preenchimento automático de parâmetros

Depois que um comando é digitado, o prompt automático começa a sugerir parâmetros. As descrições dos parâmetros incluem o tipo de valor e uma descrição do que é o parâmetro. Os parâmetros obrigatórios são listados primeiro e rotulados conforme necessário. O exemplo a seguir mostra a lista de parâmetros do `aws dynamodb describe-table`.

```
$ aws dynamodb describe-table
> aws dynamodb describe-table
--table-name (required) [string] The name of the
table to describe.
--cli-input-json [string] Reads arguments
from the JSON string provided. The JSON string follows the format provide...
--cli-input-yaml [string] Reads arguments
from the YAML string provided. The YAML string follows the format provide...
--generate-cli-skeleton [string] Prints a JSON
skeleton to standard output without sending an API request. If provided wit...
```

Preenchimento automático de recursos

O prompt automático faz chamadas de API da AWS usando propriedades de recursos da AWS disponíveis para sugerir valores de recursos. Isso permite que o prompt automático sugira possíveis recursos pertencentes a você ao inserir parâmetros. No exemplo a seguir, o prompt automático lista seus nomes de tabela ao preencher o parâmetro `--table-name` do comando `aws dynamodb describe-table`.

```
$ aws dynamodb describe-table
> aws dynamodb describe-table --table-name
Table1
Table2
Table3
```

Preenchimento automático de sintaxe abreviada

Para parâmetros que usam sintaxe abreviada, o prompt automático sugere valores a serem usados. No exemplo a seguir, o prompt automático lista os valores de sintaxe abreviada para o parâmetro `--placement` no comando `aws ec2 run-instances`.

```
$ aws ec2 run-instances
> aws ec2 run-instances --placement
```

```
AvailabilityZone=      [string] The Availability Zone of the instance. If not
specified, an Availability Zone wil...
Affinity=              [string] The affinity setting for the instance on the
Dedicated Host. This parameter is no...
GroupName=            [string] The name of the placement group the instance is in.
PartitionNumber=      [integer] The number of the partition the instance is in.
Valid only if the placement grou...
```

Preenchimento automático de nomes de arquivos

Ao preencher parâmetros em comandos da aws, o preenchimento automático sugere nomes de arquivos locais após o uso dos prefixos `file://` ou `fileb://`. No exemplo a seguir, o prompt automático sugere arquivos locais depois da inserção de `--item file://` para o comando `aws ec2 run-instances`.

```
$ aws ec2 run-instances
> aws ec2 run-instances --item file://
                           item1.txt
                           file1.json
                           file2.json
```

Preenchimento automático de regiões

Ao usar o parâmetro global `--region`, o prompt automático lista as possíveis regiões para seleção. No exemplo a seguir, o prompt automático sugere regiões depois da inserção de `--region` para o comando `aws dynamodb list-tables`.

```
$ aws dynamodb list-tables
> aws dynamodb list-tables --region
                                af-south-1
                                ap-east-1
                                ap-northeast-1
                                ap-northeast-2
```

Preenchimento automático de perfis

Quando o parâmetro global `--profile` é usado, o prompt automático lista seus perfis. No exemplo a seguir, o prompt automático sugere seus perfis depois da inserção de `--profile` para o comando `aws dynamodb list-tables`.

```
$ aws dynamodb list-tables
```



```
> aws dynamodb list-tables --profile
                                profile1
                                profile2
                                profile3
```

Pesquisas difusas

Completa comandos e valores que contêm um conjunto específico de caracteres. No exemplo a seguir, o prompt automático sugere regiões que contêm eu depois da inserção de `--region eu` para o comando `aws dynamodb list-tables`.

```
$ aws dynamodb list-tables
> aws dynamodb list-tables --region west
                                eu-west-1
                                eu-west-2
                                eu-west-3
                                us-west-1
```

Histórico

Para visualizar e executar comandos usados anteriormente no modo de prompt automático, pressione CTRL + R. O histórico lista os comandos anteriores que você pode selecionar usando as teclas de seta. No exemplo a seguir, o histórico do modo de prompt automático é exibido.

```
$ aws
> aws
    dynamodb list-tables
    s3 ls
```

Modos de prompt automático

O prompt automático para a AWS CLI versão 2 oferece 2 modos configuráveis:

- **Modo completo:** usa o prompt automático cada vez que você tenta executar um comando `aws`, seja quando chamado manualmente via parâmetro `--cli-auto-prompt`, seja quando habilitado permanentemente. Isso inclui pressionar ENTER após um comando completo ou um comando incompleto.
- **Modo parcial:** usa o prompt automático se um comando está incompleto ou não pode ser executado devido a erros de validação no lado do cliente. Esse modo é particularmente útil se

you have pre-existing scripts, runbooks or you want to receive the prompt automatically only for commands with which you are not familiar, instead of seeing the prompt for all commands.

Configurar o prompt automático

To configure the automatic prompt, you can use the following methods in order of precedence:

- Command-line options: enable or disable the automatic prompt for a single command. Use [--cli-auto-prompt](#) to enable the automatic prompt and [--no-cli-auto-prompt](#) to disable it.
- Environment variables: use the variable [aws_cli_auto_prompt](#).
- Shared configuration files: use the configuration [cli_auto_prompt](#).

Controlar a saída do comando da AWS CLI

This section describes the different ways to control the output of the AWS Command Line Interface (AWS CLI). Customizing the output of the AWS CLI in your terminal can improve readability, simplify script automation, and facilitate navigation in larger data sets.

The AWS CLI is compatible with various [output formats](#), including [json](#), [text](#), [yaml](#), and [table](#). Some services have [pagination](#) on the server side for their data, and the AWS CLI provides its own client-side pagination resources.

Finally, the AWS CLI supports [filtering on the server and client sides](#) that you can use individually or together to filter the output of the AWS CLI. Server-side filtering is processed first and returns the output for the client-side filtering. Server-side filtering is compatible with the service API. Client-side filtering is supported by the AWS CLI with the `--query` parameter.

Opções de saída do lado do servidor e do lado do cliente

Server-side output options are directly compatible with the AWS service (AWS Service) API. Filtered or paginated data is not sent to the client, which can accelerate HTTP response times and improve bandwidth for larger data sets.

As opções de saída do lado do cliente são recursos criados pela AWS CLI. Todos os dados são enviados ao cliente e, em seguida, a AWS CLI filtra ou realiza a paginação do conteúdo exibido. As operações do lado do cliente não economizam velocidade ou largura de banda para conjuntos de dados maiores.

Quando as opções do lado do servidor e do lado do cliente são usadas juntas, as operações do lado do servidor são concluídas primeiro e depois enviadas ao cliente para operações do lado do cliente. Essa ação usa a economia em potencial de velocidade e largura de banda das opções do lado do servidor, enquanto usa recursos adicionais da AWS CLI para obter a saída desejada.

Tópicos

- [Definição do formato da saída da AWS CLI](#)
- [Uso de opções de paginação da AWS CLI](#)
- [Filtragem da saída da AWS CLI](#)

Definição do formato da saída da AWS CLI

Este tópico descreve os diferentes formatos de saída para a AWS Command Line Interface (AWS CLI). A AWS CLI oferece suporte aos seguintes formatos de saída:

- **[json](#)**: a saída é formatada como uma string [JSON](#).
- **[yaml](#)**: a saída é formatada como uma string [YAML](#).
- **[yaml-stream](#)**: a saída é transmitida e formatada como uma string [YAML](#). A transmissão possibilita um manuseio mais rápido de tipos de dados grandes.
- **[text](#)**: a saída é formatada como várias linhas de valores de string separados por tabulação. Isso pode ser útil para passar a saída para um processador de texto, como `grep`, `sed` ou `awk`.
- **[table](#)**: a saída é formatada como uma tabela usando os caracteres `+` e `|` para formar as bordas da célula. Geralmente, a informação é apresentada em um formato "amigável", que é muito mais fácil de ler do que outros, mas não tão útil programaticamente.

Como selecionar o formato de saída

Como explicado no tópico sobre [configuração](#), você pode especificar o formato de saída de três maneiras:

- Usando a opção **output** em um perfil nomeado no arquivo **config**: o exemplo a seguir ajusta o formato de saída padrão como `text`.

```
[default]
output=text
```

- Usando a variável de ambiente **AWS_DEFAULT_OUTPUT**: a saída a seguir ajusta o formato como `table` para os comandos nessa sessão de linhas de comandos até que a variável seja alterada ou a sessão encerrada. Usar essa variável de ambiente substitui qualquer valor definido no arquivo `config`.

```
$ export AWS_DEFAULT_OUTPUT="table"
```

- Usando a opção **--output** na linha de comando: o exemplo a seguir define a saída apenas deste comando como `json`. Usar essa opção no comando substitui qualquer variável de ambiente definida atualmente ou o valor no arquivo `config`.

```
$ aws swf list-domains --registration-status REGISTERED --output json
```

Important

O tipo de saída especificado por você modifica a forma de operação da opção `--query`.

- Se você especificar `--output text`, a saída será paginada antes que o filtro `--query` seja aplicado, e a AWS CLI executará a consulta uma vez em cada página da saída. Consequentemente, a consulta inclui o primeiro elemento correspondente em cada página, o que pode resultar em uma saída adicional inesperada. Para filtrar ainda mais a saída, é possível usar outras ferramentas da linha de comando, como `head` ou `tail`.
- Se você especificar `--output json`, `--output yaml` ou `--output yaml-stream`, a saída será completamente processada como uma única estrutura JSON nativa antes que o filtro `--query` seja aplicado. A AWS CLI executa a consulta apenas uma vez com relação a toda a estrutura, produzindo um resultado filtrado que é, então, a saída.

Formato de saída JSON

[JSON](#) é o formato de saída padrão da AWS CLI. A maioria das linguagens de programação pode facilmente decodificar strings JSON usando funções internas ou com bibliotecas disponíveis publicamente. É possível combinar a saída JSON com a [opção --query](#) de maneiras poderosas para filtrar e formatar a saída em formato JSON da AWS CLI.

Para filtragem mais avançada que talvez você não consiga fazer com `--query`, é possível considerar `jq`, um processador JSON da linha de comando. Baixe e localize o tutorial oficial em <http://stedolan.github.io/jq/>.

Veja a seguir um exemplo de saída JSON.

```
$ aws iam list-users --output json
```

```
{
  "Users": [
    {
      "Path": "/",
      "UserName": "Admin",
      "UserId": "AIDA111111111111EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/Admin",
      "CreateDate": "2014-10-16T16:03:09+00:00",
      "PasswordLastUsed": "2016-06-03T18:37:29+00:00"
    },
    {
      "Path": "/backup/",
      "UserName": "backup-user",
      "UserId": "AIDA222222222222EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/backup/backup-user",
      "CreateDate": "2019-09-17T19:30:40+00:00"
    },
    {
      "Path": "/",
      "UserName": "cli-user",
      "UserId": "AIDA333333333333EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/cli-user",
      "CreateDate": "2019-09-17T19:11:39+00:00"
    }
  ]
}
```

Formato da saída YAML

O [YAML](#) é uma boa escolha para lidar com a saída programaticamente com serviços e ferramentas que emitem ou consomem strings no formato [YAML](#), como o AWS CloudFormation com suporte para [modelos no formato YAML](#).

Para filtragem mais avançada que talvez você não consiga fazer com `--query`, é possível considerar `yq`, um processador YAML da linha de comando. Você pode baixar `yq` no [repositório yq](#) no GitHub.

Veja a seguir um exemplo de saída YAML.

```
$ aws iam list-users --output yaml
```

```
Users:
- Arn: arn:aws:iam::123456789012:user/Admin
  CreateDate: '2014-10-16T16:03:09+00:00'
  PasswordLastUsed: '2016-06-03T18:37:29+00:00'
  Path: /
  UserId: AIDA111111111111EXAMPLE
  Username: Admin
- Arn: arn:aws:iam::123456789012:user/backup/backup-user
  CreateDate: '2019-09-17T19:30:40+00:00'
  Path: /backup/
  UserId: AIDA222222222222EXAMPLE
  Username: arq-45EFD6D1-CE56-459B-B39F-F9C1F78FBE19
- Arn: arn:aws:iam::123456789012:user/cli-user
  CreateDate: '2019-09-17T19:30:40+00:00'
  Path: /
  UserId: AIDA333333333333EXAMPLE
  Username: cli-user
```

Formato da saída de fluxo do YAML

O formato `yaml-stream` se beneficia do [YAML](#) e oferece uma visualização mais ágil/rápida de grandes conjuntos de dados ao transmitir os dados para você. Você pode começar a visualizar e usar dados de YAML antes do download de toda a consulta.

Para filtragem mais avançada que talvez você não consiga fazer com `--query`, é possível considerar `yq`, um processador YAML da linha de comando. Você pode baixar `yq` no [repositório yq](#) no GitHub.

A seguir, veja um exemplo de saída `yaml-stream`.

```
$ aws iam list-users --output yaml-stream
```

```
- IsTruncated: false
  Users:
  - Arn: arn:aws:iam::123456789012:user/Admin
    CreateDate: '2014-10-16T16:03:09+00:00'
    PasswordLastUsed: '2016-06-03T18:37:29+00:00'
    Path: /
    UserId: AIDA1111111111EXAMPLE
    Username: Admin
  - Arn: arn:aws:iam::123456789012:user/backup/backup-user
    CreateDate: '2019-09-17T19:30:40+00:00'
    Path: /backup/
    UserId: AIDA2222222222EXAMPLE
    Username: arq-45EFD6D1-CE56-459B-B39F-F9C1F78FBE19
  - Arn: arn:aws:iam::123456789012:user/cli-user
    CreateDate: '2019-09-17T19:30:40+00:00'
    Path: /
    UserId: AIDA3333333333EXAMPLE
    Username: cli-user
```

Veja a seguir um exemplo de saída de `yaml-stream` usando o parâmetro `--page-size` para pagnar o conteúdo YAML transmitido.

```
$ aws iam list-users --output yaml-stream --page-size 2
```

```
- IsTruncated: true
  Marker: ab1234cdef5ghi67jk8lmo9p/
q012rs3t445uv6789w0x1y2z/345a6b78c9d00/1efgh234ij56klmno78pqrstu90vwxyx
  Users:
  - Arn: arn:aws:iam::123456789012:user/Admin
    CreateDate: '2014-10-16T16:03:09+00:00'
    PasswordLastUsed: '2016-06-03T18:37:29+00:00'
    Path: /
    UserId: AIDA1111111111EXAMPLE
    Username: Admin
  - Arn: arn:aws:iam::123456789012:user/backup/backup-user
    CreateDate: '2019-09-17T19:30:40+00:00'
    Path: /backup/
    UserId: AIDA2222222222EXAMPLE
```

```

    UserName: arq-45EFD6D1-CE56-459B-B39F-F9C1F78FBE19
- IsTruncated: false
Users:
- Arn: arn:aws:iam::123456789012:user/cli-user
  CreateDate: '2019-09-17T19:30:40+00:00'
  Path: /
  UserId: AIDA333333333333EXAMPLE
  UserName: cli-user

```

Formato de saída de texto

O formato `text` organiza a saída da AWS CLI em linhas delimitadas por tabulação. Isso funciona bem com ferramentas de texto Unix tradicionais, como `grep`, `sed` e `awk`, bem como o processamento de texto executado pelo PowerShell.

O formato de saída `text` segue a estrutura básica mostrada abaixo. As colunas são classificados alfabeticamente por nomes de chaves correspondentes subjacentes do objeto JSON.

```

IDENTIFIER  sorted-column1 sorted-column2
IDENTIFIER2 sorted-column1 sorted-column2

```

A seguir, veja um exemplo de saída `text`. Cada campo é separado dos outros por tabulação, com uma guia adicional na qual há um campo vazio.

```
$ aws iam list-users --output text
```

```

USERS  arn:aws:iam::123456789012:user/Admin                2014-10-16T16:03:09+00:00
2016-06-03T18:37:29+00:00  /                AIDA111111111111EXAMPLE  Admin
USERS  arn:aws:iam::123456789012:user/backup/backup-user      2019-09-17T19:30:40+00:00
                        /backup/  AIDA222222222222EXAMPLE  backup-user
USERS  arn:aws:iam::123456789012:user/cli-user                  2019-09-17T19:11:39+00:00
                        /                AIDA333333333333EXAMPLE  cli-user

```

A quarta coluna é o campo `PasswordLastUsed` e está vazia nas duas últimas entradas porque esses usuários nunca fazem login no AWS Management Console.

Important

É altamente recomendável que, se você especificar uma saída `text`, também use sempre a opção `--query` para garantir um comportamento consistente.

Isso ocorre porque o formato de texto ordena alfabeticamente as colunas da saída pelo nome da chave do objeto JSON subjacente retornado pelo serviço da AWS, e recursos similares podem não têm os mesmos nomes de chave. Por exemplo, a representação JSON de uma instância do Amazon EC2 baseada em Linux pode ter elementos que não estão presentes na representação JSON de uma instância baseada em Windows ou vice-versa. Além disso, os recursos podem ter elementos de valores de chave adicionados ou removidos em futuras atualizações, alterando a ordem das colunas. Este é o local em que `--query` expande a funcionalidade da saída `text` para fornecer total controle sobre o formato de saída.

No exemplo a seguir, o comando especifica quais elementos devem ser exibidos e define a ordem das colunas com a notação da lista `[key1, key2, ...]`. Isso oferece a você a segurança de que os valores de chave corretos sempre serão exibido na coluna esperada. Por fim, observe como a AWS CLI resulta em `None` como os valores para as chaves que não existem.

```
$ aws iam list-users --output text --query 'Users[*].
[UserName,Arn,CreateDate,PasswordLastUsed,UserId]'
```

```
Admin          arn:aws:iam::123456789012:user/Admin
2014-10-16T16:03:09+00:00  2016-06-03T18:37:29+00:00  AIDA111111111111EXAMPLE
backup-user    arn:aws:iam::123456789012:user/backup-user
2019-09-17T19:30:40+00:00  None                        AIDA222222222222EXAMPLE
cli-user       arn:aws:iam::123456789012:user/cli-backup
2019-09-17T19:11:39+00:00  None                        AIDA333333333333EXAMPLE
```

O exemplo a seguir mostra como você pode usar `grep` e `awk` com a saída `text` do comando `aws ec2 describe-instances`. O primeiro comando exibe a zona de disponibilidade, o estado atual e o ID de cada instância na saída `text`. O segundo comando processa essa saída para exibir somente os IDs de todas as instâncias em execução na Zona de disponibilidade `us-west-2a`.

```
$ aws ec2 describe-instances --query 'Reservations[*].Instances[*].
[Placement.AvailabilityZone, State.Name, InstanceId]' --output text
```

```
us-west-2a      running i-4b41a37c
us-west-2a      stopped i-a071c394
us-west-2b      stopped i-97a217a0
us-west-2a      running i-3045b007
```

```
us-west-2a      running i-6fc67758
```

```
$ aws ec2 describe-instances --query 'Reservations[*].Instances[*].
[Placement.AvailabilityZone, State.Name, InstanceId]' --output text | grep us-west-2a |
grep running | awk '{print $3}'
```

```
i-4b41a37c
i-3045b007
i-6fc67758
```

O exemplo a seguir vai mais longe e mostra não apenas como filtrar a saída, mas como usar essa saída para automatizar a alteração dos tipos de instância para cada instância interrompida.

```
$ aws ec2 describe-instances --query 'Reservations[*].Instances[*].[State.Name,
InstanceId]' --output text |
> grep stopped |
> awk '{print $2}' |
> while read line;
> do aws ec2 modify-instance-attribute --instance-id $line --instance-type '{"Value":
"m1.medium"}';
> done
```

A saída text também pode ser útil no PowerShell. Como as colunas na saída text são delimitadas por tabulação, é possível dividir facilmente a saída em uma matriz usando o delimitador `t do PowerShell. O comando a seguir exibe o valor da terceira coluna (InstanceId) se a primeira coluna (AvailabilityZone) corresponder à string us-west-2a.

```
PS C:\>aws ec2 describe-instances --query 'Reservations[*].Instances[*].
[Placement.AvailabilityZone, State.Name, InstanceId]' --output text |
%{if ($_.split("`t")[0] -match "us-west-2a") { $_.split("`t")[2]; } }
```

```
-4b41a37c
i-a071c394
i-3045b007
i-6fc67758
```

Embora o exemplo anterior mostre como usar o parâmetro --query para analisar os objetos JSON subjacentes e extrair a coluna desejada, o PowerShell tem capacidade própria para lidar com o JSON, se a compatibilidade entre as plataformas não for uma preocupação. Em vez de lidar com a

saída como texto, o que é exigido pela maioria dos shells de comando, o PowerShell permite que você use o cmdlet `ConvertFrom-Json` para produzir um objeto estruturado hierarquicamente. Depois, é possível acessar diretamente o membro desejado usando esse objeto.

```
(aws ec2 describe-instances --output json | ConvertFrom-Json).Reservations.Instances.InstanceId
```

Tip

Se você enviar o texto e filtrar a saída para um único campo usando o parâmetro `--query`, a saída será uma única linha de valores separados por tabulação. Para obter cada valor em uma linha separada, você pode colocar o campo de saída entre colchetes, conforme mostrado nos exemplos a seguir.

Saída de linha única separada por tabulação:

```
$ aws iam list-groups-for-user --user-name susan --output text --query "Groups[].GroupName"
```

```
HRDepartment    Developers      SpreadsheetUsers  LocalAdmins
```

Cada valor em sua própria linha, colocando `[GroupName]` entre colchetes:

```
$ aws iam list-groups-for-user --user-name susan --output text --query "Groups[].[GroupName]"
```

```
HRDepartment
Developers
SpreadsheetUsers
LocalAdmins
```

Formato de saída de tabela

O formato `table` produz representações legíveis da saída complexa da AWS CLI em um formato tabular.

```
$ aws iam list-users --output table
```

```

-----
|
| ListUsers |
+-----+
+
||
| Users |
+-----+
+-----+-----+-----+
| | Arn | CreateDate |
| PasswordLastUsed | Path | UserId | Username |
+-----+-----+-----+
+-----+-----+-----+
| | arn:aws:iam::123456789012:user/Admin | 2014-10-16T16:03:09+00:00 | |
| 2016-06-03T18:37:29+00:00 | / | AIDA1111111111EXAMPLE | Admin |
| | arn:aws:iam::123456789012:user/backup/backup-user | 2019-09-17T19:30:40+00:00 |
| | /backup/ | AIDA2222222222EXAMPLE | backup-user |
| | arn:aws:iam::123456789012:user/cli-user | 2019-09-17T19:11:39+00:00 |
| | / | AIDA3333333333EXAMPLE | cli-user |
+-----+
+

```

É possível combinar a opção `--query` com o formato `table` para exibir um conjunto de elementos pré-selecionado na saída bruta. As diferenças da saída entre as notações de lista e dicionário: no primeiro exemplo, os nomes de coluna são ordenados alfabeticamente, e, no segundo, as colunas sem nome são ordenadas conforme definido pelo usuário. Para obter mais informações sobre a opção `--query`, consulte [Filtragem da saída da AWS CLI](#).

```

$ aws ec2 describe-volumes --query 'Volumes[*].
{ID:VolumeId,InstanceId:Attachments[0].InstanceId,AZ:AvailabilityZone,Size:Size}' --
output table

```

```

-----
| DescribeVolumes |
+-----+-----+-----+-----+
| AZ | ID | InstanceId | Size |
+-----+-----+-----+-----+
| us-west-2a | vol-e11a5288 | i-a071c394 | 30 |
| us-west-2a | vol-2e410a47 | i-4b41a37c | 8 |
+-----+-----+-----+-----+

```

```
$ aws ec2 describe-volumes --query 'Volumes[*].
[VolumeId,Attachments[0].InstanceId,AvailabilityZone,Size]' --output table
```

```
-----
|                               DescribeVolumes                               |
+-----+-----+-----+-----+
| vol-e11a5288| i-a071c394 | us-west-2a | 30 |
| vol-2e410a47| i-4b41a37c | us-west-2a | 8  |
+-----+-----+-----+-----+
```

Uso de opções de paginação da AWS CLI

Este tópico descreve as diferentes maneiras de paginar a saída pela AWS CLI.

Há duas formas principais de controlar a paginação a partir da AWS CLI.

- [Usando os parâmetros de paginação do lado do servidor.](#)
- [Usando o programa de paginação do lado do cliente de saída padrão.](#)

Os parâmetros de paginação do lado do servidor são processados primeiro, e qualquer saída é enviada para a paginação do lado do cliente.

Paginação do lado do servidor

Para comandos que podem retornar uma grande lista de itens, a AWS Command Line Interface (AWS CLI) oferece várias opções para controlar o número de itens incluídos na saída quando a AWS CLI chama a API de um serviço para preencher a lista.

As opções incluem o seguinte:

- [Como usar o parâmetro `--no-paginate`](#)
- [Como usar o parâmetro `--page-size`](#)
- [Como usar o parâmetro `--max-items`](#)
- [Como usar o parâmetro `--starting-token`](#)

Por padrão, a AWS CLI usa um tamanho de página determinado pelo serviço específico e recupera todos os itens disponíveis. Por exemplo, o Amazon S3 tem um tamanho de página padrão de mil.

Se você executar `aws s3api list-objects` em um bucket do Amazon S3 que contém 3,5 mil objetos, a AWS CLI fará automaticamente quatro chamadas para o Amazon S3, lidando com a lógica de paginação específica do serviço para você em segundo plano e retornando todos os 3,5 mil objetos na saída final.

Como usar o parâmetro `--no-paginate`

A opção `--no-paginate` desabilita os seguintes tokens de paginação no lado do cliente. Ao usar um comando, por padrão a AWS CLI faz várias chamadas automaticamente para retornar todos os resultados possíveis para criar a paginação. Uma chamada para cada página. Desabilitar a paginação faz com que a AWS CLI chame apenas uma vez pela primeira página de resultados do comando.

Por exemplo, se você executar `aws s3api list-objects` em um bucket do Amazon S3 que contém 3500 objetos, o AWS CLI fará apenas a primeira chamada para o Amazon S3, retornando apenas os primeiros 1000 objetos na saída final.

```
$ aws s3api list-objects \  
  --bucket my-bucket \  
  --no-paginate  
{  
  "Contents": [  
    ...
```

Como usar o parâmetro `--page-size`

Caso haja problemas ao executar os comandos da lista em um grande número de recursos, o tamanho da página padrão poderá ser muito alto. Isso pode fazer com que as chamadas para os serviços do AWS excedam o tempo máximo permitido e gerar um erro de “expiração”. Você pode usar a opção `--page-size` para especificar que a AWS CLI solicite um número menor de itens de cada chamada para o serviço da AWS. A AWS CLI ainda recuperará a lista completa, mas executará um número maior de chamadas de API de serviço em segundo plano e recuperará um número menor de itens em cada chamada. Isso fornece às chamadas individuais chances melhores de sucesso sem um tempo limite. Alterar o tamanho da página não afeta a saída, afeta somente o número de chamadas da API que precisam ser feitas para gerar a saída.

```
$ aws s3api list-objects \  
  --bucket my-bucket \  
  --page-size 100  
{
```

```
"Contents": [  
...
```

Como usar o parâmetro `--max-items`

Para incluir menos itens por vez na saída da AWS CLI, use a opção `--max-items`. A AWS CLI ainda lida com paginação com o serviço conforme descrito anteriormente, mas imprime apenas o número de itens que você especificar de cada vez.

```
$ aws s3api list-objects \  
  --bucket my-bucket \  
  --max-items 100  
{  
  "NextToken": "eyJNYXJrZXIiOiBudWxsLCAiYm90b190cnVuY2F0ZV9hbW91bnQiOiAxfQ==",  
  "Contents": [  
  ...
```

Como usar o parâmetro `--starting-token`

Se o número de saída de itens (`--max-items`) for menor do que o número total de itens retornados pelas chamadas de API subjacentes, a saída incluirá um `NextToken` que pode ser passado para um comando subsequente para recuperar o próximo conjunto de itens. O exemplo a seguir mostra como usar o valor `NextToken` retornado pelo exemplo anterior e permite que você recupere os segundos 100 itens.

Note

O parâmetro `--starting-token` não pode ser nulo nem vazio. Se o comando anterior não retornar um valor `NextToken`, não haverá mais itens a serem retornados e você não precisará chamar o comando novamente.

```
$ aws s3api list-objects \  
  --bucket my-bucket \  
  --max-items 100 \  
  --starting-token eyJNYXJrZXIiOiBudWxsLCAiYm90b190cnVuY2F0ZV9hbW91bnQiOiAxfQ==  
{  
  "Contents": [  
  ...
```

O serviço da AWS especificado pode não retornar itens na mesma ordem cada vez que você chamá-lo. Se você especificar valores diferentes para `--page-size` e `--max-items`, poderá obter resultados inesperados com itens ausentes ou duplicados. Para evitar que isso aconteça, use o mesmo número para `--page-size` e `--max-items` para sincronizar a paginação da AWS CLI com a paginação do serviço subjacente. Também é possível recuperar a lista completa e executar quaisquer operações necessárias de paginação localmente.

Paginação do lado do cliente

A AWS CLI versão 2 fornece um programa de paginação do lado do cliente para uso na saída. Por padrão, o recurso retorna toda a saída pelo programa de paginação padrão do sistema operacional.

Em ordem de precedência, você pode especificar a paginação de saída destas duas formas:

- Usando a configuração `cli_pager` no arquivo `config` no perfil `default` ou nomeado.
- Usando a variável de ambiente `AWS_PAGER`.
- Usando a variável de ambiente `PAGER`.

Por ordem de precedência, você pode desabilitar todo o uso de um programa de paginação externo das seguintes maneiras:

- Usar opção de linha de comando `--no-cli-pager` para desabilitar a paginação para o uso de um único comando.
- Definir a configuração `cli_pager` ou a variável `AWS_PAGER` como uma string vazia.

Tópicos de paginação do lado do cliente:

- [Como usar a configuração `cli\_pager`](#)
- [Como usar a variável de ambiente `AWS\_PAGER`](#)
- [Como usar a opção `--no-cli-pager`](#)
- [Como usar sinalizadores de paginação](#)

Como usar a configuração `cli_pager`

Você pode salvar as definições de configuração usadas com frequência e credenciais em arquivos que são mantidos pela AWS CLI. As configurações em um perfil de nome têm precedência sobre as

configurações no perfil `default`. Para obter mais informações sobre as opções de configuração, consulte [Configurações do arquivo de configuração e credenciais](#).

O exemplo a seguir define a paginação de saída padrão para o programa `less`.

```
[default]
cli_pager=less
```

O exemplo abaixo define o padrão para desativar o uso de um paginador.

```
[default]
cli_pager=
```

Como usar a variável de ambiente `AWS_PAGER`

O exemplo a seguir define a paginação de saída padrão para o programa `less`. Para obter mais informações sobre variáveis de ambiente, consulte [Variáveis de ambiente para configurar a AWS CLI](#).

Linux and macOS

```
$ export AWS_PAGER="less"
```

Windows

```
C:\> setx AWS_PAGER "less"
```

O exemplo a seguir desabilita o uso do paginador.

Linux and macOS

```
$ export AWS_PAGER=""
```

Windows

```
C:\> setx AWS_PAGER ""
```

Como usar a opção `--no-cli-pager`

Para desabilitar o uso de uma paginação em um único comando, use a opção `--no-cli-pager`. Para obter mais informações sobre as opções de linha de comando, consulte [Opções de linha de comando](#).

```
$ aws s3api list-objects \  
    --bucket my-bucket \  
    --no-cli-pager  
{  
    "Contents": [  
    ...
```

Como usar sinalizadores de paginação

Você pode especificar sinalizadores para usar automaticamente com seu programa de paginação. Os sinalizadores dependem do programa de paginação utilizado. Os exemplos abaixo são referem-se aos padrões típicos de `less` e `more`.

Linux and macOS

Se você não especificar o contrário, o paginador que a AWS CLI versão 2 usará por padrão é `less`. Se você não tiver a variável de ambiente `LESS` definida, a AWS CLI versão 2 usará os sinalizadores `FRX`. Você pode combinar sinalizadores especificando-os ao definir a paginação AWS CLI.

O exemplo a seguir usa o sinalizador `S`. Esse sinalizador então se combina aos sinalizadores `FRX` padrão para criar um sinalizador `FRXS`.

```
$ export AWS_PAGER="less -S"
```

Caso não queira usar nenhum dos sinalizadores `FRX`, você poderá negá-los. O exemplo a seguir nega o sinalizador `F` para criar um sinalizador `RX` final.

```
$ export AWS_PAGER="less -+F"
```

Para obter mais informações sobre sinalizadores `less`, consulte [less](#) em [manpages.org](#).

Windows

Se você não especificar o contrário, o paginador que a AWS CLI versão 2 usará por padrão é `more` sem sinalizadores adicionais.

O exemplo a seguir usa o parâmetro `/c`.

```
C:\> setx AWS_PAGER "more /c"
```

Para obter mais informações sobre sinalizadores `more`, consulte [more](#) os Documentos da Microsoft.

Filtragem da saída da AWS CLI

A AWS Command Line Interface (AWS CLI) conta com filtragem nos lados do servidor e do cliente que você pode usar individualmente ou em conjunto para filtrar sua saída da AWS CLI. A filtragem no lado do servidor é processada primeiro e retorna a saída para a filtragem no lado do cliente.

- A filtragem no lado do servidor é suportada pela API, e você geralmente a implementa com um parâmetro `--filter`. O serviço retorna apenas resultados correspondentes que podem acelerar os tempos de resposta de HTTP para conjuntos de dados grandes.
- A filtragem no lado do cliente é suportada pelo cliente da AWS CLI com o parâmetro `--query`. Esse parâmetro oferece recursos que a filtragem do lado do servidor pode não ter.

Tópicos

- [Filtragem no lado do servidor](#)
- [Filtragem no lado do cliente](#)
- [Combinação das filtrações no lado do servidor e no lado do cliente](#)
- [Recursos adicionais](#)

Filtragem no lado do servidor

A filtragem no lado do servidor na AWS CLI é fornecida pela API do serviço AWS. O serviço da AWS retorna apenas os registros na resposta HTTP que correspondem ao seu filtro, o que pode acelerar os tempos de resposta de HTTP para conjuntos de dados grandes. Como a filtragem no lado do servidor é definida pela API do serviço, os nomes e funções dos parâmetros variam entre os serviços. Alguns nomes de parâmetros comuns usados na filtragem são:

- `--filter` como [ses](#) e [ce](#).
- `--filters` como [ec2](#), [autoescalabilidade](#) e [rds](#).

- Nomes que começam com a palavra `filter`, por exemplo `--filter-expression`, para o comando [aws dynamodb scan](#).

Para obter informações sobre se um comando específico oferece filtragem no lado do servidor e a respeito das regras de filtragem, consulte o [AWS CLI versão 2](#).

Filtragem no lado do cliente

A AWS CLI fornece recursos de filtragem integrada baseados em JSON no lado do cliente com o parâmetro `--query`. O parâmetro `--query` é uma ferramenta poderosa que você pode usar para personalizar o conteúdo e estilo da sua saída. O parâmetro `--query` recebe a resposta HTTP retornada pelo servidor e filtra os resultados antes de exibi-los. Como toda a resposta HTTP é enviada para o cliente antes da filtragem, a filtragem no lado do cliente pode ser mais lenta do que a filtragem no lado do servidor para conjuntos de dados grandes.

A consulta usa a [sintaxe JMESPath](#) para criar expressões para filtrar sua saída. Para aprender a sintaxe JMESPath, consulte o [Tutorial](#) no site do JMESPath.

Important

O tipo de saída especificado por você modifica a forma de operação da opção `--query`.

- Se você especificar `--output text`, a saída será paginada antes que o filtro `--query` seja aplicado, e a AWS CLI executará a consulta uma vez em cada página da saída. Consequentemente, a consulta inclui o primeiro elemento correspondente em cada página, o que pode resultar em uma saída adicional inesperada. Para filtrar ainda mais a saída, é possível usar outras ferramentas da linha de comando, como `head` ou `tail`.
- Se você especificar `--output json`, `--output yaml` ou `--output yaml-stream`, a saída será completamente processada como uma única estrutura JSON nativa antes que o filtro `--query` seja aplicado. A AWS CLI executa a consulta apenas uma vez com relação a toda a estrutura, produzindo um resultado filtrado que é, então, a saída.

Tópicos de filtragem no lado do cliente

- [Antes de começar](#)
- [Identificadores](#)
- [Como selecionar em uma lista](#)

- [Como filtrar dados aninhados](#)
- [Resultados da simplificação](#)
- [Filtragem de valores específicos](#)
- [Encadeamento de expressões](#)
- [Filtragem por vários valores de identificador](#)
- [Adição de rótulos a valores de identificador](#)
- [Funções](#)
- [Exemplos avançados --query](#)

Antes de começar

Ao usar expressões de filtro usadas nesses exemplos, certifique-se de usar as regras de aspas corretas para o shell do terminal. Para obter mais informações, consulte [the section called “Aspas com strings”](#).

A saída JSON a seguir mostra um exemplo do que o parâmetro `--query` pode produzir. A saída descreve três volumes do Amazon EBS anexados a instâncias separadas do Amazon EC2.

Exemplo de saída

```
$ aws ec2 describe-volumes
{
  "Volumes": [
    {
      "AvailabilityZone": "us-west-2a",
      "Attachments": [
        {
          "AttachTime": "2013-09-17T00:55:03.000Z",
          "InstanceId": "i-a071c394",
          "VolumeId": "vol-e11a5288",
          "State": "attached",
          "DeleteOnTermination": true,
          "Device": "/dev/sda1"
        }
      ],
      "VolumeType": "standard",
      "VolumeId": "vol-e11a5288",
      "State": "in-use",
      "SnapshotId": "snap-f23ec1c8",
      "CreateTime": "2013-09-17T00:55:03.000Z",
```

```
    "Size": 30
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-18T20:26:16.000Z",
        "InstanceId": "i-4b41a37c",
        "VolumeId": "vol-2e410a47",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-2e410a47",
    "State": "in-use",
    "SnapshotId": "snap-708e8348",
    "CreateTime": "2013-09-18T20:26:15.000Z",
    "Size": 8
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2020-11-20T19:54:06.000Z",
        "InstanceId": "i-1jd73kv8",
        "VolumeId": "vol-a1b3c7nd",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-a1b3c7nd",
    "State": "in-use",
    "SnapshotId": "snap-234087fb",
    "CreateTime": "2020-11-20T19:54:05.000Z",
    "Size": 15
  }
]
```

Identificadores

Identificadores são rótulos para valores de saída. Ao criar filtros, você usa identificadores para refinar os resultados da consulta. No exemplo de saída a seguir, todos os identificadores como Volumes, AvailabilityZone e AttachTime estão destacados.

```
$ aws ec2 describe-volumes
{
  "Volumes": [
    {
      "AvailabilityZone": "us-west-2a",
      "Attachments": [
        {
          "AttachTime": "2013-09-17T00:55:03.000Z",
          "InstanceId": "i-a071c394",
          "VolumeId": "vol-e11a5288",
          "State": "attached",
          "DeleteOnTermination": true,
          "Device": "/dev/sda1"
        }
      ],
      "VolumeType": "standard",
      "VolumeId": "vol-e11a5288",
      "State": "in-use",
      "SnapshotId": "snap-f23ec1c8",
      "CreateTime": "2013-09-17T00:55:03.000Z",
      "Size": 30
    },
    {
      "AvailabilityZone": "us-west-2a",
      "Attachments": [
        {
          "AttachTime": "2013-09-18T20:26:16.000Z",
          "InstanceId": "i-4b41a37c",
          "VolumeId": "vol-2e410a47",
          "State": "attached",
          "DeleteOnTermination": true,
          "Device": "/dev/sda1"
        }
      ],
      "VolumeType": "standard",
      "VolumeId": "vol-2e410a47",
      "State": "in-use",
    }
  ]
}
```

```

    "SnapshotId": "snap-708e8348",
    "CreateTime": "2013-09-18T20:26:15.000Z",
    "Size": 8
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2020-11-20T19:54:06.000Z",
        "InstanceId": "i-1jd73kv8",
        "VolumeId": "vol-a1b3c7nd",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-a1b3c7nd",
    "State": "in-use",
    "SnapshotId": "snap-234087fb",
    "CreateTime": "2020-11-20T19:54:05.000Z",
    "Size": 15
  }
]
}

```

Para obter mais informações, consulte [Identificadores](#) no site do JMESPath.

Como selecionar em uma lista

Uma lista ou matriz é um identificador que é seguido por um colchete “[”, como Volumes e Attachments na [the section called “Antes de começar”](#).

Sintaxe

```
<listName>[ ]
```

Para filtrar toda a saída de uma matriz, você pode usar a notação curinga. Expressões [curinga](#) são expressões usadas para retornar elementos usando a notação *.

O exemplo a seguir consulta todo o conteúdo Volumes.

```
$ aws ec2 describe-volumes \
```



```
--query 'Volumes[*]'
```

```
[
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-17T00:55:03.000Z",
        "InstanceId": "i-a071c394",
        "VolumeId": "vol-e11a5288",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-e11a5288",
    "State": "in-use",
    "SnapshotId": "snap-f23ec1c8",
    "CreateTime": "2013-09-17T00:55:03.000Z",
    "Size": 30
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2020-11-20T19:54:06.000Z",
        "InstanceId": "i-1jd73kv8",
        "VolumeId": "vol-a1b3c7nd",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-a1b3c7nd",
    "State": "in-use",
    "SnapshotId": "snap-234087fb",
    "CreateTime": "2020-11-20T19:54:05.000Z",
    "Size": 15
  }
]
```

Para exibir um volume específico na matriz por índice, chame o índice da matriz. Por exemplo, o primeiro item na matriz `Volumes` tem um índice igual a 0, o que resulta na consulta `Volumes[0]`. Para obter mais informações sobre índices de matriz, consulte [Expressões de índice](#) no site do JMESPath.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[0]'
{
  "AvailabilityZone": "us-west-2a",
  "Attachments": [
    {
      "AttachTime": "2013-09-17T00:55:03.000Z",
      "InstanceId": "i-a071c394",
      "VolumeId": "vol-e11a5288",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-e11a5288",
  "State": "in-use",
  "SnapshotId": "snap-f23ec1c8",
  "CreateTime": "2013-09-17T00:55:03.000Z",
  "Size": 30
}
```

Para exibir um intervalo específico de volumes por índice, use `slice` com a seguinte sintaxe, onde `start` é o índice da matriz inicial, `stop` é o índice onde o filtro interrompe o processamento e `step` é o intervalo de salto.

Sintaxe

```
<arrayName>[<start>:<stop>:<step>]
```

Se algum deles for omitido da expressão `slice`, os seguintes valores padrão serão usados:

- Start: o primeiro índice na lista, 0.
- Stop: o último índice na lista.
- Step: sem pular etapas, onde o valor é 1.

Para retornar somente os dois primeiros volumes, use um valor de start igual a 0, um valor de stop igual 2 e um valor de step igual a 1, conforme mostrado no exemplo a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[0:2:1]'
[
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-17T00:55:03.000Z",
        "InstanceId": "i-a071c394",
        "VolumeId": "vol-e11a5288",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-e11a5288",
    "State": "in-use",
    "SnapshotId": "snap-f23ec1c8",
    "CreateTime": "2013-09-17T00:55:03.000Z",
    "Size": 30
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-18T20:26:16.000Z",
        "InstanceId": "i-4b41a37c",
        "VolumeId": "vol-2e410a47",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-2e410a47",
    "State": "in-use",
    "SnapshotId": "snap-708e8348",
    "CreateTime": "2013-09-18T20:26:15.000Z",
    "Size": 8
  }
]
```

```
}
]
```

Como este exemplo contém valores padrão, você pode encurtar o slice de `Volumes[0:2:1]` para `Volumes[:2]`.

O exemplo a seguir omite valores padrão e retorna cada dois volumes em toda a matriz.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[:2]'
[
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-17T00:55:03.000Z",
        "InstanceId": "i-a071c394",
        "VolumeId": "vol-e11a5288",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-e11a5288",
    "State": "in-use",
    "SnapshotId": "snap-f23ec1c8",
    "CreateTime": "2013-09-17T00:55:03.000Z",
    "Size": 30
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2020-11-20T19:54:06.000Z",
        "InstanceId": "i-1jd73kv8",
        "VolumeId": "vol-a1b3c7nd",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
```

```
    "VolumeId": "vol-a1b3c7nd",
    "State": "in-use",
    "SnapshotId": "snap-234087fb",
    "CreateTime": "2020-11-20T19:54:05.000Z",
    "Size": 15
  }
]
```

Os valores de `step` também podem ser números negativos, o que resulta na filtragem em ordem inversa de uma matriz, conforme mostrado no exemplo a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[::-2]'
[
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2020-11-20T19:54:06.000Z",
        "InstanceId": "i-1jd73kv8",
        "VolumeId": "vol-a1b3c7nd",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-a1b3c7nd",
    "State": "in-use",
    "SnapshotId": "snap-234087fb",
    "CreateTime": "2020-11-20T19:54:05.000Z",
    "Size": 15
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-17T00:55:03.000Z",
        "InstanceId": "i-a071c394",
        "VolumeId": "vol-e11a5288",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ]
  }
]
```

```

    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-e11a5288",
  "State": "in-use",
  "SnapshotId": "snap-f23ec1c8",
  "CreateTime": "2013-09-17T00:55:03.000Z",
  "Size": 30
}
]
```

Para obter mais informações, consulte [Slices](#) no site do JMESPath.

Como filtrar dados aninhados

Para refinar a filtragem de `Volumes[*]` para valores aninhados, use subexpressões anexando um ponto e seus critérios de filtragem.

Sintaxe

```
<expression>.<expression>
```

O exemplo a seguir mostra todas as informações de Attachments para todos os volumes.

```

$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments'
[
  [
    {
      "AttachTime": "2013-09-17T00:55:03.000Z",
      "InstanceId": "i-a071c394",
      "VolumeId": "vol-e11a5288",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  [
    {
      "AttachTime": "2013-09-18T20:26:16.000Z",
      "InstanceId": "i-4b41a37c",
      "VolumeId": "vol-2e410a47",
      "State": "attached",

```

```

    "DeleteOnTermination": true,
    "Device": "/dev/sda1"
  },
  [
    {
      "AttachTime": "2020-11-20T19:54:06.000Z",
      "InstanceId": "i-1jd73kv8",
      "VolumeId": "vol-a1b3c7nd",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ]
]
```

Para filtrar ainda mais os valores aninhados, acrescente a expressão para cada identificador aninhado. O exemplo a seguir lista o State para todos os Volumes.

```

$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[*].State'
[
  [
    "attached"
  ],
  [
    "attached"
  ],
  [
    "attached"
  ]
]
```

Resultados da simplificação

Para obter mais informações, consulte [Subexpressões](#) no site do JMESPath.

Você pode simplificar os resultados para `Volumes[*].Attachments[*].State` removendo a notação curinga, o que resultará na consulta `Volumes[*].Attachments[].State`. A simplificação muitas vezes é útil para melhorar a legibilidade dos resultados.

```
$ aws ec2 describe-volumes \
```

```
--query 'Volumes[*].Attachments[].State'
[
  "attached",
  "attached",
  "attached"
]
```

Para obter mais informações, consulte [Simplificação](#) no site do JMESPath.

Filtragem de valores específicos

Para filtrar valores específicos em uma lista, use uma expressão de filtro como mostrado na sintaxe a seguir.

Sintaxe

```
? <expression> <comparator> <expression>]
```

Os comparadores de expressão incluem ==, !=, <, <=, > e >=. O exemplo a seguir filtra VolumeIds para todos os Volumes em um State igual a Attached.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[?State==`attached`].VolumeId'
[
  [
    "vol-e11a5288"
  ],
  [
    "vol-2e410a47"
  ],
  [
    "vol-a1b3c7nd"
  ]
]
```

Os resultados podem ser simplificados, o que resulta no exemplo a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[?State==`attached`].VolumeId[]'
[
  "vol-e11a5288",
```



```
"vol-2e410a47",
"vol-a1b3c7nd"
]
```

O exemplo a seguir filtra VolumeIds para todos os Volumes com tamanho inferior a 20.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[?Size < `20`].VolumeId'
[
  "vol-2e410a47",
  "vol-a1b3c7nd"
]
```

Para obter mais informações, consulte [Expressões de filtragem](#) no site do JMESPath.

Encadeamento de expressões

Você pode encadear resultados de um filtro para uma nova lista e, em seguida, filtrar o resultado com outra expressão usando a seguinte sintaxe:

Sintaxe

```
<expression> | <expression>]
```

O exemplo a seguir leva os resultados do filtro da expressão `Volumes[*].Attachments[].InstanceId` e produz o primeiro resultado na matriz.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[].InstanceId | [0]'
"i-a071c394"
```

Esse exemplo faz isso criando primeiro a matriz a partir da expressão a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[].InstanceId'
"i-a071c394",
"i-4b41a37c",
"i-1jd73kv8"
```

Depois, devolva o primeiro elemento para essa matriz.

```
"i-a071c394"
```

Para obter mais informações, consulte [Expressões de encadeamento](#) no site do JMESPath.

Filtragem por vários valores de identificador

Para filtrar por vários identificadores, use uma lista de seleção múltipla usando a seguinte sintaxe:

Sintaxe

```
<listName>[].[<expression>, <expression>]
```

No exemplo a seguir, VolumeId e VolumeType são filtrados na lista Volumes, o que resulta na expressão abaixo.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[].[VolumeId, VolumeType]'
[
  [
    "vol-e11a5288",
    "standard"
  ],
  [
    "vol-2e410a47",
    "standard"
  ],
  [
    "vol-a1b3c7nd",
    "standard"
  ]
]
```

Para adicionar dados aninhados à lista, adicione outra lista de seleção múltipla. O exemplo a seguir expande o exemplo anterior filtrando também por InstanceId e State na lista aninhada Attachments. Isso resulta na seguinte expressão.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[].[VolumeId, VolumeType, Attachments[].[InstanceId, State]]'
[
  [
    "vol-e11a5288",
```

```

    "standard",
    [
      [
        "i-a071c394",
        "attached"
      ]
    ]
  ],
  [
    "vol-2e410a47",
    "standard",
    [
      [
        "i-4b41a37c",
        "attached"
      ]
    ]
  ],
  [
    "vol-a1b3c7nd",
    "standard",
    [
      [
        "i-1jd73kv8",
        "attached"
      ]
    ]
  ]
]

```

Para torná-la mais legível, simplifique a expressão conforme mostrado no exemplo a seguir.

```

$ aws ec2 describe-volumes \
  --query 'Volumes[][VolumeId, VolumeType, Attachments[][InstanceId, State]][]'
[
  "vol-e11a5288",
  "standard",
  [
    "i-a071c394",
    "attached"
  ],
  "vol-2e410a47",
  "standard",

```

```
[
  "i-4b41a37c",
  "attached"
],
"vol-a1b3c7nd",
"standard",
[
  "i-1jd73kv8",
  "attached"
]
]
```

Para obter mais informações, consulte [Lista de multisseleção](#) no site do JMESPath.

Adição de rótulos a valores de identificador

Para tornar essa saída mais fácil de ler, use um hash de seleção múltipla com a sintaxe a seguir.

Sintaxe

```
<listName>[].{<label>: <expression>, <label>: <expression>}
```

O rótulo do identificador não precisa ser o mesmo que o nome do identificador. O exemplo a seguir usa o rótulo Type para os valores VolumeType.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[].{VolumeType: VolumeType}'
[
  {
    "VolumeType": "standard",
  },
  {
    "VolumeType": "standard",
  },
  {
    "VolumeType": "standard",
  }
]
```

Para simplificar, o exemplo a seguir mantém os nomes dos identificadores para cada rótulo e exibe VolumeId, VolumeType, InstanceId e State para todos os volumes:

```
$ aws ec2 describe-volumes \
  --query 'Volumes[].{VolumeId: VolumeId, VolumeType: VolumeType, InstanceId:
  Attachments[0].InstanceId, State: Attachments[0].State}'
[
  {
    "VolumeId": "vol-e11a5288",
    "VolumeType": "standard",
    "InstanceId": "i-a071c394",
    "State": "attached"
  },
  {
    "VolumeId": "vol-2e410a47",
    "VolumeType": "standard",
    "InstanceId": "i-4b41a37c",
    "State": "attached"
  },
  {
    "VolumeId": "vol-a1b3c7nd",
    "VolumeType": "standard",
    "InstanceId": "i-1jd73kv8",
    "State": "attached"
  }
]
```

Para obter mais informações, consulte [Hash de multisseleção](#) no site do JMESPath.

Funções

A sintaxe JMESPath contém muitas funções que você pode usar em suas consultas. Para obter informações sobre funções JMESPath, consulte [Funções integradas](#) no site do JMESPath.

Para demonstrar como você pode incorporar uma função em suas consultas, o exemplo a seguir usa a função `sort_by`. A função `sort_by` classifica uma matriz usando uma expressão como a chave de classificação com a seguinte sintaxe:

Sintaxe

```
sort_by(<listName>, <sort expression>)[].<expression>
```

O exemplo a seguir usa o [exemplo de hash de multisseleção](#) anterior e classifica a saída por `VolumeId`.

```
$ aws ec2 describe-volumes \
  --query 'sort_by(Volumes, &VolumeId)[].{VolumeId: VolumeId, VolumeType: VolumeType,
InstanceId: Attachments[0].InstanceId, State: Attachments[0].State}'
[
  {
    "VolumeId": "vol-2e410a47",
    "VolumeType": "standard",
    "InstanceId": "i-4b41a37c",
    "State": "attached"
  },
  {
    "VolumeId": "vol-a1b3c7nd",
    "VolumeType": "standard",
    "InstanceId": "i-1jd73kv8",
    "State": "attached"
  },
  {
    "VolumeId": "vol-e11a5288",
    "VolumeType": "standard",
    "InstanceId": "i-a071c394",
    "State": "attached"
  }
]
```

Para obter mais informações, consulte [sort_by](#) no site do JMESPath.

Exemplos avançados --query

Para extrair informações de um item específico

O exemplo a seguir usa o parâmetro --query para localizar um item específico em uma lista e extrai as informações desse item. O exemplo lista todas as AvailabilityZones associadas ao endpoint de serviço especificado. Ele extrai o item da lista ServiceDetails que tem o ServiceName especificado e gera o campo AvailabilityZones do item selecionado.

```
$ aws --region us-east-1 ec2 describe-vpc-endpoint-services \
  --query 'ServiceDetails[?ServiceName==`com.amazonaws.us-
east-1.ecs`].AvailabilityZones'
[
  [
    "us-east-1a",
    "us-east-1b",
```

```
        "us-east-1c",
        "us-east-1d",
        "us-east-1e",
        "us-east-1f"
    ]
]
```

Para mostrar snapshots após a data de criação especificada

O exemplo a seguir mostra como listar todos os seus snapshots que foram criados após uma data especificada, incluindo apenas alguns dos campos disponíveis na saída.

```
$ aws ec2 describe-snapshots --owner self \
    --output json \
    --query 'Snapshots[?StartTime>=`2018-02-07`].
{Id:SnapshotId,Vid:VolumeId,Size:VolumeSize}'
[
  {
    "id": "snap-0effb42b7a1b2c3d4",
    "vid": "vol-0be9bb0bf12345678",
    "Size": 8
  }
]
```

Para mostrar as AMIs mais recentes

O exemplo a seguir lista as cinco imagens de máquina da Amazon (AMIs) mais recentes que você criou, classificadas das mais recentes para as mais antigas.

```
$ aws ec2 describe-images \
    --owners self \
    --query 'reverse(sort_by(Images,&CreationDate))[:5].{id:ImageId,date:CreationDate}'
[
  {
    "id": "ami-0a1b2c3d4e5f60001",
    "date": "2018-11-28T17:16:38.000Z"
  },
  {
    "id": "ami-0a1b2c3d4e5f60002",
    "date": "2018-09-15T13:51:22.000Z"
  },
  {

```

```

    "id": "ami-0a1b2c3d4e5f60003",
    "date": "2018-08-19T10:22:45.000Z"
  },
  {
    "id": "ami-0a1b2c3d4e5f60004",
    "date": "2018-05-03T12:04:02.000Z"
  },
  {
    "id": "ami-0a1b2c3d4e5f60005",
    "date": "2017-12-13T17:16:38.000Z"
  }
]

```

Para mostrar instâncias não íntegras do Auto Scaling

O exemplo a seguir mostra apenas o InstanceId para todas as instâncias com problemas de integridade no grupo de AutoScaling especificado.

```

$ aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-name My-AutoScaling-Group-Name \
  --output text \
  --query 'AutoScalingGroups[*].Instances[?HealthStatus==`Unhealthy`].InstanceId'

```

Para incluir volumes com a tag especificada

O exemplo a seguir descreve todas as instâncias com uma tag test. Contanto que haja outra etiqueta além de test anexada ao volume, o volume ainda é retornado nos resultados.

A expressão abaixo para retornar todas as tags com a tag test em uma matriz. Qualquer etiqueta que não é a etiqueta test contém um valor null.

```

$ aws ec2 describe-volumes \
  --query 'Volumes.Tags[?Value == `test`]'

```

Para excluir volumes com a etiqueta especificada

O exemplo a seguir descreve todas as instâncias sem uma etiqueta test. A utilização de uma expressão ?Value != `test` simples não funciona para excluir um volume, pois os volumes podem ter várias etiquetas. Contanto que haja outra etiqueta além de test anexada ao volume, o volume ainda é retornado nos resultados.

Para excluir todos os volumes com a etiqueta `test`, comece com a expressão abaixo para retornar todas as etiquetas com a etiqueta `test` em um array. Qualquer etiqueta que não é a etiqueta `test` contém um valor `null`.

```
$ aws ec2 describe-volumes \
  --query 'Volumes.Tags[?Value == `test`]'
```

Em seguida, filtrar todos os resultados test positivos usando a função `not_null`.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[!not_null(Tags[?Value == `test`].Value)]'
```

Encadeie os resultados para simplificá-los, o que resultará na consulta a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[!not_null(Tags[?Value == `test`].Value)] | []'
```

Combinação das filtrações no lado do servidor e no lado do cliente

É possível usar as filtrações no lado do servidor e no lado do cliente ao mesmo tempo. A filtração no lado do servidor é concluída primeiro, o que envia os dados para o cliente que, em seguida, são filtrados pelo parâmetro `--query`. Se você estiver usando conjuntos de dados grandes, aplicar a filtração no lado do servidor primeiro pode reduzir a quantidade de dados enviados ao cliente para cada chamada AWS CLI sem prejudicar a personalização poderosa proporcionada pela filtração no lado do cliente.

O exemplo a seguir mostra volumes do Amazon EC2 usando filtrações no lado do servidor e no lado do cliente. O serviço filtra uma lista de todos os volumes anexados na zona de disponibilidade `us-west-2a`. O parâmetro `--query` limita ainda mais a saída apenas para aqueles volumes com um valor `Size` maior que 50 e mostra apenas os campos especificados com nomes definidos pelo usuário.

```
$ aws ec2 describe-volumes \
  --filters "Name=availability-zone,Values=us-west-2a" "Name=status,Values=attached" \
  --query 'Volumes[?Size > `50`].{Id:VolumeId,Size:Size,Type:VolumeType}'
[
  {
```

```

    "Id": "vol-0be9bb0bf12345678",
    "Size": 80,
    "VolumeType": "gp2"
  }
]
```

O exemplo a seguir recupera uma lista de imagens que atendem a vários critérios. Depois, usa o parâmetro `--query` para classificar a saída por `CreationDate`, selecionando apenas as mais recentes. Por fim, ele exibe o `ImageId` dessa imagem.

```

$ aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=amzn*gp2" "Name=virtualization-type,Values=hvm"
  "Name=root-device-type,Values=ebs" \
  --query "sort_by(Images, &CreationDate)[-1].ImageId" \
  --output text
ami-00ced3122871a4921
```

O exemplo a seguir exibe o número de volumes disponíveis acima de 1000 IOPS usando `length` para contar quantos há em uma lista.

```

$ aws ec2 describe-volumes \
  --filters "Name=status,Values=available" \
  --query 'length(Volumes[?Iops > `1000`])'
3
```

Recursos adicionais

Prompt automático da AWS CLI

Ao começar a usar expressões de filtragem, você poderá usar o recurso de prompt automático na AWS CLI versão 2. O recurso de prompt automático mostra uma pré-visualização quando você pressiona a tecla F5. Para obter mais informações, consulte [the section called “Prompt automático”](#).

JMESPath Terminal

O JMESPath Terminal é um terminal de comandos interativos que permitem experimentar expressões JMESPath usadas na filtragem no lado do cliente. Usando o comando `jpterm`, o terminal mostra os resultados imediatos da consulta enquanto você está digitando. Você pode

encaminhar diretamente a saída da AWS CLI para o terminal, permitindo assim a experimentação avançada de consultas.

O exemplo a seguir encaminha a saída da `aws ec2 describe-volumes` diretamente para o JMESPath Terminal.

```
$ aws ec2 describe-volumes | jpterm
```

Para obter mais informações sobre o JMESPath Terminal e instruções de instalação, consulte [JMESPath Terminal](#) no GitHub.

Utilitário jq

O `jq` fornece uma maneira de transformar sua saída no lado do cliente em um formato de saída desejado. Para obter mais informações sobre o `jq` e instruções de instalação, consulte [jq](#) no GitHub.

Códigos de retorno da AWS CLI

O código de retorno geralmente é um código oculto enviado após a execução de um comando da AWS Command Line Interface (AWS CLI) que descreve o status do comando. Você pode usar o comando `echo` para exibir o código enviado do último comando AWS CLI e usar esses códigos para determinar se um comando foi bem-sucedido ou se falhou, e por que um comando pode ter apresentado um erro. Além dos códigos de retorno, você pode visualizar mais detalhes sobre uma falha executando seus comandos com a opção `--debug`. Essa opção produz um relatório detalhado das etapas que a AWS CLI usa para processar o comando, e o resultado que foi gerado por cada etapa.

Para determinar o código de retorno de um comando de AWS CLI, execute um dos seguintes comandos imediatamente após a execução do comando de CLI.

Linux and macOS

```
$ echo $?  
0
```

Windows PowerShell

```
PS> echo $lastexitcode
```

```
0
```

Windows Command Prompt

```
C:\> echo %errorlevel%  
0
```

Veja a seguir os valores de código de retorno que podem ser retornados ao final da execução de um comando de AWS Command Line Interface (AWS CLI).

Códig	Significado
0	O serviço respondeu com um código de status de resposta HTTP de 200, o que indica que não houve erros gerados pela AWS CLI e pelo serviço da AWS para o qual a solicitação foi enviada.
1	Uma ou mais operações de transferência do Amazon S3 falhou. Limitado a comandos do S3.
2	O significado desse código de retorno depende do comando. • Aplicável a todos os comandos da AWS CLI : não foi possível analisar o comando inserido. Falhas de análise podem ser causadas, entre outros motivos, pela ausência de subcomandos ou argumentos necessários ou pelo uso de comandos ou argumentos desconhecidos. • Limitado a comandos do S3: um ou mais arquivos marcados para transferência foram ignorados durante o processo de transferência. No entanto, todos os outros arquivos marcados para transferência foram transferidos com êxito. Os arquivos que são ignorados durante o processo de transferência incluem: arquivos inexistentes, arquivos que são dispositivos de caracteres especiais, dispositivo de bloqueio especial, filas FIFO ou soquetes, além de arquivos para os quais o usuário não tem permissões de leitura.
130	O comando foi interrompido por um SIGINT. Este é o sinal enviado por você para cancelar um comando com Ctrl+C.
252	A sintaxe do comando era inválida, um parâmetro desconhecido foi fornecido ou um valor de parâmetro estava incorreto e impediu a execução do comando.

Códig	Significado
253	O ambiente ou configuração do sistema era inválido. Embora o comando fornecido possa estar sintaticamente válido, uma configuração ou credenciais ausentes impediram a execução do comando.
254	O comando foi analisado com êxito e uma solicitação foi feita para o serviço especificado, mas o serviço retornou um erro. Isso geralmente indica o uso incorreto da API ou outros problemas específicos do serviço.
255	Ocorreu uma falha no comando. Houve erros gerados pela AWS CLI ou pelo serviço da AWS para o qual a solicitação foi enviada.

Uso dos assistentes da AWS CLI

O AWS Command Line Interface (AWS CLI) permite usar um assistente para alguns comandos. Para contribuir ou visualizar a lista completa de assistentes da AWS CLI disponíveis, consulte a [Pasta de assistentes da AWS CLI](#) no GitHub

Como funcionam

Similar ao console da AWS, a AWS CLI oferece um assistente de interface do usuário que pode orientar você no gerenciamento dos seus recursos da AWS. Para usar o assistente, você chama o subcomando `wizard` e o nome do assistente após o nome do serviço em um comando. A estrutura do comando é a seguinte:

Sintaxe:

```
$ aws <command> wizard <wizardName>
```

O exemplo a seguir chama o assistente para criar uma nova tabela do `dynamodb`.

```
$ aws dynamodb wizard new-table
```

O `aws configure` é o único assistente que não tem um nome de assistente. Ao executar o assistente, execute o comando `aws configure wizard` conforme demonstrado no exemplo a seguir.

```
$ aws configure wizard
```

Depois de chamar um assistente, um formulário será exibido no shell. Para cada parâmetro, você receberá uma lista de opções para selecionar ou será avisado para inserir em uma string. Para selecionar a partir de uma lista, use as teclas de seta para cima e para baixo e pressione ENTER. Para exibir detalhes sobre uma opção, pressione a tecla de seta para a direita. Após terminar de preencher um parâmetro, pressione ENTER.

```
$ aws configure wizard
What would you like to configure
> Static Credentials
    Assume Role
    Process Provider
    Additional CLI configuration
Enter the name of the profile:
Enter your Access Key Id:
Enter your Secret Access Key:
```

Para editar prompts anteriores, use SHIFT + TAB. Para alguns assistentes, depois de preencher todos os prompts, você poderá visualizar um modelo do AWS CloudFormation ou o comando da AWS CLI preenchido com suas informações. Este modo de visualização é útil para aprender a AWS CLI, APIs de serviço e criação de modelos para scripts.

Pressione ENTER após a pré-visualização ou o último prompt para executar o comando final.

```
$ aws configure wizard
What would you like to configure
Enter the name of the profile: testWizard
Enter your Access Key Id: AB1C2D3EF4GH5I678J90K
Enter your Secret Access Key: ab1c2def34gh5i67j8k90l1mnop2qrs45tu678v90
<ENTER>
```

Criar e usar aliases da AWS CLI

Os aliases são atalhos que você pode criar na AWS Command Line Interface (AWS CLI) para encurtar comandos ou scripts que utiliza com frequência. Os aliases são criados no arquivo `alias` localizado em sua pasta de configuração.

Tópicos

- [Pré-requisitos](#)
- [Etapa 1: Criação do arquivo de alias](#)
- [Etapa 2: Criação de um alias](#)
- [Passo 3: Como chamar um alias](#)
- [Exemplos de repositório de alias](#)
- [Recursos](#)

Pré-requisitos

Para usar comandos de alias, é necessário fazer o seguinte:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- Usar, no mínimo, a AWS CLI versão 1.11.24 ou 2.0.0.
- (Opcional) Para usar scripts bash em alias da AWS CLI, é necessário usar um terminal compatível com bash.

Etapa 1: Criação do arquivo de alias

Para criar o arquivo `alias`, você pode utilizar a navegação de arquivos e um editor de texto ou utilizar o terminal preferido seguindo o procedimento passo a passo. Para criar rapidamente seu arquivo de alias, use o seguinte bloco de comandos.

Linux and macOS

```
$ mkdir -p ~/.aws/cli
$ echo '[toplevel]' > ~/.aws/cli/alias
```

Windows

```
C:\> md %USERPROFILE%\aws\cli
C:\> echo [toplevel] > %USERPROFILE%\aws\cli\alias
```

Para criar o arquivo de alias

1. Crie uma pasta chamada `cli` na pasta de configuração da AWS CLI. Por padrão, a pasta de configuração é `~/.aws/` no Linux ou no macOS e `%USERPROFILE%\aws\` no Windows. Você pode criar isso via sua navegação de arquivos ou usando o comando a seguir.

Linux and macOS

```
$ mkdir -p ~/.aws/cli
```

Windows

```
C:\> md %USERPROFILE%\aws\cli
```

O caminho padrão da pasta `cli` resultante é `~/.aws/cli/` no Linux ou no macOS e `%USERPROFILE%\aws\cli` no Windows.

2. Na pasta `cli`, crie um arquivo de texto chamado `aliassem` extensão e adicione `[toplevel]` à primeira linha. Você pode criar esse arquivo com seu editor de texto preferido ou usar o comando a seguir.

Linux and macOS

```
$ echo '[toplevel]' > ~/.aws/cli/alias
```

Windows

```
C:\> echo [toplevel] > %USERPROFILE%\aws\cli\alias
```

Etapa 2: Criação de um alias

Você pode criar um alias usando comandos básicos ou scripts bash.

Criação um alias de comando básico

Você pode criar seu alias ao adicionar um comando usando a seguinte sintaxe no arquivo `alias` criado na etapa anterior.

Sintaxe


```
aliasname = command [--options]
```

aliasname é o que você atribui ao seu alias. **command** é o comando que você deseja chamar, o qual pode incluir outros aliases. Você pode incluir opções ou parâmetros em seu alias ou adicioná-los ao chamar seu alias.

O exemplo a seguir cria um alias chamado `aws whoami` usando o comando [aws sts get-caller-identity](#). Como esse alias chama um comando da AWS CLI existente, você pode escrever o comando sem usar o prefixo `aws`.

```
whoami = sts get-caller-identity
```

O exemplo a seguir usa o exemplo `whoami` anterior e adiciona o filtro `Account` e as opções `output` de texto.

```
whoami2 = sts get-caller-identity --query Account --output text
```

Criar um alias de subcomando

Note

O recurso de alias do subcomando requer, no mínimo, a versão da AWS CLI 1.11.24 ou 2.0.0

É possível criar um alias para subcomandos adicionando um comando usando a seguinte sintaxe no arquivo `alias` criado na etapa anterior.

Sintaxe

```
[command commandGroup]  
aliasname = command [--options]
```

O **commandGroup** é o namespace do comando, por exemplo, o comando `aws ec2 describe-regions` está sob o grupo do comando `ec2`. **aliasname** é o que você atribui ao seu alias.

command é o comando que você deseja chamar, o qual pode incluir outros aliases. Você pode incluir opções ou parâmetros em seu alias ou adicioná-los ao chamar seu alias.

O exemplo a seguir cria um alias chamado `aws ec2 regions` usando o comando [aws ec2 describe-regions](#). Como esse alias chama um comando da AWS CLI existente sob o namespace do comando `ec2`, é possível escrever o comando sem usar o prefixo `aws ec2`.

```
[command ec2]
regions = describe-regions --query Regions[].RegionName
```

Para criar aliases de comandos fora do namespace do comando, use um ponto de exclamação como prefixo do comando completo. O exemplo a seguir cria um alias chamado `aws ec2 instance-profiles` usando o comando [aws iam list-instance-profiles](#).

```
[command ec2]
instance-profiles = !aws iam list-instance-profiles
```

Note

Os aliases usam apenas namespaces de comandos existentes e não é possível criar outros. Por exemplo, você não pode criar um alias com a seção `[command johnsmith]`, pois o namespace do comando `johnsmith` ainda não existe.

Criação de um alias de script bash

Warning

Para usar scripts bash em alias da AWS CLI, é necessário usar um terminal compatível com bash.

Você pode criar um alias usando scripts bash para processos mais avançados usando a sintaxe a seguir.

Sintaxe

```
aliasname =
    !f() {
        script content
    }; f
```

aliasname é o nome do seu alias e *script content* é o script que você deseja executar ao chamar o alias.

O exemplo a seguir usa `opendns` para mostrar seu endereço IP atual. Como você pode usar aliases em outros aliases, o alias `myip` a seguir é útil para permitir ou revogar o acesso ao seu endereço IP de dentro de outros aliases.

```
myip =  
  !f() {  
    dig +short myip.opendns.com @resolver1.opendns.com  
  }; f
```

Os exemplos de script a seguir chamam o alias `aws myip` anterior para autorizar seu endereço IP para ingresso em um grupo de segurança do Amazon EC2.

```
authorize-my-ip =  
  !f() {  
    ip=$(aws myip)  
    aws ec2 authorize-security-group-ingress --group-id ${1} --cidr $ip/32 --protocol  
tcp --port 22  
  }; f
```

Quando você chama aliases que usam scripts bash, as variáveis são sempre passadas na ordem em que você as inseriu. Nos scripts bash, os nomes das variáveis não são levados em consideração, apenas a ordem em que aparecem. No exemplo de alias `textalert` a seguir, a variável para a opção `--message` é a primeira e para a opção `--phone-number` é a segunda.

```
textalert =  
  !f() {  
    aws sns publish --message "${1}" --phone-number ${2}  
  }; f
```

Passo 3: Como chamar um alias

Para executar o alias que você criou em seu arquivo `alias`, use a sintaxe a seguir. Você pode adicionar opções extras ao chamar seu alias.

Sintaxe

```
$ aws aliasname
```

O exemplo a seguir usa o alias do comando `aws whoami`.

```
$ aws
  whoami
{
  "UserId": "A12BCD34E5FGHI6JKLM",
  "Account": "1234567890987",
  "Arn": "arn:aws:iam::1234567890987:user/userName"
}
```

O exemplo a seguir usa o alias `aws whoami` com opções adicionais para retornar somente o número Account na saída de text.

```
$ aws whoami --query Account --output
text
1234567890987
```

O exemplo a seguir usa o [alias do subcomando](#) `aws ec2 regions`.

```
$ aws ec2
  regions
[
  "ap-south-1",
  "eu-north-1",
  "eu-west-3",
  "eu-west-2",
  ...
]
```

Como chamar um alias usando variáveis de script bash

Quando você chama aliases que usam scripts bash, as variáveis são passadas na ordem em que foram inseridas. Nos scripts bash, os nomes das variáveis não são levados em consideração, apenas a ordem em que aparecem. Por exemplo, no alias `textalert` a seguir, a variável para a opção `--message` é a primeira e para a opção `--phone-number` é a segunda.

```
textalert =
!f() {
  aws sns publish --message "${1}" --phone-number ${2}
}; f
```

Ao chamar o alias `textalert`, você precisa passar variáveis na mesma ordem em que elas são executadas no alias. No exemplo a seguir usamos as variáveis `$message` e `$phone`. A variável `$message` é passada como `${1}` para a opção `--message` e a variável `$phone` é passada como `${2}` para a opção `--phone-number`. Isso faz com que o alias `textalert` seja chamado com êxito para enviar uma mensagem.

```
$ aws textalert $message
$phone
{
  "MessageId": "1ab2cd3e4-fg56-7h89-i01j-2klmn34567"
}
```

No exemplo a seguir, a ordem é invertida quando o alias é chamado para `$phone` e `$message`. A variável `$phone` é passada como `${1}` para a opção `--message` e a variável `$message` é passada como `${2}` para a opção `--phone-number`. Como as variáveis estão fora de ordem, o alias passa as variáveis incorretamente. Isso causa um erro porque o conteúdo de `$message` não corresponde aos requisitos de formatação de número de telefone para a opção `--phone-number`.

```
$ aws textalert $phone
$message
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:

aws help
aws <command> help
aws <command> <subcommand> help

Unknown options: text
```

Exemplos de repositório de alias

O [repositório de alias da AWS CLI](#) no GitHub contém exemplos de alias da AWS CLI criados pela equipe de desenvolvedores da AWS CLI e pela comunidade. Você pode usar todo o arquivo de exemplo de alias ou use aliases individuais para seu próprio uso.

Warning

A execução dos comandos nesta seção exclui seu arquivo `alias`. Para evitar que o arquivo de alias existente seja sobrescrito, altere o local de download.

Para usar aliases do repositório

1. Instale o Git. Para obter instruções de instalação, consulte [Introdução: Instalação do Git](#) na Documentação do Git.
2. Instale o comando jp. O comando jp é usado no alias tostring. Para obter instruções de instalação, consulte [JMESPath \(jp\) Readme.md](#) no GitHub.
3. Instale o comando jq. O comando jq é usado no alias tostring-with-jq. Para obter instruções de instalação, consulte [Processador JSON \(jq\)](#) no GitHub.
4. Baixe o alias de uma das seguintes formas:
 - Execute os seguintes comandos que são baixados do repositório e copiam o arquivo alias para sua pasta de configuração.

Linux and macOS

```
$ git clone https://github.com/awslabs/awscli-aliases.git
$ mkdir -p ~/.aws/cli
$ cp awscli-aliases/alias ~/.aws/cli/alias
```

Windows

```
C:\> git clone https://github.com/awslabs/awscli-aliases.git
C:\> md %USERPROFILE%\aws\cli
C:\> copy awscli-aliases\alias %USERPROFILE%\aws\cli
```

- Baixe diretamente o repositório e salve na pasta cli em sua pasta de configuração da AWS CLI. Por padrão, a pasta de configuração é ~/.aws/ no Linux ou no macOS e %USERPROFILE%\aws\ no Windows.
5. Para verificar se os aliases estão funcionando, execute o alias a seguir.

```
$ aws whoami
```

Isso exibe a mesma resposta que o comando `aws sts get-caller-identity`:

```
{
  "Account": "012345678901",
  "UserId": "AIUAINBADX2VEG2TC6HD6",
  "Arn": "arn:aws:iam::012345678901:user/myuser"
}
```

Recursos

- O [repositório de alias da AWS CLI](#) no GitHub contém exemplos de alias da AWS CLI criados pela equipe de desenvolvedores da AWS CLI e contribuições da comunidade da AWS CLI.
- The alias feature announcement from [AWS re:Invent 2016: The Effective AWS CLI User](#) no YouTube.
- [aws sts get-caller-identity](#)
- [aws ec2 describe-instances](#)
- [aws sns publish](#)

Exemplos de código

Este capítulo fornece vários exemplos que mostram como usar a AWS Command Line Interface (AWS CLI) com Serviços da AWS.

Neste guia, há os seguintes tipos de exemplos da AWS CLI:

- [Exemplos de comando guiados](#): exemplos de comando guiados do Guia do usuário da AWS CLI sobre como usar a AWS CLI com alguns Serviços da AWS. Geralmente, esses exemplos são mais detalhados que os [da versão 2 do guia de referência da AWS CLI](#).
- [AWS CLI usando exemplos de código de script Bash](#): exemplos de script bash de código aberto. Os exemplos de script Bash estão hospedados no [Repositório de exemplos de código da AWS](#) no GitHub.

Exemplo de feedback

Você não consegue encontrar o que precisa? Solicite um exemplo de comando usando o link Fornecer feedback na parte inferior desta página ou na respectiva página de comando [da versão 2 do guia de referência da AWS CLI](#).

Quer contribuir? Contribua com exemplos de comandos da AWS CLI no [Repositório de códigos de exemplos da AWS](#) no GitHub. Para obter mais informações sobre contribuição, consulte [as etapas rápidas de contribuição de códigos de exemplos da AWS CLI](#) nas páginas do GitHub.

Exemplos de comando guiado da AWS CLI

Esta seção fornece exemplos que mostram como usar a AWS Command Line Interface (AWS CLI) para acessar vários serviços da AWS.

Note

Para ter uma referência completa de todos os comandos disponíveis para cada serviço, consulte o [AWS CLIGuia de referência da versão 2](#) ou use a ajuda da linha de comando integrada. Para obter mais informações, consulte [Obter ajuda com a AWS CLI](#).

Serviços

- [Uso do Amazon DynamoDB com a AWS CLI](#)
- [Usar o Amazon EC2 com a AWS CLI](#)
- [Usar o Amazon S3 Glacier com a AWS CLI](#)
- [Uso do AWS Identity and Access Management pela AWS CLI](#)
- [Uso do Amazon S3 com a AWS CLI](#)
- [Uso do Amazon SNS com a AWS CLI](#)

Uso do Amazon DynamoDB com a AWS CLI

Introdução ao Amazon DynamoDB

[O que é o Amazon DynamoDB?](#)

A AWS Command Line Interface (AWS CLI) oferece suporte a todos os serviços de banco de dados da AWS, incluindo o Amazon DynamoDB. Use a AWS CLI para operações imromptu, como a criação de uma tabela. Ela também pode ser usada para incorporar operações do DynamoDB em scripts utilitários.

Para obter mais informações sobre como usar a AWS CLI com o DynamoDB, consulte [dynamodb](#) na Referência de comandos da AWS CLI.

Para listar os comandos da AWS CLI para o DynamoDB, use o comando a seguir.

```
$ aws dynamodb help
```

Tópicos

- [Pré-requisitos](#)
- [Criação e uso de tabelas do DynamoDB](#)
- [Uso do DynamoDB Local](#)
- [Recursos](#)

Pré-requisitos

Para executar os comandos dynamodb, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).

Criação e uso de tabelas do DynamoDB

O formato da linha de comando consiste em um nome de comando do DynamoDB, seguido pelos parâmetros desse comando. A AWS CLI oferece suporte à [sintaxe simplificada](#) da CLI para os valores de parâmetro, assim como JSON completo.

O exemplo a seguir cria uma tabela chamada MusicCollection.

```
$ aws dynamodb create-table \  
  --table-name MusicCollection \  
  --attribute-definitions AttributeName=Artist,AttributeType=S  
  AttributeName=SongTitle,AttributeType=S \  
  --key-schema AttributeName=Artist,KeyType=HASH  
  AttributeName=SongTitle,KeyType=RANGE \  
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1
```

Adicione novas linhas à tabela com comandos semelhantes aos mostrados no exemplo a seguir. Esses exemplos usam uma combinação de sintaxe abreviada e JSON.

```
$ aws dynamodb put-item \  
  --table-name MusicCollection \  
  --item '{  
    "Artist": {"S": "No One You Know"},  
    "SongTitle": {"S": "Call Me Today"} ,  
    "AlbumTitle": {"S": "Somewhat Famous"}  
  }' \  
  --return-consumed-capacity TOTAL  
{  
  "ConsumedCapacity": {  
    "CapacityUnits": 1.0,  
    "TableName": "MusicCollection"  
  }  
}
```

```
$ aws dynamodb put-item \
  --table-name MusicCollection \
  --item '{
    "Artist": {"S": "Acme Band"},
    "SongTitle": {"S": "Happy Day"} ,
    "AlbumTitle": {"S": "Songs About Life"}
  }' \
  --return-consumed-capacity TOTAL
{
  "ConsumedCapacity": {
    "CapacityUnits": 1.0,
    "TableName": "MusicCollection"
  }
}
```

Pode ser difícil compor um JSON válido em um comando com uma única linha. Para tornar isso mais fácil, a AWS CLI pode ler arquivos JSON. Por exemplo, considere o seguinte trecho de código JSON, que é armazenado em um arquivo chamado `expression-attributes.json`.

```
{
  ":v1": {"S": "No One You Know"},
  ":v2": {"S": "Call Me Today"}
}
```

Você pode usar esse arquivo para emitir uma solicitação query usando a AWS CLI. No exemplo a seguir, o conteúdo do arquivo `expression-attributes.json` é usado como o valor para o parâmetro `--expression-attribute-values`.

```
$ aws dynamodb query --table-name MusicCollection \
  --key-condition-expression "Artist = :v1 AND SongTitle = :v2" \
  --expression-attribute-values file://expression-attributes.json
{
  "Count": 1,
  "Items": [
    {
      "AlbumTitle": {
        "S": "Somewhat Famous"
      },
      "SongTitle": {
        "S": "Call Me Today"
      },
    },
  ],
}
```

```
        "Artist": {
            "S": "No One You Know"
        }
    ],
    "ScannedCount": 1,
    "ConsumedCapacity": null
}
```

Uso do DynamoDB Local

Além do DynamoDB, é possível usar a AWS CLI com o DynamoDB local. O DynamoDB Local é um banco de dados e servidor pequeno no lado do cliente que copia o serviço do DynamoDB. O DynamoDB Local permite criar aplicativos que usam a API do DynamoDB sem de fato manipular tabelas ou dados no serviço da Web do DynamoDB. Em vez disso, todas as ações da API são roteadas para um banco de dados local. Isso economiza a taxa de transferência provisionada, o armazenamento de dados e as taxas de transferência de dados.

Para obter mais informações sobre o DynamoDB Local e como usá-lo com a AWS CLI, consulte as seguintes seções do [Guia do desenvolvedor do Amazon DynamoDB](#):

- [DynamoDB Local](#)
- [Como usar a AWS CLI com o DynamoDB Local](#)

Recursos

Referência da AWS CLI:

- [aws dynamodb](#)
- [aws dynamodb create-table](#)
- [aws dynamodb put-item](#)
- [aws dynamodb query](#)

Referência do serviço:

- [DynamoDB Local](#) no Guia do desenvolvedor do Amazon DynamoDB
- [Como usar a AWS CLI com o DynamoDB Local](#) no Guia do desenvolvedor do Amazon DynamoDB

Usar o Amazon EC2 com a AWS CLI

Introdução ao Amazon Elastic Compute Cloud

[Introdução ao Amazon EC2: servidor de nuvem elástico e hospedagem com a AWS](#)

Você pode acessar os recursos do Amazon Elastic Compute Cloud (Amazon EC2) usando a AWS Command Line Interface (AWS CLI). Para listar os comandos da AWS CLI para o Amazon EC2, use o comando a seguir.

```
aws ec2 help
```

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Este tópico mostra exemplos curtos de comandos da AWS CLI que executam tarefas comuns para o Amazon EC2.

Para exemplos de formas longas de comandos da AWS CLI, consulte [Repositório de exemplos de código da AWS CLI](#) no GitHub.

Tópicos

- [Criar, exibir e excluir pares de chaves do Amazon EC2](#)
- [Criar, configurar e excluir grupos de segurança para o Amazon EC2](#)
- [Inicializar, listar e encerrar instâncias do Amazon EC2](#)
- [Alterar um tipo de instância do Amazon EC2 com um script bash](#)

Criar, exibir e excluir pares de chaves do Amazon EC2

Você pode usar a AWS Command Line Interface (AWS CLI) para criar, exibir e excluir seus pares de chaves do Amazon Elastic Compute Cloud (Amazon EC2). Você usa pares de chaves para se conectar a uma instância do Amazon EC2.

Você deve fornecer o par de chaves para o AmazonEC2 ao criar a instância e, em seguida, usar esse par de chaves para autenticar ao se conectar à instância.

 Note

Para obter exemplos de outros comandos, consulte o [AWS CLI](#).

Tópicos

- [Pré-requisitos](#)
- [Criar um par de chaves](#)
- [Exibir o par de chaves](#)
- [Excluir o par de chaves](#)
- [Referências](#)

Pré-requisitos

Para executar os comandos `ec2`, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- Defina suas permissões do IAM para permitir acesso ao Amazon EC2. Para obter mais informações sobre permissões do IAM para Amazon EC2, consulte [Políticas do IAM para Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

Criar um par de chaves

Para criar um par de chaves, use o comando [aws ec2 create-key-pair](#) com a opção `--query` e a opção `--output text` para canalizar sua chave privada diretamente para um arquivo.

```
$ aws ec2 create-key-pair --key-name MyKeyPair --query 'KeyMaterial' --output text  
> MyKeyPair.pem
```

Para o PowerShell, o padrão de redirecionamento `> file` assume a codificação UTF-8, que não pode ser usada com alguns clientes SSH. Portanto, você deve converter a saída redirecionando-a para o comando `out-file` e definir explicitamente a codificação para `ascii`.

```
PS C:\>aws ec2 create-key-pair --key-name MyKeyPair --query 'KeyMaterial' --output text  
| out-file -encoding ascii -filepath MyKeyPair.pem
```

O arquivo resultante `MyKeyPair.pem` é semelhante ao seguinte.

```
-----BEGIN RSA PRIVATE KEY-----
EXAMPLEKEYKCAQEAY7WZhaDsR1W3mRlQtvhwyORRX8gnxgDAfRt/gx42kWXsT4rXE/b5CpSgie/
vBoU7jLxx92pNHoFnByP+Dc21eyyz6CvjTmWA0JwfWiW5/akH7i05dSrvC7dQkW2duV5QuUdEQW
Z/aNxMniGQE6XAgfwlnXVBwrerrQo+ZWQeqiUwwMkuEbLeJFLhMCvYURpUMSC1oehm449ilx9X1F
G50TCFe0zf18dqqCP6GzbPaIjiU19xX/az0R9V+tpU0zEL+wmXnZt3/nHPQ5xvD20JH67km6SuPW
oPzev/D8V+x4+bHthfSjR9Y7DvQFjfbVwHXigBdtZcU2/wei8D/HYwIDAQABaoIBAGZ1kaEvnrrqu
/uler7vgIn5m7lN5LkKw4hJLAIW6tUT/fzvtcHK0SkbQCQXuriHmQ2MqyJX/0kn2NfjLV/ufGxbL1
mb5qwMGUnEpJaZD6QSSs3kICLwWUYUiGfc0uiSbmJoap/GTLU0W5Mfcv36PaBUNy5p53V6G7hXb2
bahyWyJNfjLe4M86yd2YK3V2CmK+X/B0sShnJ36+hjrXPPWmV3N9zEmCdJjA+K15DYmhm/tJWSD9
81oGk9TopEp7CkIfatEATyyZiVqoRq6k64iuM9JkA30zdXzMqexXVJ1TLZVEH0E7bh1Y9d801ozR
oQs/FiZNAx2iijCWyv0lpjE73+kCgYEA9mZtyhkHkFDpwrSM1APaL8oNAbbjwEy7Z5Mqfq1+lIp1
YkriL0DbLXlvRAH+yHPRit2hH0jtUNZh4Axv+cpq09qbUI3+43eEy24B7G/Uh+GTfbjsXs0xQx/x
p9otyVwc7hsQ5TA5PZb+mvkJ50BEKzet9XcKw0NBVELGhnEPe7cCgYEA06Vgov6YHleHui9kHuws
ayav0elc5zkxjF9nfHFJRry21R1trw2Vdpn+9g481URipzWV0Eihvm+xTtmaZ1Sp//lkq75XDwnU
WA8gkn603QE3fq2yN98BURsAKdJfJ5RL1HvGQvTe10HLYXpJnEkHv+Unl2ajLivWUt5pbBrKbUC
gYBjb0+0Zk0sCcpZ29sbzjYjpIddErySIyRX5gV2uNQwAjlDp9PfN295yQ+BxMBXiIycWVQiw0bH
oMo7yykABY70zd5wQewBQ4AdSlWSX4nGDtsiFxiI5sKuAAe0CbTosy1s8w8fxoJ5Tz1sd0xNeGs
Arq6Wv/G16zQuAE9zK9vwwKBgF+09VI/1wJBirsDGz9whVwFFPrTkJNvJZzYt69qezx1sjgFKshy
WBhd4xHZtmCqpBP1AymEjr/T0lboxyARMXMnIOWIANXMGb4KGSyl1mzSVAoQ+fqr+cJ3d0dyP1lj
jjb0Ed/NY8fr1NDxAVHE8BSkdsx2f6ELEyBKJSRr9snRAoGAMrTwYneXzvTskF/S5Fyu0i0egLda
NWUH38v/nDCgEpIXD5Hn3qAEcju1IjmbwlvTW+nY2jVhv7UGd8MjwUTNGItbdb6nsYqM2asrnF3qS
VRkAKKKYegJkpUfVTTrW0YFjXkfcR/V+QFL50ndHAKJXjW7a4ejJLncTzmZSpYzwApc=
-----END RSA PRIVATE KEY-----
```

Sua chave privada não é armazenada na AWS e só pode ser recuperada quando é criada. Não será possível recuperá-la posteriormente. Ao invés disso, se você perder a chave privada, deverá criar um novo par de chaves.

Se você estiver se conectando à sua instância a partir de um computador com Linux, recomendamos que você use o seguinte comando para definir as permissões do arquivo de chave privada, de maneira que apenas você possa lê-lo.

```
$ chmod 400 MyKeyPair.pem
```

Exibir o par de chaves

A “impressão digital” é gerada a partir do par de chaves, e você pode usá-la para verificar se a chave privada presente em sua máquina local corresponde à chave pública armazenada na AWS.

A impressão digital é um hash SHA1 de uma cópia codificada DER da chave privada. Esse valor é capturado quando o par de chaves é criado e é armazenado na AWS com a chave pública. Você pode visualizar a impressão digital no console do Amazon EC2 ou executando o comando [aws ec2 describe-key-pairs](#) da AWS CLI.

O exemplo a seguir exibe a impressão digital para MyKeyPair.

```
$ aws ec2 describe-key-pairs --key-name MyKeyPair
{
  "KeyPairs": [
    {
      "KeyName": "MyKeyPair",
      "KeyFingerprint":
"1f:51:ae:28:bf:89:e9:d8:1f:25:5d:37:2d:7d:b8:ca:9f:f5:f1:6f"
    }
  ]
}
```

Para obter mais informações sobre chaves e impressões digitais, consulte [Pares de chaves do Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

Excluir o par de chaves

Para excluir um par de chaves, execute o comando [aws ec2 delete-key-pair](#), substituindo *MyKeyPair* pelo nome do par a ser excluído.

```
$ aws ec2 delete-key-pair --key-name MyKeyPair
```

Referências

Referência da AWS CLI:

- [aws ec2](#)
- [aws ec2 create-key-pair](#)
- [aws ec2 delete-key-pair](#)
- [aws ec2 describe-key-pairs](#)

Outra referência:

- [Documentação do Amazon Elastic Compute Cloud](#)

- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

Criar, configurar e excluir grupos de segurança para o Amazon EC2

Você pode criar um grupo de segurança para suas instâncias do Amazon Elastic Compute Cloud (Amazon EC2) que atua essencialmente como um firewall, com regras que determinam qual tráfego de rede pode entrar e sair.

É possível usar a AWS Command Line Interface (AWS CLI) para criar um grupo de segurança, adicionar regras a grupos de segurança existentes e excluir grupos de segurança.

Note

Para obter exemplos de outros comandos, consulte o [AWS CLI](#).

Tópicos

- [Pré-requisitos](#)
- [Crie um grupo de segurança](#)
- [Adicionar regras ao grupo de segurança](#)
- [Excluir o grupo de segurança](#)
- [Referências](#)

Pré-requisitos

Para executar os comandos ec2, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- Defina suas permissões do IAM para permitir acesso ao Amazon EC2. Para obter mais informações sobre permissões do IAM para Amazon EC2, consulte [Políticas do IAM para Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

Crie um grupo de segurança

É possível criar grupos de segurança associados a nuvens privadas virtuais (VPCs).

O exemplo do [aws ec2 create-security-group](#) a seguir mostra como criar um grupo de segurança para a VPC especificada.

```
$ aws ec2 create-security-group --group-name my-sg --description "My security group" --  
vpc-id vpc-1a2b3c4d  
{  
  "GroupId": "sg-903004f8"  
}
```

Para visualizar as informações iniciais para um grupo de segurança, execute o comando [aws ec2 describe-security-groups](#). Você pode fazer referência a um grupo de segurança do EC2-VPC somente por seu vpc-id, e não apenas por seu nome.

```
$ aws ec2 describe-security-groups --group-ids sg-903004f8  
{  
  "SecurityGroups": [  
    {  
      "IpPermissionsEgress": [  
        {  
          "IpProtocol": "-1",  
          "IpRanges": [  
            {  
              "CidrIp": "0.0.0.0/0"  
            }  
          ],  
          "UserIdGroupPairs": []  
        }  
      ],  
      "Description": "My security group"  
      "IpPermissions": [],  
      "GroupName": "my-sg",  
      "VpcId": "vpc-1a2b3c4d",  
      "OwnerId": "123456789012",  
      "GroupId": "sg-903004f8"  
    }  
  ]  
}
```

Adicionar regras ao grupo de segurança

Quando você executa uma instância do Amazon EC2, é necessário ativar regras no grupo de segurança para permitir o tráfego de rede de entrada como meio de se conectar à imagem.

Por exemplo, se estiver iniciando uma instância do Windows, normalmente você adiciona uma regra para permitir que o tráfego de entrada na porta TCP 3389 ofereça suporte ao Protocolo de Desktop Remoto (RDP). Se estiver iniciando uma instância do Linux, geralmente você adiciona uma regra para permitir que o tráfego de entrada na porta 22 ofereça suporte às conexões SSH.

Use o comando [aws ec2 authorize-security-group-ingress](#) para adicionar uma regra a um grupo de segurança. Um parâmetro obrigatório deste comando é o endereço IP público de seu computador ou rede (na forma de um intervalo de endereços) ao qual seu computador está conectado na notação [CIDR](#).

Note

Fornecemos o serviço <https://checkip.amazonaws.com/> para permitir a você determinar seu endereço IP público. Para encontrar outros serviços que podem ajudar você a identificar o endereço IP, use o navegador para pesquisar “qual é meu endereço IP”. Se você se conectar por meio de um ISP ou protegido por um firewall usando um endereço IP dinâmico (por meio de um gateway NAT de uma rede privada), seu endereço poderá mudar periodicamente. Nesse caso, você deve descobrir o intervalo de endereços IP usados por computadores cliente.

O exemplo a seguir mostra como adicionar uma regra para RDP (porta TCP 3389) em um grupo de segurança do EC2-VPC com o ID `sg-903004f8` usando seu endereço IP.

Para começar, encontre seu endereço IP.

```
$ curl https://checkip.amazonaws.com
x.x.x.x
```

Em seguida, você pode adicionar o endereço IP ao seu grupo de segurança, executando o comando [aws ec2 authorize-security-group-ingress](#).

```
$ aws ec2 authorize-security-group-ingress --group-id sg-903004f8 --protocol tcp --port 3389 --cidr x.x.x.x/x
```

O comando a seguir adiciona uma regra para habilitar o SSH para instâncias no mesmo grupo de segurança.

```
$ aws ec2 authorize-security-group-ingress --group-id sg-903004f8 --protocol tcp --port 22 --cidr x.x.x.x/x
```

Para visualizar as alterações no grupo de segurança, execute o comando [aws ec2 describe-security-groups](#).

```
$ aws ec2 describe-security-groups --group-ids sg-903004f8
{
  "SecurityGroups": [
    {
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "UserIdGroupPairs": []
        }
      ],
      "Description": "My security group"
      "IpPermissions": [
        {
          "ToPort": 22,
          "IpProtocol": "tcp",
          "IpRanges": [
            {
              "CidrIp": "x.x.x.x/x"
            }
          ],
          "UserIdGroupPairs": [],
          "FromPort": 22
        }
      ],
      "GroupName": "my-sg",
      "OwnerId": "123456789012",
      "GroupId": "sg-903004f8"
    }
  ]
}
```

Excluir o grupo de segurança

Para excluir um grupo de segurança, execute o comando [aws ec2 delete-security-group](#).

Note

Não é possível excluir um grupo de segurança se ele estiver atualmente anexado a um ambiente.

O exemplo de comando a seguir exclui um grupo de segurança do EC2-VPC.

```
$ aws ec2 delete-security-group --group-id sg-903004f8
```

Referências

Referência da AWS CLI:

- [aws ec2](#)
- [aws ec2 authorize-security-group-ingress](#)
- [aws ec2 create-security-group](#)
- [aws ec2 delete-security-group](#)
- [aws ec2 describe-security-groups](#)

Outra referência:

- [Documentação do Amazon Elastic Compute Cloud](#)
- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

Inicializar, listar e encerrar instâncias do Amazon EC2

Você pode usar a AWS Command Line Interface (AWS CLI) para iniciar, listar e encerrar instâncias do Amazon Elastic Compute Cloud (Amazon EC2). Se iniciar uma instância que não esteja no nível gratuito da AWS, você será cobrado depois de iniciar a instância e cobrado pelo tempo que a instância estiver em execução, mesmo que ela permaneça inativa.

 Note

Para obter exemplos de outros comandos, consulte o [AWS CLI](#).

Tópicos

- [Pré-requisitos](#)
- [Executar sua instância](#)
- [Adicionar um dispositivo de blocos à instância](#)
- [Adicionar uma etiqueta à instância](#)
- [Conecte-se à sua instância](#)
- [Listar as instâncias](#)
- [Encerrar a instância](#)
- [Referências](#)

Pré-requisitos

Para executar os comandos ec2 neste tópico, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- Defina suas permissões do IAM para permitir acesso ao Amazon EC2. Para obter mais informações sobre permissões do IAM para Amazon EC2, consulte [Políticas do IAM para Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.
- Crie um [par de chaves](#) e um [grupo de segurança](#).
- Selecione uma Imagem de máquina da Amazon (AMI) e anote o ID da AMI. Para obter mais informações, consulte [Como encontrar uma AMI](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

Executar sua instância

Para iniciar uma instância do Amazon EC2 usando a AMI selecionada, use o comando [aws ec2 run-instances](#). É possível iniciar a instância em uma nuvem privada virtual (VPC).

Inicialmente, a instância será exibida no estado `pending`, mas mudará para o estado `running` depois de alguns minutos.

O exemplo a seguir mostra como iniciar uma instância `t2.micro` na sub-rede especificada de uma VPC. Substitua os valores dos parâmetros *em itálico* pelos seus próprios.

```
$ aws ec2 run-instances --image-id ami-xxxxxxx --count 1 --instance-type t2.micro --  
key-name MyKeyPair --security-group-ids sg-903004f8 --subnet-id subnet-6e7f829e  
{  
  "OwnerId": "123456789012",  
  "ReservationId": "r-5875ca20",  
  "Groups": [  
    {  
      "GroupName": "my-sg",  
      "GroupId": "sg-903004f8"  
    }  
  ],  
  "Instances": [  
    {  
      "Monitoring": {  
        "State": "disabled"  
      },  
      "PublicDnsName": null,  
      "Platform": "windows",  
      "State": {  
        "Code": 0,  
        "Name": "pending"  
      },  
      "EbsOptimized": false,  
      "LaunchTime": "2013-07-19T02:42:39.000Z",  
      "PrivateIpAddress": "10.0.1.114",  
      "ProductCodes": [],  
      "VpcId": "vpc-1a2b3c4d",  
      "InstanceId": "i-5203422c",  
      "ImageId": "ami-173d747e",  
      "PrivateDnsName": "ip-10-0-1-114.ec2.internal",  
      "KeyName": "MyKeyPair",  
      "SecurityGroups": [  
        {  
          "GroupName": "my-sg",  
          "GroupId": "sg-903004f8"  
        }  
      ],  
    }  
  ],  
}
```

```
"ClientToken": null,
"SubnetId": "subnet-6e7f829e",
"InstanceType": "t2.micro",
"NetworkInterfaces": [
  {
    "Status": "in-use",
    "SourceDestCheck": true,
    "VpcId": "vpc-1a2b3c4d",
    "Description": "Primary network interface",
    "NetworkInterfaceId": "eni-a7edb1c9",
    "PrivateIpAddresses": [
      {
        "PrivateDnsName": "ip-10-0-1-114.ec2.internal",
        "Primary": true,
        "PrivateIpAddress": "10.0.1.114"
      }
    ],
    "PrivateDnsName": "ip-10-0-1-114.ec2.internal",
    "Attachment": {
      "Status": "attached",
      "DeviceIndex": 0,
      "DeleteOnTermination": true,
      "AttachmentId": "eni-attach-52193138",
      "AttachTime": "2013-07-19T02:42:39.000Z"
    },
    "Groups": [
      {
        "GroupName": "my-sg",
        "GroupId": "sg-903004f8"
      }
    ],
    "SubnetId": "subnet-6e7f829e",
    "OwnerId": "123456789012",
    "PrivateIpAddress": "10.0.1.114"
  }
],
"SourceDestCheck": true,
"Placement": {
  "Tenancy": "default",
  "GroupName": null,
  "AvailabilityZone": "us-west-2b"
},
"Hypervisor": "xen",
"BlockDeviceMappings": [
```



```

        {
            "DeviceName": "/dev/sda1",
            "Ebs": {
                "Status": "attached",
                "DeleteOnTermination": true,
                "VolumeId": "vol-877166c8",
                "AttachTime": "2013-07-19T02:42:39.000Z"
            }
        },
        "Architecture": "x86_64",
        "StateReason": {
            "Message": "pending",
            "Code": "pending"
        },
        "RootDeviceName": "/dev/sda1",
        "VirtualizationType": "hvm",
        "RootDeviceType": "ebs",
        "Tags": [
            {
                "Value": "MyInstance",
                "Key": "Name"
            }
        ],
        "AmiLaunchIndex": 0
    }
]
}

```

Adicionar um dispositivo de blocos à instância

Cada instância lançada tem um volume do dispositivo root associados. Use o mapeamento de dispositivos de blocos para especificar volumes adicionais do Amazon Elastic Block Store (Amazon EBS) ou volumes de armazenamento de instâncias para anexar a uma instância quando ela for iniciada.

Para adicionar um dispositivo de blocos em sua instância, especifique a opção `--block-device-mappings` ao usar `run-instances`.

O exemplo de parâmetro a seguir provisiona um volume do Amazon EBS padrão de 20 GB e o mapeia em sua instância usando o identificador `/dev/sdf`.

```
--block-device-mappings "[{\"DeviceName\":\"/dev/sdf\", \"Ebs\":{\"VolumeSize\":20, \"DeleteOnTermination\":false} }]"
```

O exemplo a seguir adiciona um volume do Amazon EBS mapeado em `/dev/sdf` com base em um snapshot existente. Um snapshot representa uma imagem que é carregada no volume para você. Ao especificar um snapshot, não é necessário especificar um tamanho de volume; ele será grande o suficiente para armazenar sua imagem. No entanto, se você especificar um tamanho, ele deverá ser maior que ou igual ao tamanho do snapshot.

```
--block-device-mappings "[{\"DeviceName\":\"/dev/sdf\", \"Ebs\":{\"SnapshotId\":\"snap-a1b2c3d4\"} }]"
```

O exemplo a seguir adiciona dois volumes à sua instância. O número de volumes disponíveis para sua instância depende do seu tipo de instância.

```
--block-device-mappings "[{\"DeviceName\":\"/dev/sdf\", \"VirtualName\":\"ephemeral0\"}, {\"DeviceName\":\"/dev/sdg\", \"VirtualName\":\"ephemeral1\"}]"
```

O exemplo a seguir cria o mapeamento (`/dev/sdj`), mas não provisiona um volume para a instância.

```
--block-device-mappings "[{\"DeviceName\":\"/dev/sdj\", \"NoDevice\":\"\"}]"
```

Para obter mais informações, consulte [Mapeamento de dispositivos de blocos](#) no Guia do usuário do Amazon EC2 para instâncias Linux.

Adicionar uma etiqueta à instância

Uma etiqueta é um rótulo atribuído a um recurso da AWS. Permite adicionar metadados aos seus recursos que você pode usar para diversas finalidades. Para obter mais informações, consulte [Marcação de recursos](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

O exemplo a seguir mostra como adicionar uma etiqueta com o nome da chave “Name” e o valor “MyInstance” à instância especificada com o comando [aws ec2 create-tags](#).

```
$ aws ec2 create-tags --resources i-5203422c --tags Key=Name,Value=MyInstance
```

Conecte-se à sua instância

Durante a execução da instância, é possível se conectar a ela e usá-la da mesma forma que você usaria um computador. Para obter mais informações, consulte [Conectar à sua instância do Amazon EC2](#) no Guia do usuário do Amazon EC2 para instâncias do Linux.

Listar as instâncias

Use o AWS CLI para listar suas instâncias e visualizar informações sobre elas. Liste todas as suas instâncias, ou filtre os resultados de acordo com as instâncias de interesse.

Os exemplos a seguir mostram como usar o comando [aws ec2 describe-instances](#).

O comando a seguir relaciona todas as suas instâncias.

```
$ aws ec2 describe-instances
```

O comando a seguir filtra a lista apenas para suas instâncias t2.micro e mostra apenas os valores InstanceId para cada correspondência.

```
$ aws ec2 describe-instances --filters "Name=instance-type,Values=t2.micro" --query  
"Reservations[].Instances[].InstanceId"  
[  
    "i-05e998023d9c69f9a"  
]
```

O comando a seguir lista todas as suas instâncias que tem a etiqueta Name=MyInstance.

```
$ aws ec2 describe-instances --filters "Name=tag:Name,Values=MyInstance"
```

O comando a seguir relaciona as instâncias que foram executadas usando qualquer uma das seguintes AMIs: ami-x0123456, ami-y0123456 e ami-z0123456.

```
$ aws ec2 describe-instances --filters "Name=image-id,Values=ami-x0123456,ami-  
y0123456,ami-z0123456"
```

Encerrar a instância

O encerramento de uma instância significa excluí-la. Você não pode se reconectar com uma instância depois de tê-la encerrado.

Assim que o estado da instância de mudar para shutting-down ou para terminated, não há mais custos para essa instância. Se você desejar se reconectar a ela mais tarde, use [stop-instances](#) em vez de `terminate-instances`. Para obter mais informações, consulte [Terminar a instância](#) no Guia do usuário do Amazon EC2 para instâncias do Linux.

Para excluir uma instância, use o comando [aws ec2 terminate-instances](#).

```
$ aws ec2 terminate-instances --instance-ids i-5203422c
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-5203422c",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

Referências

Referência da AWS CLI:

- [aws ec2](#)
- [aws ec2 create-tags](#)
- [aws ec2 describe-instances](#)
- [aws ec2 run-instances](#)
- [aws ec2 terminate-instances](#)

Outra referência:

- [Documentação do Amazon Elastic Compute Cloud](#)
- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

Alterar um tipo de instância do Amazon EC2 com um script bash

Este exemplo de script bash para Amazon EC2 altera o tipo de instância para uma instância do Amazon EC2 usando a AWS Command Line Interface (AWS CLI). Ele interrompe a instância se ela estiver em execução, altera o tipo de instância e, em seguida, se solicitado, a reinicia. Scripts shell são programas desenvolvidos para ser executados em uma interface de linha de comando.

Note

Para obter exemplos de outros comandos, consulte o [AWS CLI](#).

Tópicos

- [Antes de começar](#)
- [Sobre este exemplo](#)
- [Parâmetros](#)
- [Arquivos](#)
- [Referências](#)

Antes de começar

Antes que você possa executar qualquer um dos exemplos abaixo, as seguintes tarefas deverão ser concluídas.

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- O perfil utilizado deve ter permissões que garantem as operações da AWS realizadas pelos exemplos.
- Uma instância do Amazon EC2 em execução na conta para a qual você tem permissão para interromper e modificar. Se você executar o script de teste, ele iniciará uma instância para você, testará alterando o tipo e encerrará a instância.
- Como prática recomendada da AWS, conceda a esse código privilégio mínimo ou apenas as permissões necessárias para executar uma tarefa. Para obter mais informações, consulte [Conceder privilégio mínimo](#) no Manual do usuário do AWS Identity and Access Management (IAM).

- Este código não foi testado em todas as regiões da AWS. Alguns serviços da AWS só estão disponíveis em regiões específicas. Para obter mais informações, consulte [Endpoints de serviço e cotas](#) no Guia de referência geral da AWS.
- Executar este código pode resultar em cobranças em sua conta da AWS. É sua responsabilidade garantir que todos os recursos criados por este script sejam removidos após você terminar de usá-los.

Sobre este exemplo

Este exemplo é escrito como uma função no arquivo de script shell `change_ec2_instance_type.sh` que você pode usar como `source` a partir de outro script ou da linha de comando. Cada arquivo de script contém comentários descrevendo cada uma das funções. Uma vez que a função esteja na memória, você poderá chamá-la via linha de comando. Por exemplo, os seguintes comandos alteram o tipo de instância especificada para `t2.nano`:

```
$ source ./change_ec2_instance_type.sh
$ ./change_ec2_instance_type -i *instance-id* -t new-type
```

Para obter o exemplo completo e arquivos de script para download, consulte [Alterar tipo de instância do Amazon EC2](#) no Repositório de exemplos de código da AWS no GitHub.

Parâmetros

`-i:` (string) especifica o ID da instância a ser modificada.

`-t:` (string) especifica o tipo de instância do Amazon EC2 de destino.

`-r:` (opção) por padrão, não está definida. Se `-r` estiver definido, reiniciará a instância após a alteração do tipo.

`-f:` (opção) por padrão, o script solicita ao usuário que confirme o encerramento da instância antes de fazer a mudança. Se `-f` estiver definida, a função não avisará o usuário antes de encerrar a instância para realizar a alteração de tipo.

`-v:` (opção) por padrão, o script opera silenciosamente e exibe a saída somente em caso de erro. Se `-v` estiver definida, a função exibirá o status ao longo de sua operação.

Arquivos

change_ec2_instance_type.sh

O arquivo de script principal contém a função `change_ec2_instance_type()` que executa as seguintes tarefas:

- Verifica se a instância do Amazon EC2 especificada existe.
- A menos que a opção `-f` esteja selecionada, avisará o usuário antes de interromper a instância.
- Altera o tipo de instância
- Se você definir `-r`, reiniciará a instância e confirmará que a instância está sendo executada

Visualize o código para [change_ec2_instance_type.sh](#) no GitHub.

test_change_ec2_instance_type.sh

O script do arquivo `change_ec2_instance_type_test.sh` testa os vários caminhos de código para a função `change_ec2_instance_type`. Se todas as etapas no script de teste funcionarem corretamente, ele removerá todos os recursos que criou.

Você pode executar o script de teste com os seguintes parâmetros:

- `-v`: (opção) cada teste mostra um status de aprovação/falha conforme eles são executados. Por padrão, os testes são executados silenciosamente e a saída inclui apenas o status final de aprovação/falha.
- `-i`: (opção) o script faz uma pausa após cada teste para permitir que você navegue pelos resultados intermediários de cada etapa. Isso permite examinar o status atual da instância usando o console do Amazon EC2. O script avançará para a próxima etapa após você pressionar ENTER no prompt.

Visualize o código para [test_change_ec2_instance_type.sh](#) no GitHub.

awsdocs_general.sh

O arquivo de script `awsdocs_general.sh` contém funções de uso geral usadas em exemplos avançados para a AWS CLI.

Visualize o código para [awsdocs_general.sh](#) no GitHub.

Referências

Referência da AWS CLI:

- [aws ec2](#)
- [aws ec2 describe-instances](#)
- [aws ec2 modify-instance-attribute](#)
- [aws ec2 start-instances](#)
- [aws ec2 stop-instances](#)
- [aws ec2 wait instance-running](#)
- [aws ec2 wait instance-stopped](#)

Outra referência:

- [Documentação do Amazon Elastic Compute Cloud](#)
- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

Usar o Amazon S3 Glacier com a AWS CLI

Introdução ao Amazon S3 Glacier

[Introdução ao Amazon S3 Glacier](#)

Este tópico mostra exemplos de comandos da AWS CLI que executam tarefas comuns para o S3 Glacier. Os exemplos demonstram como usar a AWS CLI para fazer upload de um arquivo grande para o S3 Glacier dividindo-o em partes menores e fazendo upload a partir da linha de comando.

Você pode acessar os recursos do Amazon S3 Glacier usando a AWS Command Line Interface (AWS CLI). Para listar os comandos da AWS CLI para o S3 Glacier, use o comando a seguir.

```
aws glacier help
```


 Note

Para obter referência de comandos e exemplos adicionais, consulte [aws glacier](#) na Referência de comandos da AWS CLI.

Tópicos

- [Pré-requisitos](#)
- [Criar um cofre do Amazon S3 Glacier](#)
- [Preparar um arquivo para upload](#)
- [Iniciar um multipart upload e fazer upload de arquivos](#)
- [Concluir o upload](#)
- [Recursos](#)

Pré-requisitos

Para executar os comandos `glacier`, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- Este tutorial usa várias ferramentas de linha de comando que normalmente vêm pré-instaladas em sistemas operacionais Unix, incluindo Linux e macOS. Os usuários do Windows podem usar as mesmas ferramentas instalando o [Cygwin](#) e executando os comandos do terminal Cygwin. São observados comandos originários do Windows e utilitários que executam as mesmas funções onde disponível.

Criar um cofre do Amazon S3 Glacier

Crie um cofre com o [create-vault](#).

```
$ aws glacier create-vault --account-id - --vault-name myvault
{
  "location": "/123456789012/vaults/myvault"
}
```

 Note

Todos os comandos do S3 Glacier exigem um parâmetro de ID de conta. Use o caractere de hífen (`--account-id -`) para usar a conta atual.

Preparar um arquivo para upload

Crie um arquivo para o upload de teste. Os comandos a seguir criam um arquivo com o nome *largefile* com exatamente 3 MiB de dados aleatórios.

Linux ou macOS

```
$ dd if=/dev/urandom of=largefile bs=3145728 count=1
1+0 records in
1+0 records out
3145728 bytes (3.1 MB) copied, 0.205813 s, 15.3 MB/s
```

`dd` é um utilitário que copia um número de bytes a partir de um arquivo de entrada para um arquivo de saída. O exemplo anterior usa o arquivo de dispositivo do sistema `/dev/urandom` como uma fonte de dados aleatórios. O `fsutil` executa uma função semelhante no Windows.

Windows

```
C:\> fsutil file createnew largefile 3145728
File C:\temp\largefile is created
```

Em seguida, divida o arquivo em blocos de 1 MiB (1.048.576 bytes).

```
$ split -b 1048576 --verbose largefile chunk
creating file `chunkaa'
creating file `chunkab'
creating file `chunkac'
```

 Note

[HJ-Split](#) é um divisor de arquivos gratuito para Windows e muitas outras plataformas.

Iniciar um multipart upload e fazer upload de arquivos

Crie um upload fracionado no Amazon S3 Glacier usando o comando [initiate-multipart-upload](#).

```
$ aws glacier initiate-multipart-upload --account-id - --archive-description "multipart
upload test" --part-size 1048576 --vault-name myvault
{
  "uploadId": "19gaRezEXAMPLES6Ry5YYdqthHOC_kGRCT03L9yetr220UmPtBYKk-
0ssZtLqyFu7sY1_1R7vgFuJV6NtcV5zpsJ",
  "location": "/123456789012/vaults/myvault/multipart-
uploads/19gaRezEXAMPLES6Ry5YYdqthHOC_kGRCT03L9yetr220UmPtBYKk-
0ssZtLqyFu7sY1_1R7vgFuJV6NtcV5zpsJ"
}
```

O S3 Glacier exige o tamanho de cada parte em bytes (1 MiB neste exemplo), o nome do cofre e uma ID de conta para configurar o upload fracionado. A AWS CLI gera um ID de upload quando a operação é concluída. Salve o ID de upload para um shell variável para uso posterior.

Linux ou macOS

```
$ UPLOADID="19gaRezEXAMPLES6Ry5YYdqthHOC_kGRCT03L9yetr220UmPtBYKk-
0ssZtLqyFu7sY1_1R7vgFuJV6NtcV5zpsJ"
```

Windows

```
C:\> set UPLOADID="19gaRezEXAMPLES6Ry5YYdqthHOC_kGRCT03L9yetr220UmPtBYKk-
0ssZtLqyFu7sY1_1R7vgFuJV6NtcV5zpsJ"
```

Depois, use o comando [upload-multipart-part](#) para fazer upload das três partes.

```
$ aws glacier upload-multipart-part --upload-id $UPLOADID --body chunkaa --range 'bytes
0-1048575/*' --account-id - --vault-name myvault
{
  "checksum": "e1f2a7cd6e047fa606fe2f0280350f69b9f8cfa602097a9a026360a7edc1f553"
}
$ aws glacier upload-multipart-part --upload-id $UPLOADID --body chunkab --range 'bytes
1048576-2097151/*' --account-id - --vault-name myvault
{
  "checksum": "e1f2a7cd6e047fa606fe2f0280350f69b9f8cfa602097a9a026360a7edc1f553"
```

```
}  
$ aws glacier upload-multipart-part --upload-id $UPLOADID --body chunkac --range 'bytes  
2097152-3145727/*' --account-id - --vault-name myvault  
{  
  "checksum": "e1f2a7cd6e047fa606fe2f0280350f69b9f8cfa602097a9a026360a7edc1f553"  
}
```

Note

O exemplo anterior usa o sinal de dólar (\$) para fazer referência ao conteúdo da variável de shell UPLOADID no Linux. Na linha de comando do Windows, use um sinal de porcentagem (%) nos dois lados do nome da variável (por exemplo, %UPLOADID%).

Especifique o intervalo de bytes de cada parte ao fazer upload para que o S3 Glacier possa remontá-lo na ordem correta. Cada parte é 1.048.576 bytes, portanto, a primeira parte ocupa 0-1048575 bytes, a segunda 1048576-2097151 e a terceira 2097152-3145727.

Concluir o upload

O Amazon S3 Glacier exige uma árvore hash do arquivo original para confirmar que todas as partes carregadas chegaram à AWS intactas.

Para calcular uma árvore hash, você deve dividir o arquivo em partes de 1 MiB e calcular um binário SHA-256 hash de cada item. Em seguida, divida a lista de hashes em pares, combine os dois binários hashes em cada par e execute hashes dos resultados. Repita esse processo até que haja apenas um hash à esquerda. Se houver um número ímpar de hashes em qualquer nível, envie para o próximo nível sem modificá-lo.

A chave para calcular uma árvore hash corretamente ao usar os utilitários de linha de comando é armazenar cada hash em formato binário e converter apenas para hexadecimal na última etapa. A combinação ou hash de qualquer versão hexadecimal hash em árvore gerará um resultado incorreto.

Note

Os usuários do Windows podem usar o comando type em vez do cat. OpenSSL está disponível para Windows em [OpenSSL.org](https://www.openssl.org).

Para calcular uma árvore hash

1. Se ainda não fez isso, divida o arquivo original em partes de 1 MiB.

```
$ split --bytes=1048576 --verbose largefile chunk
creating file `chunkaa'
creating file `chunkab'
creating file `chunkac'
```

2. Calcule e armazene o hash SHA-256 binário de cada fragmento.

```
$ openssl dgst -sha256 -binary chunkaa > hash1
$ openssl dgst -sha256 -binary chunkab > hash2
$ openssl dgst -sha256 -binary chunkac > hash3
```

3. Combine os primeiros dois hashes e execute o hash binário do resultado.

```
$ cat hash1 hash2 > hash12
$ openssl dgst -sha256 -binary hash12 > hash12hash
```

4. Combine o pai de partes de hash aa e ab com o hash de bloco ac e o resultado do hash, desta vez exibindo hexadecimal. Armazene o resultado em um shell variável.

```
$ cat hash12hash hash3 > hash123
$ openssl dgst -sha256 hash123
SHA256(hash123)= 9628195fcdcbbbe76cdde932d4646fa7de5f219fb39823836d81f0cc0e18aa67
$ TREEHASH=9628195fcdcbbbe76cdde932d4646fa7de5f219fb39823836d81f0cc0e18aa67
```

Por fim, preencha o upload com o comando [complete-multipart-upload](#). Este comando usa o tamanho do arquivo original em bytes, o valor de hash em árvore final em hexadecimal, e a ID da conta e nome do cofre.

```
$ aws glacier complete-multipart-upload --checksum $TREEHASH --archive-size 3145728 --
upload-id $UPLOADID --account-id - --vault-name myvault
{
  "archiveId": "d3AbWhE0YE1m6f_fI1jPG82F8xzbMEEZmrAllGAA0NJAzo5QdP-
N83MKqd96Unspoa5H5lItWX-sK8-QS0ZhwsyGiu9-R-
kwWUyS1dSB1mgPPWkEbeFfqDSav053rU7FvVLHfRc6hg",
  "checksum": "9628195fcdcbbbe76cdde932d4646fa7de5f219fb39823836d81f0cc0e18aa67",
```

```
"location": "/123456789012/vaults/myvault/archives/
d3AbWhE0YE1m6f_fI1jPG82F8xzbMEEZmrAllGAA0NJAzo5QdP-N83MKqd96Unspoa5H5lItWX-sK8-
QS0ZhwsyGiu9-R-kwUyS1dSB1mgPPWkEbeFfqDSav053rU7FvVLHfRc6hg"
}
```

Também é possível verificar o status do cofre usando o comando [describe-vault](#).

```
$ aws glacier describe-vault --account-id - --vault-name myvault
{
  "SizeInBytes": 3178496,
  "VaultARN": "arn:aws:glacier:us-west-2:123456789012:vaults/myvault",
  "LastInventoryDate": "2018-12-07T00:26:19.028Z",
  "NumberOfArchives": 1,
  "CreationDate": "2018-12-06T21:23:45.708Z",
  "VaultName": "myvault"
}
```

Note

O status do cofre é atualizado cerca de uma vez por dia. Consulte [Como trabalhar com cofres](#) para obter mais informações.

Já é seguro remover o bloco e os arquivos de hash que você criou.

```
$ rm chunk* hash*
```

Para obter mais informações sobre uploads fracionados, consulte [Upload de arquivos grandes em partes](#) e [Como computar somas de verificação](#) no Guia do desenvolvedor do Amazon S3 Glacier.

Recursos

Referência da AWS CLI:

- [aws glacier](#)
- [aws glacier complete-multipart-upload](#)
- [aws glacier create-vault](#)
- [aws glacier describe-vault](#)
- [aws glacier initiate-multipart-upload](#)

Referência do serviço:

- [Guia do desenvolvedor do Amazon S3 Glacier](#)
- [Carregar arquivos grandes em partes](#) no Guia do desenvolvedor do Amazon S3 Glacier
- [Calcular somas de verificação](#) no Guia do desenvolvedor do Amazon S3 Glacier
- [Trabalhar com vaults](#) no Guia do desenvolvedor do Amazon S3 Glacier

Uso do AWS Identity and Access Management pela AWS CLI

Uma introdução à AWS Identity and Access Management.

[Introdução à AWS Identity and Access Management](#)

Acesse os recursos do AWS Identity and Access Management (IAM) usando a AWS Command Line Interface (AWS CLI). Para listar os comandos da AWS CLI para o IAM use o comando a seguir.

```
aws iam help
```

Este tópico mostra exemplos de comandos da AWS CLI que executam tarefas comuns para o IAM.

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Tópicos

- [Criar usuários e grupos do IAM](#)
- [Anexar uma política gerenciada do IAM a um usuário](#)
- [Definir uma senha inicial para um usuário do IAM](#)
- [Criar uma chave de acesso para um usuário do IAM](#)

Criar usuários e grupos do IAM

Este tópico descreve como usar os comandos da AWS Command Line Interface (AWS CLI) para criar um grupo do AWS Identity and Access Management (IAM) e um usuário, depois adicionar o

usuário ao grupo. Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Como criar um grupo e adicionar um novo usuário a ele

1. Use o comando [create-group](#) para criar o grupo.

```
$ aws iam create-group --group-name MyIamGroup
{
  "Group": {
    "GroupName": "MyIamGroup",
    "CreateDate": "2018-12-14T03:03:52.834Z",
    "GroupId": "AGPAJNUJ2W4IJVEXAMPLE",
    "Arn": "arn:aws:iam::123456789012:group/MyIamGroup",
    "Path": "/"
  }
}
```

2. Use o comando [create-user](#) para criar o usuário.

```
$ aws iam create-user --user-name MyUser
{
  "User": {
    "UserName": "MyUser",
    "Path": "/",
    "CreateDate": "2018-12-14T03:13:02.581Z",
    "UserId": "AIDAJY2PE5XUZ4EXAMPLE",
    "Arn": "arn:aws:iam::123456789012:user/MyUser"
  }
}
```

3. Use o comando [add-user-to-group](#) para adicionar o usuário ao grupo.

```
$ aws iam add-user-to-group --user-name MyUser --group-name MyIamGroup
```

4. Para verificar se o grupo MyIamGroup contém MyUser, use o comando [get-group](#).

```
$ aws iam get-group --group-name MyIamGroup
{
  "Group": {
```



```

    "GroupName": "MyIamGroup",
    "CreateDate": "2018-12-14T03:03:52Z",
    "GroupId": "AGPAJNUJ2W4IJVEXAMPLE",
    "Arn": "arn:aws:iam::123456789012:group/MyIamGroup",
    "Path": "/"
  },
  "Users": [
    {
      "UserName": "MyUser",
      "Path": "/",
      "CreateDate": "2018-12-14T03:13:02Z",
      "UserId": "AIDAJY2PE5XUZ4EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/MyUser"
    }
  ],
  "IsTruncated": "false"
}

```

Anexar uma política gerenciada do IAM a um usuário

Este tópico descreve como usar os comandos da AWS Command Line Interface (AWS CLI) para associar uma política do AWS Identity and Access Management (IAM) a um usuário. Neste exemplo, a política fornece ao usuário “Acesso de Usuário Power”. Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Como associar uma política gerenciada do IAM a um usuário

1. Determine o nome de recurso da Amazon (ARN) da política a ser anexada. O comando a seguir usa `list-policies` para encontrar o ARN da política com o nome `PowerUserAccess`. Depois, ele armazena o ARN em uma variável de ambiente.

```

$ export POLICYARN=$(aws iam list-policies --query 'Policies[?
PolicyName==`PowerUserAccess`].{ARN:Arn}' --output text) ~
$ echo $POLICYARN
arn:aws:iam::aws:policy/PowerUserAccess

```

2. Para anexar a política, use o comando [attach-user-policy](#) e faça referência à variável de ambiente que contém o ARN da política.

```
$ aws iam attach-user-policy --user-name MyUser --policy-arn $POLICYARN
```

3. Verifique se a política foi anexada ao usuário executando o comando [list-attached-user-policies](#).

```
$ aws iam list-attached-user-policies --user-name MyUser
{
  "AttachedPolicies": [
    {
      "PolicyName": "PowerUserAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/PowerUserAccess"
    }
  ]
}
```

Para obter mais informações, consulte [Recursos de gerenciamento de acesso](#). Este tópico fornece links para uma visão geral das permissões e políticas, bem como links para exemplos de políticas de acesso ao Amazon S3, Amazon EC2 e outros serviços.

Definir uma senha inicial para um usuário do IAM

Este tópico descreve como usar os comandos da AWS Command Line Interface (AWS CLI) para definir uma senha inicial para um usuário do AWS Identity and Access Management (IAM). Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

O comando a seguir usa [create-login-profile](#) para definir uma senha inicial para o usuário especificado. Quando o usuário faz login pela primeira vez, ele precisa alterar a senha para algo que apenas ele sabe.

```
$ aws iam create-login-profile --user-name MyUser --password My!User1Login8P@ssword --password-reset-required
{
  "LoginProfile": {
    "UserName": "MyUser",
    "CreateDate": "2018-12-14T17:27:18Z",
```

```
    "PasswordResetRequired": true
  }
}
```

Você pode usar o comando `update-login-profile` para alterar a senha para um usuário.

```
$ aws iam update-login-profile --user-name MyUser --password My!User1ADifferentP@ssword
```

Criar uma chave de acesso para um usuário do IAM

Este tópico descreve como usar os comandos da AWS Command Line Interface (AWS CLI) para criar um conjunto de chaves de acesso para um usuário do AWS Identity and Access Management (IAM). Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Você pode usar o comando [create-access-key](#) para criar uma chave de acesso para um usuário. Uma chave de acesso é um conjunto de credenciais de segurança que consiste em um ID de chave de acesso e uma chave secreta.

Um usuário pode criar apenas duas chaves de acesso ao mesmo tempo. Se você tentar criar um terceiro conjunto, o comando retornará um erro `LimitExceeded`.

```
$ aws iam create-access-key --user-name MyUser
{
  "AccessKey": {
    "UserName": "MyUser",
    "AccessKeyId": "AKIAIOSFODNN7EXAMPLE",
    "Status": "Active",
    "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY",
    "CreateDate": "2018-12-14T17:34:16Z"
  }
}
```

Use o comando [delete-access-key](#) para excluir uma chave de acesso para um usuário. Especifique qual chave de acesso deve ser excluída usando o ID de chave de acesso.

```
$ aws iam delete-access-key --user-name MyUser --access-key-id AKIAIOSFODNN7EXAMPLE
```

Uso do Amazon S3 com a AWS CLI

Introdução ao Amazon Simple Storage Service (Amazon S3)

[Introdução ao Amazon Simple Storage Service \(Amazon S3\): armazenamento na nuvem na AWS](#)

É possível acessar os recursos do Amazon Simple Storage Service (Amazon S3) usando a AWS Command Line Interface (AWS CLI). A AWS CLI oferece dois níveis de comandos para acessar o Amazon S3:

- `s3`: comandos de alto nível que simplificam a execução de tarefas comuns, como criar, manipular e excluir objetos e buckets.
- `s3api`: expõe o acesso direto a todas as operações da API do Amazon S3, o que permite realizar operações avançadas.

Tópicos neste guia:

- [Uso de comandos de alto nível \(s3\) com a AWS CLI](#)
- [Uso de comandos em nível de API \(s3api\) com a AWS CLI](#)
- [Exemplo de script de operações de ciclo de vida de bucket Amazon S3](#)

Uso de comandos de alto nível (s3) com a AWS CLI

Este tópico descreve alguns dos comandos que você pode utilizar para gerenciar buckets e objetos do Amazon S3 usando os comandos [aws s3](#) na AWS CLI. Para comandos não abordados neste tópico e exemplos de comandos adicionais, consulte os comandos [aws s3](#) na Referência da AWS CLI.

Os comandos `aws s3` de alto nível simplificam o gerenciamento de objetos do Amazon S3. Esses comandos permitem a você gerenciar o conteúdo do Amazon S3 dentro dele mesmo e com diretórios locais.

Tópicos

- [Pré-requisitos](#)
- [Antes de começar](#)
- [Criar um bucket](#)

- [Listar buckets e objetos](#)
- [Excluir buckets](#)
- [Excluir objetos](#)
- [Mover objetos](#)
- [Copiar objetos](#)
- [Sincronizar objetos](#)
- [Opções usadas com frequência para comandos s3](#)
- [Recursos](#)

Pré-requisitos

Para executar os comandos s3, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- O perfil utilizado deve ter permissões que garantem as operações da AWS realizadas pelos exemplos.
- Entenda estes termos do Amazon S3:
 - Bucket: uma pasta de nível superior do Amazon S3.
 - Prefixo: uma pasta do Amazon S3 em um bucket.
 - Objeto: qualquer item hospedado em um bucket do Amazon S3.

Antes de começar

Esta seção descreve algumas coisas a serem observadas antes de usar comandos da aws s3.

Carregamentos de objetos grandes

Quando você usa comandos da aws s3 para fazer upload de objetos grandes em um bucket do Amazon S3, a AWS CLI executa automaticamente um upload fracionado. Quando esses comandos da aws s3 são usados, não é possível continuar uploads que falharam.

Se o upload fracionado falhar devido a um tempo limite excedido, ou se você o cancelou manualmente na AWS CLI, a AWS CLI interromperá o upload e limpará todos os arquivos que foram criados. Esse processo pode levar alguns minutos.

Se o processo de upload fracionado ou limpeza for cancelado por um comando kill ou uma falha do sistema, os arquivos criados permanecerão no bucket do Amazon S3. Para limpar o carregamento fracionado, use o comando [s3api abort-multipart-upload](#)

Propriedades e tags de arquivos em cópias com várias partes

Quando você usa a versão da AWS CLI versão 1 dos comandos no namespace do `aws s3` para copiar um arquivo de um local de bucket do Amazon S3 para outro local de bucket do Amazon S3 e essa operação usa a [cópia fracionada](#), nenhuma propriedade de arquivo do objeto de origem é copiada para o objeto de destino.

Por padrão, os comandos da AWS CLI versão 2 no namespace do `s3` que executam cópias de várias partes agora transferem todas as etiquetas e o seguinte conjunto de propriedades da cópia de origem para a cópia de destino: `content-type`, `content-language`, `content-encoding`, `content-disposition`, `cache-control`, `expires` e `metadata`.

Isso pode resultar em chamadas de API da AWS adicionais para o endpoint do Amazon S3 que não teriam sido feitas se você tivesse usado a AWS CLI versão 1. Elas podem incluir: `HeadObject`, `GetObjectTagging` e `PutObjectTagging`.

Se você precisar alterar esse comportamento padrão em comandos da AWS CLI versão 2, use o parâmetro `--copy-props` para especificar uma das seguintes opções:

- `default`: o valor padrão. Especifica que a cópia inclui todas as tags anexadas ao objeto de origem e as propriedades englobadas pelo parâmetro `--metadata-directive` usado para cópias que não são multipart: `content-type`, `content-language`, `content-encoding`, `content-disposition`, `cache-control`, `expires` e `metadata`.
- `metadata-directive`: especifica que a cópia inclui apenas as propriedades que são englobadas pelo parâmetro `--metadata-directive` usado para cópias não fracionadas. Não copia nenhuma etiqueta.
- `none`: especifica que a cópia não inclui nenhuma das propriedades do objeto de origem.

Criar um bucket

Use o comando [s3 mb](#) para criar um bucket. Os nomes de bucket devem ser globalmente exclusivos (únicos em todo o Amazon S3) e devem ser compatíveis com o DNS.

Nomes de bucket podem conter letras minúsculas, números, hífen e pontos. Os nomes de bucket podem iniciar e terminar apenas com uma letra ou número e não podem conter um ponto ao lado de um hífen ou outro ponto.

Sintaxe

```
$ aws s3 mb <target> [--options]
```

Exemplos de s3 mb

O exemplo a seguir cria o bucket `s3://bucket-name`.

```
$ aws s3 mb s3://bucket-name
```

Listar buckets e objetos

Para listar seus buckets, pastas ou objetos, use o comando [s3 ls](#). O uso do comando sem um destino ou opções lista todos os buckets.

Sintaxe

```
$ aws s3 ls <target> [--options]
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter uma lista completa de opções disponíveis, consulte [s3 ls](#) na Referência de comandos da AWS CLI.

Exemplos de s3 ls

O exemplo a seguir lista todos os seus buckets do Amazon S3.

```
$ aws s3 ls
2018-12-11 17:08:50 my-bucket
2018-12-14 14:55:44 my-bucket2
```

O comando a seguir lista todos os objetos e prefixos em um bucket. Neste exemplo de saída, o prefixo `example/` tem um arquivo chamado `MyFile1.txt`.

```
$ aws s3 ls s3://bucket-name
                                PRE example/
2018-12-04 19:05:48              3 MyFile1.txt
```

Você pode filtrar a saída para um prefixo específico, incluindo-o no comando. O comando a seguir relaciona os objetos em *bucket-name/example* (ou seja, objetos em *bucket-name* filtrados pelo prefixo *example/*).

```
$ aws s3 ls s3://bucket-name/example/
2018-12-06 18:59:32          3 MyFile1.txt
```

Excluir buckets

Para excluir um bucket, use o comando [s3 rb](#).

Sintaxe

```
$ aws s3 rb <target> [--options]
```

Exemplos de s3 rb

O exemplo a seguir remove o bucket `s3://bucket-name`.

```
$ aws s3 rb s3://bucket-name
```

Por padrão, o bucket deve estar vazio para a operação ser bem-sucedida. Para remover um bucket que não está vazio, é necessário incluir a opção `--force`. Se você estiver usando um bucket com versionamento que contém objetos anteriormente excluídos, mas ainda retidos, esse comando não permitirá que você remova o bucket. Você deve primeiro remover todo o conteúdo.

O exemplo de comando a seguir exclui todos os objetos no bucket e, em seguida, exclui o bucket.

```
$ aws s3 rb s3://bucket-name --force
```

Excluir objetos

Para excluir objetos em um bucket ou seu diretório local, use o comando [s3 rm](#).

Sintaxe

```
$ aws s3 rm <target> [--options]
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter uma lista completa de opções, consulte [s3 rm](#) na Referência de comandos da AWS CLI.

Exemplos de s3 rm

O exemplo a seguir exclui `filename.txt` de `s3://bucket-name/example`.

```
$ aws s3 rm s3://bucket-name/example/filename.txt
```

O exemplo a seguir exclui todos os objetos de `s3://bucket-name/example` usando a opção `--recursive`.

```
$ aws s3 rm s3://bucket-name/example --recursive
```

Mover objetos

Use o comando [s3 mv](#) para mover objetos de um bucket ou diretório local.

Sintaxe

```
$ aws s3 mv <source> <target> [--options]
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter uma lista completa de opções disponíveis, consulte [s3 mv](#) na Referência de comandos da AWS CLI.

Exemplos de s3 mv

O exemplo a seguir move todos os objetos de `s3://bucket-name/example` para `s3://my-bucket/`.

```
$ aws s3 mv s3://bucket-name/example s3://my-bucket/
```

O exemplo a seguir move um arquivo local do diretório de trabalho atual para o bucket do Amazon S3 com o comando `s3 mv`.

```
$ aws s3 mv filename.txt s3://bucket-name
```

O exemplo a seguir move um arquivo do bucket do Amazon S3 para o diretório de trabalho atual, onde `./` especifica o diretório de trabalho atual.

```
$ aws s3 mv s3://bucket-name/filename.txt ./
```

Copiar objetos

Use o comando [s3 cp](#) para copiar objetos de um bucket ou diretório local.

Sintaxe

```
$ aws s3 cp <source> <target> [--options]
```

Você pode usar o parâmetro dash para transmitir arquivos para entrada padrão (stdin) ou saída padrão (stdout).

Warning

Se estiver usando o PowerShell, o shell poderá alterar a codificação de um CRLF ou adicionar um CRLF à entrada ou saída encadeada ou à saída redirecionada.

O comando `s3 cp` usa a sintaxe a seguir para fazer upload de um fluxo de arquivos do stdin para um bucket especificado.

Sintaxe

```
$ aws s3 cp - <target> [--options]
```

O comando `s3 cp` usa a sintaxe a seguir para baixar um fluxo de arquivos do Amazon S3 para stdout.

Sintaxe

```
$ aws s3 cp <target> [--options] -
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter a lista completa de opções, consulte [s3 cp](#) na Referência de comandos da AWS CLI.

Exemplos do `s3 cp`

O exemplo a seguir copia todos os objetos de `s3://bucket-name/example` para `s3://my-bucket/`.

```
$ aws s3 cp s3://bucket-name/example s3://my-bucket/
```

O exemplo a seguir copia um arquivo local do diretório de trabalho atual para o bucket do Amazon S3 com o comando `s3 cp`.

```
$ aws s3 cp filename.txt s3://bucket-name
```

O exemplo a seguir copia um arquivo do bucket do Amazon S3 para o diretório de trabalho atual, onde `./` especifica o diretório de trabalho atual.

```
$ aws s3 cp s3://bucket-name/filename.txt ./
```

O exemplo a seguir usa `echo` para transmitir o texto “hello world” para o arquivo `s3://bucket-name/filename.txt`.

```
$ echo "hello world" | aws s3 cp - s3://bucket-name/filename.txt
```

O exemplo a seguir transmite o arquivo `s3://bucket-name/filename.txt` para `stdout` e imprime o conteúdo no console.

```
$ aws s3 cp s3://bucket-name/filename.txt -  
hello world
```

O exemplo a seguir transmite o conteúdo de `s3://bucket-name/pre` para `stdout`, usa o comando `bzip2` para comprimir os arquivos e faz upload do novo arquivo compactado chamado `key.bz2` para `s3://bucket-name`.

```
$ aws s3 cp s3://bucket-name/pre - | bzip2 --best | aws s3 cp - s3://bucket-name/  
key.bz2
```

Sincronizar objetos

O comando [s3 sync](#) sincroniza o conteúdo de um bucket e um diretório ou o conteúdo de dois buckets. Normalmente, `s3 sync` copia arquivos que estão faltando ou desatualizadas ou objetos entre a origem e o destino. No entanto, você também pode fornecer a opção `--delete` para remover arquivos ou objetos do destino que não estão presentes na origem.

Sintaxe

```
$ aws s3 sync <source> <target> [--options]
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter uma lista completa de opções, consulte [s3 sync](#) na Referência de comandos da AWS CLI.

Exemplos de s3 sync

O exemplo a seguir sincroniza o conteúdo de um prefixo do Amazon S3 chamado path no bucket chamado my-bucket com o diretório de trabalho atual.

s3 sync atualiza todos os arquivos que têm tamanho ou hora de modificação diferentes dos arquivos com o mesmo nome no destino. A saída exibe operações específicas executadas durante a sincronização. Observe que a operação sincroniza recursivamente o subdiretório MySubdirectory e seu conteúdo com s3://my-bucket/path/MySubdirectory.

```
$ aws s3 sync . s3://my-bucket/path
upload: MySubdirectory\MyFile3.txt to s3://my-bucket/path/MySubdirectory/MyFile3.txt
upload: MyFile2.txt to s3://my-bucket/path/MyFile2.txt
upload: MyFile1.txt to s3://my-bucket/path/MyFile1.txt
```

O exemplo a seguir, uma extensão do exemplo anterior, mostra como usar a opção --delete.

```
// Delete local file
$ rm ./MyFile1.txt

// Attempt sync without --delete option - nothing happens
$ aws s3 sync . s3://my-bucket/path

// Sync with deletion - object is deleted from bucket
$ aws s3 sync . s3://my-bucket/path --delete
delete: s3://my-bucket/path/MyFile1.txt

// Delete object from bucket
$ aws s3 rm s3://my-bucket/path/MySubdirectory/MyFile3.txt
delete: s3://my-bucket/path/MySubdirectory/MyFile3.txt

// Sync with deletion - local file is deleted
$ aws s3 sync s3://my-bucket/path . --delete
```

```
delete: MySubdirectory\MyFile3.txt

// Sync with Infrequent Access storage class
$ aws s3 sync . s3://my-bucket/path --storage-class STANDARD_IA
```

Quando a opção `--delete` é usada, as opções `--exclude` e `--include` podem filtrar arquivos ou objetos a serem excluídos durante uma operação `s3 sync`. Nesse caso, a sequência de parâmetro deve especificar os arquivos a serem excluídos ou incluídos, a exclusão no contexto do diretório de destino ou bucket. Por exemplo:

```
Assume local directory and s3://my-bucket/path currently in sync and each contains 3
files:
MyFile1.txt
MyFile2.rtf
MyFile88.txt
...

// Sync with delete, excluding files that match a pattern. MyFile88.txt is deleted,
while remote MyFile1.txt is not.
$ aws s3 sync . s3://my-bucket/path --delete --exclude "path/MyFile?.txt"
delete: s3://my-bucket/path/MyFile88.txt
...

// Sync with delete, excluding MyFile2.rtf - local file is NOT deleted
$ aws s3 sync s3://my-bucket/path . --delete --exclude "./MyFile2.rtf"
download: s3://my-bucket/path/MyFile1.txt to MyFile1.txt
...

// Sync with delete, local copy of MyFile2.rtf is deleted
$ aws s3 sync s3://my-bucket/path . --delete
delete: MyFile2.rtf
```

Opções usadas com frequência para comandos s3

As opções a seguir são usadas frequentemente para os comandos descritos neste tópico. Para obter uma lista completa de opções que você pode usar em um comando, consulte o comando específico no [AWS CLI versão 2](#).

acl

`s3 sync` e `s3 cp` podem usar a opção `--acl`. Isso permite definir as permissões de acesso para arquivos copiados para o Amazon S3. A opção `--acl` aceita os valores `private`, `public-`

reade public-read-write. Para obter mais informações, consulte [ACL pré-configurada](#) no Manual do usuário do Amazon Simple Storage Service.

```
$ aws s3 sync . s3://my-bucket/path --acl public-read
```

exclude

Ao usar o comando `s3 cp`, `s3 mv`, `s3 sync` ou `s3 rm`, você pode filtrar os resultados usando a opção `--exclude` ou `--include`. A opção `--exclude` define regras para excluir apenas objetos do comando, e as opções se aplicam na ordem especificada. Isso é mostrado no exemplo a seguir.

```
Local directory contains 3 files:
MyFile1.txt
MyFile2.rtf
MyFile88.txt

// Exclude all .txt files, resulting in only MyFile2.rtf being copied
$ aws s3 cp . s3://my-bucket/path --exclude "*.txt"

// Exclude all .txt files but include all files with the "MyFile*.txt" format,
resulting in, MyFile1.txt, MyFile2.rtf, MyFile88.txt being copied
$ aws s3 cp . s3://my-bucket/path --exclude "*.txt" --include "MyFile*.txt"

// Exclude all .txt files, but include all files with the "MyFile*.txt" format,
but exclude all files with the "MyFile?.txt" format resulting in, MyFile2.rtf and
MyFile88.txt being copied
$ aws s3 cp . s3://my-bucket/path --exclude "*.txt" --include "MyFile*.txt" --
exclude "MyFile?.txt"
```

include

Ao usar o comando `s3 cp`, `s3 mv`, `s3 sync` ou `s3 rm`, você pode filtrar os resultados usando a opção `--exclude` ou `--include`. A opção `--include` define regras para incluir apenas objetos especificados para o comando, e as opções se aplicam na ordem especificada. Isso é mostrado no exemplo a seguir.

```
Local directory contains 3 files:
MyFile1.txt
MyFile2.rtf
MyFile88.txt
```

```
// Include all .txt files, resulting in MyFile1.txt and MyFile88.txt being copied
$ aws s3 cp . s3://my-bucket/path --include "*.txt"

// Include all .txt files but exclude all files with the "MyFile*.txt" format,
// resulting in no files being copied
$ aws s3 cp . s3://my-bucket/path --include "*.txt" --exclude "MyFile*.txt"

// Include all .txt files, but exclude all files with the "MyFile*.txt" format, but
// include all files with the "MyFile?.txt" format resulting in MyFile1.txt being
// copied
$ aws s3 cp . s3://my-bucket/path --include "*.txt" --exclude "MyFile*.txt" --
include "MyFile?.txt"
```

grant (conceder)

Os comandos `s3 cp`, `s3 mv` e `s3 sync` incluem uma opção `--grants` que pode ser usada para conceder permissões referentes a esse objeto para usuários ou grupos específicos. Defina a opção `--grants` como uma lista de permissões usando a sintaxe a seguir. Substitua `Permission`, `Grantee_Type` e `Grantee_ID` pelos seus próprios valores.

Sintaxe

```
--grants Permission=Grantee_Type=Grantee_ID
        [Permission=Grantee_Type=Grantee_ID ...]
```

Cada valor contém os seguintes elementos:

- *Permission* (Permissão): especifica as permissões concedidas. Pode ser definida como `read`, `readacl`, `writeacl` ou `full`.
- *Grantee_Type* (Tipo de favorecido): especifica como identificar o favorecido. Pode ser definido como `uri`, `emailaddress` ou `id`.
- *Grantee_ID* – Especifica o favorecido com base em *Grantee_Type*.
 - `uri`: o URI do grupo. Para obter mais informações, consulte [Quem é o favorecido?](#)
 - `emailaddress` - O endereço de e-mail da conta.
 - `id` - O ID canônico da conta

Para obter mais informações sobre controle de acesso do Amazon S3, consulte [Controle de acesso](#).

O exemplo a seguir copia um objeto em um bucket. Ele concede permissões `read` sobre o objeto a todos os usuários e permissões `full` (`read`, `readacl` e `writeacl`) à conta associada a `user@example.com`.

```
$ aws s3 cp file.txt s3://my-bucket/ --grants read=uri=http://acs.amazonaws.com/groups/global/AllUsers full=emailaddress=user@example.com
```

Também será possível especificar uma classe de armazenamento não padrão (`REDUCED_REDUNDANCY` ou `STANDARD_IA`) para objetos cujo upload você fizer no Amazon S3. Para fazer isso, use a opção `--storage-class`.

```
$ aws s3 cp file.txt s3://my-bucket/ --storage-class REDUCED_REDUNDANCY
```

recursive

Quando você usa essa opção, o comando é executado em todos os arquivos ou objetos sob o diretório ou prefixo especificado. O exemplo a seguir exclui `s3://my-bucket/path` e todo o seu conteúdo.

```
$ aws s3 rm s3://my-bucket/path --recursive
```

Recursos

Referência da AWS CLI:

- [aws s3](#)
- [aws s3 cp](#)
- [aws s3 mb](#)
- [aws s3 mv](#)
- [aws s3 ls](#)
- [aws s3 rb](#)
- [aws s3 rm](#)
- [aws s3 sync](#)

Referência do serviço:

- [Como trabalhar com buckets do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como trabalhar com objetos do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como listar chaves hierarquicamente usando um prefixo e delimitador](#) no Manual do usuário do Amazon Simple Storage Service
- [Anular carregamentos fracionados para um bucket do S3 usando o AWS SDK for .NET \(nível baixo\)](#) no Manual do usuário do Amazon Simple Storage Service

Uso de comandos em nível de API (s3api) com a AWS CLI

Os comandos no nível de API (contidos no conjunto de comandos `s3api`) fornecem acesso direto às APIs do Amazon Simple Storage Service (Amazon S3) e ativam algumas operações que não são expostas em comandos de alto nível do `s3`. Esses comandos são equivalentes aos outros serviços da AWS que fornecem acesso em nível de API para a funcionalidade de serviços. Para obter mais informações sobre os comando do `s3`, consulte [Uso de comandos de alto nível \(s3\) com a AWS CLI](#)

Este tópico fornece exemplos que demonstram como usar os comandos de nível inferior que são mapeados em APIs do Amazon S3. Além disso, você pode encontrar exemplos para cada comando da API do S3 na seção `s3api` do [AWS CLI versão 2](#).

Tópicos

- [Pré-requisitos](#)
- [Aplicar uma ACL personalizada](#)
- [Configurar uma política de registro em log](#)
- [Recursos](#)

Pré-requisitos

Para executar os comandos `s3api`, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- O perfil utilizado deve ter permissões que garantem as operações da AWS realizadas pelos exemplos.
- Entenda estes termos do Amazon S3:

- Bucket: uma pasta de nível superior do Amazon S3.
- Prefixo: uma pasta do Amazon S3 em um bucket.
- Objeto: qualquer item hospedado em um bucket do Amazon S3.

Aplicar uma ACL personalizada

Com comandos de alto nível, é possível usar a opção `--acl` para aplicar listas de controle de acesso (ACLs) predefinidas a objetos do Amazon S3. No entanto, não é possível usar esse comando para definir ACLs em todo o bucket. No entanto, você pode fazer isso usando o comando em nível de API [put-bucket-acl](#).

O exemplo a seguir mostra como conceder o controle total a dois usuários do AWS (user1@example.com e user2@example.com) e a permissão de leitura a todos. O identificador para “todos” vem de um URI especial que você transmite como um parâmetro.

```
$ aws s3api put-bucket-acl --bucket MyBucket --grant-full-control  
'emailaddress="user1@example.com",emailaddress="user2@example.com"' --grant-read  
'uri="http://acs.amazonaws.com/groups/global/AllUsers"'
```

Para obter detalhes sobre como construir as ACLs, consulte [PUT Bucket acl](#) na Referência da API do Amazon Simple Storage Service. Os comandos de ACL `s3api` na CLI, como `put-bucket-acl`, usam a mesma [notação de argumento abreviada](#).

Configurar uma política de registro em log

O comando da API `put-bucket-logging` configura uma política de registro de buckets.

No exemplo a seguir, ao usuário da AWS `usuario@exemple.com` recebe controle total sobre os arquivos de log, e todos os usuários terão acesso de leitura a eles. Observe que o comando `put-bucket-acl` também deve conceder ao sistema de entrega de log do Amazon S3 (especificado por um URI) as permissões necessárias para ler e gravar os logs no bucket.

```
$ aws s3api put-bucket-acl --bucket MyBucket --grant-read-acp 'URI="http://  
acs.amazonaws.com/groups/s3/LogDelivery"' --grant-write 'URI="http://acs.amazonaws.com/  
groups/s3/LogDelivery"'
```

```
$ aws s3api put-bucket-logging --bucket MyBucket --bucket-logging-status file://  
logging.json
```

O arquivo `logging.json` no comando anterior tem o seguinte conteúdo.

```
{
  "LoggingEnabled": {
    "TargetBucket": "MyBucket",
    "TargetPrefix": "MyBucketLogs/",
    "TargetGrants": [
      {
        "Grantee": {
          "Type": "AmazonCustomerByEmail",
          "EmailAddress": "user@example.com"
        },
        "Permission": "FULL_CONTROL"
      },
      {
        "Grantee": {
          "Type": "Group",
          "URI": "http://acs.amazonaws.com/groups/global/AllUsers"
        },
        "Permission": "READ"
      }
    ]
  }
}
```

Recursos

Referência da AWS CLI:

- [aws s3api](#)
- [aws s3api put-bucket-acl](#)
- [aws s3api put-bucket-logging](#)

Referência do serviço:

- [Como trabalhar com buckets do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como trabalhar com objetos do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como listar chaves hierarquicamente usando um prefixo e delimitador](#) no Manual do usuário do Amazon Simple Storage Service

- [Anular carregamentos fracionados para um bucket do S3 usando o AWS SDK for .NET \(nível baixo\)](#) no Manual do usuário do Amazon Simple Storage Service

Exemplo de script de operações de ciclo de vida de bucket Amazon S3

Este tópico usa um exemplo de script bash para operações de ciclo de vida do bucket do Amazon S3 usando a AWS Command Line Interface (AWS CLI). Este exemplo de desenvolvimento de scripts usa o conjunto de comandos [aws s3api](#). Scripts shell são programas desenvolvidos para ser executados em uma interface de linha de comando.

Tópicos

- [Antes de começar](#)
- [Sobre este exemplo](#)
- [Arquivos](#)
- [Referências](#)

Antes de começar

Antes que você possa executar qualquer um dos exemplos abaixo, as seguintes tarefas deverão ser concluídas.

- Instale e configure a AWS CLI. Para obter mais informações, consulte [the section called “Instalar/atualizar”](#) e [Autenticação e credenciais de acesso](#).
- O perfil utilizado deve ter permissões que garantem as operações da AWS realizadas pelos exemplos.
- Como prática recomendada da AWS, conceda a esse código privilégio mínimo ou apenas as permissões necessárias para executar uma tarefa. Para obter mais informações, consulte [Conceder privilégio mínimo](#) no Guia do usuário do IAM.
- Este código não foi testado em todas as regiões da AWS. Alguns serviços da AWS só estão disponíveis em regiões específicas. Para obter mais informações, consulte [Endpoints de serviço e cotas](#) no Guia de referência geral da AWS.
- Executar este código pode resultar em cobranças em sua conta da AWS. É sua responsabilidade garantir que todos os recursos criados por este script sejam removidos após você terminar de usá-los.

O serviço Amazon S3 usa os seguintes termos:

- Bucket: uma pasta de nível superior do Amazon S3.
- Prefixo: uma pasta do Amazon S3 em um bucket.
- Objeto: qualquer item hospedado em um bucket do Amazon S3.

Sobre este exemplo

Este exemplo mostra como interagir com algumas das operações básicas do Amazon S3 usando um conjunto de funções em arquivos de script de shell. As funções estão localizadas no arquivo de script shell chamado `bucket-operations.sh`. Você pode chamar essas funções em outro arquivo. Cada arquivo de script contém comentários descrevendo cada uma das funções.

Para ver os resultados intermediários de cada etapa, execute o script com um parâmetro `-i`. É possível exibir o status atual do bucket ou do conteúdo usando o console do Amazon S3. O script avançará para a próxima etapa somente após você pressionar ENTER no prompt.

Para obter o exemplo completo e os arquivos de script para download, consulte [Operações de ciclo de vida de bucket do Amazon S3](#) no Repositório de exemplos de código da AWS no GitHub.

Arquivos

O exemplo contém os seguintes arquivos:

`bucket-operations.sh`

Esse arquivo de script principal pode ser originado de outro arquivo. Ele inclui funções que executam as seguintes tarefas:

- Criar um bucket e verificar se ele existe
- Copiar um arquivo do computador local para um bucket
- Copiar um arquivo de um local de bucket para um local de bucket diferente
- Listar o conteúdo de um bucket
- Excluir um arquivo de um bucket
- Excluir um bucket

Visualize o código para [bucket-operations.sh](#) no GitHub.

test-bucket-operations.sh

O arquivo de script de shell `test-bucket-operations.sh` demonstra como chamar as funções utilizando o arquivo `bucket-operations.sh` e chamando cada uma das funções. Após chamar funções, o script de teste remove todos os recursos que ele criou.

Visualize o código para [test-bucket-operations.sh](#) no GitHub.

awsdocs-general.sh

O arquivo de script `awsdocs-general.sh` contém funções de uso geral usadas em exemplos avançados de código para a AWS CLI.

Visualize o código para [awsdocs-general.sh](#) no GitHub.

Referências

Referência da AWS CLI:

- [aws s3api](#)
- [aws s3api create-bucket](#)
- [aws s3api copy-object](#)
- [aws s3api delete-bucket](#)
- [aws s3api delete-object](#)
- [aws s3api head-bucket](#)
- [aws s3api list-objects](#)
- [aws s3api put-object](#)

Outra referência:

- [Como trabalhar com buckets do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como trabalhar com objetos do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

Uso do Amazon SNS com a AWS CLI

É possível acessar os recursos do Amazon Simple Notification Service (Amazon SNS) usando a AWS Command Line Interface (AWS CLI). Para listar os comandos da AWS CLI para o Amazon SNS, use o comando a seguir.

```
aws sns help
```

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Este tópico mostra exemplos de comandos da AWS CLI que executam tarefas comuns para o Amazon SNS.

Tópicos

- [Criar um tópico](#)
- [Assinar um tópico](#)
- [Publicar em um tópico](#)
- [Cancelar a assinatura de um tópico](#)
- [Excluir um tópico](#)

Criar um tópico

Para criar um tópico, use o comando [sns create-topic](#) e especifique o nome a ser atribuído a ele.

```
$ aws sns create-topic --name my-topic
{
  "TopicArn": "arn:aws:sns:us-west-2:123456789012:my-topic"
}
```

Anote o TopicArn da resposta que você usará mais tarde para publicar uma mensagem.

Assinar um tópico

Para assinar um tópico, use o comando [sns subscribe](#).

O exemplo a seguir especifica o protocolo email e um endereço de e-mail para o notification-endpoint.

```
$ aws sns subscribe --topic-arn arn:aws:sns:us-west-2:123456789012:my-topic --
protocol email --notification-endpoint saanvi@example.com
{
  "SubscriptionArn": "pending confirmation"
}
```

O AWS envia imediatamente uma mensagem de confirmação para o endereço de e-mail especificado no comando `subscribe`. A mensagem de e-mail tem o seguinte texto.

```
You have chosen to subscribe to the topic:
arn:aws:sns:us-west-2:123456789012:my-topic
To confirm this subscription, click or visit the following link (If this was in error
no action is necessary):
Confirm subscription
```

Assim que o destinatário clicar no link `Confirm subscription` (Confirmar assinatura), o navegador do destinatário exibirá uma mensagem de notificação com informações semelhantes à seguinte.

```
Subscription confirmed!

You have subscribed saanvi@example.com to the topic:my-topic.

Your subscription's id is:
arn:aws:sns:us-west-2:123456789012:my-topic:1328f057-de93-4c15-512e-8bb22EXAMPLE

If it was not your intention to subscribe, click here to unsubscribe.
```

Publicar em um tópico

Para enviar uma mensagem a todos os assinantes de um tópico, use o comando [sns publish](#).

O exemplo a seguir envia a mensagem “Hello World!” para todos os assinantes do tópico especificado.

```
$ aws sns publish --topic-arn arn:aws:sns:us-west-2:123456789012:my-topic --
message "Hello World!"
{
  "MessageId": "4e41661d-5eec-5ddf-8dab-2c867EXAMPLE"
}
```


Neste exemplo, a AWS envia uma mensagem de e-mail com o texto “Hello World!” para `saanvi@example.com`.

Cancelar a assinatura de um tópico

Para cancelar a assinatura de um tópico e parar de receber as mensagens publicadas nesse tópico, use o comando [sns unsubscribe](#) e especifique o ARN do tópico do qual você deseja cancelar a assinatura.

```
$ aws sns unsubscribe --subscription-arn arn:aws:sns:us-west-2:123456789012:my-topic:1328f057-de93-4c15-512e-8bb22EXAMPLE
```

Para verificar se a assinatura foi cancelada com êxito, use o comando [sns list-subscriptions](#) para confirmar que o ARN não aparece mais na lista.

```
$ aws sns list-subscriptions
```

Excluir um tópico

Para excluir um tópico, execute o comando [sns delete-topic](#).

```
$ aws sns delete-topic --topic-arn arn:aws:sns:us-west-2:123456789012:my-topic
```

Para verificar se a AWS excluiu o tópico com êxito, use o comando [sns list-topics](#) para confirmar que o tópico não aparece mais na lista.

```
$ aws sns list-topics
```

AWS CLI com exemplos de código de script Bash

Os exemplos de código neste tópico mostram como usar a AWS Command Line Interface com script Bash com a AWS.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Exemplos entre serviços são amostras de aplicações que funcionam em vários Serviços da AWS.

Exemplos

- [Ações e cenários de uso da AWS CLI com script Bash](#)

Ações e cenários de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio de Serviços da AWS.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Serviços

- [Exemplos do DynamoDB de uso da AWS CLI com script Bash](#)
- [Exemplos do HealthImaging de uso da AWS CLI com script Bash](#)
- [Exemplos do IAM que usam a AWS CLI com script Bash](#)
- [Exemplos do Amazon S3 de uso da AWS CLI com script Bash](#)
- [Exemplos do AWS STS de uso da AWS CLI com script Bash](#)

Exemplos do DynamoDB de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do DynamoDB.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

Tópicos

- [Ações](#)
- [Cenários](#)

Ações

Criar uma tabela

O exemplo de código a seguir mostra como criar uma tabela do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_create_table
#
# This function creates an Amazon DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table to create.
#     -a attribute_definitions -- JSON file path of a list of attributes and their
#     types.
#     -k key_schema -- JSON file path of a list of attributes and their key types.
#     -p provisioned_throughput -- Provisioned throughput settings for the table.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_create_table() {
    local table_name attribute_definitions key_schema provisioned_throughput response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
}
```

```

function usage() {
    echo "function dynamodb_create_table"
    echo "Creates an Amazon DynamoDB table."
    echo " -n table_name -- The name of the table to create."
    echo " -a attribute_definitions -- JSON file path of a list of attributes and
their types."
    echo " -k key_schema -- JSON file path of a list of attributes and their key
types."
    echo " -p provisioned_throughput -- Provisioned throughput settings for the
table."
    echo ""
}

# Retrieve the calling parameters.
while getopts "n:a:k:p:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        a) attribute_definitions="${OPTARG}" ;;
        k) key_schema="${OPTARG}" ;;
        p) provisioned_throughput="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$attribute_definitions" ]]; then
    errecho "ERROR: You must provide an attribute definitions json file path the -a
parameter."
    usage
    return 1
fi

```

```

fi

if [[ -z "$key_schema" ]]; then
    errecho "ERROR: You must provide a key schema json file path the -k parameter."
    usage
    return 1
fi

if [[ -z "$provisioned_throughput" ]]; then
    errecho "ERROR: You must provide a provisioned throughput json file path the -p
parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:    $table_name"
iecho "    attribute_definitions:  $attribute_definitions"
iecho "    key_schema:    $key_schema"
iecho "    provisioned_throughput:  $provisioned_throughput"
iecho ""

response=$(aws dynamodb create-table \
    --table-name "$table_name" \
    --attribute-definitions file://"${attribute_definitions}" \
    --key-schema file://"${key_schema}" \
    --provisioned-throughput "$provisioned_throughput")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-table operation failed.$response"
    return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```
#####
```

```

# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    fi
}

```

```

elif [ "$err_code" == 252 ]; then
    errecho " Command syntax invalid."
elif [ "$err_code" == 253 ]; then
    errecho " The system environment or configuration was invalid."
elif [ "$err_code" == 254 ]; then
    errecho " The service returned an error."
elif [ "$err_code" == 255 ]; then
    errecho " 255 is a catch-all error."
fi

return 0
}

```

- Para obter detalhes da API, consulte [CreateTable](#) na Referência de comandos da AWS CLI.

Excluir uma tabela

O exemplo de código a seguir mostra como excluir uma tabela do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function dynamodb_delete_table
#
# This function deletes a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_delete_table() {
    local table_name response

```

```

local option OPTARG # Required to use getopt command in a function.

# bashsupport disable=BP5008
function usage() {
    echo "function dynamodb_delete_table"
    echo "Deletes an Amazon DynamoDB table."
    echo " -n table_name  -- The name of the table to delete."
    echo ""
}

# Retrieve the calling parameters.
while getopt "n:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho ""

response=$(aws dynamodb delete-table \
    --table-name "$table_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code

```



```

    errecho "ERROR: AWS reports delete-table operation failed.$response"
    return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:

```

```
#          0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [DeleteTable](#) na Referência de comandos da AWS CLI.

Excluir um item de uma tabela

O exemplo de código a seguir mostra como excluir um item de uma tabela do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_delete_item
```

```

#
# This function deletes an item from a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys        -- Path to json file containing the keys that identify the item to
#                       delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_delete_item() {
    local table_name keys response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_delete_item"
        echo "Delete an item from a DynamoDB table."
        echo " -n table_name  -- The name of the table."
        echo " -k keys        -- Path to json file containing the keys that identify the item
to delete."
        echo ""
    }
    while getopt "n:k:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            k) keys="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

```

```

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    keys:       $keys"
iecho ""

response=$(aws dynamodb delete-item \
    --table-name "$table_name" \
    --key file://"${keys}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-item operation failed.$response"
    return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {

```

```

    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    fi
}

```

```

elif [ "$err_code" == 255 ]; then
    errecho " 255 is a catch-all error."
fi

return 0
}

```

- Para obter detalhes da API, consulte [Deleteltem](#) na Referência de comandos da AWS CLI.

Obter um lote de itens

O exemplo de código a seguir mostra como obter um lote de itens do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function dynamodb_batch_get_item
#
# This function gets a batch of items from a DynamoDB table.
#
# Parameters:
#     -i item  -- Path to json file containing the keys of the items to get.
#
# Returns:
#     The items as json output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_batch_get_item() {
    local item response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation

```

```
#####
function usage() {
    echo "function dynamodb_batch_get_item"
    echo "Get a batch of items from a DynamoDB table."
    echo " -i item  -- Path to json file containing the keys of the items to get."
    echo ""
}

while getopts "i:h" option; do
    case "${option}" in
        i) item="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

response=$(aws dynamodb batch-get-item \
    --request-items file://"${item}")
local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports batch-get-item operation failed.$response"
    return 1
fi

echo "$response"

return 0
}
```

As funções utilitárias usadas neste exemplo.

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    }
}
```



```

elif [ "$err_code" == 255 ]; then
    errecho " 255 is a catch-all error."
fi

return 0
}

```

- Para obter detalhes da API, consulte [BatchGetItem](#) na Referência de comandos da AWS CLI.

Obter um item de uma tabela

O exemplo de código a seguir mostra como obter um item de uma tabela do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function dynamodb_get_item
#
# This function gets an item from a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys        -- Path to json file containing the keys that identify the item to
#                       get.
#     [-q query]    -- Optional JMESPath query expression.
#
# Returns:
#     The item as text output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_get_item() {
    local table_name keys query response
    local option OPTARG # Required to use getopt command in a function.

```

```

# #####
# Function usage explanation
# #####
function usage() {
    echo "function dynamodb_get_item"
    echo "Get an item from a DynamoDB table."
    echo " -n table_name -- The name of the table."
    echo " -k keys -- Path to json file containing the keys that identify the item
to get."
    echo " [-q query] -- Optional JMESPath query expression."
    echo ""
}
query=""
while getopts "n:k:q:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        k) keys="${OPTARG}" ;;
        q) query="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

```

```

if [[ -n "$query" ]]; then
    response=$(aws dynamodb get-item \
        --table-name "$table_name" \
        --key file://"${keys}" \
        --output text \
        --query "$query")
else
    response=$(
        aws dynamodb get-item \
            --table-name "$table_name" \
            --key file://"${keys}" \
            --output text
    )
fi

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports get-item operation failed.$response"
    return 1
fi

if [[ -n "$query" ]]; then
    echo "$response" | sed "/^\t/s/\t//1" # Remove initial tab that the JMSEPath
query inserts on some strings.
else
    echo "$response"
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

```

```

}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}

```

- Para obter detalhes da API, consulte [GetItem](#) na Referência de comandos da AWS CLI.

Obter informações sobre uma tabela

O exemplo de código a seguir mostra como obter informações sobre uma tabela do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_describe_table
#
# This function returns the status of a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#
# Response:
#     - TableStatus:
#     And:
#     0 - Table is active.
#     1 - If it fails.
#####
function dynamodb_describe_table {
    local table_name
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_describe_table"
        echo "Describe the status of a DynamoDB table."
        echo "  -n table_name  -- The name of the table."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
```

```

    n) table_name="${OPTARG}" ;;
  h)
    usage
    return 0
    ;;
  \?)
    echo "Invalid parameter"
    usage
    return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
  errecho "ERROR: You must provide a table name with the -n parameter."
  usage
  return 1
fi

local table_status
table_status=$(
  aws dynamodb describe-table \
    --table-name "$table_name" \
    --output text \
    --query 'Table.TableStatus'
)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log "$error_code"
  errecho "ERROR: AWS reports describe-table operation failed.$table_status"
  return 1
fi

echo "$table_status"

return 0
}

```

As funções utilitárias usadas neste exemplo.

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi
}
```

```
    return 0
}
```

- Para obter detalhes da API, consulte [DescribeTable](#) na Referência de comandos da AWS CLI.

Listar tabelas

O exemplo de código a seguir mostra como listar tabelas do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_list_tables
#
# This function lists all the tables in a DynamoDB.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_list_tables() {
    response=$(aws dynamodb list-tables \
        --output text \
        --query "TableNames")

    local error_code=${?}

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log $error_code
        errecho "ERROR: AWS reports batch-write-item operation failed.$response"
        return 1
    fi

    echo "$response" | tr -s "[:space:]" "\n"
```



```

    return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    }
}

```

```

elif [ "$err_code" == 254 ]; then
    errecho " The service returned an error."
elif [ "$err_code" == 255 ]; then
    errecho " 255 is a catch-all error."
fi

return 0
}

```

- Para obter detalhes da API, consulte [ListTable](#) na Referência de comandos da AWS CLI.

Colocar um item em uma tabela

O exemplo de código a seguir mostra como colocar um item em uma tabela do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function dynamodb_put_item
#
# This function puts an item into a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -i item -- Path to json file containing the item values.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_put_item() {
    local table_name item response
    local option OPTARG # Required to use getopt command in a function.

    #####

```

```
# Function usage explanation
#####
function usage() {
    echo "function dynamodb_put_item"
    echo "Put an item into a DynamoDB table."
    echo " -n table_name -- The name of the table."
    echo " -i item -- Path to json file containing the item values."
    echo ""
}

while getopts "n:i:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        i) item="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    item:       $item"
iecho ""
iecho ""
```

```

response=$(aws dynamodb put-item \
  --table-name "$table_name" \
  --item file://"item")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports put-item operation failed.$response"
  return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
  if [[ $VERBOSE == true ]]; then
    echo "$@"
  fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
  printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()

```

```
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [PutItem](#) na Referência de comandos da AWS CLI.

Consultar uma tabela

O exemplo de código a seguir mostra como consultar uma tabela do DynamoDB.

AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_query
#
# This function queries a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -k key_condition_expression -- The key condition expression.
#     -a attribute_names -- Path to JSON file containing the attribute names.
#     -v attribute_values -- Path to JSON file containing the attribute values.
#     [-p projection_expression] -- Optional projection expression.
#
# Returns:
#     The items as json output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_query() {
    local table_name key_condition_expression attribute_names attribute_values
    projection_expression response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_query"
        echo "Query a DynamoDB table."
        echo " -n table_name -- The name of the table."
        echo " -k key_condition_expression -- The key condition expression."
        echo " -a attribute_names -- Path to JSON file containing the attribute names."
        echo " -v attribute_values -- Path to JSON file containing the attribute
values."
```

```

    echo " [-p projection_expression] -- Optional projection expression."
    echo ""
}

while getopts "n:k:a:v:p:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        k) key_condition_expression="${OPTARG}" ;;
        a) attribute_names="${OPTARG}" ;;
        v) attribute_values="${OPTARG}" ;;
        p) projection_expression="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$key_condition_expression" ]]; then
    errecho "ERROR: You must provide a key condition expression with the -k
parameter."
    usage
    return 1
fi

if [[ -z "$attribute_names" ]]; then
    errecho "ERROR: You must provide a attribute names with the -a parameter."
    usage
    return 1
fi

if [[ -z "$attribute_values" ]]; then

```

```

    errecho "ERROR: You must provide a attribute values with the -v parameter."
    usage
    return 1
fi

if [[ -z "$projection_expression" ]]; then
    response=$(aws dynamodb query \
        --table-name "$table_name" \
        --key-condition-expression "$key_condition_expression" \
        --expression-attribute-names file://"${attribute_names}" \
        --expression-attribute-values file://"${attribute_values}")
else
    response=$(aws dynamodb query \
        --table-name "$table_name" \
        --key-condition-expression "$key_condition_expression" \
        --expression-attribute-names file://"${attribute_names}" \
        --expression-attribute-values file://"${attribute_values}" \
        --projection-expression "$projection_expression")
fi

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports query operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

```



```

}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}

```

- Para obter detalhes da API, consulte [Query](#) na Referência de comandos da AWS CLI.

Verificar uma tabela

O exemplo de código a seguir mostra como verificar uma tabela do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_scan
#
# This function scans a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -f filter_expression -- The filter expression.
#     -a expression_attribute_names -- Path to JSON file containing the expression
#     attribute names.
#     -v expression_attribute_values -- Path to JSON file containing the
#     expression attribute values.
#     [-p projection_expression] -- Optional projection expression.
#
# Returns:
#     The items as json output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_scan() {
    local table_name filter_expression expression_attribute_names
    expression_attribute_values projection_expression response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_scan"
        echo "Scan a DynamoDB table."
    }
}
```

```

    echo " -n table_name  -- The name of the table."
    echo " -f filter_expression  -- The filter expression."
    echo " -a expression_attribute_names  -- Path to JSON file containing the
expression attribute names."
    echo " -v expression_attribute_values  -- Path to JSON file containing the
expression attribute values."
    echo " [-p projection_expression]  -- Optional projection expression."
    echo ""
}

while getopts "n:f:a:v:p:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        f) filter_expression="${OPTARG}" ;;
        a) expression_attribute_names="${OPTARG}" ;;
        v) expression_attribute_values="${OPTARG}" ;;
        p) projection_expression="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$filter_expression" ]]; then
    errecho "ERROR: You must provide a filter expression with the -f parameter."
    usage
    return 1
fi

if [[ -z "$expression_attribute_names" ]]; then

```

```

    errecho "ERROR: You must provide expression attribute names with the -a
parameter."
    usage
    return 1
fi

if [[ -z "$expression_attribute_values" ]]; then
    errecho "ERROR: You must provide expression attribute values with the -v
parameter."
    usage
    return 1
fi

if [[ -z "$projection_expression" ]]; then
    response=$(aws dynamodb scan \
        --table-name "$table_name" \
        --filter-expression "$filter_expression" \
        --expression-attribute-names file://"${expression_attribute_names}" \
        --expression-attribute-values file://"${expression_attribute_values}")
else
    response=$(aws dynamodb scan \
        --table-name "$table_name" \
        --filter-expression "$filter_expression" \
        --expression-attribute-names file://"${expression_attribute_names}" \
        --expression-attribute-values file://"${expression_attribute_values}" \
        --projection-expression "$projection_expression")
fi

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports scan operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

```

As funções utilitárias usadas neste exemplo.

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi
}
```

```
    return 0
}
```

- Para obter detalhes da API, consulte [Scan](#) na Referência de comandos da AWS CLI.

Atualizar um item em uma tabela

O exemplo de código a seguir mostra como atualizar um item em uma tabela do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_update_item
#
# This function updates an item in a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -k keys -- Path to json file containing the keys that identify the item to
#     update.
#     -e update expression -- An expression that defines one or more attributes
#     to be updated.
#     -v values -- Path to json file containing the update values.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_update_item() {
    local table_name keys update_expression values response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
```

```
#####
function usage() {
    echo "function dynamodb_update_item"
    echo "Update an item in a DynamoDB table."
    echo " -n table_name  -- The name of the table."
    echo " -k keys    -- Path to json file containing the keys that identify the item
to update."
    echo " -e update expression  -- An expression that defines one or more
attributes to be updated."
    echo " -v values  -- Path to json file containing the update values."
    echo ""
}

while getopts "n:k:e:v:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        k) keys="${OPTARG}" ;;
        e) update_expression="${OPTARG}" ;;
        v) values="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

if [[ -z "$update_expression" ]]; then
```

```

    errecho "ERROR: You must provide an update expression with the -e parameter."
    usage
    return 1
fi

if [[ -z "$values" ]]; then
    errecho "ERROR: You must provide a values json file path the -v parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    keys:       $keys"
iecho "    update_expression:  $update_expression"
iecho "    values:      $values"

response=$(aws dynamodb update-item \
    --table-name "$table_name" \
    --key file://"${keys}" \
    --update-expression "$update_expression" \
    --expression-attribute-values file://"${values}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports update-item operation failed.$response"
    return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.

```



```
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    fi
}
```

```

elif [ "$err_code" == 254 ]; then
    errecho " The service returned an error."
elif [ "$err_code" == 255 ]; then
    errecho " 255 is a catch-all error."
fi

return 0
}

```

- Para obter detalhes da API, consulte [UpdateItem](#) na Referência de comandos da AWS CLI.

Gravar um lote de itens

O exemplo de código a seguir mostra como gravar um lote de itens do DynamoDB.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function dynamodb_batch_write_item
#
# This function writes a batch of items into a DynamoDB table.
#
# Parameters:
#     -i item  -- Path to json file containing the items to write.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_batch_write_item() {
    local item response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation

```

```
#####
function usage() {
    echo "function dynamodb_batch_write_item"
    echo "Write a batch of items into a DynamoDB table."
    echo " -i item  -- Path to json file containing the items to write."
    echo ""
}
while getopts "i:h" option; do
    case "${option}" in
        i) item="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    item:       $item"
iecho ""

response=$(aws dynamodb batch-write-item \
    --request-items file://"${item}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports batch-write-item operation failed.$response"
    return 1
fi
```

```

    return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####

```

```
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [BatchWriteItem](#) na Referência de comandos da AWS CLI.

Cenários

Conceitos básicos de tabelas, itens e consultas

O código de exemplo a seguir mostra como:

- Criar uma tabela que possa conter dados de filmes.
- Colocar, obter e atualizar um único filme na tabela.
- Grave dados de filmes na tabela usando um arquivo JSON de exemplo.
- Consultar filmes que foram lançados em determinado ano.
- Verificar filmes que foram lançados em um intervalo de anos.
- Exclua um filme da tabela e, depois, exclua a tabela.

AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

O cenário de conceitos básicos do DynamoDB.

```
#####
# function dynamodb_getting_started_movies
#
# Scenario to create an Amazon DynamoDB table and perform a series of operations on
# the table.
#
# Returns:
#     0 - If successful.
#     1 - If an error occurred.
#####
function dynamodb_getting_started_movies() {

    source ./dynamodb_operations.sh

    key_schema_json_file="dynamodb_key_schema.json"
    attribute_definitions_json_file="dynamodb_attr_def.json"
    item_json_file="movie_item.json"
    key_json_file="movie_key.json"
    batch_json_file="batch.json"
    attribute_names_json_file="attribute_names.json"
    attributes_values_json_file="attribute_values.json"

    echo_repeat "*" 88
    echo
    echo "Welcome to the Amazon DynamoDB getting started demo."
    echo
    echo_repeat "*" 88
    echo

    local table_name
    echo -n "Enter a name for a new DynamoDB table: "
    get_input
    table_name=$get_input_result
}
```

```
local provisioned_throughput="ReadCapacityUnits=5,WriteCapacityUnits=5"

echo '[
{"AttributeName": "year", "KeyType": "HASH"},
{"AttributeName": "title", "KeyType": "RANGE"}
]' >"$key_schema_json_file"

echo '[
{"AttributeName": "year", "AttributeType": "N"},
{"AttributeName": "title", "AttributeType": "S"}
]' >"$attribute_definitions_json_file"

if dynamodb_create_table -n "$table_name" -a "$attribute_definitions_json_file" \
-k "$key_schema_json_file" -p "$provisioned_throughput" 1>/dev/null; then
    echo "Created a DynamoDB table named $table_name"
else
    errecho "The table failed to create. This demo will exit."
    clean_up
    return 1
fi

echo "Waiting for the table to become active...."

if dynamodb_wait_table_active -n "$table_name"; then
    echo "The table is now active."
else
    errecho "The table failed to become active. This demo will exit."
    cleanup "$table_name"
    return 1
fi

echo
echo_repeat "*" 88
echo

echo -n "Enter the title of a movie you want to add to the table: "
get_input
local added_title
added_title=$get_input_result

local added_year
get_int_input "What year was it released? "
added_year=$get_input_result
```

```

local rating
get_float_input "On a scale of 1 - 10, how do you rate it? " "1" "10"
rating=$get_input_result

local plot
echo -n "Summarize the plot for me: "
get_input
plot=$get_input_result

echo '{
  "year": {"N" : ""$added_year""},
  "title": {"S" : ""$added_title""},
  "info": {"M" : {"plot": {"S" : ""$plot""}, "rating": {"N" : ""$rating""} } }
}' >"$item_json_file"

if dynamodb_put_item -n "$table_name" -i "$item_json_file"; then
  echo "The movie '$added_title' was successfully added to the table
'$table_name'."
else
  errecho "Put item failed. This demo will exit."
  clean_up "$table_name"
  return 1
fi

echo
echo_repeat "*" 88
echo

echo "Let's update your movie '$added_title'."
get_float_input "You rated it $rating, what new rating would you give it? " "1"
"10"
rating=$get_input_result

echo -n "You summarized the plot as '$plot'."
echo "What would you say now? "
get_input
plot=$get_input_result

echo '{
  "year": {"N" : ""$added_year""},
  "title": {"S" : ""$added_title""}
}' >"$key_json_file"

```



```

echo '{
  "r": {"N" : ""$rating""},
  "p": {"S" : ""$plot""}
}' >"$item_json_file"

local update_expression="SET info.rating = :r, info.plot = :p"

if dynamodb_update_item -n "$table_name" -k "$key_json_file" -e
"$update_expression" -v "$item_json_file"; then
  echo "Updated '$added_title' with new attributes."
else
  errecho "Update item failed. This demo will exit."
  clean_up "$table_name"
  return 1
fi

echo
echo_repeat "*" 88
echo

echo "We will now use batch write to upload 150 movie entries into the table."

local batch_json
for batch_json in movie_files/movies_*.json; do
  echo "{ \"$table_name\" : $(<"$batch_json") }" >"$batch_json_file"
  if dynamodb_batch_write_item -i "$batch_json_file" 1>/dev/null; then
    echo "Entries in $batch_json added to table."
  else
    errecho "Batch write failed. This demo will exit."
    clean_up "$table_name"
    return 1
  fi
done

local title="The Lord of the Rings: The Fellowship of the Ring"
local year="2001"

if get_yes_no_input "Let's move on...do you want to get info about '$title'? (y/n)
"; then
  echo '{
"year": {"N" : ""$year""},
"title": {"S" : ""$title""}
}' >"$key_json_file"
  local info

```

```

info=$(dynamodb_get_item -n "$table_name" -k "$key_json_file")

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "Get item failed. This demo will exit."
    clean_up "$table_name"
    return 1
fi

echo "Here is what I found:"
echo "$info"
fi

local ask_for_year=true
while [[ "$ask_for_year" == true ]]; do
    echo "Let's get a list of movies released in a given year."
    get_int_input "Enter a year between 1972 and 2018: " "1972" "2018"
    year=$get_input_result
    echo '{
"#n": "year"
}' >"$attribute_names_json_file"

    echo '{
":v": {"N" :"""$year"""}
}' >"$attributes_values_json_file"

    response=$(dynamodb_query -n "$table_name" -k "#n=:v" -a
"$attribute_names_json_file" -v "$attributes_values_json_file")

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "Query table failed. This demo will exit."
    clean_up "$table_name"
    return 1
fi

echo "Here is what I found:"
echo "$response"

if ! get_yes_no_input "Try another year? (y/n) "; then
    ask_for_year=false
fi
done

```

```

echo "Now let's scan for movies released in a range of years. Enter a year: "
get_int_input "Enter a year between 1972 and 2018: " "1972" "2018"
local start=$get_input_result

get_int_input "Enter another year: " "1972" "2018"
local end=$get_input_result

echo '{
  "#n": "year"
}' >"$attribute_names_json_file"

echo '{
  ":v1": {"N" : ""$start""},
  ":v2": {"N" : ""$end""}
}' >"$attributes_values_json_file"

response=$(dynamodb_scan -n "$table_name" -f "#n BETWEEN :v1 AND :v2" -a
"$attribute_names_json_file" -v "$attributes_values_json_file")

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
  errecho "Scan table failed. This demo will exit."
  clean_up "$table_name"
  return 1
fi

echo "Here is what I found:"
echo "$response"

echo
echo_repeat "*" 88
echo

echo "Let's remove your movie '$added_title' from the table."

if get_yes_no_input "Do you want to remove '$added_title'? (y/n) "; then
  echo '{
"year": {"N" : ""$added_year""},
"title": {"S" : ""$added_title""}
}' >"$key_json_file"

  if ! dynamodb_delete_item -n "$table_name" -k "$key_json_file"; then
    errecho "Delete item failed. This demo will exit."
    clean_up "$table_name"
  fi
fi

```

```

        return 1
    fi
fi

if get_yes_no_input "Do you want to delete the table '$table_name'? (y/n) "; then
    if ! clean_up "$table_name"; then
        return 1
    fi
else
    if ! clean_up; then
        return 1
    fi
fi

return 0
}

```

As funções do DynamoDB usadas nesse cenário.

```

#####
# function dynamodb_create_table
#
# This function creates an Amazon DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table to create.
#     -a attribute_definitions -- JSON file path of a list of attributes and their
types.
#     -k key_schema -- JSON file path of a list of attributes and their key types.
#     -p provisioned_throughput -- Provisioned throughput settings for the table.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_create_table() {
    local table_name attribute_definitions key_schema provisioned_throughput response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####

```

```

function usage() {
    echo "function dynamodb_create_table"
    echo "Creates an Amazon DynamoDB table."
    echo " -n table_name -- The name of the table to create."
    echo " -a attribute_definitions -- JSON file path of a list of attributes and
their types."
    echo " -k key_schema -- JSON file path of a list of attributes and their key
types."
    echo " -p provisioned_throughput -- Provisioned throughput settings for the
table."
    echo ""
}

# Retrieve the calling parameters.
while getopts "n:a:k:p:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        a) attribute_definitions="${OPTARG}" ;;
        k) key_schema="${OPTARG}" ;;
        p) provisioned_throughput="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$attribute_definitions" ]]; then
    errecho "ERROR: You must provide an attribute definitions json file path the -a
parameter."
    usage
    return 1
fi

```

```

fi

if [[ -z "$key_schema" ]]; then
    errecho "ERROR: You must provide a key schema json file path the -k parameter."
    usage
    return 1
fi

if [[ -z "$provisioned_throughput" ]]; then
    errecho "ERROR: You must provide a provisioned throughput json file path the -p
parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:    $table_name"
iecho "    attribute_definitions:  $attribute_definitions"
iecho "    key_schema:    $key_schema"
iecho "    provisioned_throughput:  $provisioned_throughput"
iecho ""

response=$(aws dynamodb create-table \
    --table-name "$table_name" \
    --attribute-definitions file://"${attribute_definitions}" \
    --key-schema file://"${key_schema}" \
    --provisioned-throughput "$provisioned_throughput")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-table operation failed.$response"
    return 1
fi

return 0
}

#####
# function dynamodb_describe_table
#
# This function returns the status of a DynamoDB table.
#

```

```

# Parameters:
#     -n table_name  -- The name of the table.
#
# Response:
#     - TableStatus:
#     And:
#     0 - Table is active.
#     1 - If it fails.
#####
function dynamodb_describe_table {
    local table_name
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_describe_table"
        echo "Describe the status of a DynamoDB table."
        echo "  -n table_name  -- The name of the table."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$table_name" ]]; then
        errecho "ERROR: You must provide a table name with the -n parameter."
        usage
        return 1
    fi
}

```

```

fi

local table_status
table_status=$(
    aws dynamodb describe-table \
        --table-name "$table_name" \
        --output text \
        --query 'Table.TableStatus'
)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log "$error_code"
    errecho "ERROR: AWS reports describe-table operation failed.$table_status"
    return 1
fi

echo "$table_status"

return 0
}

#####
# function dynamodb_put_item
#
# This function puts an item into a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -i item -- Path to json file containing the item values.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_put_item() {
    local table_name item response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {

```



```

    echo "function dynamodb_put_item"
    echo "Put an item into a DynamoDB table."
    echo " -n table_name  -- The name of the table."
    echo " -i item      -- Path to json file containing the item values."
    echo ""
}

while getopts "n:i:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        i) item="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    item:       $item"
iecho ""
iecho ""

response=$(aws dynamodb put-item \
    --table-name "$table_name" \

```

```

    --item file://"${item}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports put-item operation failed.$response"
    return 1
fi

return 0

}

#####
# function dynamodb_update_item
#
# This function updates an item in a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -k keys -- Path to json file containing the keys that identify the item to
# update.
#     -e update expression -- An expression that defines one or more attributes
# to be updated.
#     -v values -- Path to json file containing the update values.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_update_item() {
    local table_name keys update_expression values response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_update_item"
        echo "Update an item in a DynamoDB table."
        echo " -n table_name -- The name of the table."
    }

```

```

    echo " -k keys  -- Path to json file containing the keys that identify the item
to update."
    echo " -e update expression  -- An expression that defines one or more
attributes to be updated."
    echo " -v values  -- Path to json file containing the update values."
    echo ""
}

while getopts "n:k:e:v:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        k) keys="${OPTARG}" ;;
        e) update_expression="${OPTARG}" ;;
        v) values="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

if [[ -z "$update_expression" ]]; then
    errecho "ERROR: You must provide an update expression with the -e parameter."
    usage
    return 1
fi

```

```

if [[ -z "$values" ]]; then
    errecho "ERROR: You must provide a values json file path the -v parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    keys:       $keys"
iecho "    update_expression:  $update_expression"
iecho "    values:      $values"

response=$(aws dynamodb update-item \
    --table-name "$table_name" \
    --key file://" $keys" \
    --update-expression "$update_expression" \
    --expression-attribute-values file://" $values")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports update-item operation failed.$response"
    return 1
fi

return 0
}

#####
# function dynamodb_batch_write_item
#
# This function writes a batch of items into a DynamoDB table.
#
# Parameters:
#     -i item  -- Path to json file containing the items to write.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_batch_write_item() {
    local item response

```

```

local option OPTARG # Required to use getopt command in a function.

#####
# Function usage explanation
#####
function usage() {
    echo "function dynamodb_batch_write_item"
    echo "Write a batch of items into a DynamoDB table."
    echo " -i item  -- Path to json file containing the items to write."
    echo ""
}
while getopt "i:h" option; do
    case "${option}" in
        i) item="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    item:       $item"
iecho ""

response=$(aws dynamodb batch-write-item \
    --request-items file://"${item}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then

```

```

    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports batch-write-item operation failed.$response"
    return 1
fi

return 0
}

#####
# function dynamodb_get_item
#
# This function gets an item from a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys       -- Path to json file containing the keys that identify the item to
# get.
#     [-q query]    -- Optional JMESPath query expression.
#
# Returns:
#     The item as text output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_get_item() {
    local table_name keys query response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_get_item"
        echo "Get an item from a DynamoDB table."
        echo " -n table_name  -- The name of the table."
        echo " -k keys       -- Path to json file containing the keys that identify the item
to get."
        echo " [-q query]    -- Optional JMESPath query expression."
        echo ""
    }
    query=""
    while getopt "n:k:q:h" option; do
        case "${option}" in

```

```

n) table_name="${OPTARG}" ;;
k) keys="${OPTARG}" ;;
q) query="${OPTARG}" ;;
h)
    usage
    return 0
    ;;
\?)
    echo "Invalid parameter"
    usage
    return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

if [[ -n "$query" ]]; then
    response=$(aws dynamodb get-item \
        --table-name "$table_name" \
        --key file://"${keys}" \
        --output text \
        --query "$query")
else
    response=$(
        aws dynamodb get-item \
            --table-name "$table_name" \
            --key file://"${keys}" \
            --output text
    )
fi

local error_code=${?}

```

```

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports get-item operation failed.$response"
    return 1
fi

if [[ -n "$query" ]]; then
    echo "$response" | sed "/^\t/s/\t//1" # Remove initial tab that the JMSEPath
query inserts on some strings.
else
    echo "$response"
fi

return 0
}

#####
# function dynamodb_query
#
# This function queries a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -k key_condition_expression -- The key condition expression.
#     -a attribute_names -- Path to JSON file containing the attribute names.
#     -v attribute_values -- Path to JSON file containing the attribute values.
#     [-p projection_expression] -- Optional projection expression.
#
# Returns:
#     The items as json output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_query() {
    local table_name key_condition_expression attribute_names attribute_values
projection_expression response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {

```



```
echo "function dynamodb_query"
echo "Query a DynamoDB table."
echo " -n table_name -- The name of the table."
echo " -k key_condition_expression -- The key condition expression."
echo " -a attribute_names -- Path to JSON file containing the attribute names."
echo " -v attribute_values -- Path to JSON file containing the attribute
values."
echo " [-p projection_expression] -- Optional projection expression."
echo ""
}

while getopts "n:k:a:v:p:h" option; do
  case "${option}" in
    n) table_name="${OPTARG}" ;;
    k) key_condition_expression="${OPTARG}" ;;
    a) attribute_names="${OPTARG}" ;;
    v) attribute_values="${OPTARG}" ;;
    p) projection_expression="${OPTARG}" ;;
    h)
      usage
      return 0
      ;;
    \?)
      echo "Invalid parameter"
      usage
      return 1
      ;;
  esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
  errecho "ERROR: You must provide a table name with the -n parameter."
  usage
  return 1
fi

if [[ -z "$key_condition_expression" ]]; then
  errecho "ERROR: You must provide a key condition expression with the -k
parameter."
  usage
  return 1
fi
```

```

if [[ -z "$attribute_names" ]]; then
    errecho "ERROR: You must provide a attribute names with the -a parameter."
    usage
    return 1
fi

if [[ -z "$attribute_values" ]]; then
    errecho "ERROR: You must provide a attribute values with the -v parameter."
    usage
    return 1
fi

if [[ -z "$projection_expression" ]]; then
    response=$(aws dynamodb query \
        --table-name "$table_name" \
        --key-condition-expression "$key_condition_expression" \
        --expression-attribute-names file://"${attribute_names}" \
        --expression-attribute-values file://"${attribute_values}")
else
    response=$(aws dynamodb query \
        --table-name "$table_name" \
        --key-condition-expression "$key_condition_expression" \
        --expression-attribute-names file://"${attribute_names}" \
        --expression-attribute-values file://"${attribute_values}" \
        --projection-expression "$projection_expression")
fi

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports query operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function dynamodb_scan
#
# This function scans a DynamoDB table.

```

```

#
# Parameters:
#     -n table_name  -- The name of the table.
#     -f filter_expression  -- The filter expression.
#     -a expression_attribute_names  -- Path to JSON file containing the expression
#     attribute names.
#     -v expression_attribute_values  -- Path to JSON file containing the
#     expression attribute values.
#     [-p projection_expression]  -- Optional projection expression.
#
# Returns:
#     The items as json output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_scan() {
    local table_name filter_expression expression_attribute_names
    expression_attribute_values projection_expression response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    # #####
    function usage() {
        echo "function dynamodb_scan"
        echo "Scan a DynamoDB table."
        echo " -n table_name  -- The name of the table."
        echo " -f filter_expression  -- The filter expression."
        echo " -a expression_attribute_names  -- Path to JSON file containing the
expression attribute names."
        echo " -v expression_attribute_values  -- Path to JSON file containing the
expression attribute values."
        echo " [-p projection_expression]  -- Optional projection expression."
        echo ""
    }

    while getopt "n:f:a:v:p:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            f) filter_expression="${OPTARG}" ;;
            a) expression_attribute_names="${OPTARG}" ;;
            v) expression_attribute_values="${OPTARG}" ;;
            p) projection_expression="${OPTARG}" ;;
        esac
    done
}

```

```
h)
    usage
    return 0
    ;;
\?)
    echo "Invalid parameter"
    usage
    return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$filter_expression" ]]; then
    errecho "ERROR: You must provide a filter expression with the -f parameter."
    usage
    return 1
fi

if [[ -z "$expression_attribute_names" ]]; then
    errecho "ERROR: You must provide expression attribute names with the -a
parameter."
    usage
    return 1
fi

if [[ -z "$expression_attribute_values" ]]; then
    errecho "ERROR: You must provide expression attribute values with the -v
parameter."
    usage
    return 1
fi

if [[ -z "$projection_expression" ]]; then
    response=$(aws dynamodb scan \
        --table-name "$table_name" \
        --filter-expression "$filter_expression" \
        --expression-attribute-names file://"${expression_attribute_names} \
```

```

        --expression-attribute-values file://"$expression_attribute_values")
    else
        response=$(aws dynamodb scan \
            --table-name "$table_name" \
            --filter-expression "$filter_expression" \
            --expression-attribute-names file://"$expression_attribute_names" \
            --expression-attribute-values file://"$expression_attribute_values" \
            --projection-expression "$projection_expression")
    fi

    local error_code=${?}

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log $error_code
        errecho "ERROR: AWS reports scan operation failed.$response"
        return 1
    fi

    echo "$response"

    return 0
}

#####
# function dynamodb_delete_item
#
# This function deletes an item from a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -k keys -- Path to json file containing the keys that identify the item to
# delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_delete_item() {
    local table_name keys response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####

```

```

function usage() {
    echo "function dynamodb_delete_item"
    echo "Delete an item from a DynamoDB table."
    echo " -n table_name  -- The name of the table."
    echo " -k keys      -- Path to json file containing the keys that identify the item
to delete."
    echo ""
}
while getopts "n:k:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        k) keys="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    keys:       $keys"
iecho ""

response=$(aws dynamodb delete-item \
    --table-name "$table_name" \

```

```

    --key file://"${keys}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-item operation failed.$response"
    return 1
fi

return 0
}

#####
# function dynamodb_delete_table
#
# This function deletes a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_delete_table() {
    local table_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function dynamodb_delete_table"
        echo "Deletes an Amazon DynamoDB table."
        echo " -n table_name  -- The name of the table to delete."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            h)
                usage

```

```

        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho ""

response=$(aws dynamodb delete-table \
    --table-name "$table_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-table operation failed.$response"
    return 1
fi

return 0
}

```

As funções utilitárias usadas nesse cenário.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.

```



```
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    fi
}
```

```
elif [ "$err_code" == 254 ]; then
    errecho " The service returned an error."
elif [ "$err_code" == 255 ]; then
    errecho " 255 is a catch-all error."
fi

return 0
}
```

- Para obter detalhes da API, consulte os tópicos a seguir na Referência de comandos da AWS CLI.
 - [BatchWriteItem](#)
 - [CreateTable](#)
 - [DeleteItem](#)
 - [DeleteTable](#)
 - [DescribeTable](#)
 - [GetItem](#)
 - [PutItem](#)
 - [Query](#)
 - [Scan](#)
 - [UpdateItem](#)

Exemplos do HealthImaging de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do HealthImaging.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

Tópicos

- [Ações](#)

Ações

Criar um datastore

O exemplo de código a seguir mostra como criar um datastore do HealthImaging.

AWS CLI com script Bash

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function imaging_create_datastore
#
# This function creates an AWS HealthImaging data store for importing DICOM P10
# files.
#
# Parameters:
#     -n data_store_name - The name of the data store.
#
# Returns:
#     The datastore ID.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function imaging_create_datastore() {
    local datastore_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function imaging_create_datastore"
        echo "Creates an AWS HealthImaging data store for importing DICOM P10 files."
    }
}
```

```

    echo "  -n data_store_name - The name of the data store."
    echo ""
}

# Retrieve the calling parameters.
while getopts "n:h" option; do
    case "${option}" in
        n) datastore_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$datastore_name" ]]; then
    errecho "ERROR: You must provide a data store name with the -n parameter."
    usage
    return 1
fi

response=$(aws medical-imaging create-datastore \
    --datastore-name "$datastore_name" \
    --output text \
    --query 'datastoreId')

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports medical-imaging create-datastore operation failed.
$response"
    return 1
fi

echo "$response"

return 0

```

```
}

```

- Para obter detalhes da API, consulte [CreateDatastore](#) na Referência de comandos da AWS CLI.

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

Excluir um datastore

O exemplo de código a seguir mostra como excluir um datastore do HealthImaging.

AWS CLI com script Bash

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function imaging_delete_datastore
#
# This function deletes an AWS HealthImaging data store.
#
# Parameters:
#     -i datastore_id - The ID of the data store.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function imaging_delete_datastore() {
    local datastore_id response
    local option OPTARG # Required to use getopt command in a function.
```

```
# bashsupport disable=BP5008
function usage() {
    echo "function imaging_delete_datastore"
    echo "Deletes an AWS HealthImaging data store."
    echo "  -i datastore_id - The ID of the data store."
    echo ""
}

# Retrieve the calling parameters.
while getopts "i:h" option; do
    case "${option}" in
        i) datastore_id="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$datastore_id" ]]; then
    errecho "ERROR: You must provide a data store ID with the -i parameter."
    usage
    return 1
fi

response=$(aws medical-imaging delete-datastore \
    --datastore-id "$datastore_id")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports medical-imaging delete-datastore operation failed.
$response"
    return 1
fi
```

```
    return 0
}
```

- Para obter detalhes da API, consulte [DeleteDatastore](#) na Referência de comandos da AWS CLI.

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

Obter propriedades do datastore

O exemplo de código a seguir mostra como obter propriedades do datastore do HealthImaging.

AWS CLI com script Bash

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function imaging_get_datastore
#
# Get a data store's properties.
#
# Parameters:
#     -i data_store_id - The ID of the data store.
#
# Returns:
#     [datastore_name, datastore_id, datastore_status, datastore_arn,  created_at,
#     updated_at]
#     And:
#     0 - If successful.
#     1 - If it fails.
```

```
#####
function imaging_get_datastore() {
    local datastore_id option OPTARG # Required to use getopt command in a function.
    local error_code
    # bashsupport disable=BP5008
    function usage() {
        echo "function imaging_get_datastore"
        echo "Gets a data store's properties."
        echo "  -i datastore_id - The ID of the data store."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "i:h" option; do
        case "${option}" in
            i) datastore_id="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$datastore_id" ]]; then
        errecho "ERROR: You must provide a data store ID with the -i parameter."
        usage
        return 1
    fi

    local response

    response=$(
        aws medical-imaging get-datastore \
            --datastore-id "$datastore_id" \
            --output text \
            --query "[ datastoreProperties.datastoreName,
datastoreProperties.datastoreId, datastoreProperties.datastoreStatus,
```



```

datastoreProperties.datastoreArn,  datastoreProperties.createdAt,
datastoreProperties.updatedAt]"
)
error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports list-datastores operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

```

- Para obter detalhes da API, consulte [GetDatastore](#) na Referência de comandos da AWS CLI.

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

Listar datastores

O exemplo de código a seguir mostra como listar datastores do HealthImaging.

AWS CLI com script Bash

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function imaging_list_datastores
#

```

```
# List the HealthImaging data stores in the account.
#
# Returns:
#     [[datastore_name, datastore_id, datastore_status]]
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function imaging_list_datastores() {
    local option OPTARG # Required to use getopt command in a function.
    local error_code
    # bashsupport disable=BP5008
    function usage() {
        echo "function imaging_list_datastores"
        echo "Lists the AWS HealthImaging data stores in the account."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "h" option; do
        case "${option}" in
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    local response
    response=$(aws medical-imaging list-datastores \
        --output text \
        --query "datastoreSummaries[*][datastoreName, datastoreId, datastoreStatus]")
    error_code=${?}

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log $error_code
        errecho "ERROR: AWS reports list-datastores operation failed.$response"
        return 1
    fi
}
```

```
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [ListDatastores](#) na Referência de comandos da AWS CLI.

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

Exemplos do IAM que usam a AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do IAM.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

Tópicos

- [Ações](#)
- [Cenários](#)

Ações

Anexar uma política a uma função

O exemplo de código a seguir mostra como anexar uma política do IAM a um perfil.

AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_attach_role_policy
#
# This function attaches an IAM policy to a role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_ARN -- The IAM policy document ARN..
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_attach_role_policy() {
    local role_name policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_attach_role_policy"
        echo "Attaches an AWS Identity and Access Management (IAM) policy to an IAM
role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_ARN -- The IAM policy document ARN."
        echo ""
    }
}
```

```
}

# Retrieve the calling parameters.
while getopts "n:p:h" option; do
    case "${option}" in
        n) role_name="${OPTARG}" ;;
        p) policy_arn="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy ARN with the -p parameter."
    usage
    return 1
fi

response=$(aws iam attach-role-policy \
    --role-name "$role_name" \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports attach-role-policy operation failed.\n$response"
    return 1
fi
```

```

    echo "$response"

    return 0
}

```

- Para obter detalhes da API, consulte [AttachRolePolicy](#) na Referência de comandos da AWS CLI.

Criar uma política

O exemplo de código a seguir mostra como criar uma política do IAM.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_create_policy
#
# This function creates an IAM policy.
#
# Parameters:
#     -n policy_name -- The name of the IAM policy.
#     -p policy_json -- The policy document.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.

```

```
#####
function iam_create_policy() {
    local policy_name policy_document response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_policy"
        echo "Creates an AWS Identity and Access Management (IAM) policy."
        echo "  -n policy_name    The name of the IAM policy."
        echo "  -p policy_json -- The policy document."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) policy_name="${OPTARG}" ;;
            p) policy_document="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$policy_name" ]]; then
        errecho "ERROR: You must provide a policy name with the -n parameter."
        usage
        return 1
    fi

    if [[ -z "$policy_document" ]]; then
        errecho "ERROR: You must provide a policy document with the -p parameter."
        usage
        return 1
    fi
}
```

```

response=$(aws iam create-policy \
  --policy-name "$policy_name" \
  --policy-document "$policy_document" \
  --output text \
  --query Policy.Arn)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports create-policy operation failed.\n$response"
  return 1
fi

echo "$response"
}

```

- Para obter detalhes da API, consulte [CreatePolicy](#) na Referência de comandos da AWS CLI.

Criar um perfil

O exemplo de código a seguir mostra como criar um perfil do IAM.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
  printf "%s\n" "$*" 1>&2
}

#####

```



```

# function iam_create_role
#
# This function creates an IAM role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_json -- The assume role policy document.
#
# Returns:
#     The ARN of the role.
#     And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_role() {
    local role_name policy_document response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user_access_key"
        echo "Creates an AWS Identity and Access Management (IAM) role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_json -- The assume role policy document."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_document="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

```

```
if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_document" ]]; then
    errecho "ERROR: You must provide a policy document with the -p parameter."
    usage
    return 1
fi

response=$(aws iam create-role \
    --role-name "$role_name" \
    --assume-role-policy-document "$policy_document" \
    --output text \
    --query Role.Arn)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-role operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [CreateRole](#) na Referência de comandos da AWS CLI.

Criar um usuário

O exemplo de código a seguir mostra como criar um usuário do IAM.

⚠ Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

AWS CLI com script Bash

i Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_create_user
#
# This function creates the specified IAM user, unless
# it already exists.
```

```

#
# Parameters:
#     -u user_name  -- The name of the user to create.
#
# Returns:
#     The ARN of the user.
#     And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_user() {
    local user_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user"
        echo "Creates an WS Identity and Access Management (IAM) user. You must supply a
username:"
        echo "  -u user_name    The name of the user. It must be unique within the
account."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$user_name" ]]; then
        errecho "ERROR: You must provide a username with the -u parameter."
        usage
    fi
}

```

```
    return 1
fi

iecho "Parameters:\n"
iecho "    User name:    $user_name"
iecho ""

# If the user already exists, we don't want to try to create it.
if (iam_user_exists "$user_name"); then
    errecho "ERROR: A user with that name already exists in the account."
    return 1
fi

response=$(aws iam create-user --user-name "$user_name" \
    --output text \
    --query 'User.Arn')

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-user operation failed.$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [CreateUser](#) na Referência de comandos da AWS CLI.

Criar uma chave de acesso

O exemplo de código a seguir mostra como criar uma chave de acesso do IAM.

⚠ Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

AWS CLI com script Bash

i Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_create_user_access_key
#
# This function creates an IAM access key for the specified user.
#
# Parameters:
#     -u user_name -- The name of the IAM user.
#     [-f file_name] -- The optional file name for the access key output.
#
# Returns:
#     [access_key_id access_key_secret]
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_user_access_key() {
    local user_name file_name response
```

```
local option OPTARG # Required to use getopt command in a function.

# bashsupport disable=BP5008
function usage() {
    echo "function iam_create_user_access_key"
    echo "Creates an AWS Identity and Access Management (IAM) key pair."
    echo "  -u user_name    The name of the IAM user."
    echo "  [-f file_name]  Optional file name for the access key output."
    echo ""
}

# Retrieve the calling parameters.
while getopt "u:f:h" option; do
    case "${option}" in
        u) user_name="${OPTARG}" ;;
        f) file_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

response=$(aws iam create-access-key \
    --user-name "$user_name" \
    --output text)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-access-key operation failed.$response"
```

```

    return 1
fi

if [[ -n "$file_name" ]]; then
    echo "$response" >"$file_name"
fi

local key_id key_secret
# shellcheck disable=SC2086
key_id=$(echo $response | cut -f 2 -d ' ')
# shellcheck disable=SC2086
key_secret=$(echo $response | cut -f 4 -d ' ')

echo "$key_id $key_secret"

return 0
}

```

- Para obter detalhes da API, consulte [CreateAccessKey](#) na Referência de comandos da AWS CLI.

Excluir uma política

O exemplo de código a seguir mostra como excluir uma política do IAM.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then

```



```

    echo "$@"
fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_delete_policy
#
# This function deletes an IAM policy.
#
# Parameters:
#     -n policy_arn -- The name of the IAM policy arn.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_policy() {
    local policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_policy"
        echo "Deletes an WS Identity and Access Management (IAM) policy"
        echo "  -n policy_arn -- The name of the IAM policy arn."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) policy_arn="${OPTARG}" ;;
            h)
                usage
                return 0
        esac
    done

```

```

        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy arn with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    Policy arn:  $policy_arn"
iecho ""

response=$(aws iam delete-policy \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-policy operation failed.\n$response"
    return 1
fi

iecho "delete-policy response:$response"
iecho

return 0
}

```

- Para obter detalhes da API, consulte [DeletePolicy](#) na Referência de comandos da AWS CLI.

Excluir uma função

O exemplo de código a seguir mostra como excluir um perfil do IAM.

AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_delete_role
#
# This function deletes an IAM role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_role() {
```

```

local role_name response
local option OPTARG # Required to use getopt command in a function.

# bashsupport disable=BP5008
function usage() {
    echo "function iam_delete_role"
    echo "Deletes an WS Identity and Access Management (IAM) role"
    echo "  -n role_name -- The name of the IAM role."
    echo ""
}

# Retrieve the calling parameters.
while getopt "n:h" option; do
    case "${option}" in
        n) role_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

echo "role_name:$role_name"
if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "  Role name:  $role_name"
iecho ""

response=$(aws iam delete-role \
    --role-name "$role_name")

local error_code=${?}

```

```

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-role operation failed.\n$response"
    return 1
fi

iecho "delete-role response:$response"
iecho

return 0
}

```

- Para obter detalhes da API, consulte [DeleteRole](#) na Referência de comandos da AWS CLI.

Excluir um usuário

O exemplo de código a seguir mostra como excluir um usuário do IAM.

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {

```

```

if [[ $VERBOSE == true ]]; then
    echo "$@"
fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_delete_user
#
# This function deletes the specified IAM user.
#
# Parameters:
#     -u user_name  -- The name of the user to create.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_user() {
    local user_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_user"
        echo "Deletes an WS Identity and Access Management (IAM) user. You must supply a
username:"
        echo "  -u user_name    The name of the user."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            h)

```

```
        usage
        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    User name:  $user_name"
iecho ""

# If the user does not exist, we don't want to try to delete it.
if (! iam_user_exists "$user_name"); then
    errecho "ERROR: A user with that name does not exist in the account."
    return 1
fi

response=$(aws iam delete-user \
    --user-name "$user_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-user operation failed.$response"
    return 1
fi

iecho "delete-user response:$response"
iecho

return 0
}
```

- Para obter detalhes da API, consulte [DeleteUser](#) na Referência de comandos da AWS CLI.

Excluir uma chave de acesso

O exemplo de código a seguir mostra como excluir uma chave de acesso do IAM.

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_delete_access_key
#
# This function deletes an IAM access key for the specified IAM user.
#
# Parameters:
#     -u user_name -- The name of the user.
#     -k access_key -- The access key to delete.
#
```



```

# Returns:
#      0 - If successful.
#      1 - If it fails.
#####
function iam_delete_access_key() {
    local user_name access_key response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_access_key"
        echo "Deletes an WS Identity and Access Management (IAM) access key for the
specified IAM user"
        echo "  -u user_name    The name of the user."
        echo "  -k access_key    The access key to delete."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:k:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            k) access_key="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$user_name" ]]; then
        errecho "ERROR: You must provide a username with the -u parameter."
        usage
        return 1
    fi

    if [[ -z "$access_key" ]]; then
        errecho "ERROR: You must provide an access key with the -k parameter."
    fi
}

```

```
usage
return 1
fi

iecho "Parameters:\n"
iecho "    Username:  $user_name"
iecho "    Access key:  $access_key"
iecho ""

response=$(aws iam delete-access-key \
    --user-name "$user_name" \
    --access-key-id "$access_key")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-access-key operation failed.\n$response"
    return 1
fi

iecho "delete-access-key response:$response"
iecho

return 0
}
```

- Para obter detalhes da API, consulte [DeleteAccessKey](#) na Referência de comandos da AWS CLI.

Desanexar uma política de uma função

O exemplo de código a seguir mostra como desanexar uma política do IAM.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_detach_role_policy
#
# This function detaches an IAM policy to a role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_ARN -- The IAM policy document ARN..
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_detach_role_policy() {
    local role_name policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_detach_role_policy"
        echo "Detaches an AWS Identity and Access Management (IAM) policy to an IAM
role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_ARN -- The IAM policy document ARN."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_arn="${OPTARG}" ;;
            h)
                usage
            ;;
        esac
    done
}
```

```

        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy ARN with the -p parameter."
    usage
    return 1
fi

response=$(aws iam detach-role-policy \
    --role-name "$role_name" \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports detach-role-policy operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}

```

- Para obter detalhes da API, consulte [DetachRolePolicy](#) na Referência de comandos da AWS CLI.

Obter um usuário

O exemplo de código a seguir mostra como obter um usuário do IAM.

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_user_exists
#
# This function checks to see if the specified AWS Identity and Access Management
# (IAM) user already exists.
#
# Parameters:
#     $1 - The name of the IAM user to check.
#
# Returns:
#     0 - If the user already exists.
#     1 - If the user doesn't exist.
#####
```

```
function iam_user_exists() {
    local user_name
    user_name=$1

    # Check whether the IAM user already exists.
    # We suppress all output - we're interested only in the return code.

    local errors
    errors=$(aws iam get-user \
        --user-name "$user_name" 2>&1 >/dev/null)

    local error_code=${?}

    if [[ $error_code -eq 0 ]]; then
        return 0 # 0 in Bash script means true.
    else
        if [[ $errors != *"error"*(NoSuchEntity)* ]]; then
            aws_cli_error_log $error_code
            errecho "Error calling iam get-user $errors"
        fi

        return 1 # 1 in Bash script means false.
    fi
}
```

- Para obter detalhes da API, consulte [GetUser](#) na Referência de comandos da AWS CLI.

Listar as chaves de acesso de um usuário

O exemplo de código a seguir mostra como listar as chaves de acesso do IAM de um usuário.

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_list_access_keys
#
# This function lists the access keys for the specified user.
#
# Parameters:
#     -u user_name -- The name of the IAM user.
#
# Returns:
#     access_key_ids
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_list_access_keys() {

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_list_access_keys"
        echo "Lists the AWS Identity and Access Management (IAM) access key IDs for the
specified user."
        echo "  -u user_name    The name of the IAM user."
        echo ""
    }
}
```

```
local user_name response
local option OPTARG # Required to use getopt command in a function.
# Retrieve the calling parameters.
while getopt "u:h" option; do
    case "${option}" in
        u) user_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

response=$(aws iam list-access-keys \
    --user-name "$user_name" \
    --output text \
    --query 'AccessKeyMetadata[].AccessKeyId')

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports list-access-keys operation failed.$response"
    return 1
fi

echo "$response"

return 0
}
```


- Para obter detalhes da API, consulte [ListAccessKeys](#) na Referência de comandos da AWS CLI.

Listar usuários

O exemplo de código a seguir mostra como listar usuários do IAM.

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_list_users
#
# List the IAM users in the account.
#
# Returns:
#     The list of users names
#     And:
#     0 - If the user already exists.
#     1 - If the user doesn't exist.
#####
```

```
function iam_list_users() {
    local option OPTARG # Required to use getopt command in a function.
    local error_code
    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_list_users"
        echo "Lists the AWS Identity and Access Management (IAM) user in the account."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "h" option; do
        case "${option}" in
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    local response

    response=$(aws iam list-users \
        --output text \
        --query "Users[].UserName")
    error_code=${?}

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log $error_code
        errecho "ERROR: AWS reports list-users operation failed.$response"
        return 1
    fi

    echo "$response"

    return 0
}
```

- Para obter detalhes da API, consulte [ListUsers](#) na Referência de comandos da AWS CLI.

Cenários

Criar um usuário e assumir uma função

O exemplo de código a seguir mostra como criar um usuário e assumir um perfil.

Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

- Crie um usuário sem permissões.
- Crie uma função que conceda permissão para listar os buckets do Amazon S3 para a conta.
- Adicione uma política para permitir que o usuário assuma a função.
- Assuma o perfil e liste buckets do S3 usando credenciais temporárias, depois limpe os recursos.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####  
# function iam_create_user_assume_role  
#  
# Scenario to create an IAM user, create an IAM role, and apply the role to the  
# user.  
#  
# "IAM access" permissions are needed to run this code.  
# "STS assume role" permissions are needed to run this code. (Note: It might be  
# necessary to
```

```

#         create a custom policy).
#
# Returns:
#     0 - If successful.
#     1 - If an error occurred.
#####
function iam_create_user_assume_role() {
{
    if [ "$IAM_OPERATIONS_SOURCED" != "True" ]; then

        source ./iam_operations.sh
    fi
}

echo_repeat "*" 88
echo "Welcome to the IAM create user and assume role demo."
echo
echo "This demo will create an IAM user, create an IAM role, and apply the role to
the user."
echo_repeat "*" 88
echo

echo -n "Enter a name for a new IAM user: "
get_input
user_name=$get_input_result

local user_arn
user_arn=$(iam_create_user -u "$user_name")

# shellcheck disable=SC2181
if [[ ${?} == 0 ]]; then
    echo "Created demo IAM user named $user_name"
else
    errecho "$user_arn"
    errecho "The user failed to create. This demo will exit."
    return 1
fi

local access_key_response
access_key_response=$(iam_create_user_access_key -u "$user_name")
# shellcheck disable=SC2181
if [[ ${?} != 0 ]]; then
    errecho "The access key failed to create. This demo will exit."
    clean_up "$user_name"

```

```
    return 1
fi

IFS=$'\t ' read -r -a access_key_values <<<"$access_key_response"
local key_name=${access_key_values[0]}
local key_secret=${access_key_values[1]}

echo "Created access key named $key_name"

echo "Wait 10 seconds for the user to be ready."
sleep 10
echo_repeat "*" 88
echo

local iam_role_name
iam_role_name=$(generate_random_name "test-role")
echo "Creating a role named $iam_role_name with user $user_name as the principal."

local assume_role_policy_document="{
  \"Version\": \"2012-10-17\",
  \"Statement\": [{
    \"Effect\": \"Allow\",
    \"Principal\": {\"AWS\": \"$user_arn\"},
    \"Action\": \"sts:AssumeRole\"
  }]
}"

local role_arn
role_arn=$(iam_create_role -n "$iam_role_name" -p "$assume_role_policy_document")

# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
  echo "Created IAM role named $iam_role_name"
else
  errecho "The role failed to create. This demo will exit."
  clean_up "$user_name" "$key_name"
  return 1
fi

local policy_name
policy_name=$(generate_random_name "test-policy")
local policy_document="{
  \"Version\": \"2012-10-17\",
  \"Statement\": [{
```

```

        \ "Effect\": \ "Allow\","
        \ "Action\": \ "s3:ListAllMyBuckets\","
        \ "Resource\": \ "arn:aws:s3:::*\"}]}"

local policy_arn
policy_arn=$(iam_create_policy -n "$policy_name" -p "$policy_document")
# shellcheck disable=SC2181
if [[ ${?} == 0 ]]; then
    echo "Created IAM policy named $policy_name"
else
    errecho "The policy failed to create."
    clean_up "$user_name" "$key_name" "$iam_role_name"
    return 1
fi

if (iam_attach_role_policy -n "$iam_role_name" -p "$policy_arn"); then
    echo "Attached policy $policy_arn to role $iam_role_name"
else
    errecho "The policy failed to attach."
    clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn"
    return 1
fi

local assume_role_policy_document="{
    \ "Version\": \ "2012-10-17\","
    \ "Statement\": [{
        \ "Effect\": \ "Allow\","
        \ "Action\": \ "sts:AssumeRole\","
        \ "Resource\": \ "$role_arn\"}]}"

local assume_role_policy_name
assume_role_policy_name=$(generate_random_name "test-assume-role-")

# shellcheck disable=SC2181
local assume_role_policy_arn
assume_role_policy_arn=$(iam_create_policy -n "$assume_role_policy_name" -p
"$assume_role_policy_document")
# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
    echo "Created IAM policy named $assume_role_policy_name for sts assume role"
else
    errecho "The policy failed to create."
    clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn" "$policy_arn"
    return 1

```

```
fi

echo "Wait 10 seconds to give AWS time to propagate these new resources and
connections."
sleep 10
echo_repeat "*" 88
echo

echo "Try to list buckets without the new user assuming the role."
echo_repeat "*" 88
echo

# Set the environment variables for the created user.
# bashsupport disable=BP2001
export AWS_ACCESS_KEY_ID=$key_name
# bashsupport disable=BP2001
export AWS_SECRET_ACCESS_KEY=$key_secret

local buckets
buckets=$(s3_list_buckets)

# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
    local bucket_count
    bucket_count=$(echo "$buckets" | wc -w | xargs)
    echo "There are $bucket_count buckets in the account. This should not have
happened."
else
    errecho "Because the role with permissions has not been assumed, listing buckets
failed."
fi

echo
echo_repeat "*" 88
echo "Now assume the role $iam_role_name and list the buckets."
echo_repeat "*" 88
echo

local credentials

credentials=$(sts_assume_role -r "$role_arn" -n "AssumeRoleDemoSession")
# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
    echo "Assumed role $iam_role_name"
```

```

else
    errecho "Failed to assume role."
    export AWS_ACCESS_KEY_ID=""
    export AWS_SECRET_ACCESS_KEY=""
    clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn" "$policy_arn"
"$assume_role_policy_arn"
    return 1
fi

IFS=$'\t ' read -r -a credentials <<<"$credentials"

export AWS_ACCESS_KEY_ID=${credentials[0]}
export AWS_SECRET_ACCESS_KEY=${credentials[1]}
# bashsupport disable=BP2001
export AWS_SESSION_TOKEN=${credentials[2]}

buckets=$(s3_list_buckets)

# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
    local bucket_count
    bucket_count=$(echo "$buckets" | wc -w | xargs)
    echo "There are $bucket_count buckets in the account. Listing buckets succeeded
because of "
    echo "the assumed role."
else
    errecho "Failed to list buckets. This should not happen."
    export AWS_ACCESS_KEY_ID=""
    export AWS_SECRET_ACCESS_KEY=""
    export AWS_SESSION_TOKEN=""
    clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn" "$policy_arn"
"$assume_role_policy_arn"
    return 1
fi

local result=0
export AWS_ACCESS_KEY_ID=""
export AWS_SECRET_ACCESS_KEY=""

echo
echo_repeat "*" 88
echo "The created resources will now be deleted."
echo_repeat "*" 88
echo

```



```

clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn" "$policy_arn"
"$assume_role_policy_arn"

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    result=1
fi

return $result
}

```

As funções do IAM usadas neste cenário.

```

#####
# function iam_user_exists
#
# This function checks to see if the specified AWS Identity and Access Management
# (IAM) user already exists.
#
# Parameters:
#     $1 - The name of the IAM user to check.
#
# Returns:
#     0 - If the user already exists.
#     1 - If the user doesn't exist.
#####
function iam_user_exists() {
    local user_name
    user_name=$1

    # Check whether the IAM user already exists.
    # We suppress all output - we're interested only in the return code.

    local errors
    errors=$(aws iam get-user \
        --user-name "$user_name" 2>&1 >/dev/null)

    local error_code=${?}

    if [[ $error_code -eq 0 ]]; then
        return 0 # 0 in Bash script means true.
    fi
}

```

```

else
    if [[ $errors != *"error"*(NoSuchEntity)* ]]; then
        aws_cli_error_log $error_code
        errecho "Error calling iam get-user $errors"
    fi

    return 1 # 1 in Bash script means false.
fi
}

#####
# function iam_create_user
#
# This function creates the specified IAM user, unless
# it already exists.
#
# Parameters:
#     -u user_name  -- The name of the user to create.
#
# Returns:
#     The ARN of the user.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_user() {
    local user_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user"
        echo "Creates an WS Identity and Access Management (IAM) user. You must supply a
username:"
        echo "  -u user_name    The name of the user. It must be unique within the
account."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            h)

```

```
        usage
        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    User name:  $user_name"
iecho ""

# If the user already exists, we don't want to try to create it.
if (iam_user_exists "$user_name"); then
    errecho "ERROR: A user with that name already exists in the account."
    return 1
fi

response=$(aws iam create-user --user-name "$user_name" \
    --output text \
    --query 'User.Arn')

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-user operation failed.$response"
    return 1
fi

echo "$response"

return 0
}
```

```
#####
# function iam_create_user_access_key
#
# This function creates an IAM access key for the specified user.
#
# Parameters:
#     -u user_name -- The name of the IAM user.
#     [-f file_name] -- The optional file name for the access key output.
#
# Returns:
#     [access_key_id access_key_secret]
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_user_access_key() {
    local user_name file_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user_access_key"
        echo "Creates an AWS Identity and Access Management (IAM) key pair."
        echo "  -u user_name    The name of the IAM user."
        echo "  [-f file_name]  Optional file name for the access key output."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:f:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            f) file_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
}
```

```

done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

response=$(aws iam create-access-key \
    --user-name "$user_name" \
    --output text)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-access-key operation failed.$response"
    return 1
fi

if [[ -n "$file_name" ]]; then
    echo "$response" >"$file_name"
fi

local key_id key_secret
# shellcheck disable=SC2086
key_id=$(echo $response | cut -f 2 -d ' ')
# shellcheck disable=SC2086
key_secret=$(echo $response | cut -f 4 -d ' ')

echo "$key_id $key_secret"

return 0
}

#####
# function iam_create_role
#
# This function creates an IAM role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_json -- The assume role policy document.

```

```

#
# Returns:
#     The ARN of the role.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_role() {
    local role_name policy_document response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user_access_key"
        echo "Creates an AWS Identity and Access Management (IAM) role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_json -- The assume role policy document."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_document="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$role_name" ]]; then
        errecho "ERROR: You must provide a role name with the -n parameter."
        usage
        return 1
    fi
}

```

```

if [[ -z "$policy_document" ]]; then
    errecho "ERROR: You must provide a policy document with the -p parameter."
    usage
    return 1
fi

response=$(aws iam create-role \
    --role-name "$role_name" \
    --assume-role-policy-document "$policy_document" \
    --output text \
    --query Role.Arn)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-role operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function iam_create_policy
#
# This function creates an IAM policy.
#
# Parameters:
#     -n policy_name -- The name of the IAM policy.
#     -p policy_json -- The policy document.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_policy() {
    local policy_name policy_document response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {

```

```

    echo "function iam_create_policy"
    echo "Creates an AWS Identity and Access Management (IAM) policy."
    echo "  -n policy_name    The name of the IAM policy."
    echo "  -p policy_json -- The policy document."
    echo ""
}

# Retrieve the calling parameters.
while getopts "n:p:h" option; do
    case "${option}" in
        n) policy_name="${OPTARG}" ;;
        p) policy_document="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$policy_name" ]]; then
    errecho "ERROR: You must provide a policy name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_document" ]]; then
    errecho "ERROR: You must provide a policy document with the -p parameter."
    usage
    return 1
fi

response=$(aws iam create-policy \
    --policy-name "$policy_name" \
    --policy-document "$policy_document" \
    --output text \
    --query Policy.Arn)

local error_code=${?}

```



```

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-policy operation failed.\n$response"
    return 1
fi

echo "$response"
}

#####
# function iam_attach_role_policy
#
# This function attaches an IAM policy to a role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_ARN -- The IAM policy document ARN..
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_attach_role_policy() {
    local role_name policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_attach_role_policy"
        echo "Attaches an AWS Identity and Access Management (IAM) policy to an IAM
role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_ARN -- The IAM policy document ARN."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_arn="${OPTARG}" ;;
            h)
                usage

```

```

        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy ARN with the -p parameter."
    usage
    return 1
fi

response=$(aws iam attach-role-policy \
    --role-name "$role_name" \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports attach-role-policy operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function iam_detach_role_policy
#
# This function detaches an IAM policy to a role.

```

```

#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_ARN -- The IAM policy document ARN..
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_detach_role_policy() {
    local role_name policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_detach_role_policy"
        echo "Detaches an AWS Identity and Access Management (IAM) policy to an IAM
role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_ARN -- The IAM policy document ARN."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_arn="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$role_name" ]]; then
        errecho "ERROR: You must provide a role name with the -n parameter."
        usage
    fi
}

```

```

    return 1
fi

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy ARN with the -p parameter."
    usage
    return 1
fi

response=$(aws iam detach-role-policy \
    --role-name "$role_name" \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports detach-role-policy operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function iam_delete_policy
#
# This function deletes an IAM policy.
#
# Parameters:
#     -n policy_arn -- The name of the IAM policy arn.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_policy() {
    local policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {

```

```

    echo "function iam_delete_policy"
    echo "Deletes an WS Identity and Access Management (IAM) policy"
    echo "  -n policy_arn -- The name of the IAM policy arn."
    echo ""
}

# Retrieve the calling parameters.
while getopts "n:h" option; do
    case "${option}" in
        n) policy_arn="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy arn with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "  Policy arn:  $policy_arn"
iecho ""

response=$(aws iam delete-policy \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-policy operation failed.\n$response"
    return 1
fi

```

```

    iecho "delete-policy response:$response"
    iecho

    return 0
}

#####
# function iam_delete_role
#
# This function deletes an IAM role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_role() {
    local role_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_role"
        echo "Deletes an WS Identity and Access Management (IAM) role"
        echo "  -n role_name -- The name of the IAM role."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
}

```

```

done
export OPTIND=1

echo "role_name:$role_name"
if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    Role name: $role_name"
iecho ""

response=$(aws iam delete-role \
    --role-name "$role_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-role operation failed.\n$response"
    return 1
fi

iecho "delete-role response:$response"
iecho

return 0
}

#####
# function iam_delete_access_key
#
# This function deletes an IAM access key for the specified IAM user.
#
# Parameters:
#     -u user_name -- The name of the user.
#     -k access_key -- The access key to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####

```

```

function iam_delete_access_key() {
    local user_name access_key response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_access_key"
        echo "Deletes an WS Identity and Access Management (IAM) access key for the
specified IAM user"
        echo "  -u user_name    The name of the user."
        echo "  -k access_key    The access key to delete."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:k:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            k) access_key="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$user_name" ]]; then
        errecho "ERROR: You must provide a username with the -u parameter."
        usage
        return 1
    fi

    if [[ -z "$access_key" ]]; then
        errecho "ERROR: You must provide an access key with the -k parameter."
        usage
        return 1
    fi
}

```



```

iecho "Parameters:\n"
iecho "    Username:  $user_name"
iecho "    Access key:  $access_key"
iecho ""

response=$(aws iam delete-access-key \
    --user-name "$user_name" \
    --access-key-id "$access_key")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-access-key operation failed.\n$response"
    return 1
fi

iecho "delete-access-key response:$response"
iecho

return 0
}

#####
# function iam_delete_user
#
# This function deletes the specified IAM user.
#
# Parameters:
#     -u user_name  -- The name of the user to create.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_user() {
    local user_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_user"
        echo "Deletes an WS Identity and Access Management (IAM) user. You must supply a
username:"

```

```
    echo "    -u user_name    The name of the user."
    echo ""
}

# Retrieve the calling parameters.
while getopts "u:h" option; do
    case "${option}" in
        u) user_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    User name:    $user_name"
iecho ""

# If the user does not exist, we don't want to try to delete it.
if (! iam_user_exists "$user_name"); then
    errecho "ERROR: A user with that name does not exist in the account."
    return 1
fi

response=$(aws iam delete-user \
    --user-name "$user_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
```

```
    errecho "ERROR: AWS reports delete-user operation failed.$response"
    return 1
fi

iecho "delete-user response:$response"
iecho

return 0
}
```

- Para obter detalhes da API, consulte os tópicos a seguir na Referência de comandos da AWS CLI.
 - [AttachRolePolicy](#)
 - [CreateAccessKey](#)
 - [CreatePolicy](#)
 - [CreateRole](#)
 - [CreateUser](#)
 - [DeleteAccessKey](#)
 - [DeletePolicy](#)
 - [DeleteRole](#)
 - [DeleteUser](#)
 - [DeleteUserPolicy](#)
 - [DetachRolePolicy](#)
 - [PutUserPolicy](#)

Exemplos do Amazon S3 de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do Amazon S3.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

Tópicos

- [Ações](#)
- [Cenários](#)

Ações

Copiar um objeto de um bucket para outro

O exemplo de código a seguir mostra como copiar um objeto do S3 de um bucket para outro.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function copy_item_in_bucket
#
# This function creates a copy of the specified file in the same bucket.
#
# Parameters:
#     $1 - The name of the bucket to copy the file from and to.
#     $2 - The key of the source file to copy.
#     $3 - The key of the destination file.
#
# Returns:
```

```
#      0 - If successful.
#      1 - If it fails.
#####
function copy_item_in_bucket() {
    local bucket_name=$1
    local source_key=$2
    local destination_key=$3
    local response

    response=$(aws s3api copy-object \
        --bucket "$bucket_name" \
        --copy-source "$bucket_name/$source_key" \
        --key "$destination_key")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api copy-object operation failed.\n$response"
        return 1
    fi
}
```

- Para obter detalhes da API, consulte [CopyObject](#) na Referência de comandos da AWS CLI.

Criar um bucket

O exemplo de código a seguir mostra como criar um bucket do S3.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
```

```

function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function create-bucket
#
# This function creates the specified bucket in the specified AWS Region, unless
# it already exists.
#
# Parameters:
#     -b bucket_name  -- The name of the bucket to create.
#     -r region_code  -- The code for an AWS Region in which to
#                        create the bucket.
#
# Returns:
#     The URL of the bucket that was created.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function create_bucket() {
    local bucket_name region_code response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function create_bucket"
        echo "Creates an Amazon S3 bucket. You must supply a bucket name:"
        echo "  -b bucket_name    The name of the bucket. It must be globally unique."
        echo "  [-r region_code]  The code for an AWS Region in which the bucket is
created."
        echo ""
    }

```

```

}

# Retrieve the calling parameters.
while getopts "b:r:h" option; do
    case "${option}" in
        b) bucket_name="${OPTARG}" ;;
        r) region_code="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done

if [[ -z "$bucket_name" ]]; then
    errecho "ERROR: You must provide a bucket name with the -b parameter."
    usage
    return 1
fi

local bucket_config_arg
# A location constraint for "us-east-1" returns an error.
if [[ -n "$region_code" ]] && [[ "$region_code" != "us-east-1" ]]; then
    bucket_config_arg="--create-bucket-configuration LocationConstraint=
$region_code"
fi

iecho "Parameters:\n"
iecho "    Bucket name:    $bucket_name"
iecho "    Region code:    $region_code"
iecho ""

# If the bucket already exists, we don't want to try to create it.
if (bucket_exists "$bucket_name"); then
    errecho "ERROR: A bucket with that name already exists. Try again."
    return 1
fi

# shellcheck disable=SC2086

```

```

response=$(aws s3api create-bucket \
  --bucket "$bucket_name" \
  $bucket_config_arg)

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
  errecho "ERROR: AWS reports create-bucket operation failed.\n$response"
  return 1
fi
}

```

- Para obter detalhes da API, consulte [CreateBucket](#) na Referência de comandos da AWS CLI.

Excluir um bucket vazio

O exemplo de código a seguir mostra como excluir um bucket vazio do S3.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
  printf "%s\n" "$*" 1>&2
}

#####
# function delete_bucket
#
# This function deletes the specified bucket.
#
# Parameters:

```



```
#      $1 - The name of the bucket.

# Returns:
#      0 - If successful.
#      1 - If it fails.
#####
function delete_bucket() {
    local bucket_name=$1
    local response

    response=$(aws s3api delete-bucket \
        --bucket "$bucket_name")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api delete-bucket failed.\n$response"
        return 1
    fi
}
```

- Para obter detalhes da API, consulte [DeleteBucket](#) na Referência de comandos da AWS CLI.

Excluir um objeto

O exemplo de código a seguir mostra como excluir um objeto do S3.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
```

```

    printf "%s\n" "$*" 1>&2
}

#####
# function delete_item_in_bucket
#
# This function deletes the specified file from the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket.
#     $2 - The key (file name) in the bucket to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function delete_item_in_bucket() {
    local bucket_name=$1
    local key=$2
    local response

    response=$(aws s3api delete-object \
        --bucket "$bucket_name" \
        --key "$key")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api delete-object operation failed.\n$response"
        return 1
    fi
}

```

- Para obter detalhes da API, consulte [DeleteObject](#) na Referência de comandos da AWS CLI.

Excluir vários objetos

O exemplo de código a seguir mostra como excluir vários objetos de um bucket do S3.

AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function delete_items_in_bucket
#
# This function deletes the specified list of keys from the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket.
#     $2 - A list of keys in the bucket to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function delete_items_in_bucket() {
    local bucket_name=$1
    local keys=$2
    local response

    # Create the JSON for the items to delete.
    local delete_items
    delete_items="{\"Objects\":["
    for key in $keys; do
        delete_items="$delete_items{\"Key\": \"$key\"},"
    done
    delete_items=${delete_items%?} # Remove the final comma.
```

```

delete_items="$delete_items]}"

response=$(aws s3api delete-objects \
  --bucket "$bucket_name" \
  --delete "$delete_items")

# shellcheck disable=SC2181
if [[ $? -ne 0 ]]; then
  errecho "ERROR: AWS reports s3api delete-object operation failed.\n$response"
  return 1
fi
}

```

- Para obter detalhes da API, consulte [DeleteObjects](#) na Referência de comandos da AWS CLI.

Determinar a existência de um bucket

O exemplo de código a seguir mostra como determinar a existência de um bucket do S3.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function bucket_exists
#
# This function checks to see if the specified bucket already exists.
#
# Parameters:
#     $1 - The name of the bucket to check.
#
# Returns:
#     0 - If the bucket already exists.
#     1 - If the bucket doesn't exist.
#####
function bucket_exists() {
  local bucket_name

```

```

bucket_name=$1

# Check whether the bucket already exists.
# We suppress all output - we're interested only in the return code.

if aws s3api head-bucket \
    --bucket "$bucket_name" \
    >/dev/null 2>&1; then
    return 0 # 0 in Bash script means true.
else
    return 1 # 1 in Bash script means false.
fi
}

```

- Para obter detalhes da API, consulte [HeadBucket](#) na Referência de comandos da AWS CLI.

Obter um objeto de um bucket

O exemplo de código a seguir mostra como ler dados de um objeto em um bucket do S3.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function download_object_from_bucket
#
# This function downloads an object in a bucket to a file.

```

```
#
# Parameters:
#     $1 - The name of the bucket to download the object from.
#     $2 - The path and file name to store the downloaded bucket.
#     $3 - The key (name) of the object in the bucket.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function download_object_from_bucket() {
    local bucket_name=$1
    local destination_file_name=$2
    local object_name=$3
    local response

    response=$(aws s3api get-object \
        --bucket "$bucket_name" \
        --key "$object_name" \
        "$destination_file_name")

    # shellcheck disable=SC2181
    if [[ ${?} -ne 0 ]]; then
        errecho "ERROR: AWS reports put-object operation failed.\n$response"
        return 1
    fi
}
```

- Para obter detalhes da API, consulte [GetObject](#) na Referência de comandos da AWS CLI.

Listar objetos em um bucket

O exemplo de código a seguir mostra como listar objetos em um bucket do S3.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function list_items_in_bucket
#
# This function displays a list of the files in the bucket with each file's
# size. The function uses the --query parameter to retrieve only the key and
# size fields from the Contents collection.
#
# Parameters:
#     $1 - The name of the bucket.
#
# Returns:
#     The list of files in text format.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function list_items_in_bucket() {
    local bucket_name=$1
    local response

    response=$(aws s3api list-objects \
        --bucket "$bucket_name" \
        --output text \
        --query 'Contents[].{Key: Key, Size: Size}')

    # shellcheck disable=SC2181
    if [[ ${?} -eq 0 ]]; then
        echo "$response"
    else
        errecho "ERROR: AWS reports s3api list-objects operation failed.\n$response"
        return 1
    fi
}
```

- Para obter detalhes da API, consulte [ListObjectsV2](#) na Referência de comandos da AWS CLI.

Carregar um objeto em um bucket

O exemplo de código a seguir mostra como carregar um objeto em um bucket do S3.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function copy_file_to_bucket
#
# This function creates a file in the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket to copy the file to.
#     $2 - The path and file name of the local file to copy to the bucket.
#     $3 - The key (name) to call the copy of the file in the bucket.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function copy_file_to_bucket() {
    local response bucket_name source_file destination_file_name
    bucket_name=$1
    source_file=$2
    destination_file_name=$3
```



```

response=$(aws s3api put-object \
  --bucket "$bucket_name" \
  --body "$source_file" \
  --key "$destination_file_name")

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
  errecho "ERROR: AWS reports put-object operation failed.\n$response"
  return 1
fi
}

```

- Para obter detalhes da API, consulte [PutObject](#) na Referência de comandos da AWS CLI.

Cenários

Conceitos básicos de buckets e objetos

O código de exemplo a seguir mostra como:

- Criar um bucket e fazer upload de um arquivo para ele.
- Baixar um objeto de um bucket.
- Copiar um objeto em uma subpasta em um bucket.
- Listar os objetos em um bucket.
- Excluir os objetos do bucket e o bucket.

AWS CLI com script Bash

Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function s3_getting_started
#

```

```
# This function creates, copies, and deletes S3 buckets and objects.
#
# Returns:
#     0 - If successful.
#     1 - If an error occurred.
#####
function s3_getting_started() {
{
    if [ "$BUCKET_OPERATIONS_SOURCED" != "True" ]; then
        cd bucket-lifecycle-operations || exit

        source ./bucket_operations.sh
        cd ..
    fi
}

echo_repeat "*" 88
echo "Welcome to the Amazon S3 getting started demo."
echo_repeat "*" 88

local bucket_name
bucket_name=$(generate_random_name "doc-example-bucket")

local region_code
region_code=$(aws configure get region)

if create_bucket -b "$bucket_name" -r "$region_code"; then
    echo "Created demo bucket named $bucket_name"
else
    errecho "The bucket failed to create. This demo will exit."
    return 1
fi

local file_name
while [ -z "$file_name" ]; do
    echo -n "Enter a file you want to upload to your bucket: "
    get_input
    file_name=$get_input_result

    if [ ! -f "$file_name" ]; then
        echo "Could not find file $file_name. Are you sure it exists?"
        file_name=""
    fi
done
```

```
local key
key="$(basename "$file_name")"

local result=0
if copy_file_to_bucket "$bucket_name" "$file_name" "$key"; then
    echo "Uploaded file $file_name into bucket $bucket_name with key $key."
else
    result=1
fi

local destination_file
destination_file="$file_name.download"
if yes_no_input "Would you like to download $key to the file $destination_file?
(y/n) "; then
    if download_object_from_bucket "$bucket_name" "$destination_file" "$key"; then
        echo "Downloaded $key in the bucket $bucket_name to the file
$destination_file."
    else
        result=1
    fi
fi

if yes_no_input "Would you like to copy $key a new object key in your bucket? (y/
n) "; then
    local to_key
    to_key="demo/$key"
    if copy_item_in_bucket "$bucket_name" "$key" "$to_key"; then
        echo "Copied $key in the bucket $bucket_name to the $to_key."
    else
        result=1
    fi
fi

local bucket_items
bucket_items=$(list_items_in_bucket "$bucket_name")

# shellcheck disable=SC2181
if [[ $? -ne 0 ]]; then
    result=1
fi

echo "Your bucket contains the following items."
echo -e "Name\t\tSize"
```

```

echo "$bucket_items"

if yes_no_input "Delete the bucket, $bucket_name, as well as the objects in it?
(y/n) "; then
    bucket_items=$(echo "$bucket_items" | cut -f 1)

    if delete_items_in_bucket "$bucket_name" "$bucket_items"; then
        echo "The following items were deleted from the bucket $bucket_name"
        echo "$bucket_items"
    else
        result=1
    fi

    if delete_bucket "$bucket_name"; then
        echo "Deleted the bucket $bucket_name"
    else
        result=1
    fi
fi

return $result
}

```

As funções do Amazon S3 usadas neste cenário.

```

#####
# function create-bucket
#
# This function creates the specified bucket in the specified AWS Region, unless
# it already exists.
#
# Parameters:
#     -b bucket_name  -- The name of the bucket to create.
#     -r region_code  -- The code for an AWS Region in which to
#                        create the bucket.
#
# Returns:
#     The URL of the bucket that was created.
#
# And:
#     0 - If successful.
#     1 - If it fails.
#####

```

```

function create_bucket() {
    local bucket_name region_code response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function create_bucket"
        echo "Creates an Amazon S3 bucket. You must supply a bucket name:"
        echo "  -b bucket_name    The name of the bucket. It must be globally unique."
        echo "  [-r region_code]    The code for an AWS Region in which the bucket is
created."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "b:r:h" option; do
        case "${option}" in
            b) bucket_name="${OPTARG}" ;;
            r) region_code="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done

    if [[ -z "$bucket_name" ]]; then
        errecho "ERROR: You must provide a bucket name with the -b parameter."
        usage
        return 1
    fi

    local bucket_config_arg
    # A location constraint for "us-east-1" returns an error.
    if [[ -n "$region_code" ]] && [[ "$region_code" != "us-east-1" ]]; then
        bucket_config_arg="--create-bucket-configuration LocationConstraint=
$region_code"
    fi

```

```

iecho "Parameters:\n"
iecho "    Bucket name:    $bucket_name"
iecho "    Region code:    $region_code"
iecho ""

# If the bucket already exists, we don't want to try to create it.
if (bucket_exists "$bucket_name"); then
    errecho "ERROR: A bucket with that name already exists. Try again."
    return 1
fi

# shellcheck disable=SC2086
response=$(aws s3api create-bucket \
    --bucket "$bucket_name" \
    $bucket_config_arg)

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "ERROR: AWS reports create-bucket operation failed.\n$response"
    return 1
fi
}

#####
# function copy_file_to_bucket
#
# This function creates a file in the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket to copy the file to.
#     $2 - The path and file name of the local file to copy to the bucket.
#     $3 - The key (name) to call the copy of the file in the bucket.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function copy_file_to_bucket() {
    local response bucket_name source_file destination_file_name
    bucket_name=$1
    source_file=$2
    destination_file_name=$3

    response=$(aws s3api put-object \

```

```

    --bucket "$bucket_name" \
    --body "$source_file" \
    --key "$destination_file_name")

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "ERROR: AWS reports put-object operation failed.\n$response"
    return 1
fi
}

#####
# function download_object_from_bucket
#
# This function downloads an object in a bucket to a file.
#
# Parameters:
#     $1 - The name of the bucket to download the object from.
#     $2 - The path and file name to store the downloaded bucket.
#     $3 - The key (name) of the object in the bucket.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function download_object_from_bucket() {
    local bucket_name=$1
    local destination_file_name=$2
    local object_name=$3
    local response

    response=$(aws s3api get-object \
        --bucket "$bucket_name" \
        --key "$object_name" \
        "$destination_file_name")

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "ERROR: AWS reports put-object operation failed.\n$response"
    return 1
fi
}

#####

```

```

# function copy_item_in_bucket
#
# This function creates a copy of the specified file in the same bucket.
#
# Parameters:
#     $1 - The name of the bucket to copy the file from and to.
#     $2 - The key of the source file to copy.
#     $3 - The key of the destination file.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function copy_item_in_bucket() {
    local bucket_name=$1
    local source_key=$2
    local destination_key=$3
    local response

    response=$(aws s3api copy-object \
        --bucket "$bucket_name" \
        --copy-source "$bucket_name/$source_key" \
        --key "$destination_key")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api copy-object operation failed.\n$response"
        return 1
    fi
}

#####
# function list_items_in_bucket
#
# This function displays a list of the files in the bucket with each file's
# size. The function uses the --query parameter to retrieve only the key and
# size fields from the Contents collection.
#
# Parameters:
#     $1 - The name of the bucket.
#
# Returns:
#     The list of files in text format.
#
# And:

```



```

#      0 - If successful.
#      1 - If it fails.
#####
function list_items_in_bucket() {
    local bucket_name=$1
    local response

    response=$(aws s3api list-objects \
        --bucket "$bucket_name" \
        --output text \
        --query 'Contents[].{Key: Key, Size: Size}')

    # shellcheck disable=SC2181
    if [[ ${?} -eq 0 ]]; then
        echo "$response"
    else
        errecho "ERROR: AWS reports s3api list-objects operation failed.\n$response"
        return 1
    fi
}

#####
# function delete_items_in_bucket
#
# This function deletes the specified list of keys from the specified bucket.
#
# Parameters:
#      $1 - The name of the bucket.
#      $2 - A list of keys in the bucket to delete.
#
# Returns:
#      0 - If successful.
#      1 - If it fails.
#####
function delete_items_in_bucket() {
    local bucket_name=$1
    local keys=$2
    local response

    # Create the JSON for the items to delete.
    local delete_items
    delete_items="{\"Objects\":["
    for key in $keys; do
        delete_items="$delete_items{\"Key\": \"$key\"},"
    done

```

```

done
delete_items=${delete_items%?} # Remove the final comma.
delete_items="$delete_items]}"

response=$(aws s3api delete-objects \
  --bucket "$bucket_name" \
  --delete "$delete_items")

# shellcheck disable=SC2181
if [[ $? -ne 0 ]]; then
  errecho "ERROR: AWS reports s3api delete-object operation failed.\n$response"
  return 1
fi
}

#####
# function delete_bucket
#
# This function deletes the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function delete_bucket() {
  local bucket_name=$1
  local response

  response=$(aws s3api delete-bucket \
    --bucket "$bucket_name")

  # shellcheck disable=SC2181
  if [[ $? -ne 0 ]]; then
    errecho "ERROR: AWS reports s3api delete-bucket failed.\n$response"
    return 1
  fi
}

```

- Para obter detalhes da API, consulte os tópicos a seguir na Referência de comandos da AWS CLI.
 - [CopyObject](#)
 - [CreateBucket](#)
 - [DeleteBucket](#)
 - [DeleteObjects](#)
 - [GetObject](#)
 - [ListObjectsV2](#)
 - [PutObject](#)

Exemplos do AWS STS de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do AWS STS.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

Tópicos

- [Ações](#)

Ações

Assumir uma função

O exemplo de código a seguir mostra como assumir um perfil com o AWS STS.

AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function sts_assume_role
#
# This function assumes a role in the AWS account and returns the temporary
# credentials.
#
# Parameters:
#     -n role_session_name -- The name of the session.
#     -r role_arn -- The ARN of the role to assume.
#
# Returns:
#     [access_key_id, secret_access_key, session_token]
#
# And:
```

```

#      0 - If successful.
#      1 - If an error occurred.
#####
function sts_assume_role() {
    local role_session_name role_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function sts_assume_role"
        echo "Assumes a role in the AWS account and returns the temporary credentials:"
        echo "  -n role_session_name -- The name of the session."
        echo "  -r role_arn -- The ARN of the role to assume."
        echo ""
    }

    while getopt n:r:h option; do
        case "${option}" in
            n) role_session_name=${OPTARG} ;;
            r) role_arn=${OPTARG} ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done

    response=$(aws sts assume-role \
        --role-session-name "$role_session_name" \
        --role-arn "$role_arn" \
        --output text \
        --query "Credentials.[AccessKeyId, SecretAccessKey, SessionToken]")

    local error_code=${?}

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log $error_code
        errecho "ERROR: AWS reports create-role operation failed.\n$response"
        return 1
    fi
}

```

```
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [AssumeRole](#) na Referência de comandos da AWS CLI.

Segurança na AWS Command Line Interface

A segurança na nuvem na AWS é a nossa maior prioridade. Como cliente da AWS, você contará com um datacenter e uma arquitetura de rede criados para atender aos requisitos das organizações com as maiores exigências de segurança.

A segurança é uma responsabilidade compartilhada entre a AWS e você. O [modelo de responsabilidade compartilhada](#) descreve isso como segurança da nuvem e segurança na nuvem:

- Segurança da nuvem — a AWS é responsável pela proteção da infraestrutura que executa serviços AWS na Nuvem AWS. A AWS também fornece serviços que podem ser usados com segurança. Auditores de terceiros testam e verificam regularmente a eficácia da nossa segurança como parte dos [AWS Programas de Conformidade](#). Para saber mais sobre os programas de conformidade que se aplicam ao AWS Command Line Interface, consulte [AWS Serviços da em escopo por programa de conformidade](#).
- Segurança na nuvem: sua responsabilidade é determinada pelo serviço da AWS que você usa. Você também é responsável por outros fatores, incluindo a confidencialidade de seus dados, os requisitos da empresa e as leis e regulamentos aplicáveis.

Esta documentação ajuda você a entender como aplicar o modelo de responsabilidade compartilhada ao usar a AWS Command Line Interface (AWS CLI). Os tópicos a seguir mostram como configurar a AWS CLI para atender aos seus objetivos de segurança e conformidade. Você também aprende como usar a AWS CLI para ajudar a monitorar e a proteger os recursos da AWS.

Tópicos

- [Proteção de dados no AWS CLI](#)
- [Identity and Access Management](#)
- [Validação de conformidade para este produto ou serviço da AWS](#)
- [Resiliência para este produto ou serviço da AWS](#)
- [Segurança da infraestrutura para esse produto ou serviço da AWS](#)
- [Impor uma versão mínima do TLS](#)

Proteção de dados no AWS CLI

O AWS [modelo de responsabilidade compartilhada](#) se aplica à proteção de dados no AWS Command Line Interface. Conforme descrito nesse modelo, a AWS é responsável por proteger a infraestrutura global que executa toda a Nuvem AWS. Você é responsável por manter o controle sobre seu conteúdo hospedado nessa infraestrutura. Você também é responsável pelas tarefas de configuração e gerenciamento de segurança dos Serviços da AWS que usa. Para obter mais informações sobre a privacidade de dados, consulte as [Perguntas frequentes sobre privacidade de dados](#). Para mais informações sobre a proteção de dados na Europa, consulte o artigo [AWS Shared Responsibility Model and GDPR](#) no Blog de segurança da AWS.

Para fins de proteção de dados, recomendamos que você proteja as credenciais da Conta da AWS e configure as contas de usuário individuais com o AWS IAM Identity Center ou o AWS Identity and Access Management (IAM). Dessa maneira, cada usuário receberá apenas as permissões necessárias para cumprir suas obrigações de trabalho. Recomendamos também que você proteja seus dados das seguintes formas:

- Use uma autenticação multifator (MFA [multi-factor authentication]) com cada conta.
- Use SSL/TLS para se comunicar com os atributos da AWS. Exigimos TLS 1.2 e recomendamos TLS 1.3.
- Configure o registro em log das atividades da API e do usuário com o AWS CloudTrail
- Use as soluções de criptografia da AWS, juntamente com todos os controles de segurança padrão dos Serviços da AWS.
- Use serviços gerenciados de segurança avançada, como o Amazon Macie, que ajuda a localizar e proteger dados sigilosos armazenados no Amazon S3.
- Se você precisar de módulos criptográficos validados pelo FIPS 140-2 ao acessar a AWS por meio de uma interface de linha de comandos ou uma API, use um endpoint do FIPS. Para ter mais informações sobre endpoints do FIPS, consulte [Federal Information Processing Standard \(FIPS\) 140-2](#).

É altamente recomendável que nunca sejam colocadas informações de identificação confidenciais, como endereços de e-mail dos seus clientes, em marcações ou campos de formato livre, como um campo Nome. Isso inclui trabalhar com a AWS CLI ou outros Serviços da AWS usando o console, a API, a AWS CLI ou os AWS SDKs. Quaisquer dados inseridos em tags ou campos de texto de formato livre usados para nomes podem ser usados para logs de faturamento ou de diagnóstico. Se

você fornecer um URL para um servidor externo, recomendamos fortemente que não sejam incluídas informações de credenciais no URL para validar a solicitação a esse servidor.

Criptografia de dados

Um atributo fundamental de qualquer serviço seguro é que as informações sejam criptografadas quando não estão sendo usadas ativamente.

Criptografia inativa

A própria AWS CLI não armazena nenhum dado do cliente além das credenciais de que precisa para interagir com os serviços da AWS em nome do usuário.

Se você usar a AWS CLI para invocar um serviço da AWS que transmita dados do cliente ao computador local para armazenamento, consulte o capítulo Segurança e conformidade no Guia do usuário desse serviço para obter informações sobre como esses dados são armazenados, protegidos e criptografados.

Criptografia em trânsito

Por padrão, todos os dados transmitidos do computador cliente que executa os endpoints de serviço da AWS CLI e da AWS são criptografados enviando tudo por meio de uma conexão HTTPS/TLS.

Você não precisa fazer nada para ativar o uso do HTTPS/TLS. Ele está sempre ativado, a menos que você o desative explicitamente para um comando individual usando a opção de linha de comando `--no-verify-ssl`.

Identity and Access Management

O AWS Identity and Access Management (IAM) é um AWS service (Serviço da AWS) que ajuda a controlar o acesso aos recursos da AWS de forma segura. Os administradores do IAM controlam quem pode ser autenticado (conectado) e autorizado (ter permissões) a usar os recursos do AWS. O IAM é um AWS service (Serviço da AWS) que pode ser usado sem custo adicional.

Tópicos

- [Público](#)
- [Autenticando com identidades](#)
- [Gerenciamento do acesso usando políticas](#)
- [Como Serviços da AWS funcionam com o IAM](#)

- [Solução de problemas de identidade e acesso do AWS](#)

Público

O uso do AWS Identity and Access Management (IAM) varia dependendo do trabalho que for realizado no AWS.

Usuário do serviço: se você usar Serviços da AWS para fazer seu trabalho, o administrador fornecerá as credenciais e as permissões necessárias. À medida que usar mais recursos do AWS para fazer seu trabalho, você poderá precisar de permissões adicionais. Entender como o acesso é gerenciado pode ajudá-lo a solicitar as permissões corretas ao seu administrador. Se você não conseguir acessar um atributo na AWS, consulte [Solução de problemas de identidade e acesso do AWS](#) ou o guia do usuário do AWS service (Serviço da AWS) que você está usando.

Administrador do serviço – Se você for o responsável pelos recursos do AWS na empresa, provavelmente terá acesso total ao AWS. Cabe a você determinar quais funcionalidades e recursos do AWS os usuários do serviço devem acessar. Assim, você deve enviar solicitações ao administrador do IAM para alterar as permissões dos usuários de seu serviço. Revise as informações nesta página para entender os Introdução ao IAM. Para saber mais sobre como sua empresa pode usar o IAM com a AWS, consulte o guia do usuário do AWS service (Serviço da AWS) que você está usando.

Administrador do IAM – Se você for um administrador do IAM, talvez queira saber detalhes sobre como pode gravar políticas para gerenciar o acesso ao AWS. Para visualizar exemplos de políticas baseadas em identidade da AWS que podem ser usadas no IAM, consulte o guia do usuário do AWS service (Serviço da AWS) que você está usando.

Autenticando com identidades

A autenticação é a forma como você faz login na AWS usando suas credenciais de identidade. É necessário ser autenticado (fazer login na AWS) como o usuário raiz da Usuário raiz da conta da AWS, como usuário do IAM ou assumindo um perfil do IAM.

Você pode fazer login na AWS como uma identidade federada usando credenciais fornecidas por uma fonte de identidades. AWS IAM Identity Center Os usuários do IAM Identity Center, a autenticação única da empresa e as suas credenciais do Google ou do Facebook são exemplos de identidades federadas. Quando você faz login como uma identidade federada, o administrador já configurou anteriormente a federação de identidades usando perfis do IAM. Quando você acessa a AWS usando a federação, está indiretamente assumindo um perfil.

É possível fazer login no AWS Management Console ou no portal de acesso da AWS dependendo do tipo de usuário que você é. Para obter mais informações sobre como fazer login na AWS, consulte [Como fazer login na conta da Conta da AWS](#) no Guia do usuário do Início de Sessão da AWS.

Se você acessar a AWS programaticamente, a AWS fornecerá um kit de desenvolvimento de software (SDK) e uma interface da linha de comando (CLI) para você assinar criptograficamente as solicitações usando as suas credenciais. Se você não utilizar as ferramentas da AWS, deverá assinar as solicitações por conta própria. Para obter mais informações sobre como usar o método recomendado para assinar solicitações por conta própria, consulte [Assinar solicitações de API da AWS](#) no Guia do usuário do IAM.

Independentemente do método de autenticação usado, também pode ser exigido que você forneça mais informações de segurança. Por exemplo, a AWS recomenda o uso da autenticação multifator (MFA) para aumentar a segurança de sua conta. Para saber mais, consulte [Autenticação multifator](#) no Guia AWS IAM Identity Center do usuário. [Usar a autenticação multifator \(MFA\) na AWS](#) no Guia do usuário do IAM.

Usuário root da Conta da AWS

Ao criar uma Conta da AWS, você começa com uma identidade de login que tem acesso completo a todos os recursos e Serviços da AWS na conta. Essa identidade, denominada usuário raiz da Conta da AWS, é acessada por login com o endereço de e-mail e a senha que você usou para criar a conta. É altamente recomendável não usar o usuário raiz para tarefas diárias. Proteja as credenciais do usuário raiz e use-as para executar as tarefas que somente ele pode executar. Para obter a lista completa das tarefas que exigem login como usuário raiz, consulte [Tarefas que exigem credenciais de usuário raiz](#) no Guia do usuário do IAM.

Identidade federada

Como prática recomendada, exija que os usuários, inclusive os que precisam de acesso de administrador, usem a federação com um provedor de identidades para acessar Serviços da AWS usando credenciais temporárias.

Identidade federada é um usuário de seu diretório de usuários corporativos, um provedor de identidades da web, o AWS Directory Service, o diretório do Centro de Identidade ou qualquer usuário que acesse os Serviços da AWS usando credenciais fornecidas por meio de uma fonte de identidade. Quando as identidades federadas acessam Contas da AWS, elas assumem funções que fornecem credenciais temporárias.

Para o gerenciamento de acesso centralizado, recomendamos usar o AWS IAM Identity Center. Você pode criar usuários e grupos no Centro de Identidade do IAM ou se conectar e sincronizar com um conjunto de usuários e grupos em sua própria fonte de identidade para uso em todas as suas Contas da AWS e aplicações. Para obter mais informações sobre o Centro de Identidade do IAM, consulte [O que é o Centro de Identidade do IAM?](#) no Guia do usuário do AWS IAM Identity Center.

Grupos e usuários do IAM

Um [usuário do IAM](#) é uma identidade dentro da Conta da AWS que tem permissões específicas para uma única pessoa ou aplicação. Sempre que possível, recomendamos depender de credenciais temporárias em vez de criar usuários do IAM com credenciais de longo prazo, como senhas e chaves de acesso. No entanto, se você tiver casos de uso específicos que exijam credenciais de longo prazo com usuários do IAM, recomendamos alternar as chaves de acesso. Para obter mais informações, consulte [Altere as chaves de acesso regularmente para casos de uso que exijam credenciais de longo prazo](#) no Guia do usuário do IAM.

Um [grupo do IAM](#) é uma identidade que especifica uma coleção de usuários do IAM. Não é possível fazer login como um grupo. É possível usar grupos para especificar permissões para vários usuários de uma vez. Os grupos facilitam o gerenciamento de permissões para grandes conjuntos de usuários. Por exemplo, você pode ter um grupo chamado IAMAdmins e atribuir a esse grupo permissões para administrar recursos do IAM.

Usuários são diferentes de funções. Um usuário é exclusivamente associado a uma pessoa ou a uma aplicação, mas uma função pode ser assumida por qualquer pessoa que precisar dela. Os usuários têm credenciais permanentes de longo prazo, mas as funções fornecem credenciais temporárias. Para saber mais, consulte [Quando criar um usuário do IAM \(em vez de uma função\)](#) no Guia do usuário do IAM.

Funções do IAM

Um [perfil do IAM](#) é uma identidade dentro da Conta da AWS que tem permissões específicas. Ela é semelhante a um usuário do IAM, mas não está associada a uma pessoa específica. É possível assumir temporariamente um perfil do IAM no AWS Management Console [alternando perfis](#). É possível assumir um perfil chamando uma operação de API da AWS CLI ou da AWS, ou usando um URL personalizado. Para obter mais informações sobre métodos para o uso de perfis, consulte [Usar perfis do IAM](#) no Guia do usuário do IAM.

Os perfis do IAM com credenciais temporárias são úteis nas seguintes situações:

- **Acesso de usuário federado:** para atribuir permissões a identidades federadas, você pode criar um perfil e definir permissões para ele. Quando uma identidade federada é autenticada, essa identidade é associada ao perfil e recebe as permissões definidas pelo mesmo. Para obter mais informações sobre perfis para federação, consulte [Criar um perfil para um provedor de identidades de terceiros](#) no Guia do usuário do IAM. Se você usar o IAM Identity Center, configure um conjunto de permissões. Para controlar o que suas identidades podem acessar após a autenticação, o IAM Identity Center correlaciona o conjunto de permissões a um perfil no IAM. Para obter informações sobre conjuntos de permissões, consulte [Conjuntos de permissões](#) no Guia do usuário do AWS IAM Identity Center.
- **Permissões temporárias para usuários do IAM:** um usuário ou um perfil do IAM pode assumir um perfil do IAM para obter temporariamente permissões diferentes para uma tarefa específica.
- **Acesso entre contas:** é possível usar um perfil do IAM para permitir que alguém (uma entidade principal confiável) em outra conta acesse recursos em sua conta. As funções são a principal forma de conceder acesso entre contas. No entanto, alguns Serviços da AWS permitem que você anexe uma política diretamente a um recurso (em vez de usar uma função como proxy). Para saber a diferença entre funções e políticas baseadas em recurso para acesso entre contas, consulte [Como as funções do IAM diferem das políticas baseadas em recurso](#) no Guia do usuário do IAM.
- **Acesso entre serviços:** alguns Serviços da AWS usam recursos em outros Serviços da AWS. Por exemplo, quando você faz uma chamada em um serviço, é comum que esse serviço execute aplicações no Amazon EC2 ou armazene objetos no Amazon S3. Um serviço pode fazer isso usando as permissões do principal de chamada, usando um perfil de serviço ou um perfil vinculado ao serviço.
- **Encaminhamento de sessões de acesso (FAS):** qualquer pessoa que utilizar uma função ou usuário do IAM para realizar ações na AWS é considerada uma entidade principal. Ao usar alguns serviços, você pode executar uma ação que inicia outra ação em um serviço diferente. O recurso FAS utiliza as permissões da entidade principal que chama um AWS service (Serviço da AWS), combinadas às permissões do AWS service (Serviço da AWS) solicitante, para realizar solicitações para serviços downstream. As solicitações de FAS só são feitas quando um serviço recebe uma solicitação que exige interações com outros Serviços da AWS ou com recursos para serem concluídas. Nesse caso, você precisa ter permissões para executar ambas as ações. Para obter detalhes da política ao fazer solicitações de FAS, consulte [Encaminhar sessões de acesso](#).
- **Perfil de serviço:** um perfil de serviço é um [perfil do IAM](#) que um serviço assume para realizar ações em seu nome. Um administrador do IAM pode criar, modificar e excluir um perfil de

serviço do IAM. Para obter mais informações, consulte [Criar um perfil para delegar permissões a um AWS service \(Serviço da AWS\)](#) no Guia do usuário do IAM.

- Perfil vinculado ao serviço: um perfil vinculado ao serviço é um tipo de perfil de serviço vinculado a um AWS service (Serviço da AWS). O serviço pode assumir o perfil para executar uma ação em seu nome. Os perfis vinculados ao serviço aparecem em sua Conta da AWS e são de propriedade do serviço. Um administrador do IAM pode visualizar, mas não pode editar as permissões para perfis vinculados a serviço.
- Aplicações em execução no Amazon EC2: é possível usar um perfil do IAM para gerenciar credenciais temporárias para aplicações em execução em uma instância do EC2 e fazer solicitações da AWS CLI ou da AWS API. É preferível fazer isso e armazenar chaves de acesso na instância do EC2. Para atribuir uma função da AWS a uma instância do EC2 e disponibilizá-la para todas as suas aplicações, crie um perfil de instância que esteja anexado a ela. Um perfil de instância contém a função e permite que os programas em execução na instância do EC2 obtenham credenciais temporárias. Para mais informações, consulte [Usar um perfil do IAM para conceder permissões a aplicações em execução nas instâncias do Amazon EC2](#) no Guia do usuário do IAM.

Para saber se deseja usar os perfis do IAM, consulte [Quando criar um perfil do IAM \(em vez de um usuário\)](#) no Guia do usuário do IAM.

Gerenciamento do acesso usando políticas

Você controla o acesso na AWS criando políticas e anexando-as a identidades ou recursos da AWS. Uma política é um objeto na AWS que, quando associado a uma identidade ou recurso, define suas permissões. A AWS avalia essas políticas quando uma entidade principal (usuário, usuário raiz ou sessão de função) faz uma solicitação. As permissões nas políticas determinam se a solicitação será permitida ou negada. A maioria das políticas são armazenadas na AWS como documentos JSON. Para obter mais informações sobre a estrutura e o conteúdo de documentos de políticas JSON, consulte [Visão geral das políticas JSON](#) no Guia do usuário do IAM.

Os administradores podem usar AWS as políticas JSON para especificar quem tem acesso a quê. Ou seja, qual entidade principal pode executar ações em quais recursos e em que condições.

Por padrão, usuários e funções não têm permissões. Para conceder aos usuários permissão para executar ações nos recursos de que eles precisam, um administrador do IAM pode criar políticas do IAM. O administrador pode então adicionar as políticas do IAM a perfis, e os usuários podem assumir os perfis.

As políticas do IAM definem permissões para uma ação, independentemente do método usado para executar a operação. Por exemplo, suponha que você tenha uma política que permite a ação `iam:GetRole`. Um usuário com essa política pode obter informações de perfis do AWS Management Console, da AWS CLI ou da API da AWS.

Políticas baseadas em identidade

As políticas baseadas em identidade são documentos de políticas de permissões JSON que você pode anexar a uma identidade, como usuário, grupo de usuários ou perfil do IAM. Essas políticas controlam quais ações os usuários e funções podem realizar, em quais recursos e em que condições. Para saber como criar uma política baseada em identidade, consulte [Criar políticas do IAM](#) no Guia do usuário do IAM.

As políticas baseadas em identidade podem ser categorizadas ainda mais como políticas em linha ou políticas gerenciadas. As políticas em linha são anexadas diretamente a um único usuário, grupo ou função. As políticas gerenciadas são políticas independentes que podem ser anexadas a vários usuários, grupos e funções na Conta da AWS. As políticas gerenciadas incluem políticas gerenciadas pela AWS e políticas gerenciadas pelo cliente. Para saber como escolher entre uma política gerenciada ou uma política em linha, consulte [Escolher entre políticas gerenciadas e políticas em linha](#) no Guia do usuário do IAM.

Políticas baseadas em recurso

Políticas baseadas em recurso são documentos de políticas JSON que você anexa a um recurso. São exemplos de políticas baseadas em recursos as políticas de confiança de perfil do IAM e as políticas de bucket do Amazon S3. Em serviços compatíveis com políticas baseadas em recursos, os administradores de serviço podem usá-las para controlar o acesso a um recurso específico. Para o recurso ao qual a política está anexada, a política define quais ações um principal especificado pode executar nesse recurso e em que condições. Você deve [especificar um principal](#) em uma política baseada em recursos. As entidades principais podem incluir contas, usuários, perfis, usuários federados ou Serviços da AWS.

Políticas baseadas em recursos são políticas em linha que estão localizadas nesse serviço. Não é possível usar as políticas gerenciadas da AWS do IAM em uma política baseada em recursos.

Listas de controle de acesso (ACLs)

As listas de controle de acesso (ACLs) controlam quais entidades principais (membros, usuários ou funções da conta) têm permissões para acessar um recurso. As ACLs são semelhantes às políticas baseadas em recursos, embora não usem o formato de documento de política JSON.

Amazon S3, AWS WAF e Amazon VPC são exemplos de serviços que oferecem suporte a ACLs. Para saber mais sobre ACLs, consulte [Visão geral da lista de controle de acesso \(ACL\)](#) no Guia do desenvolvedor do Amazon Simple Storage Service.

Outros tipos de política

A AWS aceita tipos de política menos comuns. Esses tipos de política podem definir o máximo de permissões concedidas a você pelos tipos de política mais comuns.

- **Limites de permissões:** um limite de permissões é um recurso avançado no qual você define o máximo de permissões que uma política baseada em identidade pode conceder a uma entidade do IAM (usuário ou perfil do IAM). É possível definir um limite de permissões para uma entidade. As permissões resultantes são a interseção das políticas baseadas em identidade de uma entidade e dos seus limites de permissões. As políticas baseadas em recurso que especificam o usuário ou a função no campo `Principal` não são limitadas pelo limite de permissões. Uma negação explícita em qualquer uma dessas políticas substitui a permissão. Para obter mais informações sobre limites de permissões, consulte [Limites de permissões para identidades do IAM](#) no Guia do usuário do IAM.
- **Políticas de controle de serviço (SCPs):** SCPs são políticas JSON que especificam as permissões máximas para uma organização ou unidade organizacional (UO) no AWS Organizations. O AWS Organizations é um serviço para agrupar e gerenciar centralmente várias Contas da AWS pertencentes à sua empresa. Se você habilitar todos os recursos em uma organização, poderá aplicar políticas de controle de serviço (SCPs) a qualquer uma ou a todas as contas. O SCP limita as permissões para entidades em contas-membro, incluindo cada Usuário raiz da conta da AWS. Para obter mais informações sobre o Organizations e SCPs, consulte [Como os SCPs funcionam](#) no Guia do usuário do AWS Organizations.
- **Políticas de sessão:** são políticas avançadas que você transmite como um parâmetro quando cria de forma programática uma sessão temporária para uma função ou um usuário federado. As permissões da sessão resultante são a interseção das políticas baseadas em identidade do usuário ou da função e das políticas de sessão. As permissões também podem ser provenientes de uma política baseada em recurso. Uma negação explícita em qualquer uma dessas políticas substitui a permissão. Para obter mais informações, consulte [Políticas de sessão](#) no Guia do usuário do IAM.

Vários tipos de política

Quando vários tipos de política são aplicáveis a uma solicitação, é mais complicado compreender as permissões resultantes. Para saber como a AWS determina se deve permitir uma solicitação quando há vários tipos de política envolvidos, consulte [Lógica da avaliação](#) de políticas no Guia do usuário do IAM.

Como Serviços da AWS funcionam com o IAM

Para obter uma visão geral de como Serviços da AWS funcionam com a maioria dos atributos do IAM, consulte [Serviços da AWS compatíveis com o IAM](#) no Guia do usuário do IAM.

Para saber como usar um AWS service (Serviço da AWS) específico com o IAM, consulte a seção de segurança do Guia do usuário do serviço relevante.

Solução de problemas de identidade e acesso do AWS

Use as seguintes informações para ajudar a diagnosticar e corrigir problemas comuns que podem ser encontrados ao trabalhar com o e o IAM.AWS

Tópicos

- [Não tenho autorização para executar uma ação no AWS](#)
- [Não estou autorizado a executar iam:PassRole](#)
- [Quero permitir que pessoas fora da minha Conta da AWS acessem meus recursos AWS](#)

Não tenho autorização para executar uma ação no AWS

Se você receber uma mensagem de erro informando que não tem autorização para executar uma ação, suas políticas deverão ser atualizadas para permitir que você realize a ação.

O erro do exemplo a seguir ocorre quando o usuário do IAM mateojackson tenta usar o console para visualizar detalhes sobre um atributo *my-example-widget* fictício, mas não tem as permissões *awes:GetWidget* fictícias.

```
User: arn:aws:iam::123456789012:user/mateojackson is not authorized to perform:
awes:GetWidget on resource: my-example-widget
```

Nesse caso, a política do usuário mateojackson deve ser atualizada para permitir o acesso ao atributo *my-example-widget* usando a ação *awes:GetWidget*.

Se você precisar de ajuda, entre em contato com seu administrador da AWS. Seu administrador é a pessoa que forneceu suas credenciais de login.

Não estou autorizado a executar `iam:PassRole`

Se você receber uma mensagem de erro informando que não está autorizado a executar a ação `iam:PassRole`, as suas políticas devem ser atualizadas para permitir que você passe uma função para o AWS.

Alguns Serviços da AWS permitem que você passe uma função existente para o serviço, em vez de criar uma nova função de serviço ou função vinculada ao serviço. Para fazê-lo, você deve ter permissões para passar o perfil para o serviço.

O exemplo de erro a seguir ocorre quando uma usuária do IAM chamada `marymajor` tenta utilizar o console para executar uma ação no AWS. No entanto, a ação exige que o serviço tenha permissões concedidas por um perfil de serviço. Mary não tem permissões para passar a função para o serviço.

```
User: arn:aws:iam::123456789012:user/marymajor is not authorized to perform:
iam:PassRole
```

Nesse caso, as políticas de Mary devem ser atualizadas para permitir que ela realize a ação `iam:PassRole`.

Se você precisar de ajuda, entre em contato com seu administrador da AWS. Seu administrador é a pessoa que forneceu suas credenciais de login.

Quero permitir que pessoas fora da minha Conta da AWS acessem meus recursos AWS

Você pode criar uma função que os usuários de outras contas ou pessoas fora da organização podem usar para acessar seus recursos. Você pode especificar quem é confiável para assumir a função. Para serviços que oferecem suporte a políticas baseadas em recursos ou listas de controle de acesso (ACLs), você pode usar essas políticas para conceder às pessoas acesso aos seus recursos.

Saiba mais consultando o seguinte:

- Para saber se o AWS suporta esses recursos, consulte [Como Serviços da AWS funcionam com o IAM](#).

- Saiba como conceder acesso a seus recursos em todos os Contas da AWS pertencentes a você, consulte [Fornecendo Acesso a um Usuário do IAM em Outra Conta da AWS Pertencente a Você](#) no Guia de Usuário do IAM.
- Para saber como conceder acesso a seus recursos para terceiros Contas da AWS, consulte [Fornecimento de acesso a Contas da AWS pertencentes a terceiros](#) no Guia do usuário do IAM.
- Para saber como conceder acesso por meio da federação de identidades, consulte [Conceder acesso a usuários autenticados externamente \(federação de identidades\)](#) no Guia do usuário do IAM.
- Para saber a diferença entre usar perfis e políticas baseadas em recursos para acesso entre contas, consulte [Como os perfis do IAM diferem de políticas baseadas em recursos](#) no Guia do usuário do IAM.

Validação de conformidade para este produto ou serviço da AWS

Para saber se um AWS service (Serviço da AWS) está no escopo de programas de conformidade específicos, consulte [Serviços da AWS no escopo por programa de conformidade](#) em que você está interessado. Para obter informações gerais, consulte [Programas de conformidade da AWS](#).

É possível fazer download de relatórios de auditoria de terceiros usando o AWS Artifact. Para obter mais informações, consulte [Downloading Reports in AWS Artifact](#).

Sua responsabilidade de conformidade ao usar o Serviços da AWS é determinada pela confidencialidade dos seus dados, pelos objetivos de conformidade da sua empresa e pelos regulamentos e leis aplicáveis. A AWS fornece os seguintes recursos para ajudar com a conformidade:

- [Guias de referência rápida de conformidade e segurança](#) - estes guias de implantação discutem considerações sobre arquitetura e fornecem as etapas para a implantação de ambientes de linha de base focados em segurança e conformidade na AWS.
- [Arquitetura para segurança e conformidade com HIPAA no Amazon Web Services](#): esse whitepaper descreve como as empresas podem usar a AWS para criar aplicações adequadas aos padrões HIPAA.

 Note

Nem todos os Serviços da AWS estão qualificados pela HIPAA. Para mais informações, consulte a [Referência dos serviços qualificados pela HIPAA](#).

- [Recursos de conformidade da AWS](#): essa coleção de manuais e guias pode ser aplicada a seu setor e local.
- [Guias de conformidade do cliente da AWS](#): entenda o modelo de responsabilidade compartilhada sob a ótica da conformidade. Os guias resumem as práticas recomendadas para proteção de Serviços da AWS e mapeiam as diretrizes para controles de segurança em várias estruturas (incluindo o Instituto Nacional de Padrões e Tecnologia (NIST), o Conselho de Padrões de Segurança do Setor de Cartões de Pagamento (PCI) e a Organização Internacional de Padronização (ISO)).
- [Avaliar recursos com regras](#) no Guia do desenvolvedor do AWS Config: o serviço AWS Config avalia como as configurações de recursos estão em conformidade com práticas internas, diretrizes do setor e regulamentos.
- [AWS Security Hub](#): este AWS service (Serviço da AWS) fornece uma visão abrangente do seu estado de segurança na AWS. O Security Hub usa controles de segurança para avaliar os recursos da AWS e verificar a conformidade com os padrões e as práticas recomendadas do setor de segurança. Para obter uma lista dos serviços e controles aceitos, consulte a [Referência de controles do Security Hub](#).
- [AWS Audit Manager](#): esse AWS service (Serviço da AWS) ajuda a auditar continuamente seu uso da AWS para simplificar a forma como você gerencia os riscos e a conformidade com regulamentos e padrões do setor.

Esse produto ou serviço da AWS segue o [modelo de responsabilidade compartilhada](#) por meio dos serviços específicos da Amazon Web Services (AWS) compatíveis. Para obter informações de segurança sobre o serviço da AWS, consulte a [página de documentação de segurança do serviço da AWS](#) e [Serviços da AWS que estão no escopo dos esforços de conformidade da AWS por programa de conformidade](#).

Resiliência para este produto ou serviço da AWS

A infraestrutura global da AWS é criada com base em Regiões da AWS e zonas de disponibilidade.

As Regiões da AWS fornecem várias zonas de disponibilidade separadas e isoladas fisicamente, que são conectadas com baixa latência, throughputs elevadas e redes altamente redundantes.

Com as zonas de disponibilidade, é possível projetar e operar aplicações e bancos de dados que automaticamente executam o failover entre as zonas sem interrupção. As zonas de disponibilidade são mais altamente disponíveis, tolerantes a falhas e escaláveis que uma ou várias infraestruturas de datacenter tradicionais.

Para obter mais informações sobre AWS regiões e zonas de disponibilidade, consulte [AWS Infraestrutura Global](#).

Esse produto ou serviço da AWS segue o [modelo de responsabilidade compartilhada](#) por meio dos serviços específicos da Amazon Web Services (AWS) compatíveis. Para obter informações de segurança sobre o serviço da AWS, consulte a [página de documentação de segurança do serviço da AWS](#) e [Serviços da AWS que estão no escopo dos esforços de conformidade da AWS por programa de conformidade](#).

Segurança da infraestrutura para esse produto ou serviço da AWS

Esse produto ou serviço da AWS usa serviços gerenciados e, portanto, é protegido pela segurança de rede global da AWS. Para obter informações sobre serviços de segurança da AWS e como a AWS protege a infraestrutura, consulte [Segurança na nuvem da AWS](#). Para projetar o ambiente da AWS utilizando as práticas recomendadas de segurança de infraestrutura, consulte [Proteção de infraestrutura](#) em Pilar segurança: AWS Well-Architected Framework.

Você usa chamadas de API publicadas pela AWS para acessar esse produto ou serviço da AWS pela rede. Os clientes precisam oferecer suporte para:

- Transport Layer Security (TLS). Exigimos TLS 1.2 e recomendamos TLS 1.3.
- Conjuntos de criptografia com sigilo de encaminhamento perfeito (perfect forward secrecy, ou PFS) como DHE (Ephemeral Diffie-Hellman, ou Efêmero Diffie-Hellman) ou ECDHE (Ephemeral Elliptic Curve Diffie-Hellman, ou Curva elíptica efêmera Diffie-Hellman). A maioria dos sistemas modernos, como Java 7 e versões posteriores, comporta esses modos.

Além disso, as solicitações devem ser assinadas utilizando um ID da chave de acesso e uma chave de acesso secreta associada a uma entidade principal do IAM. Ou é possível usar o [AWS Security Token Service](#) (AWS STS) para gerar credenciais de segurança temporárias para assinar solicitações.

Esse produto ou serviço da AWS segue o [modelo de responsabilidade compartilhada](#) por meio dos serviços específicos da Amazon Web Services (AWS) compatíveis. Para obter informações de segurança sobre o serviço da AWS, consulte a [página de documentação de segurança do serviço da AWS](#) e [Serviços da AWS que estão no escopo dos esforços de conformidade da AWS por programa de conformidade](#).

Impor uma versão mínima do TLS

Para aumentar a segurança ao se comunicar com serviços da AWS, você deve usar o TLS 1.2 ou posterior. Quando você usa a AWS CLI, o Python é usado para definir a versão do TLS.

A AWS CLI versão 2 usa um script Python interno que é compilado para usar no mínimo o TLS 1.2 quando compatível com o serviço com o qual ele está conversando. Contanto que você use a AWS CLI versão 2, não serão necessárias outras etapas para aplicar esse mínimo.

Solucionar erros da AWS CLI

Esta seção aborda erros comuns e os passos para a solução de problemas a serem seguidas para resolver o problema. Sugerimos seguir a [Solução geral de problemas](#) primeiro.

Sumário

- [Solução geral de problemas para tentar primeiro](#)
 - [Verificar a formatação do comando AWS CLI](#)
 - [Confirme se você está executando uma versão recente da AWS CLI](#)
 - [Como usar a opção --debug](#)
 - [Habilitar e analisar logs de histórico de comandos da AWS CLI](#)
 - [Confirmar se a AWS CLI está configurada](#)
- [Erros de comando não encontrado](#)
- [O comando “aws --version” retorna uma versão diferente da que você instalou](#)
- [O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI](#)
- [A AWS CLI processou um comando com um nome de parâmetro incompleto](#)
- [Erros de acesso negado](#)
- [Credenciais inválidas e erros de chave](#)
- [Assinatura não corresponde aos erros](#)
- [Erros de certificado SSL](#)
- [Erros JSON inválidos](#)
- [Recursos adicionais](#)

Solução geral de problemas para tentar primeiro

Se você receber um erro ou encontrar um problema com a AWS CLI, sugerimos as dicas gerais a seguir para ajudar na solução de problemas.

[Voltar ao início](#)

Verificar a formatação do comando AWS CLI

Se você receber um erro indicando que um comando não existe ou que ele não reconhece um parâmetro que a documentação indica estar disponível, é provável que o comando esteja formatado incorretamente. Sugerimos verificar o seguinte:

- Verifique se há erros de ortografia e de formatação no comando.
- Confirme se todos os [escapes e citações apropriados para o seu terminal](#) estão corretos em seu comando.
- Gere um [esqueleto de AWS CLI](#) para confirmar a estrutura do comando.
- Para JSON, consulte a [solução adicional de problemas referente a valores JSON](#). Se você estiver tendo problemas com a formatação JSON de processamento de terminal, sugerimos ignorar as regras de cotação do terminal usando [Blobs para passar dados JSON diretamente para a AWS CLI](#).

Para obter mais informações sobre como um comando específico deve ser estruturado, consulte o [AWS CLIGuia de referência da versão 2](#).

[Voltar ao início](#)

Confirme se você está executando uma versão recente da AWS CLI

Se você receber um erro indicando que um comando não existe ou que ele não reconhece um parâmetro que o [AWS CLIGuia de referência da versão 2](#) diz que está disponível, primeiro confirme se o comando está formatado corretamente. Se a formatação estiver correta, recomendamos atualizar para a versão mais recente da AWS CLI. Versões atualizadas da AWS CLI são lançadas quase todos os dias úteis. Novos serviços da AWS, recursos e parâmetros são apresentados nessas novas versões da AWS CLI. A única maneira de obter acesso a esses novos serviços, recursos ou parâmetros é atualizar para uma versão que foi lançada depois que esse elemento foi apresentado pela primeira vez.

A maneira como você atualiza sua versão da AWS CLI depende de como ela foi instalada originalmente, conforme descrito em [the section called “Instalar/atualizar”](#).

Se você tiver usado um dos instaladores empacotados, talvez precise remover a instalação existente antes de baixar e instalar a versão mais recente para seu sistema operacional.

[Voltar ao início](#)

Como usar a opção **--debug**

Quando a AWS CLI informar um erro que você não compreende imediatamente ou produzir resultados inesperados, você poderá obter mais detalhes sobre o erro executando o comando novamente com a opção **--debug**. Com esta opção, a AWS CLI gera detalhes sobre cada passo necessário para processar seu comando. Os detalhes na saída podem ajudar a determinar quando o erro ocorreu e fornecem dicas sobre o que o acionou o erro.

É possível enviar a saída para um arquivo de texto para análise posterior ou enviá-lo para AWS Support quando solicitado.

Quando você inclui a opção **--debug**, os detalhes incluem:

- Procurar credenciais
- Analisar os parâmetros fornecidos
- Criar a solicitação enviada aos servidores da AWS
- O conteúdo da solicitação enviada para a AWS
- O conteúdo da resposta não formatada
- A saída formatada

Veja a seguir um exemplo de um comando executado com e sem a opção **--debug**.

```
$ aws iam list-groups --profile MyTestProfile
{
  "Groups": [
    {
      "Path": "/",
      "GroupName": "MyTestGroup",
      "GroupId": "AGPA0123456789EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:group/MyTestGroup",
      "CreateDate": "2019-08-12T19:34:04Z"
    }
  ]
}
```

```
$ aws iam list-groups --profile MyTestProfile --debug
2019-08-12 12:36:18,305 - MainThread - awscli.clidriver - DEBUG - CLI version: aws-
cli/1.16.215 Python/3.7.3 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.12.205
```

```
2019-08-12 12:36:18,305 - MainThread - awscli.clidriver - DEBUG - Arguments entered to
CLI: ['iam', 'list-groups', '--debug']
2019-08-12 12:36:18,305 - MainThread - botocore.hooks - DEBUG - Event session-
initialized: calling handler <function add_scalar_parsers at 0x7fdf173161e0>
2019-08-12 12:36:18,305 - MainThread - botocore.hooks - DEBUG - Event session-
initialized: calling handler <function register_uri_param_handler at 0x7fdf17dec400>
2019-08-12 12:36:18,305 - MainThread - botocore.hooks - DEBUG - Event session-
initialized: calling handler <function inject_assume_role_provider_cache at
0x7fdf17da9378>
2019-08-12 12:36:18,307 - MainThread - botocore.credentials - DEBUG - Skipping
environment variable credential check because profile name was explicitly set.
2019-08-12 12:36:18,307 - MainThread - botocore.hooks - DEBUG - Event session-
initialized: calling handler <function attach_history_handler at 0x7fdf173ed9d8>
2019-08-12 12:36:18,308 - MainThread - botocore.loaders - DEBUG - Loading JSON
file: /home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/iam/2010-05-08/
service-2.json
2019-08-12 12:36:18,317 - MainThread - botocore.hooks - DEBUG - Event building-command-
table.iam: calling handler <function add_waiters at 0x7fdf1731a840>
2019-08-12 12:36:18,320 - MainThread - botocore.loaders - DEBUG - Loading JSON
file: /home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/iam/2010-05-08/
waiters-2.json
2019-08-12 12:36:18,321 - MainThread - awscli.clidriver - DEBUG - OrderedDict([('path-
prefix', <awscli.arguments.CLIArgument object at 0x7fdf171ac780>), ('marker',
<awscli.arguments.CLIArgument object at 0x7fdf171b09e8>), ('max-items',
<awscli.arguments.CLIArgument object at 0x7fdf171b09b0>)])
2019-08-12 12:36:18,322 - MainThread - botocore.hooks - DEBUG - Event building-
argument-table.iam.list-groups: calling handler <function add_streaming_output_arg at
0x7fdf17316510>
2019-08-12 12:36:18,322 - MainThread - botocore.hooks - DEBUG - Event building-
argument-table.iam.list-groups: calling handler <function add_cli_input_json at
0x7fdf17da9d90>
2019-08-12 12:36:18,322 - MainThread - botocore.hooks - DEBUG - Event building-
argument-table.iam.list-groups: calling handler <function unify_paging_params at
0x7fdf17328048>
2019-08-12 12:36:18,326 - MainThread - botocore.loaders - DEBUG - Loading JSON
file: /home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/iam/2010-05-08/
paginators-1.json
2019-08-12 12:36:18,326 - MainThread - awscli.customizations.paginate - DEBUG -
Modifying paging parameters for operation: ListGroups
2019-08-12 12:36:18,326 - MainThread - botocore.hooks - DEBUG - Event building-
argument-table.iam.list-groups: calling handler <function add_generate_skeleton at
0x7fdf1737eae8>
2019-08-12 12:36:18,326 - MainThread - botocore.hooks - DEBUG - Event
before-building-argument-table-parser.iam.list-groups: calling handler
```

```

<bound method OverrideRequiredArgsArgument.override_required_args of
<awscli.customizations.cliinputjson.CliInputJSONArgument object at 0x7fdf171b0a58>>
2019-08-12 12:36:18,327 - MainThread - botocore.hooks - DEBUG - Event
before-building-argument-table-parser.iam.list-groups: calling handler
<bound method GenerateCliSkeletonArgument.override_required_args of
<awscli.customizations.generatecliskeleton.GenerateCliSkeletonArgument object at
0x7fdf171c5978>>
2019-08-12 12:36:18,327 - MainThread - botocore.hooks - DEBUG - Event operation-
args-parsed.iam.list-groups: calling handler functools.partial(<function
check_should_enable_pagination at 0x7fdf17328158>, ['marker', 'max-items'], {'max-
items': <awscli.arguments.CLIArgument object at 0x7fdf171b09b0>}, OrderedDict([('path-
prefix', <awscli.arguments.CLIArgument object at 0x7fdf171ac780>), ('marker',
<awscli.arguments.CLIArgument object at 0x7fdf171b09e8>), ('max-items',
<awscli.customizations.paginate.PageArgument object at 0x7fdf171c58d0>), ('cli-
input-json', <awscli.customizations.cliinputjson.CliInputJSONArgument object at
0x7fdf171b0a58>), ('starting-token', <awscli.customizations.paginate.PageArgument
object at 0x7fdf171b0a20>), ('page-size', <awscli.customizations.paginate.PageArgument
object at 0x7fdf171c5828>), ('generate-cli-skeleton',
<awscli.customizations.generatecliskeleton.GenerateCliSkeletonArgument object at
0x7fdf171c5978>)]))
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event load-cli-
arg.iam.list-groups.path-prefix: calling handler <awscli.paramfile.URIArgumentHandler
object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event load-cli-
arg.iam.list-groups.marker: calling handler <awscli.paramfile.URIArgumentHandler object
at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event load-cli-
arg.iam.list-groups.max-items: calling handler <awscli.paramfile.URIArgumentHandler
object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG -
Event load-cli-arg.iam.list-groups.cli-input-json: calling handler
<awscli.paramfile.URIArgumentHandler object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG -
Event load-cli-arg.iam.list-groups.starting-token: calling handler
<awscli.paramfile.URIArgumentHandler object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event load-cli-
arg.iam.list-groups.page-size: calling handler <awscli.paramfile.URIArgumentHandler
object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event
load-cli-arg.iam.list-groups.generate-cli-skeleton: calling handler
<awscli.paramfile.URIArgumentHandler object at 0x7fdf1725c978>
2019-08-12 12:36:18,329 - MainThread - botocore.hooks - DEBUG
- Event calling-command.iam.list-groups: calling handler

```

```

<bound method CliInputJSONArgument.add_to_call_parameters of
<awscli.customizations.cliinputjson.CliInputJSONArgument object at 0x7fdf171b0a58>>
2019-08-12 12:36:18,329 - MainThread - botocore.hooks - DEBUG -
Event calling-command.iam.list-groups: calling handler <bound
method GenerateCliSkeletonArgument.generate_json_skeleton of
<awscli.customizations.generatecliskeleton.GenerateCliSkeletonArgument object at
0x7fdf171c5978>>
2019-08-12 12:36:18,329 - MainThread - botocore.credentials - DEBUG - Looking for
credentials via: assume-role
2019-08-12 12:36:18,329 - MainThread - botocore.credentials - DEBUG - Looking for
credentials via: assume-role-with-web-identity
2019-08-12 12:36:18,329 - MainThread - botocore.credentials - DEBUG - Looking for
credentials via: shared-credentials-file
2019-08-12 12:36:18,329 - MainThread - botocore.credentials - INFO - Found credentials
in shared credentials file: ~/.aws/credentials
2019-08-12 12:36:18,330 - MainThread - botocore.loaders - DEBUG - Loading JSON file: /
home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/endpoints.json
2019-08-12 12:36:18,334 - MainThread - botocore.hooks - DEBUG - Event choose-service-
name: calling handler <function handle_service_name_alias at 0x7fdf1898eb70>
2019-08-12 12:36:18,337 - MainThread - botocore.hooks - DEBUG - Event creating-client-
class.iam: calling handler <function add_generate_presigned_url at 0x7fdf18a028c8>
2019-08-12 12:36:18,337 - MainThread - botocore.regions - DEBUG - Using partition
endpoint for iam, us-west-2: aws-global
2019-08-12 12:36:18,337 - MainThread - botocore.args - DEBUG - The s3 config key is not
a dictionary type, ignoring its value of: None
2019-08-12 12:36:18,340 - MainThread - botocore.endpoint - DEBUG - Setting iam timeout
as (60, 60)
2019-08-12 12:36:18,341 - MainThread - botocore.loaders - DEBUG - Loading JSON file: /
home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/_retry.json
2019-08-12 12:36:18,341 - MainThread - botocore.client - DEBUG - Registering retry
handlers for service: iam
2019-08-12 12:36:18,342 - MainThread - botocore.hooks - DEBUG - Event before-
parameter-build.iam.ListGroups: calling handler <function generate_idempotent_uuid at
0x7fdf189b10d0>
2019-08-12 12:36:18,342 - MainThread - botocore.hooks - DEBUG - Event before-
call.iam.ListGroups: calling handler <function inject_api_version_header_if_needed at
0x7fdf189b2a60>
2019-08-12 12:36:18,343 - MainThread - botocore.endpoint - DEBUG - Making
request for OperationModel(name=ListGroups) with params: {'url_path': '/',
'query_string': '', 'method': 'POST', 'headers': {'Content-Type': 'application/x-
www-form-urlencoded; charset=utf-8', 'User-Agent': 'aws-cli/1.16.215 Python/3.7.3
Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.12.205'}, 'body': {'Action':
'ListGroups', 'Version': '2010-05-08'}, 'url': 'https://iam.amazonaws.com/',

```

```

'context': {'client_region': 'aws-global', 'client_config': <botocore.config.Config
object at 0x7fdf16e9a4a8>, 'has_streaming_input': False, 'auth_type': None}}
2019-08-12 12:36:18,343 - MainThread - botocore.hooks - DEBUG - Event request-
created.iam.ListGroups: calling handler <bound method RequestSigner.handler of
<botocore.signers.RequestSigner object at 0x7fdf16e9a470>>
2019-08-12 12:36:18,343 - MainThread - botocore.hooks - DEBUG - Event choose-
signer.iam.ListGroups: calling handler <function set_operation_specific_signer at
0x7fdf18996f28>
2019-08-12 12:36:18,343 - MainThread - botocore.auth - DEBUG - Calculating signature
using v4 auth.
2019-08-12 12:36:18,343 - MainThread - botocore.auth - DEBUG - CanonicalRequest:
POST
/

content-type:application/x-www-form-urlencoded; charset=utf-8
host:iam.amazonaws.com
x-amz-date:20190812T193618Z

content-type;host;x-amz-date
5f776d91EXAMPLE9b8cb5eb5d6d4a787a33ae41c8cd6eEXAMPLEEca69080e1e1f
2019-08-12 12:36:18,344 - MainThread - botocore.auth - DEBUG - StringToSign:
AWS4-HMAC-SHA256
20190812T193618Z
20190812/us-east-1/iam/aws4_request
ab7e367eEXAMPLE2769f178ea509978cf8bfa054874b3EXAMPLE8d043fab6cc9
2019-08-12 12:36:18,344 - MainThread - botocore.auth - DEBUG - Signature:
d85a0EXAMPLEb40164f2f539cdc76d4f294fe822EXAMPLE18ad1ddf58a1a3ce7
2019-08-12 12:36:18,344 - MainThread - botocore.endpoint - DEBUG - Sending
http request: <AWSPreparedRequest stream_output=False, method=POST,
url=https://iam.amazonaws.com/, headers={'Content-Type': b'application/
x-www-form-urlencoded; charset=utf-8', 'User-Agent': b'aws-cli/1.16.215
Python/3.7.3 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.12.205',
'X-Amz-Date': b'20190812T193618Z', 'Authorization': b'AWS4-HMAC-SHA256
Credential=AKIA01234567890EXAMPLE-east-1/iam/aws4_request, SignedHeaders=content-
type;host;x-amz-date, Signature=d85a07692aceb401EXAMPLEa1b18ad1ddf58a1a3ce7EXAMPLE',
'Content-Length': '36'}>
2019-08-12 12:36:18,344 - MainThread - urllib3.util.retry - DEBUG - Converted retries
value: False -> Retry(total=False, connect=None, read=None, redirect=0, status=None)
2019-08-12 12:36:18,344 - MainThread - urllib3.connectionpool - DEBUG - Starting new
HTTPS connection (1): iam.amazonaws.com:443
2019-08-12 12:36:18,664 - MainThread - urllib3.connectionpool - DEBUG - https://
iam.amazonaws.com:443 "POST / HTTP/1.1" 200 570

```

```

2019-08-12 12:36:18,664 - MainThread - botocore.parsers - DEBUG - Response headers:
{'x-amzn-RequestId': '74c11606-bd38-11e9-9c82-559da0adb349', 'Content-Type': 'text/
xml', 'Content-Length': '570', 'Date': 'Mon, 12 Aug 2019 19:36:18 GMT'}
2019-08-12 12:36:18,664 - MainThread - botocore.parsers - DEBUG - Response body:
b'<ListGroupResponse xmlns="https://iam.amazonaws.com/doc/2010-05-08/">\n
  <ListGroupResult>\n    <IsTruncated>false</IsTruncated>\n    <Groups>\n
    <member>\n      <Path>/</Path>\n      <GroupName>MyTestGroup</GroupName>
\n      <Arn>arn:aws:iam::123456789012:group/MyTestGroup</Arn>\n
    <GroupId>AGPA1234567890EXAMPLE</GroupId>\n      <CreateDate>2019-08-12T19:34:04Z</
CreateDate>\n    </member>\n    </Groups>\n  </ListGroupResult>\n
  <ResponseMetadata>\n    <RequestId>74c11606-bd38-11e9-9c82-559da0adb349</RequestId>\n
  </ResponseMetadata>\n</ListGroupResponse>\n'
2019-08-12 12:36:18,665 - MainThread - botocore.hooks - DEBUG - Event needs-
retry.iam.ListGroups: calling handler <botocore.retryhandler.RetryHandler object at
0x7fdf16e9a780>
2019-08-12 12:36:18,665 - MainThread - botocore.retryhandler - DEBUG - No retry needed.
2019-08-12 12:36:18,665 - MainThread - botocore.hooks - DEBUG - Event after-
call.iam.ListGroups: calling handler <function json_decode_policies at 0x7fdf189b1d90>
{
  "Groups": [
    {
      "Path": "/",
      "GroupName": "MyTestGroup",
      "GroupId": "AGPA123456789012EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:group/MyTestGroup",
      "CreateDate": "2019-08-12T19:34:04Z"
    }
  ]
}

```

[Voltar ao início](#)

Habilitar e analisar logs de histórico de comandos da AWS CLI

É possível habilitar os logs do histórico de comandos da AWS CLI usando a configuração de arquivo [cli_history](#). Após habilitar essa configuração, a AWS CLI registra o histórico de comandos da aws.

Você pode listar seu histórico usando o comando `aws history list` e usar os `command_ids` resultantes no comando `aws history show` para obter detalhes. Para obter mais informações, consulte [aws history](#) no guia de referência da AWS CLI.

Quando você inclui a opção `--debug`, os detalhes incluem:

- Chamadas de API feitas para o botocore
- Códigos de status
- Respostas HTTP
- Cabeçalhos
- Códigos de retorno

Você pode usar essas informações para confirmar que os dados do parâmetro e as chamadas de API estão se comportando da maneira esperada e deduzir em que etapa do processo seu comando está falhando.

[Voltar ao início](#)

Confirmar se a AWS CLI está configurada

Vários erros podem ocorrer se os arquivos `config` e `credentials` ou o perfil ou usuário do IAM não estiverem configurados corretamente. Para obter mais informações sobre como resolver erros nos arquivos `config` e `credentials` ou no seu usuário ou perfis do IAM, consulte [the section called “Erros de acesso negado”](#) e [the section called “Credenciais inválidas e erros de chave”](#).

[Voltar ao início](#)

Erros de comando não encontrado

Esse erro significa que o sistema operacional não pode encontrar o comando da AWS CLI. A instalação pode estar incompleta ou exigir atualização.

Causa possível: você está tentando usar um recurso mais recente da AWS CLI do que a versão instalada ou tem formatação incorreta

Exemplo de texto de erro:

```
$ aws s3 copy
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:

aws help
aws <command> help
```

```
aws <command> <subcommand> help
aws: error: argument subcommand: Invalid choice, valid choices are:
ls                               | website
cp                               | mv
....
```

Vários erros podem ocorrer se o comando estiver formatado incorretamente ou se você estiver usando uma versão anterior ao lançamento do recurso. Para obter mais informações sobre a solução de erros em torno desses dois problemas, consulte [the section called “Verificar a formatação do comando AWS CLI”](#) e [the section called “Confirme se você está executando uma versão recente da AWS CLI”](#).

[Voltar ao início](#)

Causa possível: o terminal precisa ser reiniciado após a instalação

Exemplo de texto de erro:

```
$ aws --version
command not found: aws
```

Se o comando `aws` não puder ser encontrado após a primeira instalação ou atualização da AWS CLI, talvez seja necessário reiniciar o terminal para que ele reconheça todas as atualizações da PATH.

[Voltar ao início](#)

Possível causa: AWS CLI não foi totalmente instalada

Exemplo de texto de erro:

```
$ aws --version
command not found: aws
```

Se o comando `aws` não puder ser encontrado após a primeira instalação ou atualização da AWS CLI, talvez a instalação não tenha sido totalmente concluída. Tente reinstalar seguindo os passos da sua plataforma em [the section called “Instalar/atualizar”](#).

[Voltar ao início](#)

Possível causa: a AWS CLI não tem permissões (Linux)

Se não for possível encontrar o comando `aws` após a primeira instalação ou atualização da AWS CLI no Linux, talvez ela não tenha permissões execute na pasta em que está instalada. Execute o seguinte comando com o `PATH` para a instalação da AWS CLI para fornecer permissões [chmod](#) para a AWS CLI:

```
$ sudo chmod -R 755 /usr/local/aws-cli/
```

[Voltar ao início](#)

Causa possível: o **PATH** do sistema operacional não foi atualizado durante a instalação.

Exemplo de texto de erro:

```
$ aws --version
command not found: aws
```

Talvez seja necessário adicionar o executável `aws` à variável de ambiente `PATH` do sistema operacional. Para adicionar a AWS CLI ao seu `PATH`, use as instruções a seguir para seu sistema operacional.

Linux and macOS

1. Encontre o script de perfil do shell no diretório de usuário. Se não tiver certeza de qual shell você tem, execute `echo $SHELL`.

```
$ ls -a ~
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash – `.bash_profile`, `.profile`, ou `.bash_login`
- Zsh – `.zshrc`
- Tcsh – `.tcshrc`, `.cshrc`, ou `.login`

2. Adicione um comando de exportação ao script de perfil. O comando a seguir adiciona seu compartimento local à variável `PATH` atual.

```
export PATH=/usr/local/bin:$PATH
```

3. Recarregue o perfil atualizado em sua sessão atual.

```
$ source ~/.bash_profile
```

Windows

1. Em um prompt de comando do Windows, use o comando `where` com o parâmetro `/R` *path* para encontrar o local do arquivo `aws`. Os resultados retornam todas as pastas que contêm `aws`.

```
C:\> where /R c:\ aws
c:\Program Files\Amazon\AWSCLIV2\aws.exe
...
```

Por padrão, a AWS CLI versão 2 está localizada em:

```
c:\Program Files\Amazon\AWSCLIV2\aws.exe
```

2. Pressione a tecla Windows e digite **environment variables**.
3. Na lista de sugestões, escolha Edit environment variables for your account (Editar variáveis de ambiente para sua conta).
4. Selecione PATH e, em seguida, Edit (Editar).
5. Adicione o caminho encontrado no campo Variable value (Valor da variável). Por exemplo, **C:\Program Files\Amazon\AWSCLIV2\aws.exe**.
6. Escolha OK duas vezes para aplicar as novas configurações.
7. Feche todos os prompts de comando em execução e abra novamente a janela do prompt de comando.

[Voltar ao início](#)

O comando “**aws --version**” retorna uma versão diferente da que você instalou

Seu terminal pode estar retornando um PATH diferente do AWS CLI que você espera.

Causa possível: o terminal precisa ser reiniciado após a instalação

Se o comando `aws` mostrar a versão errada, talvez seja necessário reiniciar o terminal para que ele reconheça todas as atualizações de PATH. Todos os terminais abertos precisam estar fechados, não apenas o terminal ativo.

[Voltar ao início](#)

Causa possível: o sistema precisa ser reiniciado após a instalação

Se o comando `aws` mostrar a versão errada e reiniciar o terminal não funcionar, talvez seja necessário reiniciar o sistema para que ele reconheça as atualizações de PATH.

[Voltar ao início](#)

Causa possível: você tem várias versões da AWS CLI

Se você atualizou a AWS CLI e usou um método de instalação diferente da instalação pré-existente, isso poderá fazer com que várias versões sejam instaladas. Por exemplo, se no Linux ou no macOS você usou `pip` para a instalação atual, mas tentou atualizar usando o arquivo de instalação `.pkg`, isso poderá causar alguns conflitos, especialmente com o PATH apontando para a versão antiga.

Para resolver isso, [desinstale todas as versões da AWS CLI](#) e execute uma instalação limpa.

Depois de desinstalar todas as versões, siga as instruções apropriadas do seu sistema operacional para instalar a versão desejada da [AWS CLI versão 1](#) ou da [AWS CLI versão 2](#).

 Note

Se isso estiver acontecendo depois que você instalou a AWS CLI versão 2 com uma instalação pré-existente da AWS CLI versão 1, siga as instruções de migração em [the section called “Instruções para a migração”](#).

[Voltar ao início](#)

O comando “`aws --version`” retorna uma versão após a desinstalação da AWS CLI

Isso geralmente ocorre quando ainda há uma AWS CLI instalada em algum lugar do sistema.

Causa possível: o terminal precisa ser reiniciado após a instalação

Se o comando `aws --version` mostrar a versão errada, talvez seja necessário reiniciar o terminal para que ele reconheça todas as atualizações.

[Voltar ao início](#)

Causa possível: você tem várias versões da AWS CLI em seu sistema, ou não usou o mesmo método de desinstalação usado para instalar a AWS CLI original.

A AWS CLI talvez não seja desinstalada corretamente se você desinstalou a AWS CLI usando um método diferente do usado para instalá-la ou se você instalou várias versões. Por exemplo, se você usou `pip` para a instalação atual, deverá usar `pip` para desinstalá-la. Para resolver isso, desinstale a AWS CLI usando o mesmo método usado para instalá-la.

1. Siga as instruções apropriadas do seu sistema operacional e do método de instalação original para desinstalar a [AWS CLI versão 1](#) e a [AWS CLI versão 2](#).
2. Feche todos os terminais que você abriu.
3. Abra seu terminal preferido, insira o seguinte comando e confirme que nenhuma versão é retornada.

```
$ aws --version
command not found: aws
```

Se você ainda tiver uma versão listada na saída, a AWS CLI provavelmente foi instalada usando um método diferente, ou há várias versões. Se você não souber com qual método você instalou a AWS CLI, siga as instruções de cada método de desinstalação para a [AWS CLI versão 1](#) e a [AWS CLI versão 2](#) apropriado para o sistema operacional até que nenhuma saída de versão seja recebida.

Note

Se você usou um gerenciador de pacotes para instalar a AWS CLI (`pip`, `apt`, `brew` etc.), use o mesmo gerenciador de pacotes para desinstalá-la. Siga as instruções fornecidas pelo gerenciador de pacotes sobre como desinstalar todas as versões de um pacote.

[Voltar ao início](#)

A AWS CLI processou um comando com um nome de parâmetro incompleto

Possível causa: você usou uma abreviação reconhecida do parâmetro AWS CLI

Como a AWS CLI é criada usando Python, a AWS CLI usa a biblioteca `argparse` de Python, incluindo o argumento [allow_abbrev](#). As abreviações dos parâmetros são reconhecidas pela AWS CLI e processadas.

O exemplo de comando [create-change-set](#) a seguir altera o nome da pilha do CloudFormation. O parâmetro `--change-set-n` é reconhecido como uma abreviação de `--change-set-name` e a AWS CLI processa o comando.

```
$ aws cloudformation create-change-set --stack-name my-stack --change-set-n my-change-set
```

Quando a abreviação pode ser de vários comandos, o parâmetro não será reconhecido como uma abreviação.

O exemplo de comando [create-change-set](#) a seguir altera o nome da pilha do CloudFormation. O parâmetro `--change-set-` não é reconhecido como uma abreviação, pois há vários parâmetros dos quais ele pode ser uma abreviação, como `--change-set-name` e `--change-set-type`. Portanto, a AWS CLI não processa o comando.

```
$ aws cloudformation create-change-set --stack-name my-stack --change-set- my-change-set
```

Warning

Não use abreviações de parâmetros propositalmente. Elas não são confiáveis e não são compatíveis com versões anteriores. Se algum novo parâmetro for adicionado a um comando que confunda as abreviações, os comandos serão interrompidos.

Além disso, se o parâmetro for um argumento de valor único, ele poderá causar um comportamento inesperado com os comandos. Se várias instâncias de um argumento de valor único forem transmitidas, somente a última instância será executada. No exemplo a seguir, o parâmetro `--filters` é um argumento de valor único. Os parâmetros `--filters` e `--filter` são especificados. O parâmetro `--filter` é uma abreviação

de `--filters`. Isso faz com que duas instâncias de `--filters` sejam aplicadas e somente o último argumento `--filter` se aplica.

```
$ aws ec2 describe-vpc-peering-connections \
  --filters Name=tag:TagName,Values=VpcPeeringConnection \
  --filter Name=status-code,Values=active
```

Confirme se você está usando parâmetros válidos antes de executar um comando para evitar comportamentos inesperados.

[Voltar ao início](#)

Erros de acesso negado

Causa possível: o arquivo do programa AWS CLI não tem permissão de “execução”

No Linux ou no macOS, verifique se o programa `aws` tem permissões de execução para o usuário que está chamando. Normalmente, as permissões são definidas como 755.

Para adicionar permissão de execução ao usuário, execute o comando a seguir, substituindo `~/.local/bin/aws` pelo caminho para o programa no computador:

```
$ chmod +x ~/.local/bin/aws
```

[Voltar ao início](#)

Causa possível: sua identidade do IAM não tem permissão para executar a operação.

Exemplo de texto de erro:

```
$ aws s3 ls
An error occurred (AccessDenied) when calling the ListBuckets operation: Access
denied.
```

Quando você executa um comando da AWS CLI, as operações da AWS são realizadas em seu nome, usando credenciais que associam você a uma conta ou perfil do IAM. As políticas associadas devem conceder permissão para chamar as ações de API que correspondem aos comandos executados com a AWS CLI.

A maioria dos comandos chama uma única ação com um nome que corresponde ao nome do comando. No entanto, comandos personalizados como `aws s3 sync` chamam várias APIs. É possível ver quais APIs um comando chama usando a opção `--debug`.

Se você tiver certeza de que o usuário ou o perfil tem as permissões apropriadas atribuídas pela política, verifique se o comando da AWS CLI está usando as credenciais esperadas. Consulte a [próxima seção sobre credenciais](#) para verificar se as credenciais que a AWS CLI está usando são as esperadas.

Para obter informações sobre como atribuir permissões do IAM, consulte [Visão geral do gerenciamento de acesso: permissões e políticas](#) no Manual do usuário do IAM.

[Voltar ao início](#)

Credenciais inválidas e erros de chave

Exemplo de texto de erro:

```
$ aws s3 ls
An error occurred (InvalidAccessKeyId) when calling the ListBuckets operation: The AWS
Access Key Id
you provided does not exist in our records.
```

```
$ aws s3 ls
An error occurred (InvalidClientTokenId) when calling the ListBuckets operation: The
security token
included in the request is invalid.
```

Causa possível: a AWS CLI está lendo credenciais de um local inesperado.

A AWS CLI pode estar lendo credenciais de um local diferente do que o esperado ou as informações de par de chaves estão incorretas. Execute `aws configure list` para confirmar quais credenciais são usadas.

O exemplo a seguir mostra como verificar as credenciais usadas para o perfil padrão.

```
$ aws configure list
```

Name	Value	Type	Location
----	-----	----	-----

profile	<not set>	None	None
access_key	*****XYVA	shared-credentials-file	
secret_key	*****ZAGY	shared-credentials-file	
region	us-west-2	config-file	~/.aws/config

O exemplo a seguir mostra como verificar as credenciais de um perfil nomeado.

```
$ aws configure list --profile saanvi
```

Name	Value	Type	Location
----	-----	----	-----
profile	saanvi	manual	--profile
access_key	*****	shared-credentials-file	
secret_key	*****	shared-credentials-file	
region	us-west-2	config-file	~/.aws/config

Para confirmar os detalhes do par de chaves, verifique os arquivos config e credentials. Para obter mais informações sobre os arquivos config e credentials, consulte [the section called “Configurações do arquivo de configuração e credenciais”](#). Para obter mais informações sobre credenciais e autenticação, incluindo a precedência de credenciais, consulte [Autenticação e credenciais de acesso](#).

[Voltar ao início](#)

Causa possível: o relógio do computador está fora de sincronia.

Se você estiver usando credenciais válidas, seu relógio poderá estar fora de sincronia. No Linux ou macOS, execute date para verificar a hora.

```
$ date
```

Se o relógio do sistema não estiver correto dentro de alguns minutos, use ntpd para sincronizar.

```
$ sudo service ntpd stop
$ sudo ntpdate time.nist.gov
$ sudo service ntpd start
$ ntpstat
```

No Windows, use as opções de data e hora no Painel de controle para configurar o relógio do sistema.

[Voltar ao início](#)

Assinatura não corresponde aos erros

Exemplo de texto de erro:

```
$ aws s3 ls
```

```
An error occurred (SignatureDoesNotMatch) when calling the ListBuckets operation: The request signature we calculated does not match the signature you provided. Check your key and signing method.
```

Quando a AWS CLI executa um comando, ela envia uma solicitação criptografada para os servidores da AWS a fim de executar as operações de serviço apropriadas da AWS. Suas credenciais (a chave de acesso e a chave secreta) estão envolvidas na criptografia e permitem à AWS autenticar a pessoa que está fazendo a solicitação. Há vários fatores que podem interferir na operação correta desse processo, conforme indicado a seguir.

Causa possível: o relógio está fora de sincronia com os servidores da AWS

Para ajudar a proteger contra [ataques de reprodução](#), a hora atual pode ser usada durante o processo de criptografia/descriptografia. Se a hora do cliente e a hora do servidor não coincidirem além do permitido, o processo poderá falhar e a solicitação será rejeitada. Isso também pode acontecer quando você executa um comando em uma máquina virtual cujo relógio está fora de sincronia com o relógio da máquina host. Uma causa possível é quando a máquina virtual hiberna e, depois de ativada, demora algum tempo para sincronizar o relógio com a máquina host.

No Linux ou macOS, execute `date` para verificar a hora.

```
$ date
```

Se o relógio do sistema não estiver correto dentro de alguns minutos, use `ntpd` para sincronizar.

```
$ sudo service ntpd stop
$ sudo ntpdate time.nist.gov
$ sudo service ntpd start
$ ntpstat
```

No Windows, use as opções de data e hora no Painel de controle para configurar o relógio do sistema.

[Voltar ao início](#)

Causa possível: o sistema operacional está processando incorretamente chaves da AWS que contêm determinados caracteres especiais

Se a chave da AWS incluir determinados caracteres especiais, como -, +, / ou %, algumas variantes do sistema operacional processarão a string incorretamente e farão com que a string de chave seja interpretada incorretamente.

Se você processar as chaves usando outras ferramentas ou scripts, como ferramentas que criam o arquivo de credenciais em uma nova instância como parte de sua criação, essas ferramentas e scripts poderão ter seu próprio tratamento de caracteres especiais que faça com que eles sejam transformados em algo não reconhecido pela AWS.

Sugerimos gerar novamente a chave secreta para obter uma que não inclua o caractere especial que está causando problemas.

[Voltar ao início](#)

Erros de certificado SSL

Possível causa: a AWS CLI não confia no certificado do proxy

Exemplo de texto de erro:

```
$ aws s3 ls
[SSL: CERTIFICATE_ VERIFY_FAILED] certificate verify failed
```

Quando você usa um comando AWS CLI, você recebe uma mensagem de erro de [SSL: CERTIFICATE_ VERIFY_FAILED] certificate verify failed. Isso ocorre porque a AWS CLI não confia no certificado do seu proxy devido a fatores como ele ser autoassinado, com sua empresa definida como a autoridade de certificação (CA). Isso impede que a AWS CLI encontre o certificado raiz de CA da sua empresa no registro de CA local .

Para corrigir isso, instrua a AWS CLI sobre onde encontrar o arquivo .pem da sua empresa usando a opção de arquivo de configuração [ca_bundle](#), a opção de linha de comando [--ca-bundle](#) ou a variável de ambiente [AWS_CA_BUNDLE](#).

[Voltar ao início](#)

Causa possível: sua configuração não está apontando para o local correto do certificado raiz da CA

Exemplo de texto de erro:

```
$ aws s3 ls
SSL validation failed for regionname [Errno 2] No such file or directory
```

Isso é causado devido ao local do arquivo do pacote da Autoridade de Certificação (CA) estar configurado incorretamente na AWS CLI. Para corrigir isso, confirme o local em que o arquivo `.pem` está localizado e atualize a configuração da AWS CLI usando o arquivo de configuração [ca_bundle](#), a opção de linha de comando da [--ca-bundle](#) ou a variável de ambiente [AWS_CA_BUNDLE](#).

[Voltar ao início](#)

Erros JSON inválidos

Exemplo de texto de erro:

```
$ aws dynamodb update-table \
  --provisioned-throughput '{"ReadCapacityUnits":15,WriteCapacityUnits":10}' \
  --table-name MyDDBTable
Error parsing parameter '--provisioned-throughput': Invalid JSON: Expecting property
name enclosed in
double quotes: line 1 column 25 (char 24)
JSON received: {"ReadCapacityUnits":15,WriteCapacityUnits":10}
```

Ao usar um comando da AWS CLI, você recebe a mensagem de erro “Invalid JSON”. Esse erro geralmente aparece quando você insere um comando com um formato JSON esperado e a AWS CLI não consegue ler o JSON corretamente.

Possível causa: você não inseriu um JSON válido para a AWS CLI usar

Confirme se você inseriu um JSON válido para seu comando. Sugerimos usar um validador JSON para o JSON que você está tendo dificuldade de formatar.

Para uso mais avançado de JSON na linha de comando, considere usar um processador JSON de linha de comando, como `jq`, para criar strings JSON. Para obter mais informações sobre o `jq`, consulte o [repositório do jq](#) no GitHub.

[Voltar ao início](#)

Possível causa: as regras de aspas do seu terminal estão impedindo que o JSON válido seja enviado à AWS CLI

Antes de a AWS CLI receber qualquer coisa de um comando, seu terminal processa o comando usando regras próprias de aspas e escape. Devido às regras de formatação de um terminal, parte do seu conteúdo JSON pode ser removido antes de o comando ser passado para a AWS CLI. Ao formular comandos, use as [regras de aspas do seu terminal](#).

Para solucionar problemas, use o echo para ver como o shell está lidando com seus parâmetros:

```
$ echo {"ReadCapacityUnits":15,"WriteCapacityUnits":10}
ReadCapacityUnits:15 WriteCapacityUnits:10
```

```
$ echo '{"ReadCapacityUnits":15,"WriteCapacityUnits":10}'
{"ReadCapacityUnits":15,"WriteCapacityUnits":10}
```

Modifique seu comando até que o JSON válido seja retornado.

Para uma solução de problemas mais detalhada, use o parâmetro `--debug` para visualizar os logs de depuração, pois eles exibirão exatamente o que foi passado para a AWS CLI:

```
$ aws dynamodb update-table \
  --provisioned-throughput '{"ReadCapacityUnits":15,WriteCapacityUnits":10}' \
  --table-name MyDDBTable \
  --debug
2022-07-19 22:25:07,741 - MainThread - awscli.clidriver - DEBUG - CLI version: aws-
cli/1.18.147
Python/2.7.18 Linux/5.4.196-119.356.amzn2int.x86_64 botocore/1.18.6
2022-07-19 22:25:07,741 - MainThread - awscli.clidriver - DEBUG - Arguments entered
to CLI:
['dynamodb', 'update-table', '--provisioned-throughput',
 '{"ReadCapacityUnits":15,WriteCapacityUnits":10}',
 '--table-name', 'MyDDBTable', '--debug']
```

Use as regras de aspas do seu terminal para corrigir quaisquer problemas que a entrada JSON tenha ao ser enviada à AWS CLI. Para obter mais informações sobre regras de aspas, consulte [the section called “Aspas com strings”](#).

 Note

Se estiver enfrentando problemas para obter um JSON válido para a AWS CLI, recomendamos ignorar as regras de aspas do terminal referentes à entrada de dados JSON usando blobs para passar seus dados JSON diretamente à AWS CLI. Para obter mais informações sobre o Blobs, consulte [Blob](#).

[Voltar ao início](#)

Recursos adicionais

Para obter ajuda adicional para problemas de AWS CLI, acesse a [Comunidade da AWS CLI](#) no GitHub ou a [Comunidade da AWS re:Post](#).

[Voltar ao início](#)

Migrar da AWS CLI versão 1 para a versão 2

Esta seção contém instruções para atualizar o AWS CLI versão 1 para o AWS CLI versão 2. A versão 2 da AWS CLI se baseia na versão 1 da AWS CLI e inclui recursos e aprimoramentos com base no feedback da comunidade.

Antes de migrar para a versão 2, [saiba mais sobre as diferenças entre as versões](#). A versão 2 da AWS CLI inclui novos recursos e outras alterações que podem exigir que você atualize seus scripts ou comandos para compatibilidade com versões anteriores.

As versões 1 e 2 da AWS CLI usam o mesmo nome de comando da `aws`. Se você tiver as duas versões instaladas, o computador usará a primeira encontrada no caminho de pesquisa.

Se você instalou anteriormente a versão 1 da AWS CLI, siga nossas [instruções de migração para começar a usar a versão 2](#).

Se você ainda não instalou a versão 1 da AWS CLI, siga as instruções em [Conceitos básicos](#).

Tópicos

- [Novos recursos e alterações na AWS CLI versão 2](#)
- [Instruções para a migração para a AWS CLI versão 2](#)

Novos recursos e alterações na AWS CLI versão 2

Este tópico descreve os novos recursos e as alterações no comportamento entre a AWS CLI versão 1 e a AWS CLI versão 2. Essas alterações podem exigir que você atualize seus scripts ou comandos para obter o mesmo comportamento na versão 2 que tinha na versão 1.

Tópicos

- [Novos recursos da AWS CLI versão 2](#)
- [Alterações de última hora entre a AWS CLI versão 1 e a AWS CLI versão 2](#)

Novos recursos da AWS CLI versão 2

A AWS CLI versão 2 é a versão principal mais recente da AWS CLI e oferece suporte a todos os recursos mais recentes. Alguns recursos apresentados na versão 2 não são compatíveis com a versão 1, e você deve fazer a atualização para acessá-los. Esses recursos incluem o seguinte:

O intérprete Python não é necessário

A AWS CLI versão 2 não precisa de uma instalação separada do Python. Ela inclui uma versão incorporada.

[Assistentes](#)

É possível usar um assistente com a AWS CLI versão 2. O assistente orienta você ao longo do processo para a construção de determinados comandos.

[Autenticação do IAM Identity Center](#)

Se sua organização usar o AWS IAM Identity Center [IAM Identity Center], os usuários poderão entrar no Active Directory, um diretório integrado do IAM Identity Center, ou em [outro IdP conectado ao IAM Identity Center](#). Eles são mapeados para um perfil do AWS Identity and Access Management (IAM) que permite executar comandos da AWS CLI.

[Prompt automático](#)

Quando habilitada, a AWS CLI versão 2 pode solicitar comandos, parâmetros e recursos quando você executa um comando da aws.

[Execute a AWS CLI pelas imagens oficiais do Amazon ECR Public ou do Docker](#)

As imagens oficiais do Docker da AWS CLI fornecem isolamento, portabilidade e segurança com os a AWS é compatível e mantém diretamente. Dessa forma, é possível usar a AWS CLI versão 2 em um ambiente baseado em contêiner sem precisar gerenciar a instalação sozinho.

[Paginação do lado do cliente](#)

A AWS CLI versão 2 fornece um programa de paginação do lado do cliente para uso na saída. Por padrão, esse recurso está ativado e retorna toda a saída pelo programa de pager padrão do sistema operacional.

[aws configure import](#)

Importe as credenciais de .csv geradas no AWS Management Console. Um arquivo .csv é importado com o nome do perfil correspondente ao nome do usuário do IAM.

[aws configure list-profiles](#)

Lista os nomes de todos os perfis que você configurou.

[the section called “Formato da saída de fluxo do YAML”](#)

Os formatos yaml e yaml-stream se beneficiam do formato [YAML](#) e oferecem uma visualização mais responsiva de grandes conjuntos de dados ao fazer streaming de dados para

you. You can start visualizing and using YAML data before downloading the entire query.

[Novos comandos de alto nível de **ddb** para o DynamoDB](#)

The AWS CLI version 2 has the Amazon DynamoDB high-level [ddb put](#) and [ddb select](#) commands. These commands provide a simplified interface for putting items in DynamoDB tables and for querying a table or index in DynamoDB.

[aws logs tail](#)

The AWS CLI version 2 has a `aws logs tail` command that follows the logs of a group in Amazon CloudWatch Logs. By default, the command returns logs from all log streams associated with the CloudWatch group during the last 10 minutes.

[Adicionado suporte a metadados para comandos de alto nível da **s3**](#)

The AWS CLI version 2 adds the `--copy-props` parameter to the high-level `s3` command. With this parameter, you can configure additional metadata and tags for Amazon Simple Storage Service (Amazon S3).

[AWS_REGION](#)

The AWS CLI version 2 has an environment variable compatible with the AWS SDK called `AWS_REGION`. This variable specifies the AWS Region for which to send requests. It replaces the `AWS_DEFAULT_REGION` environment variable, which is applicable only to the AWS CLI.

Alterações de última hora entre a AWS CLI versão 1 e a AWS CLI versão 2

This section describes the changes in behavior between the AWS CLI version 1 and the AWS CLI version 2. These changes may require you to update your scripts or commands to get the same behavior in version 2 that you had in version 1.

Tópicos

- [Variável de ambiente adicionada para definir codificação de arquivo de texto](#)
- [Os parâmetros binários são passados como strings codificadas em base64, por padrão.](#)
- [Manuseio aprimorado do Amazon S3 de propriedades e tags de arquivos cópias fracionadas.](#)
- [Nenhuma recuperação automática de URLs `http://` ou `https://` para parâmetros](#)

- [Pager usado para todas as saídas por padrão](#)
- [Os valores de saída de carimbo de timestamp são padronizados para o formato ISO 8601](#)
- [Manuseio aprimorado das implantações do CloudFormation que resulta em nenhuma alteração](#)
- [Comportamento padrão alterado para endpoint do Amazon S3 regional para a região us-east-1](#)
- [Comportamento padrão alterado para endpoints regionais do AWS STS](#)
- [ecr get-login removido e substituído por ecr get-login-password](#)
- [O suporte da AWS CLI versão 2 a plugins está sendo alterado](#)
- [Suporte a alias oculto removido](#)
- [A configuração do arquivo de configuração api_versions não é compatível](#)
- [A AWS CLI versão 2 usa apenas o Signature v4 para autenticar solicitações do Amazon S3.](#)
- [A AWS CLI versão 2 é mais consistente com os parâmetros de paginação](#)
- [A AWS CLI versão 2 fornece códigos de retorno mais consistentes em todos os comandos](#)

Variável de ambiente adicionada para definir codificação de arquivo de texto

Por padrão, os arquivos de texto para [the section called “Blob”](#) usam a mesma codificação que o locale instalado. Como a AWS CLI versão 2 usa uma versão incorporada do Python, as variáveis de ambiente PYTHONUTF8 e PYTHONIOENCODING não são compatíveis. Para definir uma codificação para arquivos de texto diferente da localidade, use a variável de ambiente AWS_CLI_FILE_ENCODING. O exemplo a seguir define a AWS CLI para abrir arquivos de texto usando o UTF-8 do Windows.

```
AWS_CLI_FILE_ENCODING=UTF-8
```

Para obter mais informações, consulte [Variáveis de ambiente para configurar a AWS CLI](#)

Os parâmetros binários são passados como strings codificadas em base64, por padrão.

Na AWS CLI, alguns comandos exigiam strings codificadas em [base64](#), enquanto outras exigiam strings de bytes codificadas em UTF-8. Na AWS CLI versão 1, passar dados entre dois tipos de string codificados, muitas vezes exigia algum processamento intermediário. A AWS CLI versão 2 torna o manuseio de parâmetros binários mais consistente, o que ajuda a passar valores de um comando para outro de forma mais confiável.

Por padrão, a AWS CLI versão 2 passa todos os parâmetros de entrada e saída binária como strings codificadas em base64 blobs (objeto binário grande). Para obter mais informações, consulte [the section called “Blob”](#).

Para reverter para o comportamento da AWS CLI versão 1, use a configuração de arquivo [cli_binary_format](#) ou o parâmetro [--cli-binary-format](#).

Manuseio aprimorado do Amazon S3 de propriedades e tags de arquivos cópias fracionadas.

Quando você usa os comandos da AWS CLI versão 1 no namespace do `aws s3` para copiar um arquivo de um local de bucket do S3 para outro, e essa operação usa [multipart copy](#), nenhuma propriedade de arquivo do objeto de origem é copiada para o objeto de destino.

Por padrão, os comandos correspondentes na AWS CLI versão 2 transferem todas as tags e algumas das propriedades da origem para a cópia de destino. Em comparação com a AWS CLI versão 1, isso pode resultar em mais chamadas da API da AWS sendo feitas para o endpoint do Amazon S3. Para alterar o comportamento padrão dos comandos do `s3` na AWS CLI versão 2, use o parâmetro `--copy-props`.

Para obter mais informações, consulte [the section called “Propriedades e tags de arquivos em cópias com várias partes”](#).

Nenhuma recuperação automática de URLs **http://** ou **https://** para parâmetros

A AWS CLI versão 2 não executa uma operação GET quando um valor de parâmetro começa com `http://` ou `https://` e não usa o conteúdo retornado como o valor do parâmetro. Como resultado, a opção de linha do comando associado `cli_follow_urlparam` é removida da AWS CLI versão 2.

Se você precisar recuperar um URL e passar o conteúdo desse URL como o valor de um parâmetro, recomendamos usar `curl` ou uma ferramenta semelhante para baixar o conteúdo do URL em um arquivo local. A seguir, use a sintaxe `file://` para ler o conteúdo desse arquivo e usá-lo como o valor do parâmetro.

Por exemplo, o comando a seguir não tenta mais recuperar o conteúdo da página encontrada em `http://www.example.com` e passar esses conteúdos como o parâmetro. Em vez disso, ele passa a string de texto literal `https://example.com` como o parâmetro.

```
$ aws ssm put-parameter \
```

```
--value http://www.example.com \  
--name prod.microservice1.db.secret \  
--type String 2
```

Se precisar recuperar e usar o conteúdo de um URL da web como um parâmetro, você poderá fazer o seguinte na versão 2.

```
$ curl https://my.example.com/mypolicyfile.json -o mypolicyfile.json  
$ aws iam put-role-policy \  
  --policy-document file:///./mypolicyfile.json \  
  --role-name MyRole \  
  --policy-name MyReadOnlyPolicy
```

No exemplo anterior, o parâmetro `-o` diz ao `curl` para salvar o arquivo na pasta atual com o mesmo nome que o arquivo de origem. O segundo comando recupera o conteúdo desse arquivo baixado e passa-o como o valor de `--policy-document`.

Pager usado para todas as saídas por padrão

Por padrão, a AWS CLI versão 2 retorna toda a saída pelo programa de paginação padrão do sistema operacional. Por padrão, esse programa é o [less](#) no Linux ou no macOS, e o programa [more](#) no Windows. Isso ajuda na navegação de uma grande quantidade de saída de um serviço, exibindo essa saída uma página de cada vez.

É possível configurar a AWS CLI versão 2 para usar um programa de paginação diferente, ou não usar nenhum. Para obter mais informações, consulte [the section called “Paginação do lado do cliente”](#).

Os valores de saída de carimbo de timestamp são padronizados para o formato ISO 8601

Por padrão, a AWS CLI versão 2 retorna todos os valores de resposta de timestamp no [formato ISO 8601](#). Na AWS CLI versão 1, os comandos retornavam valores de marca de data e hora em qualquer formato que era retornado pela resposta da API HTTP, que poderia variar de serviço para serviço.

Para ver timestamps no formato retornado pela resposta da API HTTP, use o valor `wire` no arquivo de config. Para obter mais informações, consulte [cli_timestamp_format](#).

Manuseio aprimorado das implantações do CloudFormation que resulta em nenhuma alteração

Na AWS CLI versão 1, se você implantava um modelo do AWS CloudFormation que resultava em nenhuma alteração, por padrão, a AWS CLI retornava um código de erro de falha. Isso causa problemas se você não considerar isso como um erro e quiser que o script continue. Você pode contornar isso na AWS CLI versão 1 adicionando o sinalizador `--no-fail-on-empty-changeset`, que retorna `0`.

Como esse é o cenário de caso comum, a AWS CLI versão 2 padroniza o retorno de um código de saída bem-sucedido de `0` quando não há alterações causadas por uma implantação e a operação retorna um conjunto de alterações vazio.

Para reverter para o comportamento original, adicione o sinalizador `--fail-on-empty-changeset`.

Comportamento padrão alterado para endpoint do Amazon S3 regional para a região **us-east-1**

Ao configurar a AWS CLI versão 1 para usar a região `us-east-1`, a região AWS CLI usa o endpoint global da `s3.amazonaws.com`, que estava fisicamente hospedado na região `us-east-1`. A AWS CLI versão 2 usa o verdadeiro endpoint regional `s3.us-east-1.amazonaws.com` quando essa região é especificada. Para forçar a AWS CLI versão 2 a usar o endpoint global, é possível definir a região para um comando como `aws-global`.

Comportamento padrão alterado para endpoints regionais do AWS STS

Por padrão, a AWS CLI versão 2 envia todas as solicitações de API do AWS Security Token Service (AWS STS) para o endpoint regional da Região da AWS atualmente configurada.

Por padrão, a AWS CLI versão 1 envia solicitações do AWS STS para o endpoint global do AWS STS. É possível controlar esse comportamento padrão na versão 1 usando a configuração [sts_regional_endpoints](#).

ecr get-login removido e substituído por **ecr get-login-password**

A AWS CLI versão 2 substitui o comando `aws ecr get-login` pelo novo comando `aws ecr get-login-password` para melhorar a integração automatizada com a autenticação do contêiner.

O comando `aws ecr get-login-password` reduz o risco de exposição das suas credenciais na lista de processos, histórico de shell ou outros arquivos de log. Também melhora a compatibilidade com o comando `docker login` para oferecer uma melhor automação.

O comando `aws ecr get-login-password` está disponível na AWS CLI versão 1.17.10 e posterior e a AWS CLI versão 2. O comando `aws ecr get-login` mais antigo ainda está disponível na AWS CLI versão 1 para compatibilidade com versões anteriores.

Com o comando `aws ecr get-login-password`, é possível substituir o seguinte código para recuperar uma senha.

```
$ (aws ecr get-login --no-include-email)
```

Para reduzir o risco de exposição de senhas no histórico do shell ou logs, use o comando de exemplo a seguir. Neste exemplo, a senha é canalizada diretamente para o comando `docker login`, em que é atribuída ao parâmetro de senha pela opção `--password-stdin`.

```
$ aws ecr get-login-password | docker login --username AWS --password-stdin MY-REGISTRY-URL
```

Para obter mais informações, consulte [aws ecr get-login-password](#) no Guia de referência da AWS CLI versão 2.

O suporte da AWS CLI versão 2 a plugins está sendo alterado

O suporte a plug-ins na AWS CLI versão 2 é completamente provisório e destina-se a ajudar os usuários a migrar da AWS CLI versão 1 até que uma interface estável e atualizada de plug-ins seja liberada. Não há garantia de que um plug-in específico ou até mesmo a interface de plug-ins da AWS CLI será compatível em versões futuras da AWS CLI versão 2. Se você depender de plug-ins, restrinja o seu uso a uma versão específica da AWS CLI e teste a funcionalidade do plug-in após a atualização.

Para habilitar o suporte a plugins, crie uma seção `[plugins]` no `~/.aws/config`.

```
[plugins]
cli_legacy_plugin_path = <path-to-plugins>/python3.7/site-packages
<plugin-name> = <plugin-module>
```

Na seção `[plugins]`, defina a variável `cli_legacy_plugin_path` e seu valor para o caminho dos pacotes do site do Python em que se encontra o módulo do plugin. Depois, é possível configurar

um plug-in fornecendo um nome a ele (`plugin-name`) e o nome do arquivo do módulo Python, (`plugin-module`), que contém o código-fonte do plug-in. A AWS CLI carrega cada plug-in importando sua `plugin-module` e chamando a função `awscli_initialize`.

Suporte a alias oculto removido

A AWS CLI versão 2 não oferece mais suporte aos aliases ocultos a seguir que eram compatíveis na versão 1.

Na tabela a seguir, a primeira coluna exibe o serviço, o comando e o parâmetro que funcionam em todas as versões, incluindo a AWS CLI versão 2. A segunda coluna exibe os alias que não funciona mais na AWS CLI versão 2.

Serviço de trabalho, comando e parâmetro	Alias obsoleto
<code>cognito-identity create-identity-pool open-id-connect-provider-ar-ns</code>	<code>open-id-connect-provider-ar-ns</code>
<code>storagegateway describe-tapes tape-ar-ns</code>	<code>tape-ar-ns</code>
<code>storagegateway.describe-tape-archives.tape-ar-ns</code>	<code>tape-ar-ns</code>
<code>storagegateway.describe-vtl-devices.vtl-device-ar-ns</code>	<code>vtl-device-ar-ns</code>
<code>storagegateway.describe-cached-iscsi-volumes.volume-ar-ns</code>	<code>volume-ar-ns</code>
<code>storagegateway.describe-stored-iscsi-volumes.volume-ar-ns</code>	<code>volume-ar-ns</code>
<code>route53domains.view-billing.start-time</code>	<code>start</code>
<code>deploy.create-deployment-group.ec2-tag-set</code>	<code>ec-2-tag-set</code>
<code>deploy.list-application-revisions.s3-bucket</code>	<code>s-3-bucket</code>
<code>deploy.list-application-revisions.s3-key-prefix</code>	<code>s-3-key-prefix</code>
<code>deploy.update-deployment-group.ec2-tag-set</code>	<code>ec-2-tag-set</code>
<code>iam.enable-mfa-device.authentication-code1</code>	<code>authentication-code-1</code>
<code>iam.enable-mfa-device.authentication-code2</code>	<code>authentication-code-2</code>

Serviço de trabalho, comando e parâmetro	Alias obsoleto
iam.resync-mfa-device.authentication-code1	authentication-code-1
iam.resync-mfa-device.authentication-code2	authentication-code-2
importexport.get-shipping-label.street1	street-1
importexport.get-shipping-label.street2	street-2
importexport.get-shipping-label.street3	street-3
lambda.publish-version.code-sha256	code-sha-256
lightsail.import-key-pair.public-key-base64	public-key-base-64
opsworks.register-volume.ec2-volume-id	ec-2-volume-id

A configuração do arquivo de configuração **api_versions** não é compatível

A AWS CLI versão 2 não oferece mais suporte a chamadas a versões mais antigas das APIs de serviço da AWS usando a definição do arquivo de configuração `api_versions`. Todos os comandos da AWS CLI agora chamam a versão mais recente das APIs de serviço atualmente suportadas pelo endpoint.

A AWS CLI versão 2 usa apenas o Signature v4 para autenticar solicitações do Amazon S3.

A AWS CLI versão 2 não oferece suporte a algoritmos de assinatura anteriores para autenticar criptograficamente solicitações de serviço enviadas para endpoints do Amazon S3. Essa assinatura acontece automaticamente com todas as solicitações do Amazon S3 e somente o [processo de assinatura do Signature versão 4](#) é compatível. Você não pode configurar a versão da assinatura. Todos os URLs pré-assinados de bucket do Amazon S3 agora usam apenas o SigV4 e têm uma duração máxima de expiração de uma semana.

A AWS CLI versão 2 é mais consistente com os parâmetros de paginação

Na AWS CLI versão 1, se você especificar parâmetros de paginação na linha de comando, a paginação automática será desativada conforme o esperado. No entanto, quando você especifica parâmetros de paginação usando um arquivo com o parâmetro `--cli-input-json`, a paginação

automática não foi desativada, o que poderia resultar em uma saída inesperada. A AWS CLI versão 2 desativa a paginação automática, independentemente de como você fornece os parâmetros.

A AWS CLI versão 2 fornece códigos de retorno mais consistentes em todos os comandos

A AWS CLI versão 2 é mais consistente em todos os comandos e retorna corretamente um código de saída apropriado em comparação com a AWS CLI versão 1. Também adicionamos os códigos de saída 252, 253 e 254. Para obter mais informações sobre códigos de saída, consulte [the section called “Códigos de retorno”](#).

Se você tiver uma dependência de como a AWS CLI versão 1 usa valores de código de retorno, recomendamos verificar os códigos de saída para garantir que você esteja obtendo os valores esperados.

Instruções para a migração para a AWS CLI versão 2

Este tópico fornece instruções para a migração da AWS CLI versão 1 para a AWS CLI versão 2.

As versões 1 e 2 da AWS CLI usam o mesmo nome de comando da `aws`. Se você tiver as duas versões instaladas, o computador usará a primeira encontrada no caminho de pesquisa. Se você já instalou a AWS CLI versão 1, recomendamos executar um dos seguintes procedimentos para usar a AWS CLI versão 2:

- Recomendado: [desinstale a AWS CLI versão 1 e use apenas a AWS CLI versão 2](#).
- [Para ter as duas versões instaladas](#), use a capacidade do sistema operacional de criar um link simbólico (symlink) ou um alias com um nome diferente para um dos dois comandos da `aws`.

Para obter informações sobre as principais diferenças entre a versão 1 e a versão 2, consulte [the section called “Novos recursos e alterações”](#).

Substituindo a versão 1 pela versão 2

Siga estes passos para substituir a AWS CLI versão 1 pela AWS CLI versão 2.

Como substituir a AWS CLI versão 1 pela AWS CLI versão 2

1. Prepare todos os scripts existentes para a migração, confirmando quaisquer todas as últimas alterações entre a versão 1 e a versão 2 na [the section called “Novos recursos e alterações”](#).

2. Desinstale a AWS CLI versão 1 seguindo as instruções de desinstalação do seu sistema operacional em [Instalar, atualizar e desinstalar a AWS CLI versão 1](#).
3. Confirme que a AWS CLI está completamente desinstalada usando o comando a seguir.

```
$ aws --version
```

Conclua uma das seguintes opções com base na saída:

- Nenhuma versão retornada: você desinstalou com sucesso a AWS CLI Versão 1 e pode prosseguir para o próximo passo.
 - Uma versão é retornada: você ainda tem uma instalação da AWS CLI versão 1. Para obter etapas sobre a solução de problemas, consulte [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#). Execute os passos de solução de problemas até que nenhuma saída de versão seja recebida.
4. Instale a AWS CLI versão 2 seguindo as instruções de instalação apropriadas do seu sistema operacional em [Instalar ou atualizar para a versão mais recente da AWS CLI](#).

Instalação lado a lado

Para ter as duas versões instaladas, use a capacidade do sistema operacional de criar um link simbólico (symlink) ou um alias com um nome diferente para um de dois comandos da aws.

1. Instale a AWS CLI versão 2 seguindo as instruções de instalação apropriadas do seu sistema operacional em [Instalar ou atualizar para a versão mais recente da AWS CLI](#).
2. Use a capacidade do seu sistema operacional para criar um alias ou um symlink com um nome diferente para um de dois comandos da aws, como usar **aws2** para a AWS CLI versão 2. A seguir estão exemplos de symlinks para a AWS CLI versão 2. Substitua o **CAMINHO** com pelo seu local de instalação.

Linux and macOS

Você pode usar um [link simbólico](#) ou um [alias](#) no Linux e no macOS.

```
$ alias aws2='PATH'
```

Windows command prompt

[DOSKEY](#) no Windows.

```
C:\> doskey aws2=PATH
```

Desinstalar a AWS CLI versão 2

Este tópico descreve como desinstalar a AWS Command Line Interface versão 2 (AWS CLI versão 2).

Instruções de instalação da AWS CLI versão 2:

Linux

Para desinstalar a AWS CLI versão 2, execute os comandos a seguir.

1. Localize o symlink e instale caminhos.

- Use o comando `which` para encontrar o symlink. Isso mostra o caminho usado com o parâmetro `--bin-dir`.

```
$ which aws
/usr/local/bin/aws
```

- Use o comando `ls` para localizar o diretório para o qual o symlink aponta. Isso fornece o caminho usado com o parâmetro `--install-dir`.

```
$ ls -l /usr/local/bin/aws
lrwxrwxrwx 1 ec2-user ec2-user 49 Oct 22 09:49 /usr/local/bin/aws -> /usr/local/
aws-cli/v2/current/bin/aws
```

2. Exclua os dois symlinks no diretório `--bin-dir`. Se o seu usuário tiver permissão de gravação nesses diretórios, não será necessário usar `sudo`.

```
$ sudo rm /usr/local/bin/aws
$ sudo rm /usr/local/bin/aws_completer
```

3. Exclua o diretório `--install-dir`. Se o seu usuário tiver permissão de gravação nesse diretório, não será necessário usar `sudo`.

```
$ sudo rm -rf /usr/local/aws-cli
```

4. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

⚠ Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `~/.aws/`. Se o seu usuário tiver permissão de gravação nesse diretório, não será necessário usar `sudo`.

```
$ sudo rm -rf ~/.aws/
```

macOS

Para desinstalar a AWS CLI versão 2, execute os comandos a seguir, substituindo os caminhos usados para instalar. Os comandos de exemplo usam os caminhos de instalação padrão.

1. Localize a pasta que contém os symlinks para o programa principal e o completer.

```
$ which aws  
/usr/local/bin/aws
```

2. Com essas informações, execute o comando a seguir para localizar a pasta de instalação para a qual os symlinks apontam.

```
$ ls -l /usr/local/bin/aws  
lrwxrwxrwx 1 ec2-user ec2-user 49 Oct 22 09:49 /usr/local/bin/aws -> /usr/local/  
aws-cli/aws
```

3. Exclua os dois symlinks na primeira pasta. Se o seu usuário tiver permissão de gravação nessas pastas, você não precisará usar `sudo`.

```
$ sudo rm /usr/local/bin/aws  
$ sudo rm /usr/local/bin/aws_completer
```

4. Exclua a pasta de instalação principal. Use `sudo` para obter acesso de gravação à pasta `/usr/local`.

```
$ sudo rm -rf /usr/local/aws-cli
```

5. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

 Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `~/.aws/`. Se o seu usuário tiver permissão de gravação nesse diretório, não será necessário usar `sudo`.

```
$ sudo rm -rf ~/.aws/
```

Windows

1. Abra Programas e Recursos seguindo um destes procedimentos:

- Abra o Painel de Controle e selecione Programas e Recursos.
- Abra um prompt de comando e insira o comando a seguir.

```
C:\> appwiz.cpl
```

2. Selecione a entrada denominada AWS Command Line Interface e escolha Uninstall (Desinstalar) para executar o desinstalador.
3. Confirme que deseja desinstalar a AWS CLI.
4. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

 Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `%UserProfile%\aws`.

```
$ rmdir %UserProfile%\aws
```

Solução de problemas de erros de instalação e desinstalação da AWS CLI

Se você encontrar problemas após instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas. Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).

Histórico do documento do guia do usuário da AWS CLI

A tabela a seguir descreve adições importantes feitas ao Manual do usuário da AWS Command Line Interface a partir de janeiro de 2019. Para receber notificações sobre atualizações dessa documentação, você pode se inscrever em o feed RSS.

Alteração	Descrição	Data
Informações atualizadas sobre credenciais e autenticação.	Instruções e exemplos atualizados de credenciais e métodos de autenticação. Isso inclui a atualização de páginas de conceitos básicos e páginas de configuração relevantes. Para acomodar esse aumento na documentação, tópicos relevantes sobre credenciais foram movidos para a nova seção Autenticação e credenciais de acesso .	31 de março de 2023
Configuração do provedor de tokens com atualização automática de autenticação para AWS IAM Identity Center adicionada	O novo processo de configuração da AWS CLI para autenticar usuários com o AWS IAM Identity Center (Centro de Identidade do IAM) usando a configuração do provedor de tokens do SSO, que podem recuperar automaticamente tokens de autenticação atualizados.	7 de dezembro de 2022
Imagem pública oficial do Amazon ECR para a AWS CLI versão 2 lançada	A imagem pública oficial compatível do Amazon ECR para a AWS CLI versão 2 é	18 de novembro de 2022

lançada para Linux, macOS e Windows.

[Atualização do guia para migrar da AWS CLI V1 para a V2](#)

Ampliado o guia de alterações mais recentes para incluir instruções de migração para ir da AWS CLI versão 1 para a AWS CLI versão 2. Contém atualizações na página Troubleshooting (Solução de problemas) para ajudar com problemas de instalação.

13 de maio de 2022

[Novo processo para criar um instalador da AWS CLI pela fonte.](#)

Novo processo para instalar ou atualizar da fonte para a versão mais recente da AWS CLI em sistemas operacionais compatíveis.

17 de fevereiro de 2022

[Os conteúdos referentes à AWS CLI V1 e V2 agora estão separados nos respectivos guias](#)

Para maior clareza e facilidade e, o conteúdo da AWS CLI versão 1 e da AWS CLI versão 2 agora está separado nos próprios guias. Para a AWS CLI versão 1, consulte o [Guia do usuário da AWS CLI versão 1](#), mais recente

2 de novembro de 2021

[Adição de informações sobre o alias AWS CLI.](#)

Adição de informações sobre o alias AWS CLI. Os aliases são atalhos que você pode criar na AWS Command Line Interface (AWS CLI) para encurtar comandos ou scripts que utiliza com frequência.

11 de março de 2021

Informações atualizadas sobre saídas de filtros	Informações sobre filtros atualizadas e movidas para suas próprias páginas.	1º de fevereiro de 2021
Adição de informações sobre assistentes	Adição de informações sobre assistentes na AWS CLI versão 2.	20 de novembro de 2020
Prompt automático atualizado	Atualização das informações sobre prompt automático da AWS CLI versão 2 com recursos atuais.	10 de novembro de 2020
Adição de exemplo de script do Amazon S3	Adição de um exemplo de script de ciclo de vida do Amazon S3.	15 de outubro de 2020
Adição de exemplo de script do Amazon EC2	Adição de um exemplo de script de tipo de instância do Amazon EC2.	15 de outubro de 2020
Adição de informações sobre novas tentativas	Adição de uma página sobre novas tentativas para recursos e comportamento de novas tentativas na AWS CLI.	17 de setembro de 2020
Página sobre paginação nos lados do servidor e do cliente	Informações sobre paginação atualizadas e centralizadas em uma única página.	17 de agosto de 2020
Página de comandos do s3 atualizada	Página de comandos de alto nível do s3 atualizada com novos exemplos e recursos.	30 de julho de 2020
Atualização de informações sobre a instalação	As informações de instalação, atualização e desinstalação para Linux, macOS e Windows foram atualizadas.	19 de maio de 2020

Adição de informações para codificação de arquivos de texto na AWS CLI versão 2	Por padrão, a AWS CLI versão 2 usa a mesma codificação de arquivos de texto que o local. Agora você pode usar variáveis de ambiente para definir a codificação de arquivo de texto.	14 de maio de 2020
Imagem oficial do Docker para a AWS CLI versão 2 lançada	A imagem oficial de suporte do Docker para a AWS CLI versão 2 é lançada para todos os Linux, macOS e Windows.	31 de março de 2020
Adição de informações sobre pagers no lado do cliente para a AWS CLI versão 2	Por padrão, a AWS CLI versão 2 usa o programa de paginação less para todas as saídas no lado do cliente.	19 de fevereiro de 2020
A AWS Command Line Interface (AWS CLI) versão 2 foi lançada oficialmente	A AWS CLI versão 2 está disponível para o público e é a versão recomendada para ser instalada pelos clientes.	10 de fevereiro de 2020
O instalador do macOS da AWS CLI versão 2 agora é um arquivo .pkg do instalador de pacotes da Apple.	O instalador AWS CLI versão 2 para macOS foi atualizado de um arquivo .zip com um script de shell para o pacote do instalador para macOS completo. Isso simplifica a instalação e a torna compatível com as versões mais recentes do macOS.	3 de fevereiro de 2020

<u>Adição de conteúdo para o tratamento padrão aprimorado da AWS CLI versão 2 para S3 e endpoints regionais do STS.</u>	Por padrão, a AWS CLI versão 2 agora direciona as solicitações para o Amazon S3 e os serviços da AWS STS para o endpoint regional configurado no momento, em vez do endpoint global.	13 de janeiro de 2020
<u>Versão de visualização do desenvolvedor da AWS CLI versão 2</u>	Apresentação da versão de pré-visualização da AWS CLI versão 2. Adicionamos instruções sobre como instalar a versão 2. Adicione o tópico de migração para discutir as diferenças entre as versões 1 e 2.	7 de novembro de 2019
<u>Adição de suporte para AWS IAM Identity Center aos perfis nomeados da AWS CLI</u>	A versão 2 da AWS CLI adiciona compatibilidade para criar um perfil nomeado que pode fazer login diretamente no IAM Identity Center e obter credenciais temporárias da AWS para uso em comandos subsequentes da AWS CLI.	7 de novembro de 2019
<u>Nova seção MFA</u>	Adicionada uma nova seção que descreve como acessar a CLI usando a autenticação multifator e funções.	3 de maio de 2019
<u>Atualização da seção “Uso da CLI”</u>	Principais melhorias e adições às instruções e aos procedimentos de uso.	7 de março de 2019

[Atualização da seção "Instalar a CLI"](#)

Principais melhorias e adições às instruções e aos procedimentos de instalação da AWS CLI.

7 de março de 2019

[Atualização da seção "Configurar a CLI"](#)

Principais melhorias e adições às instruções e aos procedimentos de configuração da AWS CLI.

7 de março de 2019

AWS Glossário

Para obter a terminologia mais recente da AWS, consulte o [glossário da AWS](#) na Referência do Glossário da AWS.